Frontend Design Patterns

A Complete Guide

Master JavaScript and React Patterns for Scalable Applications

Design Patterns • Component Patterns • Architecture

2025 Edition • 32 Chapters

Table of Contents

Chapter 1 — What Are Design Patterns
Chapter 2 — Module Pattern
Chapter 3 — Singleton Pattern
Chapter 4 — Observer Pattern
Chapter 5 — Pub/Sub Pattern
Chapter 6 — Strategy Pattern
Chapter 7 — Iterator Pattern
Chapter 8 — Proxy Pattern
Chapter 9 — Decorator Pattern
Chapter 10 — Facade Pattern
Chapter 11 — Factory Pattern
Chapter 12 — Mediator Pattern
Chapter 13 — Mixin Pattern
Chapter 14 — Component Composition
Chapter 15 — Container/Presentational
Chapter 16 — Higher-Order Components
Chapter 17 — Render Props
Chapter 18 — Custom Hooks Pattern
Chapter 19 — Compound Components
Chapter 20 — Controlled & Uncontrolled
Chapter 21 — Provider Pattern
Chapter 22 — State Reducer Pattern
Chapter 23 — State Machine Pattern
Chapter 24 — Flux/Redux Pattern
Chapter 25 — Rendering Patterns
Chapter 26 — Code Splitting & Lazy Loading
Chapter 27 — Virtualization Pattern

Chapter 28 — Memoization Pattern
Chapter 29 — Error Boundary Pattern
Chapter 30 — Micro-Frontends
Chapter 31 — Project Pattern Library
Chapter 32 — Glossary

Chapter 1: What Are Design Patterns?

What You Will Learn

- What design patterns are and where they came from
- Why patterns matter in frontend development
- The main categories of design patterns
- How to recognize when you need a pattern
- What anti-patterns are and how to avoid them
- How to read the pattern chapters in this book

Why This Chapter Matters

Imagine you just moved into a new apartment. You need to hang a picture on the wall. You could invent your own method from scratch. Maybe glue the picture directly to the wall? Tape it with duct tape? Drill random holes and hope for the best?

Or you could use the method that millions of people before you have figured out works really well: a nail, a hammer, and a picture hook.

That picture hook method is a design pattern. Someone solved this problem before. They figured out the best approach. They shared it with others. Now everyone benefits.

Software design patterns work the same way. They are proven solutions to problems that developers face again and again. You do not need to reinvent the wheel every time you build a feature. Instead, you reach into your toolbox and pick the right pattern.

This chapter sets the foundation for everything else in this book. Before we dive into specific patterns, you need to understand what patterns are, why they exist, and how to think about them.

The Origin Story: Gang of Four

In 1994, four computer scientists published a book that changed software engineering forever. Their names were Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. The book was called Design Patterns: Elements of Reusable Object-Oriented Software.

Because remembering four names is hard, developers just call them the Gang of Four (or GoF for short).

The Gang of Four documented 23 design patterns. They did not invent these patterns. They observed what experienced developers were already doing, gave each pattern a name, and wrote down when and how to use it.

Think of it like a cookbook. The Gang of Four did not invent cooking. They watched great chefs, wrote down their recipes, and organized them so anyone could follow along.

```
Before the Gang of Four:

Developer A: "I made this thing where only one instance exists..."
Developer B: "I did something similar but I call it different..."
Developer C: "What are you two talking about?"

After the Gang of Four:

Developer A: "I used the Singleton pattern."
Developer B: "Me too!"
Developer C: "Oh, I know that one. Makes sense."
```

The GoF book was written for object-oriented languages like C++ and Java. But the core ideas translate beautifully to JavaScript and frontend development. Some patterns map directly. Others inspired new patterns specific to the frontend world.

What Exactly Is a Design Pattern?

A design pattern has four essential parts:

1. A name -- so developers can talk about it quickly
2. A problem -- the situation where this pattern helps
3. A solution -- the approach to solving the problem

4. Trade-offs -- what you gain and what you give up

Let us use a real-life analogy. Think about a traffic roundabout.

- Name: Roundabout
- Problem: Cars arriving from multiple directions need to pass through an intersection without crashing
- Solution: Build a circular road. Cars enter, go around, and exit at their desired road
- Trade-offs: Slower than a green light on an empty road, but much safer and no waiting at red lights

A design pattern is NOT:

- A library you install with npm
- A framework feature
- Copy-paste code that works everywhere
- A rigid rule you must always follow

A design pattern IS:

- A template for solving a problem
- A common vocabulary for developers
- A starting point you adapt to your situation
- A way to think about code organization

Why Patterns Matter in Frontend Development

Frontend development has unique challenges:

```
+-----------------------------------------+
|          Frontend Challenges            |
+-----------------------------------------+
|                                         |
|  1. User interfaces are complex         |
|     - Many components talking to each   |
|       other                             |
|     - State changes everywhere          |
|     - User interactions are unpredictable |
|                                         |
|  2. Code grows fast                     |
|     - Small apps become big apps        |
|     - Teams grow, more people touch     |
|       the same code                     |
|     - Features pile up                  |
```

```
|                                          |
|  3. Performance matters                  |
|     - Users notice slow interfaces       |
|     - Mobile devices have less power     |
|     - Every millisecond counts           |
|                                          |
|  4. Maintenance is ongoing               |
|     - Bugs need fixing                   |
|     - Features need adding              |
|     - Code needs updating                |
|                                          |
+------------------------------------------+
```

Design patterns help with all of these. Here is a concrete example.

The Problem Without Patterns

```javascript
// app.js -- everything in one place, no organization

// Global variables everywhere
let userName = '';
let userEmail = '';
let cartItems = [];
let cartTotal = 0;
let isLoggedIn = false;
let theme = 'light';
let language = 'en';
let notifications = [];

// Functions that touch everything
function addToCart(item) {
  cartItems.push(item);
  cartTotal += item.price;
  updateCartUI();
  sendAnalytics('add_to_cart', item);
  if (isLoggedIn) {
    saveCartToServer(cartItems);
  }
  showNotification('Item added to cart!');
}

function updateCartUI() {
  // Directly manipulates the DOM
  document.getElementById('cart-count').textContent = cartItems.length;
  document.getElementById('cart-total').textContent = '$' + cartTotal;
  // ... 50 more lines of DOM manipulation
}

function sendAnalytics(event, data) {
  // Tightly coupled to a specific analytics provider
```

```
  window.google_analytics.send(event, data);
}
```

This code works. But it has serious problems:

- Every function can access and change any variable
- Adding a new feature means touching many parts of the code
- Testing one function requires setting up the entire app
- Changing the analytics provider means finding every place it is used
- Two developers working on different features will step on each other

The Same Code With Patterns Applied

```javascript
// cart-store.js -- Module Pattern for encapsulation
const CartStore = (() => {
  let items = [];    // private -- nothing outside can touch this

  return {
    addItem(item) {
      items = [...items, item];
    },
    getItems() {
      return [...items];  // return a copy, not the original
    },
    getTotal() {
      return items.reduce((sum, item) => sum + item.price, 0);
    }
  };
})();

// analytics.js -- Observer Pattern for loose coupling
class AnalyticsTracker {
  constructor(provider) {
    this.provider = provider;
  }

  track(event, data) {
    this.provider.send(event, data);
  }
}

// event-bus.js -- Pub/Sub Pattern for communication
const EventBus = {
  listeners: {},

  subscribe(event, callback) {
    if (!this.listeners[event]) {
      this.listeners[event] = [];
```

```
    }
    this.listeners[event].push(callback);
  },

  publish(event, data) {
    if (this.listeners[event]) {
      this.listeners[event].forEach(cb => cb(data));
    }
  }
};

// Now adding to cart is clean and organized
function addToCart(item) {
  CartStore.addItem(item);
  EventBus.publish('cart:updated', CartStore.getItems());
}

// Each concern listens independently
EventBus.subscribe('cart:updated', (items) => updateCartUI(items));
EventBus.subscribe('cart:updated', (items) => sendAnalytics(items));
EventBus.subscribe('cart:updated', (items) => saveToServer(items));
```

Output when `addToCart({ name: 'Book', price: 15 })` is called:

```
Cart UI updated: 1 item, total: $15
Analytics event sent: cart:updated
Cart saved to server
```

The difference is dramatic:

Aspect	Without Patterns	With Patterns
Variables	Global, accessible everywhere	Private, controlled access
Dependencies	Tightly coupled	Loosely coupled
Testing	Hard, need entire app	Easy, test each piece alone
Adding features	Risky, might break things	Safe, add new subscriber
Team work	Conflicts everywhere	Clear boundaries

Categories of Design Patterns

Design patterns are organized into categories based on what kind of problem they solve. Think of it like a toolbox with different drawers.

```
+==================================================================+
|                    DESIGN PATTERN CATEGORIES                     |
+==================================================================+
|                                                                  |
|  +-------------------+    +---------------------+               |
|  |    CREATIONAL     |    |    STRUCTURAL       |               |
|  +-------------------+    +---------------------+              |
|  | HOW objects are   |    | HOW objects are     |              |
|  | created           |    | composed together   |              |
|  |                   |    |                     |               |
|  | - Singleton       |    | - Module            |              |
|  | - Factory         |    | - Facade            |              |
|  | - Builder         |    | - Decorator         |              |
|  | - Prototype       |    | - Proxy             |              |
|  |                   |    | - Mixin             |              |
|  +-------------------+    +---------------------+              |
|                                                                  |
|  +-------------------+    +---------------------+               |
|  |    BEHAVIORAL     |    |   REACT-SPECIFIC    |              |
|  +-------------------+    +---------------------+              |
|  | HOW objects       |    | Patterns unique to  |              |
|  | communicate       |    | component-based UI  |              |
|  |                   |    |                     |               |
|  | - Observer        |    | - Hooks             |              |
|  | - Pub/Sub         |    | - HOC               |              |
|  | - Mediator        |    | - Render Props      |              |
|  | - Strategy        |    | - Compound          |              |
|  | - Iterator        |    |   Components        |              |
|  | - State Machine   |    | - Provider          |              |
|  +-------------------+    +---------------------+              |
|                                                                  |
|  +---------------------------------------------+                |
|  |          PERFORMANCE PATTERNS               |               |
|  +---------------------------------------------+                |
|  | HOW to make things fast                     |               |
|  |                                             |               |
|  | - Memoization    - Code Splitting           |               |
|  | - Virtualization - Lazy Loading             |               |
|  +---------------------------------------------+                |
|                                                                  |
+==================================================================+
```

Creational Patterns

What they do: Control how objects are created.

Real-life analogy: A car factory. You do not build a car from raw metal every time. The factory has a process to create cars efficiently and consistently.

Frontend example: A Singleton pattern ensures your app only has one authentication manager. You do not want two different objects tracking who is logged in.

Structural Patterns

What they do: Define how pieces of code are organized and composed together.

Real-life analogy: Building with LEGO bricks. Each brick has a standard shape. You combine bricks in different ways to build different structures. The way bricks connect is the structural pattern.

Frontend example: The Module pattern organizes related code into self-contained units. Your API calls go in one module. Your form validation goes in another. They do not leak into each other.

Behavioral Patterns

What they do: Define how objects communicate with each other.

Real-life analogy: A newsletter subscription. You sign up (subscribe). When a new issue comes out (an event), you get notified. You did not need to keep checking the website. The publisher did not need to know your daily schedule.

Frontend example: The Observer pattern lets your UI components react to data changes. When the shopping cart changes, the cart icon, the total price, and the checkout button all update automatically.

React-Specific Patterns

What they do: Solve problems unique to component-based UI frameworks.

These patterns did not exist in the original Gang of Four book. They emerged as developers built complex user interfaces with React and similar libraries.

Frontend example: Custom Hooks let you extract and reuse stateful logic. Instead of writing the same "fetch data and handle loading/error states" code in every component, you write it once as a hook.

Performance Patterns

What they do: Make your application faster and more efficient.

Real-life analogy: A library. Instead of buying every book you might want to read, you borrow them when needed and return them when done. This saves money (memory) and space (processing power).

Frontend example: Memoization remembers the result of expensive calculations. If you already computed the filtered and sorted list of 10,000 products, you do not compute it again unless the data changes.

How to Read Pattern Chapters in This Book

Every pattern chapter in this book follows the same structure. Once you know the structure, you can quickly find what you need.

```
+-----------------------------------------+
|         Pattern Chapter Structure         |
+-----------------------------------------+
|                                           |
|  1. What You Will Learn                   |
|     (Quick overview of the chapter)       |
|                                           |
|  2. The Problem                           |
|     (What pain point does this solve?)    |
|                                           |
|  3. The Solution                          |
|     (The pattern itself, explained        |
|      step by step)                        |
|                                           |
|  4. Before vs After Code                  |
|     (Side-by-side comparison)             |
|                                           |
|  5. Real-World Use Cases                  |
|     (Where you will actually use this)    |
|                                           |
|  6. When to Use / When NOT to Use         |
|     (Decision guide)                      |
|                                           |
|  7. Common Mistakes                       |
|     (Traps to avoid)                      |
|                                           |
|  8. Best Practices                        |
|     (Tips from experienced developers)    |
|                                           |
|  9. Practice Questions + Exercises        |
|     (Hands-on learning)                   |
|                                           |
+-----------------------------------------+
```

You do not have to read this book front to back. Each pattern chapter is designed to stand on its own. However, if you are new to patterns, I recommend reading the first few chapters in order, as later chapters sometimes reference earlier ones.

Anti-Patterns: What NOT to Do

An anti-pattern is a common approach that seems like a good idea but actually causes more problems than it solves. Knowing anti-patterns is just as important as knowing patterns.

Here are some common frontend anti-patterns:

Anti-Pattern 1: God Component

```javascript
// BAD: One component that does everything
function Dashboard() {
  const [users, setUsers] = useState([]);
  const [orders, setOrders] = useState([]);
  const [analytics, setAnalytics] = useState({});
  const [notifications, setNotifications] = useState([]);
  const [settings, setSettings] = useState({});
  // ... 20 more state variables

  useEffect(() => { fetchUsers(); }, []);
  useEffect(() => { fetchOrders(); }, []);
  useEffect(() => { fetchAnalytics(); }, []);
  // ... 10 more effects

  function handleUserUpdate(user) { /* 50 lines */ }
  function handleOrderRefund(order) { /* 40 lines */ }
  function handleSettingsChange(key, value) { /* 30 lines */ }
  // ... 20 more handlers

  return (
    <div>
      {/* 500 lines of JSX */}
    </div>
  );
}
```

Why it is bad: This component is impossible to test, hard to understand, and will cause merge conflicts when multiple developers work on it.

The fix: Break it into smaller, focused components. Use composition patterns (covered in later chapters).

Anti-Pattern 2: Prop Drilling

```javascript
// BAD: Passing props through many layers
function App() {
  const [theme, setTheme] = useState('light');

  return <Layout theme={theme} setTheme={setTheme} />;
}

function Layout({ theme, setTheme }) {
  // Layout does not use theme, just passes it down
  return <Sidebar theme={theme} setTheme={setTheme} />;
}

function Sidebar({ theme, setTheme }) {
  // Sidebar does not use theme either
  return <ThemeToggle theme={theme} setTheme={setTheme} />;
}

function ThemeToggle({ theme, setTheme }) {
  // Finally! This component actually uses the props
  return (
    <button onClick={() => setTheme(theme === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

Why it is bad: Layout and Sidebar are forced to know about theme even though they do not use it. If you rename the prop, you must update every component in the chain.

The fix: Use the Provider pattern or React Context (covered in Chapter 21).

Anti-Pattern 3: Premature Optimization

```javascript
// BAD: Over-engineering a simple list
function SimpleList({ items }) {
  // This list has 10 items. Memoization adds complexity for zero benefit.
  const sortedItems = useMemo(() => {
    return [...items].sort((a, b) => a.name.localeCompare(b.name));
  }, [items]);

  const renderedItems = useMemo(() => {
    return sortedItems.map(item => (
      <li key={item.id}>{item.name}</li>
    ));
  }, [sortedItems]);

  return <ul>{renderedItems}</ul>;
}
```

Why it is bad: `useMemo` itself has a cost. For a list of 10 items, the sorting takes microseconds. The memoization overhead might actually make this slower, and it definitely makes the code harder to read.

The fix: Write simple code first. Optimize only when you measure a real performance problem.

Patterns Are Guidelines, Not Rules

This is one of the most important things to understand about design patterns:

> A pattern is a tool. Use the right tool for the job. Do not use a hammer to screw in a lightbulb.

Here is what this means in practice:

Do Not Force Patterns Where They Are Not Needed

```javascript
// OVER-ENGINEERED: Using the Singleton pattern for a simple config
class ConfigSingleton {
  static instance = null;

  static getInstance() {
    if (!ConfigSingleton.instance) {
      ConfigSingleton.instance = new ConfigSingleton();
    }
    return ConfigSingleton.instance;
  }

  constructor() {
    if (ConfigSingleton.instance) {
      throw new Error('Use getInstance()');
    }
    this.apiUrl = 'https://api.example.com';
    this.timeout = 5000;
  }
}

// SIMPLE AND FINE: Just export an object
// config.js
export const config = {
  apiUrl: 'https://api.example.com',
  timeout: 5000
};
```

The simple object works perfectly. ES modules are already singletons (they are cached after the first import). Adding a Singleton class on top adds complexity with zero benefit.

Combine Patterns When It Makes Sense

Patterns are not mutually exclusive. Real applications often use several patterns together.

```javascript
// A React component using multiple patterns together:
// - Custom Hook pattern (useProducts)
// - Observer pattern (useEffect watching dependencies)
// - Module pattern (encapsulated API calls)
// - Memoization pattern (useMemo for expensive filtering)

function ProductList({ category, sortBy }) {
  const { products, loading, error } = useProducts(category);

  const sortedProducts = useMemo(
    () => sortProducts(products, sortBy),
    [products, sortBy]
  );

  if (loading) return <Spinner />;
  if (error) return <ErrorMessage error={error} />;

  return (
    <ul>
      {sortedProducts.map(p => (
        <ProductCard key={p.id} product={p} />
      ))}
    </ul>
  );
}
```

Adapt Patterns to Your Context

The patterns in this book are templates. Your actual implementation should fit your project's needs, your team's conventions, and your application's scale.

A pattern used in a small personal project will look different from the same pattern in a large enterprise application. That is perfectly fine.

When to Use / When NOT to Use Design Patterns

Use Patterns When

- You recognize a problem you have seen before
- Your code is getting hard to maintain or extend
- Your team needs a shared vocabulary
- You are building something that will grow over time
- You need to make your code testable

Do NOT Use Patterns When

- Your code is simple and works fine without them
- You are building a quick prototype that will be thrown away
- Adding a pattern would make the code harder to understand
- You are using a pattern just because it sounds impressive
- The problem does not match what the pattern solves

Common Mistakes

1. Using patterns you do not understand -- Learn the pattern thoroughly before applying it. A misapplied pattern is worse than no pattern.
2. Treating patterns as the only solution -- Patterns are one tool among many. Sometimes a straightforward if/else is better than a Strategy pattern.
3. Ignoring the trade-offs -- Every pattern has costs. Know them before you commit.
4. Pattern name-dropping in code reviews -- "You should use the Abstract Factory here" is not helpful if the reviewer cannot explain why. Always explain the problem you are solving.
5. Not adapting patterns to JavaScript -- The GoF book was written for C++ and Java. JavaScript has closures, first-class functions, and prototypal inheritance. Patterns in JavaScript often look different (and simpler) than their classical versions.

Best Practices

1. Start with the problem, not the pattern -- Ask "What problem am I solving?" before asking "What pattern should I use?"

2. Learn patterns gradually -- Do not try to memorize all patterns at once. Learn one, use it in a project, then learn the next.
3. Read other people's code -- Open-source projects are full of patterns. Reading real code is one of the best ways to see patterns in action.
4. Refactor toward patterns -- You do not need to design with patterns from day one. Write working code first, then refactor when patterns become obvious.
5. Keep it simple -- The best code is code that is easy to read. If a pattern makes your code harder to understand, reconsider.

Quick Summary

Design patterns are proven solutions to recurring software problems. They originated from the Gang of Four's 1994 book, which documented 23 patterns observed in experienced developers' code. In frontend development, patterns help manage complexity, improve code organization, enable team collaboration, and make code testable and maintainable. Patterns fall into five categories: creational (how things are made), structural (how things are organized), behavioral (how things communicate), React-specific (component patterns), and performance (how to make things fast). Anti-patterns are common approaches that seem helpful but cause problems. The most important rule: patterns are tools, not rules. Use them when they solve a real problem, and skip them when they add unnecessary complexity.

Key Points

- Design patterns are proven recipes for common software problems
- The Gang of Four documented 23 foundational patterns in 1994
- Patterns give developers a shared vocabulary to communicate
- Frontend patterns fall into five categories: creational, structural, behavioral, React-specific, and performance
- Anti-patterns are common mistakes disguised as solutions
- Always start with the problem, not the pattern
- Patterns are guidelines, not rules -- adapt them to your context
- Simple code that works is better than complex code with forced patterns
- JavaScript patterns often look simpler than their classical counterparts
- You can combine multiple patterns in a single piece of code

Practice Questions

1. In your own words, explain what a design pattern is. Give a non-programming analogy.
2. What is the difference between a creational pattern and a behavioral pattern? Give one example of each.
3. Look at this code. What anti-pattern does it demonstrate, and how would you fix it?

```javascript
function App({ user }) {
  return <Page user={user} />;
}
function Page({ user }) {
  return <Header user={user} />;
}
function Header({ user }) {
  return <Avatar user={user} />;
}
function Avatar({ user }) {
  return <img src={user.avatar} alt={user.name} />;
}
```

4. Why did the Gang of Four's book have such a big impact on software development? What problem did it solve for developers?
5. A teammate says, "We should use the Observer pattern here because it is a best practice." What questions would you ask before agreeing?

Exercises

Exercise 1: Pattern Identification

Look at a JavaScript project you have worked on (or an open-source project on GitHub). Find at least three places where design patterns are used, even if the original developer did not name them explicitly. Write down:

- What pattern you see
- What problem it solves in that project
- Whether you think the pattern was the right choice

Exercise 2: Anti-Pattern Hunt

Review a piece of code you wrote more than six months ago. Find at least one anti-pattern. Write a brief explanation of:

- What the anti-pattern is
- Why it causes problems
- How you would fix it using a design pattern from this book

Exercise 3: Pattern Category Match

For each of these frontend problems, decide which pattern category (creational, structural, behavioral, React-specific, or performance) would most likely help. Explain your reasoning.

1. Your app takes 3 seconds to render a list of 50,000 items
2. Five different components need to update when the user logs in
3. You have utility functions scattered across 20 different files
4. You need to create different types of form fields (text, email, select, checkbox) based on a configuration object
5. Three React components share the same "fetch data, show loading, handle error" logic

---

What Is Next?

Now that you understand what design patterns are and why they matter, it is time to learn your first pattern. In Chapter 2, we will dive into the Module Pattern -- one of the most fundamental patterns in JavaScript. You will learn how to organize your code into self-contained units that keep their internals private and expose only what the outside world needs. This pattern is the foundation for writing clean, maintainable JavaScript.

Chapter 2: Module Pattern

What You Will Learn

- Why global variables are dangerous in JavaScript
- How the Immediately Invoked Function Expression (IIFE) solves the global pollution problem
- How closures enable private and public members
- The Revealing Module Pattern and when to use it
- ES6 modules with import and export
- How barrel files organize your codebase
- Real-world module patterns in React projects

Why This Chapter Matters

Imagine you live in a house with no rooms. No walls, no doors, no separation at all. Your bedroom is your kitchen is your bathroom. Everything is in one open space. Your toothbrush is next to the frying pan. Your bed is next to the toilet. Every person in the house can see and touch everything.

That is what JavaScript code looks like without modules. Every variable, every function, every piece of data lives in one giant shared space called the global scope. Any piece of code can read or overwrite anything else. One careless variable name and your entire application breaks.

The Module Pattern fixes this. It gives your code walls, doors, and rooms. It lets you decide what is private (hidden inside a room) and what is public (available through the door). It is one of the most fundamental patterns in JavaScript, and understanding it is essential before learning any other pattern in this book.

Every modern JavaScript application uses modules. Every React project, every Node.js server, every npm package relies on modules to stay organized. This chapter takes you from

the problem (global chaos) to the solution (clean, isolated, organized code).

The Problem: Global Scope Pollution

JavaScript was originally designed for small scripts on web pages. Early JavaScript ran in a single global scope. Every variable you declared with `var` became a property of the `window` object in browsers.

```javascript
// file1.js
var userName = 'Alice';
var count = 0;

function greet() {
  console.log('Hello, ' + userName);
}

// file2.js -- written by a different developer
var userName = 'ProductDB';  // Oops! Overwrites Alice
var count = 42;              // Oops! Overwrites the counter

function greet() {           // Oops! Overwrites the greeting function
  console.log('Connecting to ' + userName);
}
```

Output when file2.js loads after file1.js:

```
// Calling greet() now runs file2's version
Connecting to ProductDB
// The original greet() is gone forever
```

This is called global scope pollution or namespace collision. It is not a theoretical problem. It happened constantly in the early days of JavaScript.

```
┌──────────────────────────────────────────────────────────┐
│                    Global Scope (window)                 │
│                                                          │
│  userName = ???     <── Who set this? file1? file2?      │
│  count = ???        <── 0 or 42? No way to tell          │
│  greet = ???        <── Which greet? Last one wins       │
│                                                          │
│  Every script dumps variables here.                      │
│  Every script can overwrite every other script.          │
│  Pure chaos.                                             │
```

```
└──────────────────────────────────────────────────────────────────────┘
```

Why This Breaks Real Applications

Consider loading multiple scripts in an HTML file:

```html
<!-- index.html -->
<script src="jquery.js"></script>
<script src="analytics.js"></script>
<script src="chat-widget.js"></script>
<script src="your-app.js"></script>
```

Each of these scripts shares the same global scope. If `analytics.js` defines a function called `track`, and `chat-widget.js` also defines a function called `track`, the second one silently overwrites the first. No error. No warning. Just broken analytics.

The real-world impact:

- Silent bugs: Variables get overwritten with no error message
- Naming fear: Developers resort to absurdly long names like `myCompany_analytics_track_event_v2` to avoid collisions
- Dependency nightmares: Script loading order matters and one wrong order breaks everything
- Untestable code: You cannot isolate and test pieces independently

---

Solution 1: The IIFE (Immediately Invoked Function Expression)

Before ES6 modules existed, JavaScript developers found a clever solution: wrap your code in a function and call it immediately. This creates a private scope.

How a Regular Function Creates Scope

In JavaScript, variables declared inside a function are not visible outside that function.

```javascript
function myModule() {
  var secret = 'hidden';
  console.log(secret); // 'hidden' -- works inside
}

myModule();
console.log(secret); // ReferenceError: secret is not defined
```

Output:

```
hidden
ReferenceError: secret is not defined
```

This is useful, but the function name `myModule` itself is a global variable. We still pollute the global scope with that name.

The IIFE Removes Even the Function Name

An IIFE wraps a function in parentheses and calls it immediately. No name, no global variable.

```javascript
// An IIFE -- Immediately Invoked Function Expression
(function () {
  var secret = 'hidden';
  var count = 0;

  function increment() {
    count++;
    console.log('Count is now: ' + count);
  }

  increment();
  increment();

  console.log('Secret inside: ' + secret);
})();

// Outside the IIFE, nothing is accessible
// console.log(secret);  // ReferenceError
// console.log(count);   // ReferenceError
// increment();          // ReferenceError
```

Output:

```
Count is now: 1
Count is now: 2
Secret inside: hidden
```

Anatomy of an IIFE

```
  (function () {        <── Wrapping in ( ) makes it an expression
    // Your code here   <── Everything in here is PRIVATE
  })();                 <── The () at the end CALLS it immediately
   │
   └── This is why it is "Immediately Invoked"
```

You can also write IIFEs with arrow functions:

```javascript
(() => {
  const secret = 'also hidden';
  console.log(secret);
})();
```

Output:

```
also hidden
```

IIFE with Parameters

IIFEs can accept arguments, which is useful for injecting dependencies:

```javascript
(function (window, document) {
  // Use window and document safely here
  // Even if someone overwrites the global window variable,
  // this code still has the original references

  const button = document.createElement('button');
  button.textContent = 'Click me';
  console.log('Button created:', button.textContent);
})(window, document);
```

Output:

```
Button created: Click me
```

Solution 2: The Classic Module Pattern (IIFE + Closures)

The IIFE hides everything. But what if you want to hide some things and expose others? That is where closures come in.

Quick Refresher: What Is a Closure?

A closure is when a function "remembers" the variables from the scope where it was created, even after that scope has finished executing.

```javascript
function createCounter() {
  let count = 0; // This variable is "closed over"

  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
console.log(counter()); // 3
// count is not accessible directly
// console.log(count); // ReferenceError
```

Output:

```
1
2
3
```

The returned function "closes over" the `count` variable. It remembers `count` even though `createCounter` has already finished running. The variable `count` is effectively private -- nothing outside can access it except through the returned function.

The Module Pattern: Private and Public Members

Combine the IIFE with closures, and you get the Module Pattern. Return an object whose methods have access to the private variables via closure.

```javascript
const CounterModule = (function () {
  // PRIVATE -- not accessible from outside
  let count = 0;
  const MAX_COUNT = 100;

  function validateCount(newCount) {
    return newCount >= 0 && newCount <= MAX_COUNT;
  }

  // PUBLIC -- returned object is the module's public API
  return {
    increment() {
      if (validateCount(count + 1)) {
        count++;
      }
      return count;
    },

    decrement() {
      if (validateCount(count - 1)) {
        count--;
      }
      return count;
    },

    getCount() {
      return count;
    },

    reset() {
      count = 0;
      return count;
    }
  };
})();

console.log(CounterModule.getCount());   // 0
console.log(CounterModule.increment());  // 1
console.log(CounterModule.increment());  // 2
console.log(CounterModule.decrement());  // 1
console.log(CounterModule.reset());      // 0

// Trying to access private members
console.log(CounterModule.count);          // undefined
console.log(CounterModule.MAX_COUNT);      // undefined
console.log(CounterModule.validateCount);  // undefined
```

Output:

```
0
1
2
1
```

```
0
undefined
undefined
undefined
```

How It Works: A Visual Map

```
┌──────────────────────────────────────────────────────┐
│              CounterModule (IIFE)                    │
│                                                      │
│   PRIVATE (hidden by closure):                       │
│   ┌──────────────────────────────────┐               │
│   │  let count = 0                   │               │
│   │  const MAX_COUNT = 100           │               │
│   │  function validateCount() {...}  │               │
│   └──────────────────────────────────┘               │
│                    │                                 │
│                    │ closure (remembers these)       │
│                    ▼                                 │
│   PUBLIC (returned object):                          │
│   ┌──────────────────────────────────┐               │
│   │  increment()  -- can read/write  │◄── Outside    │
│   │  decrement()     count via       │    code       │
│   │  getCount()      closure         │    uses       │
│   │  reset()                         │    these      │
│   └──────────────────────────────────┘               │
└──────────────────────────────────────────────────────┘
```

The Revealing Module Pattern

The Revealing Module Pattern is a cleaner variation of the Module Pattern. Instead of defining methods inline in the returned object, you define all functions as private first, then "reveal" selected ones in the return statement.

Before: Classic Module Pattern

```javascript
const UserModule = (function () {
  let users = [];

  return {
```

```
    addUser(name) {
      users.push({ name, id: Date.now() });
      console.log(`Added user: ${name}`);
    },
    getUsers() {
      return [...users]; // Return a copy
    },
    getUserCount() {
      return users.length;
    }
  };
})();
```

After: Revealing Module Pattern

```javascript
const UserModule = (function () {
  // ALL variables and functions defined as private
  let users = [];

  function addUser(name) {
    users.push({ name, id: Date.now() });
    console.log(`Added user: ${name}`);
  }

  function getUsers() {
    return [...users];
  }

  function getUserCount() {
    return users.length;
  }

  function clearUsers() {
    users = [];
    console.log('All users cleared');
  }

  // "Reveal" only the public API
  return {
    addUser,
    getUsers,
    getUserCount
    // clearUsers is NOT revealed -- stays private
  };
})();

UserModule.addUser('Alice');
UserModule.addUser('Bob');
console.log('User count:', UserModule.getUserCount());
console.log('Users:', UserModule.getUsers());
```

```
// clearUsers is private, cannot be called
console.log(UserModule.clearUsers); // undefined
```

Output:

```
Added user: Alice
Added user: Bob
User count: 2
Users: [ { name: 'Alice', id: 1698765432100 }, { name: 'Bob', id: 1698765432101 } ]
undefined
```

Why Revealing Is Better

```
  Classic Module Pattern          Revealing Module Pattern
  ______________________          ________________________


  return {                        function addUser() {...}
    addUser(name) {               function getUsers() {...}
      // definition here          function getUserCount() {...}
    },                            function clearUsers() {...}
    getUsers() {
      // definition here          return {
    },                              addUser,      // just names
    getUserCount() {                getUsers,
      // definition here            getUserCount
    }                             };
  };

  Methods defined INSIDE          Methods defined OUTSIDE
  the return object               then mapped in return

  Harder to see what              Easy to see what is
  is public vs private            public (in the return)
```

Advantages of the Revealing Module Pattern:

1. Consistent style: All functions are defined the same way
2. Easy to see the API: The return statement is a clean list of public names
3. Easy to change: Moving a function from public to private means removing one line from the return
4. Named functions: Better stack traces for debugging

Real-World Module Pattern: Shopping Cart

Here is a practical example that combines everything you have learned.

The Problem

You need a shopping cart that:

- Keeps items private (no direct manipulation)
- Validates before adding items
- Calculates totals
- Tracks discount codes
- Provides a clean API

The Solution

```javascript
const ShoppingCart = (function () {
  // Private state
  let items = [];
  let discountCode = null;

  const discountCodes = {
    'SAVE10': 0.10,
    'SAVE20': 0.20,
    'HALF': 0.50
  };

  // Private functions
  function calculateSubtotal() {
    return items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  }

  function calculateDiscount(subtotal) {
    if (discountCode && discountCodes[discountCode]) {
      return subtotal * discountCodes[discountCode];
    }
    return 0;
  }

  function findItemIndex(productId) {
    return items.findIndex(item => item.productId === productId);
  }

  // Public API
  function addItem(productId, name, price, quantity = 1) {
    if (price <= 0) {
      console.log('Error: Price must be positive');
```

```
    return false;
  }
  if (quantity <= 0) {
    console.log('Error: Quantity must be positive');
    return false;
  }

  const existingIndex = findItemIndex(productId);
  if (existingIndex > -1) {
    items[existingIndex].quantity += quantity;
    console.log(`Updated ${name}: quantity is now ${items[existingIndex].quantity}`);
  } else {
    items.push({ productId, name, price, quantity });
    console.log(`Added ${name} to cart`);
  }
  return true;
}

function removeItem(productId) {
  const index = findItemIndex(productId);
  if (index > -1) {
    const removed = items.splice(index, 1)[0];
    console.log(`Removed ${removed.name} from cart`);
    return true;
  }
  console.log('Item not found in cart');
  return false;
}

function applyDiscount(code) {
  if (discountCodes[code]) {
    discountCode = code;
    console.log(`Discount code ${code} applied: ${discountCodes[code] * 100}% off`);
    return true;
  }
  console.log('Invalid discount code');
  return false;
}

function getTotal() {
  const subtotal = calculateSubtotal();
  const discount = calculateDiscount(subtotal);
  return {
    subtotal: subtotal.toFixed(2),
    discount: discount.toFixed(2),
    total: (subtotal - discount).toFixed(2),
    itemCount: items.reduce((sum, item) => sum + item.quantity, 0)
  };
}

function getItems() {
  return items.map(item => ({ ...item })); // Return copies
}

// Reveal public API
```

```
  return {
    addItem,
    removeItem,
    applyDiscount,
    getTotal,
    getItems
  };
})();

// Usage
ShoppingCart.addItem('SKU001', 'Wireless Mouse', 29.99);
ShoppingCart.addItem('SKU002', 'USB Keyboard', 49.99);
ShoppingCart.addItem('SKU001', 'Wireless Mouse', 29.99); // Adds to quantity
ShoppingCart.applyDiscount('SAVE10');
console.log(ShoppingCart.getTotal());

// Cannot access internals
console.log(ShoppingCart.items);         // undefined
console.log(ShoppingCart.discountCodes); // undefined
```

Output:

```
Added Wireless Mouse to cart
Added USB Keyboard to cart
Updated Wireless Mouse: quantity is now 2
Discount code SAVE10 applied: 10% off
{ subtotal: '109.97', discount: '11.00', total: '98.97', itemCount: 3 }
undefined
undefined
```

ES6 Modules: The Modern Standard

In 2015, JavaScript got native module support with ES6. This is now the standard way to write modules in JavaScript. No more IIFE tricks needed.

Named Exports

```javascript
// math-utils.js
export function add(a, b) {
  return a + b;
}
```

```
export function multiply(a, b) {
  return a * b;
}

export const PI = 3.14159;

// Private -- not exported, not accessible from outside
function internalHelper(x) {
  return x * x;
}
```

```javascript
// app.js
import { add, multiply, PI } from './math-utils.js';

console.log(add(2, 3));        // 5
console.log(multiply(4, 5));   // 20
console.log(PI);               // 3.14159
// internalHelper is not accessible -- it is private to math-utils.js
```

Output:

```
5
20
3.14159
```

Default Exports

Each module can have one default export. This is useful when a module has one main thing to export.

```javascript
// Logger.js
class Logger {
  constructor(prefix) {
    this.prefix = prefix;
  }

  log(message) {
    console.log(`[${this.prefix}] ${message}`);
  }

  error(message) {
    console.error(`[${this.prefix}] ERROR: ${message}`);
  }

  warn(message) {
```

```
    console.warn(`[${this.prefix}] WARN: ${message}`);
  }
}

export default Logger;
```

```javascript
// app.js
import Logger from './Logger.js';
// You can name default imports anything you want
// import MyLogger from './Logger.js'; // Also works!

const logger = new Logger('App');
logger.log('Application started');
logger.error('Something went wrong');
```

Output:

```
[App] Application started
[App] ERROR: Something went wrong
```

Named vs Default Exports

```
  Named Exports                    Default Exports
  ─────────────                    ───────────────

  export function add() {}         export default function add() {}

  import { add } from './math';    import add from './math';
  import { add, multiply }         import whateverName from './math';
    from './math';

  Must use exact names             Can use any name on import
  Can have many per file           Only ONE per file
  Use { } in import                No { } in import

  Best for utility files           Best for single-purpose files
  with multiple exports             (a class, a component, etc.)
```

Mixing Named and Default Exports

```javascript
// api-client.js
export default class ApiClient {
```

```
  constructor(baseUrl) {
    this.baseUrl = baseUrl;
  }

  async get(endpoint) {
    const response = await fetch(`${this.baseUrl}${endpoint}`);
    return response.json();
  }
}

// Named exports alongside the default
export const DEFAULT_BASE_URL = 'https://api.example.com';
export const TIMEOUT = 5000;
```

```javascript
// app.js
import ApiClient, { DEFAULT_BASE_URL, TIMEOUT } from './api-client.js';

const client = new ApiClient(DEFAULT_BASE_URL);
console.log('Base URL:', DEFAULT_BASE_URL);
console.log('Timeout:', TIMEOUT);
```

Output:

```
Base URL: https://api.example.com
Timeout: 5000
```

ES Modules vs IIFE: How Privacy Works

Both approaches give you private and public members. The mechanism is different, but the result is the same.

```
  IIFE Module                        ES Module
  ___________                        _________

  const Mod = (function () {         // utils.js
    // Private                       // Private (not exported)
    let secret = 42;                 let secret = 42;

    function helper() {              function helper() {
      return secret;                   return secret;
    }                                }

    // Public                        // Public (exported)
```

```
    return {                          export function getSecret() {
      getSecret() {                     return helper();
        return helper();              }
      }
    };
  })();

  Privacy via: closure                Privacy via: module scope
  Works in: any JS environment         Works in: modern browsers, bundlers
  Created at: runtime                 Created at: parse time
```

In ES modules, anything you do not export is private by default. You do not need closures or IIFEs. The module system handles scoping for you.

Barrel Files: Organizing Exports

As your project grows, you end up with many modules. Importing from deep paths gets messy.

The Problem: Deep Import Paths

```javascript
// Without barrel files, imports are messy and fragile
import { validateEmail } from '../utils/validation/email.js';
import { validatePhone } from '../utils/validation/phone.js';
import { formatDate } from '../utils/formatting/date.js';
import { formatCurrency } from '../utils/formatting/currency.js';
import { fetchUser } from '../services/api/user.js';
import { fetchProducts } from '../services/api/products.js';
```

The Solution: Barrel Files (index.js)

A barrel file is an `index.js` file that re-exports from multiple modules. It acts as a single entry point for a folder.

```javascript
// utils/validation/index.js (barrel file)
export { validateEmail } from './email.js';
export { validatePhone } from './phone.js';
export { validatePassword } from './password.js';
export { validateUsername } from './username.js';
```

```javascript
// utils/formatting/index.js (barrel file)
export { formatDate } from './date.js';
export { formatCurrency } from './currency.js';
export { formatNumber } from './number.js';
```

```javascript
// utils/index.js (top-level barrel)
export * from './validation/index.js';
export * from './formatting/index.js';
```

```javascript
// Now importing is clean and simple
import { validateEmail, validatePhone, formatDate, formatCurrency } from '../utils';
```

Barrel File Structure

```
src/
├── utils/
│   ├── index.js          <── Top-level barrel
│   ├── validation/
│   │   ├── index.js      <── Barrel for validation
│   │   ├── email.js
│   │   ├── phone.js
│   │   └── password.js
│   └── formatting/
│       ├── index.js      <── Barrel for formatting
│       ├── date.js
│       ├── currency.js
│       └── number.js
├── components/
│   ├── index.js          <── Barrel for components
│   ├── Button.jsx
│   ├── Input.jsx
│   └── Modal.jsx
└── hooks/
    ├── index.js          <── Barrel for hooks
    ├── useAuth.js
    ├── useLocalStorage.js
    └── useDebounce.js
```

React Component Barrel File

```javascript
// components/index.js
export { default as Button } from './Button';
export { default as Input } from './Input';
```

```
export { default as Modal } from './Modal';
export { default as Card } from './Card';
export { default as Spinner } from './Spinner';
```

```javascript
// pages/Dashboard.jsx
import { Button, Card, Spinner } from '../components';
// Instead of:
// import Button from '../components/Button';
// import Card from '../components/Card';
// import Spinner from '../components/Spinner';
```

Barrel File Warning: Tree Shaking

Barrel files can hurt performance if your bundler cannot tree-shake properly. When you import one thing from a barrel file, the bundler might load everything the barrel re-exports.

```javascript
// If you only need formatDate, but the barrel exports 50 functions,
// a poorly configured bundler loads all 50
import { formatDate } from '../utils';

// Direct import is always safe for tree shaking
import { formatDate } from '../utils/formatting/date.js';
```

Rule of thumb: Use barrel files for convenience during development. If bundle size becomes an issue, profile and switch to direct imports where needed.

Module Pattern in React

React components are already modules (each `.jsx` file is an ES module). But the Module Pattern thinking still applies when you organize state, utilities, and custom hooks.

Example: A Feature Module

```javascript
// features/auth/auth-service.js

// Private state (module-scoped, not exported)
let cachedToken = null;
let tokenExpiry = null;
```

```
// Private helper
function isTokenExpired() {
  return !tokenExpiry || Date.now() > tokenExpiry;
}

// Public API
export async function login(email, password) {
  const response = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  const data = await response.json();
  cachedToken = data.token;
  tokenExpiry = Date.now() + data.expiresIn * 1000;
  return data.user;
}

export function getToken() {
  if (isTokenExpired()) {
    cachedToken = null;
    tokenExpiry = null;
    return null;
  }
  return cachedToken;
}

export function logout() {
  cachedToken = null;
  tokenExpiry = null;
}

export function isAuthenticated() {
  return !isTokenExpired() && cachedToken !== null;
}
```

```javascript
// components/LoginForm.jsx
import { useState } from 'react';
import { login } from '../features/auth/auth-service';

export default function LoginForm({ onSuccess }) {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState(null);

  async function handleSubmit(e) {
    e.preventDefault();
    try {
      const user = await login(email, password);
      onSuccess(user);
```

```
    } catch (err) {
      setError('Login failed. Please try again.');
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="Password"
      />
      {error && <p style={{ color: 'red' }}>{error}</p>}
      <button type="submit">Log In</button>
    </form>
  );
}
```

When to Use / When NOT to Use

When to Use the Module Pattern

- Organizing utility functions: Group related helpers into a module with a clean API
- Encapsulating state: When you need private variables that should not be directly modified
- Library/SDK development: Expose a public API while hiding implementation details
- Legacy codebases: IIFEs work without a build system -- great for adding structure to old projects
- Configuration management: Store config values privately, expose getters

When NOT to Use the Module Pattern

- Simple, stateless functions: If you just have pure utility functions, a plain ES module with exports is enough. No need for IIFEs or the Revealing Module Pattern
- React component state: Use `useState` and `useReducer`, not module-level variables, for UI state

- Shared mutable state across components: Use Context, Redux, or Zustand instead of module-scoped variables
- When you need multiple instances: The module pattern creates a single instance. If you need multiple instances, use a class or factory function instead

---

Common Mistakes

Mistake 1: Using Module-Scoped Variables for React UI State

```javascript
// BAD -- module variable does not trigger re-renders
let count = 0;

export default function Counter() {
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => count++}>Increment</button>
      {/* The UI will NOT update because React does not know count changed */}
    </div>
  );
}
```

```javascript
// GOOD -- useState triggers re-renders
import { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
  );
}
```

Mistake 2: Circular Dependencies in Barrel Files

```javascript
// utils/index.js
export * from './a.js';
```

```
export * from './b.js';

// a.js
import { helperB } from './index.js'; // Imports from barrel
export function helperA() { return helperB(); }

// b.js
import { helperA } from './index.js'; // Circular!
export function helperB() { return helperA(); }

// This creates an infinite loop or undefined imports
```

Fix: Import directly from the source file, not the barrel, when you have cross-dependencies.

```javascript
// a.js
import { helperB } from './b.js'; // Direct import, no circular issue
```

Mistake 3: Forgetting That Module Variables Are Shared

```javascript
// counter-module.js
let count = 0;
export function increment() { return ++count; }
export function getCount() { return count; }

// component-a.js
import { increment, getCount } from './counter-module.js';
increment(); // count is now 1

// component-b.js
import { getCount } from './counter-module.js';
console.log(getCount()); // 1 -- NOT 0!
// Both files share the SAME module instance
```

This is a feature, not a bug, but it surprises beginners. ES modules are singletons -- they execute once and every import gets the same instance.

Best Practices

1. Prefer ES modules over IIFEs in modern projects. IIFEs are for legacy code or environments without a module system.

2. Export the minimum necessary API. The less you expose, the easier your module is to maintain and the harder it is to misuse.
3. Use named exports for most things. They enable auto-import in IDEs and make refactoring safer (renaming is caught by the editor).
4. Use default exports for single-purpose modules like React components or classes.
5. Keep barrel files shallow. Re-export from the immediate children only, not from deeply nested modules.
6. Avoid mutable module-scoped state for UI. Use React state management for anything that should trigger re-renders.
7. Name your files after what they export. `formatDate.js` should export `formatDate`. `UserService.js` should export `UserService`. Predictability matters.

Quick Summary

The Module Pattern solves global scope pollution by creating isolated, private scopes with controlled public APIs.

```
Evolution of Modules in JavaScript:
────────────────────────────────────


1. Global Scope (chaos)
   └──► 2. IIFE (private scope, single expression)
         └──► 3. Module Pattern (IIFE + closures, private/public)
               └──► 4. Revealing Module (cleaner return object)
                     └──► 5. ES6 Modules (native, standard, best)
```

Key Points

- Global scope pollution is when multiple scripts share and overwrite variables in the global namespace
- IIFEs create a private scope by wrapping code in a function that runs immediately
- Closures let inner functions access outer variables, enabling private state
- The Module Pattern combines IIFEs and closures to create modules with private internals and a public API

- The Revealing Module Pattern defines all members privately first, then reveals selected ones in the return statement
- ES6 modules use `export` and `import` keywords. They are the modern standard and should be preferred
- Barrel files (`index.js`) re-export from multiple modules to simplify import paths
- ES modules are singletons -- every import of the same module gets the same instance
- Module-scoped variables are shared across all importers. Do not use them for React UI state

---

Practice Questions

1. What is the difference between a regular function and an IIFE? Why does wrapping a function in parentheses and calling it immediately prevent global scope pollution?
2. Explain how closures enable private variables in the Module Pattern. Why can outside code not access the private variables directly?
3. What is the difference between the classic Module Pattern and the Revealing Module Pattern? When would you choose one over the other?
4. Why are ES modules considered singletons? If two different files import the same module, do they each get a separate copy or the same instance?
5. What are the potential downsides of barrel files? How can they affect bundle size and tree shaking?

Exercises

Exercise 1: Build a Temperature Converter Module

Create a module (using either the Revealing Module Pattern or ES6 exports) that:

- Converts Celsius to Fahrenheit
- Converts Fahrenheit to Celsius
- Keeps a private history of all conversions performed
- Has a public method to retrieve the conversion history
- Has a public method to clear the history

Test it by performing several conversions and then checking the history.

Exercise 2: Refactor Global Code into Modules

Take this global code and refactor it into a clean module:

```javascript
var todos = [];
var nextId = 1;

function addTodo(text) {
  todos.push({ id: nextId++, text: text, done: false });
}

function toggleTodo(id) {
  var todo = todos.find(function(t) { return t.id === id; });
  if (todo) todo.done = !todo.done;
}

function getTodos() {
  return todos;
}
```

Make `todos` and `nextId` private. Expose `addTodo`, `toggleTodo`, `getTodos`, and add a new `removeTodo` function. Ensure `getTodos` returns copies, not the original array.

Exercise 3: Create a Barrel File Structure

Create a file structure for a React project with the following:

- A `hooks/` folder with `useAuth.js`, `useLocalStorage.js`, and `useDebounce.js`
- A `utils/` folder with `formatDate.js`, `formatCurrency.js`, and `validateEmail.js`
- Barrel files (`index.js`) for both folders
- A component that imports from both barrel files in a single clean import line

---

What Is Next?

Now that you understand how to organize code into isolated modules with private and public members, you are ready for the next question: what if you need exactly one instance of something across your entire application? One configuration object. One logger. One authentication store.

That is the Singleton Pattern, and it is the focus of Chapter 3.

Chapter 3: Singleton Pattern

What You Will Learn

- What the Singleton Pattern is and why "one instance only" matters
- How ES modules naturally create singletons
- How to use `Object.freeze` to make singletons immutable
- When the Singleton Pattern hurts (especially in testing)
- Real-world singletons: auth store, logger, theme manager
- How to implement singletons with and without classes

Why This Chapter Matters

Think about a restaurant kitchen. There is one head chef. Not two, not three -- exactly one. Every waiter delivers orders to the same head chef. Every sous-chef reports to the same head chef. If there were two head chefs giving contradictory instructions, the kitchen would descend into chaos. Two people seasoning the same dish. Two people deciding the menu. Two sources of truth.

The Singleton Pattern works the same way. Some things in your application should exist exactly once. One authentication state. One logger writing to the console. One theme configuration. One database connection pool. If you accidentally create two of these, you get inconsistent state, duplicated connections, and bugs that are incredibly hard to track down.

This pattern is both powerful and controversial. It solves real problems, but it can also create new ones when misused. By the end of this chapter, you will know exactly when to use it, when to avoid it, and how to implement it correctly in modern JavaScript.

The Problem: Accidental Multiple Instances

Imagine you build an authentication service for your app. It tracks whether the user is logged in and stores their token.

```javascript
// auth-service.js -- a plain class

class AuthService {
  constructor() {
    this.user = null;
    this.token = null;
    this.isAuthenticated = false;
    console.log('AuthService created');
  }

  login(user, token) {
    this.user = user;
    this.token = token;
    this.isAuthenticated = true;
    console.log(`Logged in as ${user.name}`);
  }

  logout() {
    this.user = null;
    this.token = null;
    this.isAuthenticated = false;
    console.log('Logged out');
  }

  getToken() {
    return this.token;
  }
}

export default AuthService;
```

Now different parts of your app use it:

```javascript
// navbar.js
import AuthService from './auth-service.js';
const auth = new AuthService(); // Creates instance #1

function showUserName() {
  if (auth.isAuthenticated) {
    console.log('Welcome, ' + auth.user.name);
  } else {
    console.log('Please log in');
  }
}
```

```
// api-client.js
import AuthService from './auth-service.js';
const auth = new AuthService(); // Creates instance #2 !!

function makeRequest(url) {
  const token = auth.getToken();
  console.log('Using token:', token);
  // fetch(url, { headers: { Authorization: `Bearer ${token}` } });
}
```

Output:

```
AuthService created
AuthService created
```

Two instances! When `navbar.js` logs the user in, `api-client.js` still thinks nobody is logged in. Its `auth` is a completely separate object with its own state.

```
┌──────────────────┐       ┌──────────────────┐
│    navbar.js     │       │   api-client.js  │
│                  │       │                  │
│  const auth =    │       │  const auth =    │
│    new Auth()    │       │    new Auth()    │
│                  │       │                  │
│  auth.user =     │       │  auth.user =     │
│    'Alice'       │       │    null          │
│  auth.token =    │       │  auth.token =    │
│    'abc123'      │       │    null          │
└──────────────────┘       └──────────────────┘
     Instance #1               Instance #2

  Two instances = two sources of truth = BUGS
```

The Solution: Singleton Pattern

The Singleton Pattern ensures a class or module has only one instance and provides a global point of access to it.

```
┌──────────────────┐       ┌──────────────────┐
│    navbar.js     │       │   api-client.js  │
│                  │       │                  │
```

```
  │  auth.user =     │      │  auth.token =     │
  │    'Alice'  ─────┼──┐   │    'abc123' ──────┼──┐
  └──────────────────┘  │   └───────────────────┘  │
                        │                          │
                        ▼                          ▼
                   ┌──────────────────────────┐
                   │   Single Auth Instance   │
                   │                          │
                   │   user: 'Alice'          │
                   │   token: 'abc123'        │
                   │   isAuthenticated: true  │
                   └──────────────────────────┘

  One instance = one source of truth = consistency
```

Implementation 1: ES Module Singleton (Recommended)

The simplest and most modern way to create a singleton in JavaScript is to use ES modules. Remember from Chapter 2: ES modules are evaluated once and cached. Every file that imports the module gets the same instance.

```javascript
// auth-service.js -- ES module singleton

class AuthService {
  constructor() {
    this.user = null;
    this.token = null;
    this.isAuthenticated = false;
    console.log('AuthService created');
  }

  login(user, token) {
    this.user = user;
    this.token = token;
    this.isAuthenticated = true;
    console.log(`Logged in as ${user.name}`);
  }

  logout() {
    this.user = null;
    this.token = null;
    this.isAuthenticated = false;
    console.log('Logged out');
```

```
  }

  getToken() {
    return this.token;
  }
}

// Create ONE instance and export it
// NOT the class -- the instance
const authService = new AuthService();
export default authService;
```

```javascript
// navbar.js
import authService from './auth-service.js';

authService.login({ name: 'Alice' }, 'token-abc-123');
console.log('Navbar - Is authenticated:', authService.isAuthenticated);
```

```javascript
// api-client.js
import authService from './auth-service.js';

// This is the SAME instance that navbar.js imported
console.log('API Client - Is authenticated:', authService.isAuthenticated);
console.log('API Client - Token:', authService.getToken());
```

Output (when navbar.js runs first):

```
AuthService created
Logged in as Alice
Navbar - Is authenticated: true
API Client - Is authenticated: true
API Client - Token: token-abc-123
```

Notice: "AuthService created" appears only once. Both files share the same instance.

Why This Works

```
  ES Module Loading Process:
  ──────────────────────────

  1. navbar.js says: import authService from './auth-service.js'
     └──► Module system checks: "Have I loaded auth-service.js before?"
          └──► No. Load it, execute it, cache the result.
               └──► The cached result includes the authService instance.

  2. api-client.js says: import authService from './auth-service.js'
```

```
      └──► Module system checks: "Have I loaded auth-service.js before?"
           └──► Yes! Return the cached result.
                └──► Same authService instance. No new AuthService() call.
```

Implementation 2: Object.freeze for Immutable Singletons

Sometimes you want a singleton whose structure cannot be changed. `Object.freeze` prevents adding, removing, or modifying properties.

```javascript
// config.js -- frozen singleton

const config = Object.freeze({
  apiUrl: 'https://api.example.com',
  apiVersion: 'v2',
  timeout: 5000,
  maxRetries: 3,
  features: Object.freeze({
    darkMode: true,
    betaFeatures: false,
    analytics: true
  })
});

export default config;
```

```javascript
// app.js
import config from './config.js';

console.log(config.apiUrl);     // 'https://api.example.com'
console.log(config.timeout);    // 5000

// Attempting to modify
config.apiUrl = 'https://evil.com';
console.log(config.apiUrl);     // Still 'https://api.example.com'

config.newProp = 'hello';
console.log(config.newProp);    // undefined

// Nested freeze works too
config.features.darkMode = false;
console.log(config.features.darkMode); // Still true
```

Output:

```
https://api.example.com
5000
https://api.example.com
undefined
true
```

Object.freeze Is Shallow by Default

`Object.freeze` only freezes the top level. Nested objects are still mutable unless you freeze them too.

```javascript
// WRONG -- shallow freeze
const settings = Object.freeze({
  theme: {
    primary: '#007bff'
  }
});

settings.theme.primary = '#ff0000'; // This WORKS -- nested object is not frozen!
console.log(settings.theme.primary); // '#ff0000'
```

```javascript
// RIGHT -- deep freeze
function deepFreeze(obj) {
  Object.freeze(obj);
  Object.keys(obj).forEach(key => {
    if (typeof obj[key] === 'object' && obj[key] !== null) {
      deepFreeze(obj[key]);
    }
  });
  return obj;
}

const settings = deepFreeze({
  theme: {
    primary: '#007bff',
    fonts: {
      heading: 'Georgia',
      body: 'Arial'
    }
  }
});

settings.theme.primary = '#ff0000';
console.log(settings.theme.primary); // '#007bff' -- stays frozen
```

Output:

```
#007bff
```

Implementation 3: Classic Singleton Class

In environments without ES modules, or when you want explicit control, you can implement the Singleton Pattern with a class that manages its own instance.

```javascript
// database-connection.js

class DatabaseConnection {
  constructor(connectionString) {
    if (DatabaseConnection.instance) {
      console.log('Returning existing connection');
      return DatabaseConnection.instance;
    }

    this.connectionString = connectionString;
    this.connected = false;
    this.queryCount = 0;
    console.log('Creating new database connection');

    DatabaseConnection.instance = this;
  }

  connect() {
    this.connected = true;
    console.log(`Connected to ${this.connectionString}`);
  }

  query(sql) {
    if (!this.connected) {
      throw new Error('Not connected to database');
    }
    this.queryCount++;
    console.log(`Query #${this.queryCount}: ${sql}`);
    return { rows: [], queryCount: this.queryCount };
  }

  disconnect() {
    this.connected = false;
    console.log('Disconnected');
  }
}

// Usage
const db1 = new DatabaseConnection('postgres://localhost:5432/mydb');
```

```
db1.connect();
db1.query('SELECT * FROM users');

const db2 = new DatabaseConnection('postgres://localhost:5432/otherdb');
// db2 is the SAME instance as db1!
console.log('Same instance?', db1 === db2);
console.log('Connection string:', db2.connectionString);
db2.query('SELECT * FROM products');
```

Output:

```
Creating new database connection
Connected to postgres://localhost:5432/mydb
Query #1: SELECT * FROM users
Returning existing connection
Same instance? true
Connection string: postgres://localhost:5432/mydb
Query #2: SELECT * FROM products
```

Notice that `db2` silently received the same instance as `db1`. The second connection string (`otherdb`) was ignored. This can be surprising and is one reason some developers dislike this approach.

Real-World Singleton: Logger

A logger is a classic singleton use case. You want one logger that every part of your application writes to, with consistent formatting and a single configuration.

```javascript
// logger.js

const LOG_LEVELS = Object.freeze({
  DEBUG: 0,
  INFO: 1,
  WARN: 2,
  ERROR: 3,
  NONE: 4
});

class Logger {
  constructor() {
    this.level = LOG_LEVELS.INFO;
    this.logs = [];
```

```
    this.prefix = '';
  }

  setLevel(level) {
    this.level = level;
    console.log(`Log level set to ${Object.keys(LOG_LEVELS).find(k => LOG_LEVELS[k] ===...
  }

  setPrefix(prefix) {
    this.prefix = prefix;
  }

  _formatMessage(level, message) {
    const timestamp = new Date().toISOString().slice(11, 23);
    const label = Object.keys(LOG_LEVELS).find(k => LOG_LEVELS[k] === level);
    const prefixStr = this.prefix ? `[${this.prefix}] ` : '';
    return `${timestamp} [${label}] ${prefixStr}${message}`;
  }

  _log(level, message, ...args) {
    if (level < this.level) return;

    const formatted = this._formatMessage(level, message);
    this.logs.push({ level, message: formatted, timestamp: Date.now() });

    if (level === LOG_LEVELS.ERROR) {
      console.error(formatted, ...args);
    } else if (level === LOG_LEVELS.WARN) {
      console.warn(formatted, ...args);
    } else {
      console.log(formatted, ...args);
    }
  }

  debug(message, ...args) { this._log(LOG_LEVELS.DEBUG, message, ...args); }
  info(message, ...args)  { this._log(LOG_LEVELS.INFO, message, ...args); }
  warn(message, ...args)  { this._log(LOG_LEVELS.WARN, message, ...args); }
  error(message, ...args) { this._log(LOG_LEVELS.ERROR, message, ...args); }

  getHistory() {
    return [...this.logs];
  }

  clear() {
    this.logs = [];
  }
}

// Export a single instance
const logger = new Logger();
export { LOG_LEVELS };
export default logger;
```

```javascript
// auth.js
import logger from './logger.js';

function login(email) {
  logger.info(`Login attempt for ${email}`);
  // ... authentication logic
  logger.info(`Login successful for ${email}`);
}

login('alice@example.com');
```

```javascript
// api-client.js
import logger from './logger.js';

function fetchData(url) {
  logger.info(`Fetching ${url}`);
  // ... fetch logic
  logger.warn('Response was slow (> 2s)');
}

fetchData('/api/users');
```

```javascript
// app.js
import logger from './logger.js';

// All modules wrote to the SAME logger
console.log('Total log entries:', logger.getHistory().length);
```

Output:

```
14:32:01.234 [INFO] Login attempt for alice@example.com
14:32:01.235 [INFO] Login successful for alice@example.com
14:32:01.236 [INFO] Fetching /api/users
14:32:01.237 [WARN] Response was slow (> 2s)
Total log entries: 4
```

Every module imported the same `logger` instance. All four messages ended up in the same log history.

Real-World Singleton: Theme Manager

```javascript
// theme-manager.js

const THEMES = {
  light: Object.freeze({
    name: 'light',
    background: '#ffffff',
    text: '#1a1a1a',
    primary: '#007bff',
    secondary: '#6c757d',
    border: '#dee2e6'
  }),
  dark: Object.freeze({
    name: 'dark',
    background: '#1a1a1a',
    text: '#f8f9fa',
    primary: '#4dabf7',
    secondary: '#adb5bd',
    border: '#495057'
  })
};

class ThemeManager {
  constructor() {
    this.currentTheme = THEMES.light;
    this.listeners = [];
  }

  getTheme() {
    return this.currentTheme;
  }

  setTheme(themeName) {
    if (!THEMES[themeName]) {
      console.error(`Theme "${themeName}" not found`);
      return;
    }

    this.currentTheme = THEMES[themeName];
    console.log(`Theme changed to: ${themeName}`);

    // Notify all listeners
    this.listeners.forEach(listener => listener(this.currentTheme));
  }

  toggle() {
    const next = this.currentTheme.name === 'light' ? 'dark' : 'light';
    this.setTheme(next);
  }

  subscribe(listener) {
    this.listeners.push(listener);
    // Return unsubscribe function
    return () => {
      this.listeners = this.listeners.filter(l => l !== listener);
```

```
    };
  }
}

const themeManager = new ThemeManager();
export { THEMES };
export default themeManager;
```

```javascript
// header.js
import themeManager from './theme-manager.js';

// Subscribe to theme changes
themeManager.subscribe((theme) => {
  console.log(`Header updating to ${theme.name} theme`);
  console.log(`  Background: ${theme.background}`);
  console.log(`  Text: ${theme.text}`);
});

// sidebar.js
import themeManager from './theme-manager.js';

themeManager.subscribe((theme) => {
  console.log(`Sidebar updating to ${theme.name} theme`);
});

// Toggle the theme -- both subscribers get notified
themeManager.toggle();
```

Output:

```
Theme changed to: dark
Header updating to dark theme
  Background: #1a1a1a
  Text: #f8f9fa
Sidebar updating to dark theme
```

One `themeManager` instance. All components listen to the same source of truth.

Real-World Singleton: Auth Store

```javascript
// auth-store.js
```

```
class AuthStore {
  constructor() {
    this.user = null;
    this.token = null;
    this.listeners = new Set();
  }

  getState() {
    return {
      user: this.user,
      token: this.token,
      isAuthenticated: this.token !== null
    };
  }

  login(user, token) {
    this.user = user;
    this.token = token;
    this._notify();
    console.log(`Auth: Logged in as ${user.name}`);
  }

  logout() {
    this.user = null;
    this.token = null;
    this._notify();
    console.log('Auth: Logged out');
  }

  subscribe(listener) {
    this.listeners.add(listener);
    return () => this.listeners.delete(listener);
  }

  _notify() {
    const state = this.getState();
    this.listeners.forEach(listener => listener(state));
  }
}

const authStore = new AuthStore();
export default authStore;
```

```javascript
// Usage across your app
import authStore from './auth-store.js';

// Component A subscribes
const unsubscribe = authStore.subscribe((state) => {
  console.log('Component A:', state.isAuthenticated ? 'Show dashboard' : 'Show login');
});

// Component B subscribes
```

```
authStore.subscribe((state) => {
  console.log('Component B:', state.isAuthenticated ? `Welcome ${state.user.name}` : 'G...
});

// Login triggers both components
authStore.login({ name: 'Alice', email: 'alice@example.com' }, 'jwt-token-xyz');

// Logout triggers both components
authStore.logout();

// Unsubscribe Component A
unsubscribe();

// Only Component B reacts now
authStore.login({ name: 'Bob', email: 'bob@example.com' }, 'jwt-token-abc');
```

Output:

```
Component A: Show dashboard
Component B: Welcome Alice
Auth: Logged in as Alice
Component A: Show login
Component B: Guest
Auth: Logged out
Component B: Welcome Bob
Auth: Logged in as Bob
```

When the Singleton Pattern Is Bad: Testing

The biggest criticism of the Singleton Pattern is that it makes testing difficult. Because every test shares the same instance, state leaks between tests.

The Problem

```javascript
// auth-store.test.js

import authStore from './auth-store.js';

// Test 1
test('login sets user', () => {
  authStore.login({ name: 'Alice' }, 'token-1');
  expect(authStore.getState().isAuthenticated).toBe(true); // PASS
```

```
});

// Test 2
test('starts logged out', () => {
  // FAIL! authStore still has Alice from Test 1
  expect(authStore.getState().isAuthenticated).toBe(false); // OOPS
});
```

```
  Test 1 runs:                    Test 2 runs:
  ┌──────────────┐                ┌──────────────┐
  │  Login Alice  │               │  Expects     │
  │  token: xyz   │──── SAME ────►│  empty state │
  │               │   INSTANCE    │  but gets    │
  │               │               │  Alice!      │
  └──────────────┘                └──────────────┘

  Singleton state leaks between tests
```

The Fix: Add a Reset Method

```javascript
// auth-store.js -- with reset for testing

class AuthStore {
  constructor() {
    this._reset();
  }

  _reset() {
    this.user = null;
    this.token = null;
    this.listeners = new Set();
  }

  // ... other methods ...

  // Only for testing!
  __resetForTesting() {
    this._reset();
  }
}

const authStore = new AuthStore();
export default authStore;
```

```javascript
// auth-store.test.js -- fixed
```

```
import authStore from './auth-store.js';

beforeEach(() => {
  authStore.__resetForTesting(); // Clean state before each test
});

test('login sets user', () => {
  authStore.login({ name: 'Alice' }, 'token-1');
  expect(authStore.getState().isAuthenticated).toBe(true); // PASS
});

test('starts logged out', () => {
  expect(authStore.getState().isAuthenticated).toBe(false); // PASS
});
```

The Alternative: Dependency Injection

Instead of importing the singleton directly, pass it as a parameter. This makes testing easy because you can pass a mock.

```javascript
// api-client.js -- dependency injection version

export function createApiClient(authStore) {
  return {
    async fetchData(url) {
      const { token } = authStore.getState();
      const headers = token ? { Authorization: `Bearer ${token}` } : {};
      console.log('Fetching with token:', token);
      // return fetch(url, { headers });
    }
  };
}
```

```javascript
// In production
import authStore from './auth-store.js';
import { createApiClient } from './api-client.js';

const api = createApiClient(authStore);
```

```javascript
// In tests -- easy to mock!
import { createApiClient } from './api-client.js';

test('sends auth header when logged in', async () => {
  const mockAuthStore = {
    getState: () => ({ token: 'test-token-123', isAuthenticated: true })
  };
```

```
  const api = createApiClient(mockAuthStore);
  await api.fetchData('/api/users'); // Uses mock, not real singleton
});
```

Output:

```
Fetching with token: test-token-123
```

Singleton vs Module-Level Object: What Is the Difference?

You might wonder: "If I just export a plain object, is that a singleton?"

Yes, effectively. In ES modules, exporting an object literal gives you a singleton for free.

```javascript
// Simple object singleton -- works perfectly for many cases
export const analytics = {
  events: [],

  track(event, data) {
    this.events.push({ event, data, timestamp: Date.now() });
    console.log(`Tracked: ${event}`);
  },

  getEvents() {
    return [...this.events];
  }
};
```

This is simpler than a class-based singleton and works in most cases. Use a class when you need:

- Constructor logic
- Private methods (with `#private` syntax)
- Inheritance
- A reset method for testing

```
  When to Use What:
  ─────────────────
```

```
Plain Object Export        Class Instance Export
___________________        _____________________

Simple key-value config    Has initialization logic
Stateless utilities        Manages internal state
Small, simple API          Needs private methods
No constructor needed      Needs a reset for testing

const config = {           const logger = new Logger();
  apiUrl: '...',           export default logger;
  timeout: 5000
};
export default config;
```

When to Use / When NOT to Use

When to Use the Singleton Pattern

- Configuration objects: App-wide settings that should not be duplicated
- Loggers: One log destination for the entire application
- Caches: A shared cache that all modules read from and write to
- Connection pools: Database or WebSocket connections
- Theme managers: One source of truth for the current theme
- Auth state: One place that knows if the user is logged in
- Feature flags: One object that tells every component which features are enabled

When NOT to Use the Singleton Pattern

- React component state: Use `useState`, Context, or a state management library. Singletons do not trigger re-renders
- When you need multiple instances: If you might ever need two loggers or two themes simultaneously, a singleton is the wrong choice
- When testability is critical: Singletons make unit testing harder. If your module needs heavy testing, prefer dependency injection
- When the "single instance" constraint is artificial: If you are making something a singleton just because you currently only need one, you are over-constraining your design
- Server-side rendering (SSR): Singletons persist across requests on the server, leaking one user's data to another. This is a serious security issue in Next.js and similar

frameworks

Common Mistakes

Mistake 1: Singleton State in Server-Side Rendering

```javascript
// DANGEROUS in Next.js or any SSR framework
// auth-store.js
const authStore = new AuthStore();
export default authStore;

// On the server, this singleton is shared across ALL requests.
// User A logs in, then User B's request sees User A's auth state!
```

Fix: In SSR, create instances per request or use framework-provided state management.

Mistake 2: Hidden Coupling

```javascript
// Every file that imports the singleton is now coupled to it
import logger from './logger.js';

// If you ever want to replace the logger (say, for a different
// logging service), you have to change every file that imports it.
```

Fix: Use dependency injection for critical dependencies, especially in libraries.

Mistake 3: Using Singletons for Everything

```javascript
// Do NOT make everything a singleton
const StringUtils = new StringUtilsClass();     // No! Just export functions
const MathHelper = new MathHelperClass();       // No! Just export functions
const Formatter = new FormatterClass();          // No! Just export functions
```

Stateless utility functions do not need to be singletons. Just export them as regular functions.

Mistake 4: Forgetting That Singletons Are Mutable by Default

```javascript
// config.js
const config = { apiUrl: 'https://api.example.com', timeout: 5000 };
export default config;

// some-module.js
import config from './config.js';
config.apiUrl = 'https://evil.com'; // Silently changes it for everyone!
```

Fix: Use `Object.freeze` for configuration singletons.

Best Practices

1. Prefer ES module singletons over class-based singletons with static instance checking. They are simpler and idiomatic in modern JavaScript.
2. Freeze configuration singletons with `Object.freeze` (deeply if needed) to prevent accidental mutations.
3. Add a reset method for testing. Name it obviously (like `__resetForTesting`) so it is not called in production.
4. Use dependency injection alongside singletons. Import the singleton in your entry point and pass it to modules that need it.
5. Be careful with SSR. In server-side rendering, singletons persist across requests. Use per-request state instead.
6. Keep singletons small. A singleton should have a focused responsibility. If it is growing beyond one concern, split it into multiple singletons.
7. Document that something is a singleton. Future developers need to know they are sharing state.

Quick Summary

The Singleton Pattern ensures exactly one instance of an object exists across your application.

```
Singleton Creation Methods:
──────────────────────────
```

```
1. ES Module Export (simplest, recommended)
   const instance = new MyClass();
   export default instance;

2. Object.freeze (for immutable config)
   export default Object.freeze({ key: 'value' });

3. Class with Static Instance (explicit control)
   class MyClass {
     constructor() {
       if (MyClass.instance) return MyClass.instance;
       MyClass.instance = this;
     }
   }

Biggest Pitfall: Testing
────────────────────────
Solution: Reset methods + dependency injection
```

Key Points

- The Singleton Pattern restricts a class or module to exactly one instance
- ES modules are natural singletons because they are evaluated once and cached by the module system
- Object.freeze makes a singleton immutable, but it is shallow -- use deep freeze for nested objects
- Singletons make testing harder because state leaks between tests. Always add a reset method
- Dependency injection is the antidote to tight singleton coupling -- pass the instance instead of importing it directly
- Real-world singletons include loggers, auth stores, theme managers, config objects, and caches
- Do not use singletons in SSR without per-request isolation
- Stateless utility functions do not need to be singletons -- just export them normally

---

Practice Questions

1. What happens if two files both call `new AuthService()` instead of importing a shared instance? How does the Singleton Pattern prevent this?
2. Why does `Object.freeze` only freeze the top level of an object? Write a function that deeply freezes an object.
3. Explain why singletons are problematic in server-side rendering. What happens when two different users' requests share the same singleton?
4. How does dependency injection help solve the testing problem with singletons? Give a concrete example.
5. When should you use a plain exported object versus a class-based singleton? What are the trade-offs?

Exercises

Exercise 1: Build a Feature Flags Singleton

Create a `FeatureFlags` singleton that:

- Stores a set of feature flags (key-value pairs where values are booleans)
- Has a method to check if a feature is enabled
- Has a method to enable or disable a feature
- Uses `Object.freeze` on the initial default flags
- Allows overrides at runtime (hint: keep defaults frozen but store overrides separately)
- Includes a `__resetForTesting` method

Test it by enabling a feature in one file and checking it from another.

Exercise 2: Singleton with Lazy Initialization

Sometimes you do not want to create the singleton until it is first needed (lazy initialization). Build a `DatabasePool` singleton where:

- The connection pool is not created until `getPool()` is called for the first time
- Subsequent calls to `getPool()` return the same pool
- The pool has `connect()`, `query()`, and `disconnect()` methods
- Track and display the total number of queries across the application

Exercise 3: Convert Singleton to Dependency Injection

Take this tightly coupled code and refactor it to use dependency injection:

```javascript
import logger from './logger.js';
import authStore from './auth-store.js';

function fetchUserProfile(userId) {
  logger.info(`Fetching profile for ${userId}`);
  const token = authStore.getToken();
  // ... fetch logic
}
```

Make `fetchUserProfile` testable without importing any singletons.

What Is Next?

You now know how to ensure exactly one instance exists. But here is the next question: what happens when that single instance changes, and other parts of your application need to know about it?

The theme manager and auth store examples in this chapter already hinted at the answer: subscribers that listen for changes. That is the Observer Pattern, and it is one of the most important patterns in all of frontend development. Chapter 4 dives deep into it.

Chapter 4: Observer Pattern

What You Will Learn

- What the Observer Pattern is and how it models real-world subscriptions
- The two roles: Subject (the thing being watched) and Observer (the watcher)
- How to implement the Observer Pattern from scratch in JavaScript
- How `addEventListener` is the Observer Pattern in disguise
- Why unsubscribing is critical for preventing memory leaks
- How the Observer Pattern powers React state management
- Real-world applications: form validation, real-time data, and component communication

Why This Chapter Matters

Think about a YouTube channel. When a creator uploads a new video, you do not have to check their page every five minutes to see if something new is available. Instead, you hit the Subscribe button. Now YouTube automatically notifies you whenever a new video drops.

You are the observer. The YouTube channel is the subject. You subscribed to changes. When a change happens, you get notified. When you no longer care, you unsubscribe.

This is the Observer Pattern, and it is everywhere in frontend development. Every time you write `addEventListener`, you are using the Observer Pattern. Every time a React component re-renders because state changed, the Observer Pattern is working behind the scenes. Every time a Redux store notifies connected components, that is the Observer Pattern.

Understanding this pattern deeply will change how you think about communication between parts of your application. Instead of one piece of code directly calling another, the Observer Pattern lets pieces of code say "tell me when something happens" and react accordingly.

The Problem: Tight Coupling and Polling

Imagine you have a weather station that measures temperature. Several displays need to show the current temperature: a dashboard widget, a mobile notification, and an analytics logger.

The Painful Way (Before the Pattern)

```javascript
// weather-station.js -- tightly coupled version

class WeatherStation {
  constructor() {
    this.temperature = 0;
    // The station has to KNOW about every display
    this.dashboard = null;
    this.mobileNotifier = null;
    this.analyticsLogger = null;
  }

  setTemperature(temp) {
    this.temperature = temp;
    console.log(`Temperature updated to ${temp}°C`);

    // Manually update each display
    if (this.dashboard) {
      this.dashboard.update(temp);
    }
    if (this.mobileNotifier) {
      this.mobileNotifier.notify(temp);
    }
    if (this.analyticsLogger) {
      this.analyticsLogger.log(temp);
    }
    // What if we add 10 more displays? 20 more?
    // This function grows forever.
  }
}

class Dashboard {
  update(temp) {
    console.log(`Dashboard: Displaying ${temp}°C`);
  }
}

class MobileNotifier {
  notify(temp) {
    console.log(`Mobile: Push notification - ${temp}°C`);
  }
}
```

```
const station = new WeatherStation();
station.dashboard = new Dashboard();
station.mobileNotifier = new MobileNotifier();
station.setTemperature(22);
```

Output:

```
Temperature updated to 22°C
Dashboard: Displaying 22°C
Mobile: Push notification - 22°C
```

What Is Wrong Here?

```
┌──────────────────────────────────────────────┐
│            WeatherStation                    │
│                                              │
│  setTemperature() {                          │
│    if (this.dashboard) {...}      ←── KNOWS  │
│    if (this.mobileNotifier) {...} ←── ABOUT  │
│    if (this.analyticsLogger) {...} ←── EACH  │
│    if (this.newThing1) {...}      ←── SINGLE │
│    if (this.newThing2) {...}      ←── ONE    │
│  }                                           │
│                                              │
│  The station is tightly coupled              │
│  to every consumer.                          │
│                                              │
│  Adding a new consumer means                 │
│  modifying the station code.                 │
└──────────────────────────────────────────────┘
```

Problems:

1. Tight coupling: The `WeatherStation` must know about every display class and its method names
2. Violation of Open/Closed Principle: Adding a new display requires modifying `WeatherStation`
3. Different interfaces: Each display has a different method name (`update`, `notify`, `log`), making the code inconsistent
4. Hard to remove displays: No clean way to stop notifying a display

The Solution: Observer Pattern

The Observer Pattern defines a one-to-many relationship between objects. When the subject (the thing being watched) changes state, all its observers (the watchers) are notified automatically.

```
┌──────────────┐          ┌──────────────┐
│              │ subscribe│              │
│              │◄─────────│  Observer A  │
│              │          │  (Dashboard) │
│              │          └──────────────┘
│              │
│   Subject    │          ┌──────────────┐
│  (Weather    │ subscribe│              │
│   Station)   │◄─────────│  Observer B  │
│              │          │  (Mobile)    │
│              │          └──────────────┘
│              │
│   When state │          ┌──────────────┐
│   changes,   │ subscribe│              │
│   notify all │◄─────────│  Observer C  │
│              │          │  (Logger)    │
└──────────────┘          └──────────────┘
        │
        │ notify("temperature changed!")
        │
        ├──────────────────► Observer A gets notified
        ├──────────────────► Observer B gets notified
        └──────────────────► Observer C gets notified
```

The subject does not know or care what its observers are. It just maintains a list and notifies everyone on that list.

Building the Observer Pattern from Scratch

Step 1: The Subject (Observable)

```javascript
// observable.js

class Observable {
  constructor() {
    this.observers = [];
```

```
  }

  subscribe(observer) {
    this.observers.push(observer);
    console.log(`New observer subscribed. Total: ${this.observers.length}`);
  }

  unsubscribe(observer) {
    this.observers = this.observers.filter(obs => obs !== observer);
    console.log(`Observer unsubscribed. Total: ${this.observers.length}`);
  }

  notify(data) {
    console.log(`Notifying ${this.observers.length} observer(s)`);
    this.observers.forEach(observer => observer(data));
  }
}

export default Observable;
```

Step 2: Using the Observable

```javascript
// weather-station.js -- Observer Pattern version
import Observable from './observable.js';

class WeatherStation extends Observable {
  constructor() {
    super();
    this.temperature = 0;
    this.humidity = 0;
  }

  setMeasurements(temperature, humidity) {
    this.temperature = temperature;
    this.humidity = humidity;
    console.log(`\nMeasurements: ${temperature}°C, ${humidity}% humidity`);

    // Notify all observers with the new data
    this.notify({
      temperature: this.temperature,
      humidity: this.humidity,
      timestamp: Date.now()
    });
  }
}

// Create the station
const station = new WeatherStation();

// Create observers (just functions!)
```

```
function dashboardDisplay(data) {
  console.log(`Dashboard: ${data.temperature}°C | ${data.humidity}% humidity`);
}

function mobileAlert(data) {
  if (data.temperature > 35) {
    console.log(`MOBILE ALERT: High temperature warning! ${data.temperature}°C`);
  } else {
    console.log(`Mobile: Temperature is ${data.temperature}°C -- normal`);
  }
}

function analyticsLogger(data) {
  console.log(`Analytics: Logged temp=${data.temperature}, humidity=${data.humidity}`);
}

// Subscribe observers
station.subscribe(dashboardDisplay);
station.subscribe(mobileAlert);
station.subscribe(analyticsLogger);

// Update measurements -- all observers get notified
station.setMeasurements(22, 65);
station.setMeasurements(38, 40);

// Unsubscribe the mobile alert
station.unsubscribe(mobileAlert);

// Only dashboard and analytics get notified now
station.setMeasurements(15, 80);
```

Output:

```
New observer subscribed. Total: 1
New observer subscribed. Total: 2
New observer subscribed. Total: 3

Measurements: 22°C, 65% humidity
Notifying 3 observer(s)
Dashboard: 22°C | 65% humidity
Mobile: Temperature is 22°C -- normal
Analytics: Logged temp=22, humidity=65

Measurements: 38°C, 40% humidity
Notifying 3 observer(s)
Dashboard: 38°C | 40% humidity
MOBILE ALERT: High temperature warning! 38°C
Analytics: Logged temp=38, humidity=40
Observer unsubscribed. Total: 2

Measurements: 15°C, 80% humidity
Notifying 2 observer(s)
Dashboard: 15°C | 80% humidity
```

```
Analytics: Logged temp=15, humidity=80
```

Before vs After

```
BEFORE (Tight Coupling)          AFTER (Observer Pattern)
────────────────────────         ────────────────────────


WeatherStation knows about:      WeatherStation knows about:
- Dashboard                      - A list of functions
- MobileNotifier                 - That is all
- AnalyticsLogger
- FutureDisplay1                 Adding a new observer:
- FutureDisplay2                 station.subscribe(newFn)
                                 Zero changes to WeatherStation
Adding a new display:
1. Create the class               Removing an observer:
2. Add property to station         station.unsubscribe(fn)
3. Add if-check in notify          Zero changes to WeatherStation
4. Modify WeatherStation code
```

---

addEventListener: The Observer Pattern in Your Browser

You have been using the Observer Pattern since your first `addEventListener` call.

```javascript
// The DOM element is the SUBJECT
const button = document.querySelector('#myButton');

// Each event listener is an OBSERVER
function handleClick(event) {
  console.log('Button clicked!', event.target);
}

function trackClick(event) {
  console.log('Analytics: button_click event tracked');
}

function animateClick(event) {
  console.log('Animation: ripple effect triggered');
}
```

```
// SUBSCRIBE observers to the subject
button.addEventListener('click', handleClick);
button.addEventListener('click', trackClick);
button.addEventListener('click', animateClick);

// When the button is clicked, ALL observers are notified
// Just like our Observable.notify() method!

// UNSUBSCRIBE an observer
button.removeEventListener('click', trackClick);
// Now only handleClick and animateClick will fire on click
```

```
  DOM Event System = Observer Pattern

  ┌──────────────┐           ┌──────────────────────┐
  │  <button>    │           │  addEventListener    │
  │  (Subject)   │           │  = subscribe()       │
  │              │           │                      │
  │  Has a list  │           │  removeEventListener│
  │  of event    │           │  = unsubscribe()     │
  │  listeners   │           │                      │
  │              │           │  Event fires         │
  │              │           │  = notify()           │
  └──────────────┘           └──────────────────────┘
```

Custom Events: Building Your Own DOM Events

You can create custom events using the same Observer Pattern mechanism built into the browser.

```javascript
// Create a custom event target
const appEvents = new EventTarget();

// Subscribe to custom events
appEvents.addEventListener('user:login', (e) => {
  console.log(`User logged in: ${e.detail.name}`);
});

appEvents.addEventListener('user:login', (e) => {
  console.log(`Updating navbar for ${e.detail.name}`);
});

// Fire the event -- notifies all listeners
appEvents.dispatchEvent(new CustomEvent('user:login', {
  detail: { name: 'Alice', email: 'alice@example.com' }
}));
```

Output:

```
User logged in: Alice
Updating navbar for Alice
```

A More Complete Observable Implementation

Let us build a production-quality Observable with useful features.

```javascript
// observable.js -- production version

class Observable {
  constructor() {
    this._observers = new Map(); // event -> Set of callbacks
  }

  /**
   * Subscribe to a specific event
   * Returns an unsubscribe function
   */
  on(event, callback) {
    if (!this._observers.has(event)) {
      this._observers.set(event, new Set());
    }
    this._observers.get(event).add(callback);

    // Return unsubscribe function (like React useEffect cleanup)
    return () => {
      this._observers.get(event)?.delete(callback);
      // Clean up empty sets
      if (this._observers.get(event)?.size === 0) {
        this._observers.delete(event);
      }
    };
  }

  /**
   * Subscribe to an event, but only fire once
   */
  once(event, callback) {
    const unsubscribe = this.on(event, (data) => {
      callback(data);
      unsubscribe(); // Auto-remove after first call
    });
    return unsubscribe;
  }

  /**
```

```
   * Emit an event with data
   */
  emit(event, data) {
    const callbacks = this._observers.get(event);
    if (callbacks) {
      callbacks.forEach(callback => {
        try {
          callback(data);
        } catch (error) {
          console.error(`Error in observer for "${event}":`, error);
        }
      });
    }
  }

  /**
   * Remove all observers for a specific event, or all events
   */
  removeAll(event) {
    if (event) {
      this._observers.delete(event);
    } else {
      this._observers.clear();
    }
  }

  /**
   * Get the count of observers for an event
   */
  listenerCount(event) {
    return this._observers.get(event)?.size || 0;
  }
}

export default Observable;
```

Using the Production Observable

```javascript
import Observable from './observable.js';

const store = new Observable();

// Subscribe to 'change' events
const unsubName = store.on('change', (data) => {
  console.log('Name observer:', data);
});

store.on('change', (data) => {
  console.log('Logger:', JSON.stringify(data));
});
```

```
// Subscribe to 'error' event, but only fire once
store.once('error', (err) => {
  console.log('One-time error handler:', err.message);
});

console.log('Change listeners:', store.listenerCount('change'));
console.log('Error listeners:', store.listenerCount('error'));

// Emit events
store.emit('change', { field: 'name', value: 'Alice' });
store.emit('error', { message: 'Network timeout' });
store.emit('error', { message: 'Second error' }); // once() already removed, no handler

// Unsubscribe name observer
unsubName();
console.log('\nAfter unsubscribing name observer:');
console.log('Change listeners:', store.listenerCount('change'));

store.emit('change', { field: 'email', value: 'alice@example.com' });
```

Output:

```
Change listeners: 2
Error listeners: 1
Name observer: { field: 'name', value: 'Alice' }
Logger: {"field":"name","value":"Alice"}
One-time error handler: Network timeout

After unsubscribing name observer:
Change listeners: 1
Logger: {"field":"email","value":"alice@example.com"}
```

Memory Leaks: The Silent Killer

The most dangerous mistake with the Observer Pattern is forgetting to unsubscribe. If an observer subscribes but never unsubscribes, it stays in memory forever.

The Problem

```javascript
// MEMORY LEAK EXAMPLE
```

```
class UserProfile {
  constructor(userId) {
    this.userId = userId;

    // Subscribe to auth changes
    authStore.on('change', (state) => {
      console.log(`Profile ${this.userId}: auth state changed`);
      // This callback holds a reference to `this` (the UserProfile instance)
    });
  }

  destroy() {
    // Oops! We forgot to unsubscribe!
    // The callback is still in authStore's observer list
    // The garbage collector cannot clean up this UserProfile
    console.log(`Profile ${this.userId} destroyed... or is it?`);
  }
}

// Create and destroy 1000 profiles
for (let i = 0; i < 1000; i++) {
  const profile = new UserProfile(i);
  profile.destroy();
}

// authStore now has 1000 callbacks!
// Each callback holds a reference to a UserProfile
// None of them can be garbage collected
// This is a memory leak
```

```
┌──────────────────────────────────────────────────────┐
│                   Memory Over Time                   │
│                                                      │
│  Memory                                              │
│  Usage     ////////////////////  ← Leaked    │
│    ▲      /                        observers   │
│    │     /                         keep        │
│    │    /                          growing     │
│    │   /                                       │
│    │  /                                        │
│    │ /                                         │
│    │/                                          │
│    └──────────────────────────────► Time          │
│                                                      │
│    Without unsubscribe, memory usage only goes       │
│    UP. Never down. Eventually: crash.                │
└──────────────────────────────────────────────────────┘
```

The Fix: Always Unsubscribe

```javascript
class UserProfile {
  constructor(userId) {
    this.userId = userId;

    // Store the unsubscribe function
    this._unsubscribeAuth = authStore.on('change', (state) => {
      console.log(`Profile ${this.userId}: auth state changed`);
    });
  }

  destroy() {
    // Clean up the subscription!
    this._unsubscribeAuth();
    console.log(`Profile ${this.userId} properly destroyed`);
  }
}
```

Memory Leaks in React: The useEffect Cleanup

React components face the same problem. When a component unmounts, its subscriptions must be cleaned up.

```javascript
// BAD -- memory leak
import { useEffect, useState } from 'react';
import chatService from './chat-service';

function ChatMessages() {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    // Subscribe but never unsubscribe!
    chatService.on('message', (msg) => {
      setMessages(prev => [...prev, msg]);
    });
    // When this component unmounts, the callback stays registered
    // It will try to call setMessages on an unmounted component
  }, []);

  return (
    <ul>
      {messages.map((msg, i) => <li key={i}>{msg.text}</li>)}
    </ul>
  );
}
```

```javascript
// GOOD -- cleanup on unmount
import { useEffect, useState } from 'react';
import chatService from './chat-service';
```

```
function ChatMessages() {
  const [messages, setMessages] = useState([]);

  useEffect(() => {
    const unsubscribe = chatService.on('message', (msg) => {
      setMessages(prev => [...prev, msg]);
    });

    // Return cleanup function -- React calls this on unmount
    return () => {
      unsubscribe();
    };
  }, []);

  return (
    <ul>
      {messages.map((msg, i) => <li key={i}>{msg.text}</li>)}
    </ul>
  );
}
```

```
  React Component Lifecycle + Observer Pattern:
  ─────────────────────────────────────────────

  Mount ──► useEffect runs ──► subscribe to observable
                                        │
  Live  ──► Receives notifications ─┘
                                        │
  Unmount ► cleanup function runs ──► unsubscribe from observable
                                        │
  Result: No memory leak. Clean. ───┘
```

Real-World Example: Form Validation with Observers

```javascript
// form-validator.js
import Observable from './observable.js';

class FormValidator extends Observable {
  constructor() {
    super();
    this.fields = {};
```

```
    this.errors = {};
  }

  registerField(name, rules) {
    this.fields[name] = { value: '', rules, touched: false };
    this.errors[name] = [];
  }

  setValue(name, value) {
    if (!this.fields[name]) return;

    this.fields[name].value = value;
    this.fields[name].touched = true;

    // Validate this field
    this.errors[name] = this._validate(name, value);

    // Notify observers with current state
    this.emit('fieldChange', {
      field: name,
      value,
      errors: this.errors[name],
      isValid: this.errors[name].length === 0,
      allValid: this.isFormValid()
    });
  }

  _validate(name, value) {
    const errors = [];
    const rules = this.fields[name].rules;

    if (rules.required && !value.trim()) {
      errors.push(`${name} is required`);
    }
    if (rules.minLength && value.length < rules.minLength) {
      errors.push(`${name} must be at least ${rules.minLength} characters`);
    }
    if (rules.pattern && !rules.pattern.test(value)) {
      errors.push(`${name} format is invalid`);
    }

    return errors;
  }

  isFormValid() {
    return Object.values(this.errors).every(fieldErrors => fieldErrors.length === 0)
      && Object.values(this.fields).every(field => field.touched);
  }
}

// Usage
const form = new FormValidator();

// Register fields with rules
form.registerField('email', {
```

```
  required: true,
  pattern: /^[^\s@]+@[^\s@]+\.[^\s@]+$/
});

form.registerField('password', {
  required: true,
  minLength: 8
});

// Observer: Error display
form.on('fieldChange', ({ field, errors }) => {
  if (errors.length > 0) {
    console.log(`[Error Display] ${field}: ${errors.join(', ')}`);
  } else {
    console.log(`[Error Display] ${field}: valid`);
  }
});

// Observer: Submit button state
form.on('fieldChange', ({ allValid }) => {
  console.log(`[Submit Button] ${allValid ? 'Enabled' : 'Disabled'}`);
});

// Simulate user typing
console.log('--- User types email ---');
form.setValue('email', 'bad');
console.log('\n--- User fixes email ---');
form.setValue('email', 'alice@example.com');
console.log('\n--- User types short password ---');
form.setValue('password', '123');
console.log('\n--- User types valid password ---');
form.setValue('password', 'securepass123');
```

Output:

```
--- User types email ---
[Error Display] email: email format is invalid
[Submit Button] Disabled

--- User fixes email ---
[Error Display] email: valid
[Submit Button] Disabled

--- User types short password ---
[Error Display] password: password must be at least 8 characters
[Submit Button] Disabled

--- User types valid password ---
[Error Display] password: valid
[Submit Button] Enabled
```

Real-World Example: Online/Offline Status Monitor

```javascript
// connection-monitor.js
import Observable from './observable.js';

class ConnectionMonitor extends Observable {
  constructor() {
    super();
    this.isOnline = typeof navigator !== 'undefined' ? navigator.onLine : true;

    if (typeof window !== 'undefined') {
      window.addEventListener('online', () => this._setStatus(true));
      window.addEventListener('offline', () => this._setStatus(false));
    }
  }

  _setStatus(online) {
    const wasOnline = this.isOnline;
    this.isOnline = online;

    if (wasOnline !== online) {
      this.emit('statusChange', {
        isOnline: online,
        timestamp: new Date().toISOString()
      });
    }
  }

  getStatus() {
    return this.isOnline;
  }
}

const monitor = new ConnectionMonitor();

// Observer: Show banner
monitor.on('statusChange', ({ isOnline }) => {
  if (isOnline) {
    console.log('Banner: Back online! Syncing data...');
  } else {
    console.log('Banner: You are offline. Changes will be saved locally.');
  }
});

// Observer: Queue or send requests
monitor.on('statusChange', ({ isOnline }) => {
  if (isOnline) {
    console.log('API Client: Flushing queued requests');
  } else {
    console.log('API Client: Switching to offline queue');
  }
});
```

```
// Simulate going offline and back online
monitor._setStatus(false);
monitor._setStatus(true);
```

Output:

```
Banner: You are offline. Changes will be saved locally.
API Client: Switching to offline queue
Banner: Back online! Syncing data...
API Client: Flushing queued requests
```

Observer Pattern in React: A Custom Hook

You can wrap the Observer Pattern in a custom hook so React components automatically subscribe and unsubscribe.

```javascript
// useObservable.js
import { useState, useEffect } from 'react';

export function useObservable(observable, event, initialValue) {
  const [value, setValue] = useState(initialValue);

  useEffect(() => {
    const unsubscribe = observable.on(event, (data) => {
      setValue(data);
    });

    // Cleanup on unmount
    return unsubscribe;
  }, [observable, event]);

  return value;
}
```

```javascript
// TemperatureDisplay.jsx
import { useObservable } from './useObservable';
import weatherStation from './weather-station';

function TemperatureDisplay() {
  const data = useObservable(weatherStation, 'measurement', {
    temperature: 0,
```

```
    humidity: 0
  });

  return (
    <div>
      <h2>Current Weather</h2>
      <p>Temperature: {data.temperature}°C</p>
      <p>Humidity: {data.humidity}%</p>
    </div>
  );
}

// Multiple components can use the same observable
function WeatherAlerts() {
  const data = useObservable(weatherStation, 'measurement', {
    temperature: 0
  });

  if (data.temperature > 35) {
    return <div className="alert">High temperature warning!</div>;
  }
  return null;
}
```

When `TemperatureDisplay` or `WeatherAlerts` unmounts, the cleanup function in `useEffect` runs `unsubscribe()`. No memory leaks.

When to Use / When NOT to Use

When to Use the Observer Pattern

- DOM event handling: Any situation where elements emit events (clicks, scrolls, resizes, key presses)
- Real-time data: WebSocket messages, server-sent events, live updates
- Cross-component communication: When one component's action should affect several other components
- Form validation: Changes to form fields trigger validation, error display, and submit button state
- State management: Stores that notify UI components of changes
- Undo/redo systems: Record changes and notify the UI to update

When NOT to Use the Observer Pattern

- Simple parent-child communication: In React, just pass props. Do not add an observer system for one-way data flow
- Synchronous one-to-one communication: If only one thing needs to know about a change, call it directly. The Observer Pattern adds complexity for one-to-one relationships
- When you forget to unsubscribe: If your team struggles with cleanup, consider simpler alternatives like React Context or a state management library that handles subscriptions for you
- Performance-critical hot paths: Notifying many observers in a tight loop (like during animation frames) can cause performance issues. Profile and optimize

---

Common Mistakes

Mistake 1: Subscribing in a Loop Without Unsubscribing

```javascript
// BAD -- creates a new subscription every render cycle
function BadComponent() {
  // This runs on EVERY render
  store.on('change', (data) => {
    console.log(data); // 1 call on first render, 2 on second, 3 on third...
  });

  return <div>...</div>;
}
```

```javascript
// GOOD -- subscribe once in useEffect with cleanup
function GoodComponent() {
  useEffect(() => {
    const unsubscribe = store.on('change', (data) => {
      console.log(data); // Always just 1 call
    });
    return unsubscribe;
  }, []);

  return <div>...</div>;
}
```

Mistake 2: Modifying the Observer List During Notification

```javascript
// DANGEROUS -- modifying the list while iterating
class BadObservable {
  notify(data) {
    this.observers.forEach(observer => {
      observer(data); // What if this observer calls unsubscribe()?
      // The array changes WHILE we are iterating over it!
    });
  }
}
```

```javascript
// SAFE -- iterate over a copy
class SafeObservable {
  notify(data) {
    const observersCopy = [...this.observers]; // Copy the array
    observersCopy.forEach(observer => observer(data));
  }
}
```

Mistake 3: Not Handling Errors in Observers

```javascript
// BAD -- one broken observer stops all subsequent observers
class FragileObservable {
  notify(data) {
    this.observers.forEach(observer => observer(data));
    // If observer #2 throws, observers #3, #4, #5 never get called
  }
}
```

```javascript
// GOOD -- catch errors per observer
class RobustObservable {
  notify(data) {
    this.observers.forEach(observer => {
      try {
        observer(data);
      } catch (error) {
        console.error('Observer error:', error);
      }
    });
  }
}
```

Best Practices

1. Always return an unsubscribe function from your `subscribe`/`on` method. This makes cleanup easy and follows the pattern used by React's `useEffect`.
2. Use `useEffect` cleanup in React components. Every subscription in a `useEffect` should have a corresponding unsubscribe in the returned cleanup function.
3. Wrap errors in try/catch inside your notify loop. One broken observer should not prevent others from being notified.
4. Iterate over a copy of the observer list during notification. This prevents bugs when observers add or remove themselves during notification.
5. Name your events clearly. Use descriptive event names like `'user:login'`, `'cart:itemAdded'`, `'theme:changed'` instead of generic names like `'update'` or `'change'`.
6. Keep observer callbacks lightweight. Heavy processing in an observer can block other observers and the UI. Offload expensive work with `requestAnimationFrame` or `setTimeout`.
7. Consider using a `once` method for events that should only be handled once (initialization, one-time setup, etc.).

Quick Summary

The Observer Pattern establishes a subscription mechanism where observers register interest in a subject's changes and get notified automatically.

```
Observer Pattern Flow:
──────────────────────

1. Subject maintains a list of observers
2. Observers subscribe: subject.on('event', callback)
3. Subject state changes
4. Subject notifies all observers: subject.emit('event', data)
5. Each observer reacts to the data
6. When done, observer unsubscribes: unsubscribe()

Built-in Examples:
──────────────────
- addEventListener / removeEventListener
- React's useState (triggers re-render for all dependent components)
- Node.js EventEmitter
- RxJS Observables
```

Key Points

- The Observer Pattern defines a one-to-many dependency so that when the subject changes, all observers are notified
- The subject (observable) maintains a list of observers and notifies them of changes
- Observers are callbacks that react to the subject's changes
- addEventListener is the Observer Pattern built into the browser's DOM API
- Unsubscribing is critical -- failing to remove observers causes memory leaks
- In React, use useEffect cleanup functions to unsubscribe when components unmount
- Always handle errors in observer callbacks to prevent one broken observer from blocking others
- The Observer Pattern decouples the subject from its observers -- the subject never needs to know what its observers are

---

Practice Questions

1. What is the difference between tight coupling (calling observers directly) and the Observer Pattern? What advantage does loose coupling provide?
2. Why does `addEventListener` qualify as an implementation of the Observer Pattern? Identify the subject, observer, subscribe, and notify operations.
3. What causes memory leaks with the Observer Pattern? How does React's `useEffect` cleanup function prevent them?
4. Why should you iterate over a copy of the observer list during notification? What bug can occur if you iterate over the original list?
5. What is the difference between `on` (subscribe) and `once` (subscribe once)? Give a real-world scenario where `once` is more appropriate.

Exercises

Exercise 1: Build a Stock Price Ticker

Create a `StockTicker` observable that:

- Tracks the price of multiple stocks (e.g., AAPL, GOOGL, MSFT)

- Allows observers to subscribe to specific stocks by name
- Notifies only the relevant observers when a specific stock price changes
- Has a `once` feature for one-time price alerts (e.g., "notify me when AAPL hits $200")

Test it with at least three observers watching different stocks.

Exercise 2: Observable with History

Extend the `Observable` class to include:

- A history buffer that stores the last N emitted values (configurable)
- A `replay` method that immediately sends all buffered values to a new subscriber
- This is useful when a component subscribes after events have already been emitted

This pattern is similar to RxJS `ReplaySubject`.

Exercise 3: React useObservable Hook with Selector

Build a custom React hook `useObservable(observable, event, selector)` where:

- `selector` is a function that extracts only the data the component needs
- The hook only triggers a re-render if the selected data actually changed (shallow comparison)
- Include proper cleanup on unmount

Example usage:

```javascript
// Only re-renders when temperature changes, ignores humidity changes
const temp = useObservable(weatherStation, 'measurement', data => data.temperature);
```

What Is Next?

The Observer Pattern creates a direct relationship between subjects and observers. The observer subscribes directly to the subject. But what if two parts of your application need to communicate without knowing each other at all?

That is where the Publish/Subscribe (Pub/Sub) Pattern comes in. Chapter 5 introduces an intermediary -- an event bus -- that lets components communicate without any direct connection.

Chapter 5: Publish/Subscribe Pattern

What You Will Learn

- What the Pub/Sub Pattern is and how it differs from the Observer Pattern
- How to build an EventBus class from scratch
- How components communicate without knowing each other
- Cross-component communication in frontend applications
- Real-world uses: analytics tracking, notification systems, and micro-frontends
- How to avoid common Pub/Sub pitfalls like event storms and debugging nightmares

Why This Chapter Matters

Think about a post office. When you mail a letter, you do not walk to the recipient's house and hand it to them. You drop it in the mailbox, and the postal service handles delivery. You do not know the recipient's schedule, their exact address layout, or whether they are home. The postal service is the middleman that decouples sender from receiver.

The Pub/Sub Pattern works exactly like this. Components publish messages (drop letters in the mailbox) and other components subscribe to messages (get mail delivered to them). The key difference from the Observer Pattern is the middleman -- the event bus -- which sits between publishers and subscribers.

In the Observer Pattern from Chapter 4, observers subscribe directly to the subject. They know about each other. In Pub/Sub, publishers and subscribers have zero knowledge of each other. They only know about the event bus.

This distinction matters a great deal in large applications. When you have dozens of components that need to communicate, the Pub/Sub Pattern keeps them loosely coupled. A shopping cart component does not need to know that an analytics tracker is listening. A

notification panel does not need to know where notifications come from. Each component does its own job and communicates through the event bus.

Observer vs Pub/Sub: The Key Difference

Before diving into implementation, let us clearly distinguish these two patterns.

Observer Pattern (Chapter 4)

```
Observer Pattern: DIRECT relationship

┌──────────┐   subscribe()   ┌──────────┐
│ Observer │──────────────►│ Subject  │
│ (knows   │                 │ (knows its │
│  subject) │◄──────────────│  observers)│
└──────────┘   notify()      └──────────┘

- Observer subscribes directly to the subject
- Subject maintains its own observer list
- They know about each other
```

Pub/Sub Pattern

```
Pub/Sub Pattern: INDIRECT relationship via middleman

┌──────────┐                                  ┌──────────┐
│ Publisher │   publish('event', data)         │ Subscriber │
│ (does NOT │──────────┐                      │ (does NOT  │
│  know who │          │                      │  know who  │
│  listens) │          ▼                      │  publishes)│
└──────────┘                                  └──────────┘
                 ┌──────────┐
                 │          │  deliver        ▲
                 │ Event Bus │──────────────┘
                 │ (middleman) │
                 │          │                 ┌──────────┐
                 │          │  deliver        │ Subscriber │
                 │          │──────────►│  2       │
                 └──────────┘                 └──────────┘

- Publishers and subscribers never interact directly
- The event bus handles all routing
- Complete decoupling
```

Side-by-Side Comparison

```
Feature              Observer Pattern         Pub/Sub Pattern
_________            _________________        _________________

Coupling             Subject knows its        Publishers and subscribers
                     observers                are completely decoupled

Middleman            None                     Event bus / message broker

Subscribe to         The subject directly     An event name/topic on
                                              the event bus

Communication        Synchronous              Can be sync or async

Best for             One source, many         Many sources, many
                     listeners                listeners, no dependencies

Debugging            Easier (direct link)     Harder (indirect link)
```

Building an EventBus

The EventBus is the heart of the Pub/Sub Pattern. It manages subscriptions and routes messages.

```javascript
// event-bus.js

class EventBus {
  constructor() {
    this._events = new Map();
  }

  /**
   * Subscribe to an event topic
   * Returns an unsubscribe function
   */
  subscribe(event, callback) {
    if (!this._events.has(event)) {
      this._events.set(event, new Set());
    }

    this._events.get(event).add(callback);

    // Return unsubscribe function
    return () => {
```

```
      this._events.get(event)?.delete(callback);
      if (this._events.get(event)?.size === 0) {
        this._events.delete(event);
      }
    };
  }

  /**
   * Subscribe to an event, fire callback only once
   */
  subscribeOnce(event, callback) {
    const unsubscribe = this.subscribe(event, (data) => {
      callback(data);
      unsubscribe();
    });
    return unsubscribe;
  }

  /**
   * Publish an event with data to all subscribers
   */
  publish(event, data) {
    const subscribers = this._events.get(event);
    if (!subscribers || subscribers.size === 0) {
      return;
    }

    // Iterate over a copy to avoid issues if subscribers
    // unsubscribe during notification
    const subscribersCopy = new Set(subscribers);
    subscribersCopy.forEach(callback => {
      try {
        callback(data);
      } catch (error) {
        console.error(`Error in subscriber for "${event}":`, error);
      }
    });
  }

  /**
   * Remove all subscribers for a specific event
   */
  clear(event) {
    if (event) {
      this._events.delete(event);
    } else {
      this._events.clear();
    }
  }

  /**
   * Check if an event has any subscribers
   */
  hasSubscribers(event) {
    return (this._events.get(event)?.size || 0) > 0;
```

```
  }

  /**
   * Get subscriber count for an event
   */
  subscriberCount(event) {
    return this._events.get(event)?.size || 0;
  }

  /**
   * List all active event names
   */
  getEventNames() {
    return [...this._events.keys()];
  }
}

// Export a single EventBus instance (singleton)
const eventBus = new EventBus();
export { EventBus }; // Export class for creating separate buses
export default eventBus;
```

Using the EventBus: Components That Do Not Know Each Other

Let us build a scenario where three features communicate through the EventBus without importing each other.

The Shopping Cart

```javascript
// features/cart/cart.js
import eventBus from '../../event-bus.js';

const cart = {
  items: [],

  addItem(product) {
    const existing = this.items.find(item => item.id === product.id);
    if (existing) {
      existing.quantity++;
    } else {
      this.items.push({ ...product, quantity: 1 });
    }
```

```
    console.log(`Cart: Added "${product.name}"`);

    // Publish event -- cart does NOT know who listens
    eventBus.publish('cart:itemAdded', {
      product,
      cartTotal: this.getTotal(),
      itemCount: this.getItemCount()
    });
  },

  removeItem(productId) {
    const index = this.items.findIndex(item => item.id === productId);
    if (index > -1) {
      const removed = this.items.splice(index, 1)[0];
      console.log(`Cart: Removed "${removed.name}"`);

      eventBus.publish('cart:itemRemoved', {
        product: removed,
        cartTotal: this.getTotal(),
        itemCount: this.getItemCount()
      });
    }
  },

  getTotal() {
    return this.items.reduce((sum, item) => sum + item.price * item.quantity, 0);
  },

  getItemCount() {
    return this.items.reduce((sum, item) => sum + item.quantity, 0);
  }
};

export default cart;
```

The Analytics Tracker

```javascript
// features/analytics/analytics.js
import eventBus from '../../event-bus.js';

// Analytics subscribes to events it cares about
// It has NO idea where these events come from

eventBus.subscribe('cart:itemAdded', (data) => {
  console.log(`Analytics: track "add_to_cart" - ${data.product.name} ($${data.product.p...
  // sendToAnalyticsService('add_to_cart', data);
});

eventBus.subscribe('cart:itemRemoved', (data) => {
```

```
  console.log(`Analytics: track "remove_from_cart" - ${data.product.name}`);
  // sendToAnalyticsService('remove_from_cart', data);
});

eventBus.subscribe('user:login', (data) => {
  console.log(`Analytics: track "login" - ${data.email}`);
});

eventBus.subscribe('user:logout', () => {
  console.log('Analytics: track "logout"');
});

console.log('Analytics tracker initialized');
```

The Notification Panel

```javascript
// features/notifications/notifications.js
import eventBus from '../../event-bus.js';

const notifications = {
  list: [],

  init() {
    // Subscribe to events that should show notifications
    eventBus.subscribe('cart:itemAdded', (data) => {
      this.show(`"${data.product.name}" added to cart. Total: $${data.cartTotal.toFixed...
    });

    eventBus.subscribe('user:login', (data) => {
      this.show(`Welcome back, ${data.name}!`);
    });

    eventBus.subscribe('order:placed', (data) => {
      this.show(`Order #${data.orderId} confirmed! Estimated delivery: ${data.deliveryD...
    });

    console.log('Notification panel initialized');
  },

  show(message) {
    this.list.push({ message, timestamp: Date.now() });
    console.log(`Notification: ${message}`);
  }
};

export default notifications;
```

Putting It All Together

```javascript
// app.js
import cart from './features/cart/cart.js';
import './features/analytics/analytics.js';    // Just importing initializes it
import notifications from './features/notifications/notifications.js';
import eventBus from './event-bus.js';

notifications.init();

console.log('\n--- Adding items to cart ---');
cart.addItem({ id: 'P001', name: 'Wireless Headphones', price: 79.99 });
cart.addItem({ id: 'P002', name: 'Phone Case', price: 19.99 });

console.log('\n--- User logs in ---');
eventBus.publish('user:login', { name: 'Alice', email: 'alice@example.com' });

console.log('\n--- Removing item from cart ---');
cart.removeItem('P001');
```

Output:

```
Analytics tracker initialized
Notification panel initialized

--- Adding items to cart ---
Cart: Added "Wireless Headphones"
Analytics: track "add_to_cart" - Wireless Headphones ($79.99)
Notification: "Wireless Headphones" added to cart. Total: $79.99

Cart: Added "Phone Case"
Analytics: track "add_to_cart" - Phone Case ($19.99)
Notification: "Phone Case" added to cart. Total: $99.98

--- User logs in ---
Analytics: track "login" - alice@example.com
Notification: Welcome back, Alice!

--- Removing item from cart ---
Cart: Removed "Wireless Headphones"
Analytics: track "remove_from_cart" - Wireless Headphones
```

The Dependency Map

```
  WITHOUT Pub/Sub:                    WITH Pub/Sub:

  ┌─────────┐   ┌─────────────┐       ┌─────────┐   ┌─────────────┐
  │  Cart   │──►│ Analytics │       │  Cart   │   │ Analytics │
  │         │──►│             │       │         │   │             │
  └─────────┘   └─────────────┘       └────┬────┘   └──────┬──────┘
       │                                   │               │
```

```
        │
        └──────►┌──────────┐
                │ Notific. │
                │          │
                └──────────┘

  Cart imports Analytics
  Cart imports Notifications
  Cart imports anything that
  cares about cart changes

  Tight coupling.
  Cart knows too much.
```

```
        │  publish    │ subscribe
        ▼             ▼
┌──────────────────────────┐
│       Event Bus          │
└──────────────────────────┘

          ▲
     subscribe │
               │
       ┌───────┴──┐
       │ Notific. │
       └──────────┘

Loose coupling.
Nobody knows each other.
```

Advanced EventBus: Wildcard Subscriptions

Sometimes you want to listen to all events in a category.

```javascript
// event-bus-advanced.js

class AdvancedEventBus {
  constructor() {
    this._events = new Map();
    this._wildcardSubscribers = new Map(); // pattern -> Set of callbacks
  }

  subscribe(event, callback) {
    // Check if this is a wildcard pattern
    if (event.includes('*')) {
      if (!this._wildcardSubscribers.has(event)) {
        this._wildcardSubscribers.set(event, new Set());
      }
      this._wildcardSubscribers.get(event).add(callback);
    } else {
      if (!this._events.has(event)) {
        this._events.set(event, new Set());
      }
      this._events.get(event).add(callback);
    }

    return () => {
      if (event.includes('*')) {
        this._wildcardSubscribers.get(event)?.delete(callback);
      } else {
        this._events.get(event)?.delete(callback);
```

```
      }
    };
  }

  publish(event, data) {
    // Notify exact matches
    const exact = this._events.get(event);
    if (exact) {
      exact.forEach(cb => {
        try { cb(data, event); } catch (e) { console.error(e); }
      });
    }

    // Notify wildcard matches
    this._wildcardSubscribers.forEach((callbacks, pattern) => {
      if (this._matchWildcard(event, pattern)) {
        callbacks.forEach(cb => {
          try { cb(data, event); } catch (e) { console.error(e); }
        });
      }
    });
  }

  _matchWildcard(event, pattern) {
    const regex = new RegExp('^' + pattern.replace(/\*/g, '.*') + '$');
    return regex.test(event);
  }
}

// Usage
const bus = new AdvancedEventBus();

// Subscribe to ALL cart events
bus.subscribe('cart:*', (data, eventName) => {
  console.log(`[Cart Logger] Event: ${eventName}`, data);
});

// Subscribe to ALL events (global logger)
bus.subscribe('*', (data, eventName) => {
  console.log(`[Global] ${eventName}`);
});

bus.publish('cart:itemAdded', { name: 'Mouse' });
bus.publish('cart:itemRemoved', { name: 'Keyboard' });
bus.publish('user:login', { email: 'alice@example.com' });
```

Output:

```
[Cart Logger] Event: cart:itemAdded { name: 'Mouse' }
[Global] cart:itemAdded
[Cart Logger] Event: cart:itemRemoved { name: 'Keyboard' }
[Global] cart:itemRemoved
[Global] user:login
```

Real-World Use Case: Analytics Tracking System

Analytics is one of the most natural use cases for Pub/Sub. Many different user actions need to be tracked, and the tracking logic should not be sprinkled across every component.

```javascript
// analytics/tracker.js
import eventBus from '../event-bus.js';

class AnalyticsTracker {
  constructor() {
    this.queue = [];
    this.batchSize = 5;
    this.batchInterval = 10000; // 10 seconds
  }

  init() {
    // Page views
    eventBus.subscribe('page:view', (data) => {
      this._track('page_view', {
        path: data.path,
        title: data.title,
        referrer: data.referrer
      });
    });

    // User actions
    eventBus.subscribe('user:signup', (data) => {
      this._track('sign_up', { method: data.method });
    });

    eventBus.subscribe('user:login', (data) => {
      this._track('login', { method: data.method });
    });

    // E-commerce
    eventBus.subscribe('cart:itemAdded', (data) => {
      this._track('add_to_cart', {
        item_id: data.product.id,
        item_name: data.product.name,
        value: data.product.price
      });
    });

    eventBus.subscribe('checkout:started', (data) => {
      this._track('begin_checkout', {
        value: data.total,
        item_count: data.itemCount
```

```
      });
    });

    eventBus.subscribe('order:placed', (data) => {
      this._track('purchase', {
        transaction_id: data.orderId,
        value: data.total
      });
    });

    // Start batch sending
    this._startBatchTimer();
    console.log('Analytics tracker initialized');
  }

  _track(eventName, params) {
    const event = {
      name: eventName,
      params,
      timestamp: Date.now(),
      sessionId: this._getSessionId()
    };

    this.queue.push(event);
    console.log(`Analytics: Queued "${eventName}"`, params);

    // Flush immediately if batch is full
    if (this.queue.length >= this.batchSize) {
      this._flush();
    }
  }

  _flush() {
    if (this.queue.length === 0) return;

    const batch = [...this.queue];
    this.queue = [];
    console.log(`Analytics: Sending batch of ${batch.length} events`);
    // In real code: fetch('/api/analytics', { method: 'POST', body: JSON.stringify(bat...
  }

  _startBatchTimer() {
    setInterval(() => this._flush(), this.batchInterval);
  }

  _getSessionId() {
    return 'session-' + Math.random().toString(36).slice(2, 10);
  }
}

const tracker = new AnalyticsTracker();
export default tracker;
```

```javascript
// app.js
import tracker from './analytics/tracker.js';
import eventBus from './event-bus.js';

tracker.init();

// These events can come from ANYWHERE in the app
// The analytics tracker picks them up automatically
eventBus.publish('page:view', { path: '/products', title: 'Products' });
eventBus.publish('cart:itemAdded', { product: { id: 'P1', name: 'Laptop', price: 999 } });
eventBus.publish('cart:itemAdded', { product: { id: 'P2', name: 'Mouse', price: 29 } });
eventBus.publish('checkout:started', { total: 1028, itemCount: 2 });
eventBus.publish('order:placed', { orderId: 'ORD-1234', total: 1028 });
```

Output:

```
Analytics tracker initialized
Analytics: Queued "page_view" { path: '/products', title: 'Products' }
Analytics: Queued "add_to_cart" { item_id: 'P1', item_name: 'Laptop', value: 999 }
Analytics: Queued "add_to_cart" { item_id: 'P2', item_name: 'Mouse', value: 29 }
Analytics: Queued "begin_checkout" { value: 1028, item_count: 2 }
Analytics: Queued "purchase" { transaction_id: 'ORD-1234', value: 1028 }
Analytics: Sending batch of 5 events
```

Cross-Component Communication in React

Pub/Sub can bridge React components that are far apart in the component tree without prop drilling.

```javascript
// hooks/useEventBus.js
import { useEffect, useCallback } from 'react';
import eventBus from '../event-bus';

/**
 * Subscribe to an event bus event within a React component
 * Automatically cleans up on unmount
 */
export function useSubscribe(event, callback) {
  useEffect(() => {
    const unsubscribe = eventBus.subscribe(event, callback);
    return unsubscribe;
  }, [event, callback]);
}
```

```
/**
 * Get a publish function for use in React components
 */
export function usePublish() {
  return useCallback((event, data) => {
    eventBus.publish(event, data);
  }, []);
}
```

```javascript
// components/ProductCard.jsx
import { usePublish } from '../hooks/useEventBus';

export default function ProductCard({ product }) {
  const publish = usePublish();

  function handleAddToCart() {
    // Just publish the event -- ProductCard does not know who listens
    publish('cart:itemAdded', { product });
  }

  return (
    <div className="product-card">
      <h3>{product.name}</h3>
      <p>${product.price}</p>
      <button onClick={handleAddToCart}>Add to Cart</button>
    </div>
  );
}
```

```javascript
// components/CartBadge.jsx
import { useState, useCallback } from 'react';
import { useSubscribe } from '../hooks/useEventBus';

export default function CartBadge() {
  const [count, setCount] = useState(0);

  // Subscribe to cart events
  useSubscribe('cart:itemAdded', useCallback(() => {
    setCount(c => c + 1);
  }, []));

  useSubscribe('cart:itemRemoved', useCallback(() => {
    setCount(c => Math.max(0, c - 1));
  }, []));

  return (
    <div className="cart-badge">
      Cart ({count})
    </div>
  );
}
```

```javascript
// components/ToastNotification.jsx
import { useState, useCallback } from 'react';
import { useSubscribe } from '../hooks/useEventBus';

export default function ToastNotification() {
  const [toasts, setToasts] = useState([]);

  useSubscribe('cart:itemAdded', useCallback((data) => {
    const toast = {
      id: Date.now(),
      message: `"${data.product.name}" added to cart`
    };
    setToasts(prev => [...prev, toast]);

    // Auto-remove after 3 seconds
    setTimeout(() => {
      setToasts(prev => prev.filter(t => t.id !== toast.id));
    }, 3000);
  }, []));

  return (
    <div className="toast-container">
      {toasts.map(toast => (
        <div key={toast.id} className="toast">{toast.message}</div>
      ))}
    </div>
  );
}
```

```
  Component Tree:

  <App>
  ├── <Header>
  │   └── <CartBadge />      ◄── subscribes to 'cart:itemAdded'
  ├── <ProductList>
  │   ├── <ProductCard />     ◄── publishes 'cart:itemAdded'
  │   ├── <ProductCard />     ◄── publishes 'cart:itemAdded'
  │   └── <ProductCard />     ◄── publishes 'cart:itemAdded'
  └── <ToastNotification />   ◄── subscribes to 'cart:itemAdded'

  ProductCard, CartBadge, and ToastNotification
  have ZERO knowledge of each other.
  They communicate entirely through the event bus.
```

Pub/Sub with Typed Events (Preventing Typos)

A common bug with Pub/Sub is typos in event names. If you publish `'cart:itmAdded'` but subscribe to `'cart:itemAdded'`, nothing happens and there is no error.

```javascript
// events.js -- Define event names as constants

export const EVENTS = Object.freeze({
  CART: Object.freeze({
    ITEM_ADDED: 'cart:itemAdded',
    ITEM_REMOVED: 'cart:itemRemoved',
    CLEARED: 'cart:cleared'
  }),
  USER: Object.freeze({
    LOGIN: 'user:login',
    LOGOUT: 'user:logout',
    PROFILE_UPDATED: 'user:profileUpdated'
  }),
  ORDER: Object.freeze({
    PLACED: 'order:placed',
    SHIPPED: 'order:shipped',
    DELIVERED: 'order:delivered'
  }),
  PAGE: Object.freeze({
    VIEW: 'page:view',
    SCROLL: 'page:scroll'
  })
});
```

```javascript
// Now use constants instead of strings
import { EVENTS } from './events.js';
import eventBus from './event-bus.js';

// Publishing
eventBus.publish(EVENTS.CART.ITEM_ADDED, { product });

// Subscribing
eventBus.subscribe(EVENTS.CART.ITEM_ADDED, (data) => {
  console.log('Item added:', data.product.name);
});

// If you typo the constant name, your IDE catches it immediately
// eventBus.publish(EVENTS.CART.ITM_ADDED, data);
// TypeError: Cannot read property 'ITM_ADDED' of undefined
```

When to Use / When NOT to Use

When to Use the Pub/Sub Pattern

- Analytics and telemetry: Track user actions without polluting component code
- Cross-feature communication: When features in different parts of the app need to react to each other's events
- Plugin/extension systems: Let third-party code hook into your application via events
- Micro-frontends: Communication between independently deployed frontend apps
- Toast notifications: Any component can trigger a notification without knowing the notification system
- Logging and debugging: Subscribe to events to log them without modifying source components

When NOT to Use the Pub/Sub Pattern

- Parent-child React communication: Use props and callbacks. Pub/Sub is overkill for direct relationships
- When you need guaranteed delivery: Pub/Sub is fire-and-forget. If no one subscribes, the message is lost
- When debugging matters most: Pub/Sub makes it hard to trace where an event originated. If your team frequently debugs event flow, consider more explicit patterns
- State management: Do not replace Redux, Zustand, or React Context with Pub/Sub for state management. State management libraries provide predictable state updates, dev tools, and time-travel debugging. Pub/Sub provides none of that
- Two components talking directly: If Component A always and only talks to Component B, just import and call directly. Pub/Sub adds indirection for no benefit

---

Common Mistakes

Mistake 1: Event Name Typos (The Silent Failure)

```javascript
// Publisher
eventBus.publish('cart:itemAdded', data);

// Subscriber -- typo in event name!
eventBus.subscribe('cart:itemAdd', (data) => {
  // This NEVER fires. No error. No warning. Silent failure.
  console.log(data);
});
```

Fix: Use constant event names as shown above. Your IDE will catch the typo.

Mistake 2: Forgetting to Unsubscribe in React

```javascript
// BAD -- subscribes on every render, never unsubscribes
function BadComponent() {
  const [count, setCount] = useState(0);

  // This runs on every render!
  eventBus.subscribe('cart:itemAdded', () => {
    setCount(c => c + 1);
  });

  return <span>{count}</span>;
}
```

```javascript
// GOOD -- subscribe in useEffect, cleanup on unmount
function GoodComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    const unsubscribe = eventBus.subscribe('cart:itemAdded', () => {
      setCount(c => c + 1);
    });
    return unsubscribe; // Cleanup!
  }, []);

  return <span>{count}</span>;
}
```

Mistake 3: Event Storms (Cascading Events)

```javascript
// Event A triggers Event B, which triggers Event C,
// which triggers Event A again = infinite loop

eventBus.subscribe('data:updated', () => {
  eventBus.publish('ui:refresh');  // Triggers next subscriber
});

eventBus.subscribe('ui:refresh', () => {
  eventBus.publish('data:validate'); // Triggers next subscriber
});

eventBus.subscribe('data:validate', () => {
  eventBus.publish('data:updated'); // Goes back to the start!
  // Infinite loop!
});
```

Fix: Never publish an event in response to an event that could circle back. If you need chains, use a clear, linear flow with distinct event names.

Mistake 4: Passing Mutable Data

```javascript
// BAD -- subscribers can modify shared data
const product = { name: 'Mouse', price: 29.99 };
eventBus.publish('cart:itemAdded', { product });

// Subscriber A modifies the data
eventBus.subscribe('cart:itemAdded', (data) => {
  data.product.price = 0; // FREE STUFF!
  // This affects ALL other subscribers that receive the same object
});
```

Fix: Publish copies of data, or freeze the published data.

```javascript
// GOOD -- publish a frozen copy
eventBus.publish('cart:itemAdded', Object.freeze({
  product: { ...product }
}));
```

Best Practices

1. Use a naming convention for events. Adopt a consistent pattern like `domain:action` (e.g., `cart:itemAdded`, `user:login`, `page:scroll`). This makes events discoverable and groupable.
2. Define event names as constants in a central file. This prevents typos and makes it easy to see all events in the system.
3. Always unsubscribe when a component unmounts or a feature is disabled. Use the returned unsubscribe function.
4. Do not use Pub/Sub for state management. It is great for notifications and side effects, but terrible for managing application state that the UI depends on.
5. Keep event payloads small and serializable. Avoid publishing DOM elements, class instances, or functions in event data.
6. Document your events. Maintain a list of all event names, their payload shapes, and who publishes/subscribes to each. Without documentation, Pub/Sub systems become impossible to maintain.
7. Avoid event chains that can loop. If event A triggers event B, make sure B cannot trigger A (directly or indirectly).

8. Consider using multiple event buses for different domains (one for UI events, one for data events) to reduce noise and improve separation.

Quick Summary

The Pub/Sub Pattern decouples communication between components by routing messages through an intermediary event bus.

```
Pub/Sub vs Observer:
────────────────────

Observer:  Subject ◄─── Observer (direct link)
Pub/Sub:   Publisher ──► EventBus ──► Subscriber (no direct link)

When to use which:
──────────────────
Observer:  When the "what changed" matters and there is one clear source
Pub/Sub:   When "something happened" and many unrelated parts need to react
```

Key Points

- The Pub/Sub Pattern uses a middleman (event bus) so publishers and subscribers never interact directly
- The key difference from Observer is the complete decoupling -- publishers do not know about subscribers and vice versa
- An EventBus manages subscriptions and routes messages between publishers and subscribers
- Always unsubscribe when a component or feature is destroyed to prevent memory leaks
- Use constant event names to avoid typos and silent failures
- Pub/Sub is ideal for analytics, notifications, logging, and cross-feature communication
- Do not use Pub/Sub for state management -- use proper state management tools instead
- Watch out for event storms (cascading events that create infinite loops)
- Document your events -- without documentation, Pub/Sub systems are very hard to debug

---

Practice Questions

1. What is the key structural difference between the Observer Pattern and the Pub/Sub Pattern? Draw the relationship between the objects in each.
2. Why is Pub/Sub considered "fire-and-forget"? What happens if an event is published but nobody has subscribed to it?
3. How do wildcard subscriptions work? Give an example where subscribing to `'cart:*'` would be useful.
4. Why is Pub/Sub a poor choice for state management in a React application? What problems arise from using it that way?
5. Explain the "event storm" problem. How can you design your event system to prevent cascading event loops?

Exercises

Exercise 1: Build a Chat Application Event System

Create a Pub/Sub-based chat system with these events:

- `chat:messageSent` -- when a user sends a message
- `chat:messageReceived` -- when a message arrives from another user
- `chat:userJoined` -- when a user enters the chat room
- `chat:userLeft` -- when a user leaves the chat room

Build three subscribers:

1. A message list that displays messages
2. A user count display that tracks online users
3. A notification sound player that logs "DING!" for every new message

Test by simulating several users joining, chatting, and leaving.

Exercise 2: EventBus with Middleware

Extend the EventBus to support middleware -- functions that run before every event is delivered to subscribers. Use cases:

- A logging middleware that logs every event
- A filter middleware that blocks certain events in production

- A transform middleware that adds a timestamp to every event payload

Exercise 3: Scoped Event Buses

Create a factory function that produces separate EventBus instances for different domains:

```javascript
const uiBus = createEventBus('ui');
const dataBus = createEventBus('data');
const analyticsBus = createEventBus('analytics');
```

Each bus should:

- Prefix all event names with its domain (so `uiBus.publish('click')` actually publishes `'ui:click'`)
- Have its own independent subscriber list
- Optionally forward events to a global bus for debugging

What Is Next?

You now understand two powerful communication patterns: Observer (direct) and Pub/Sub (indirect). But communication is just one challenge in frontend development. Another common challenge is working with data collections -- arrays of items from APIs, paginated results, tree structures.

How do you traverse these data structures uniformly, without caring about their internal shape? How do you lazily load pages of data from a server? That is the Iterator Pattern, covered in Chapter 7.

Chapter 6: Strategy Pattern

What You Will Learn

- What the Strategy Pattern is and why it exists
- How to replace messy if/else chains with interchangeable behaviors
- How to implement strategies as plain functions in JavaScript
- How to use the Strategy Pattern with React components
- Real-world applications: form validation, pricing, sorting, and API requests

Why This Chapter Matters

Picture a GPS navigation app. You type in a destination and it gives you a route. But what kind of route? The fastest route avoids traffic. The shortest route saves fuel. The scenic route goes past beautiful views. The walking route avoids highways entirely.

Each routing method is a strategy. The app does not care which one you pick. It just says, "Give me a strategy, and I will follow it." You can swap strategies at any time without rewriting the navigation engine.

That is the Strategy Pattern. It takes a chunk of behavior -- an algorithm, a rule, a method of doing something -- and packages it so you can swap it in and out like changing a battery. No giant if/else chains. No rewriting your core logic. Just clean, interchangeable pieces.

In frontend development, you run into this constantly. Payment processing with multiple methods. Form validation with different rules for different fields. Sorting data in different orders. Fetching data from REST or GraphQL. Every one of these is a perfect fit for the Strategy Pattern.

The Problem: if/else Chains That Never Stop Growing

Let us start with a real scenario. You are building an e-commerce checkout page. Customers can pay with a credit card, PayPal, or cryptocurrency.

The Painful Way (Before the Pattern)

```javascript
// payment-processor.js -- the messy version

function processPayment(method, amount, details) {
  if (method === 'credit-card') {
    // Validate card number
    if (!details.cardNumber || details.cardNumber.length !== 16) {
      throw new Error('Invalid card number');
    }
    // Validate expiry
    if (!details.expiry) {
      throw new Error('Expiry date required');
    }
    // Process credit card payment
    console.log(`Charging $${amount} to card ending in ${details.cardNumber.slice(-4)}`);
    // ... credit card API call logic
    return { success: true, transactionId: 'CC-' + Date.now() };

  } else if (method === 'paypal') {
    // Validate PayPal email
    if (!details.email || !details.email.includes('@')) {
      throw new Error('Invalid PayPal email');
    }
    // Process PayPal payment
    console.log(`Charging $${amount} via PayPal to ${details.email}`);
    // ... PayPal API call logic
    return { success: true, transactionId: 'PP-' + Date.now() };

  } else if (method === 'crypto') {
    // Validate wallet address
    if (!details.walletAddress || details.walletAddress.length < 26) {
      throw new Error('Invalid wallet address');
    }
    // Process crypto payment
    console.log(`Sending $${amount} to wallet ${details.walletAddress.slice(0, 8)}...`);
    // ... crypto API call logic
    return { success: true, transactionId: 'CR-' + Date.now() };

  } else {
    throw new Error(`Unknown payment method: ${method}`);
  }
}
```

Output when calling `processPayment('credit-card', 49.99, { cardNumber: '4111111111111234', expiry: '12/25' })`:

```
Charging $49.99 to card ending in 1234
// Returns: { success: true, transactionId: 'CC-1698765432100' }
```

What Is Wrong Here?

Look at that function. It is already 40 lines long and it only handles three payment methods. What happens when the business says:

- "Add Apple Pay"
- "Add bank transfer"
- "Add gift cards"
- "Add buy-now-pay-later"

Every new method means diving into this function, adding another `else if` block, and hoping you do not break the existing ones. This violates the Open/Closed Principle: software should be open for extension but closed for modification.

```
┌──────────────────────────────────────────────┐
│          processPayment() Function           │
│                                              │
│  if (credit-card) {                          │
│    ... 15 lines of credit card logic ...     │
│  } else if (paypal) {                        │
│    ... 12 lines of PayPal logic ...          │
│  } else if (crypto) {                        │
│    ... 13 lines of crypto logic ...          │
│  } else if (apple-pay) {     <── NEW         │
│    ... more lines ...                        │
│  } else if (bank-transfer) { <── NEW         │
│    ... more lines ...                        │
│  } else if (gift-card) {     <── NEW         │
│    ... more lines ...                        │
│  }                                           │
│                                              │
│  This function grows FOREVER                 │
└──────────────────────────────────────────────┘
```

The Solution: Strategy Pattern

The Strategy Pattern says: extract each algorithm into its own function (or class), and let the caller pick which one to use.

Think of it like a toolbox. Instead of one giant multi-tool that tries to be a hammer, screwdriver, and wrench all at once, you have separate tools. You pick the right one for the job.

```
┌──────────────────────────────────────────────────────┐
│                PaymentProcessor                      │
│                                                      │
│  "I don't know HOW to process payments.              │
│   I just call whatever strategy you give me."        │
│                                                      │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐           │
│  │ Credit   │  │ PayPal   │  │ Crypto   │           │
│  │ Card     │  │ Strategy │  │ Strategy │           │
│  │ Strategy │  │          │  │          │           │
│  └──────────┘  └──────────┘  └──────────┘           │
│                                                      │
│  Want to add Apple Pay?                              │
│  Just create a new strategy. Done.                   │
└──────────────────────────────────────────────────────┘
```

The Strategy Interface

In TypeScript, you would define an interface. In plain JavaScript, you follow a convention: every strategy function has the same signature.

```javascript
// The "interface" -- every strategy must follow this shape:
//
// function strategyName(amount, details) {
//   // 1. Validate the details
//   // 2. Process the payment
//   // 3. Return { success, transactionId }
// }
```

The Clean Way (After the Pattern)

```javascript
// strategies/credit-card.js
function creditCardStrategy(amount, details) {
  // Line 1: Validate the card number exists and is 16 digits
  if (!details.cardNumber || details.cardNumber.length !== 16) {
    throw new Error('Invalid card number');
  }
  // Line 2: Validate the expiry date exists
```

```
  if (!details.expiry) {
    throw new Error('Expiry date required');
  }
  // Line 3: Process the payment
  console.log(`Charging $${amount} to card ending in ${details.cardNumber.slice(-4)}`);
  // Line 4: Return a result with a unique transaction ID
  return { success: true, transactionId: 'CC-' + Date.now() };
}

// strategies/paypal.js
function paypalStrategy(amount, details) {
  if (!details.email || !details.email.includes('@')) {
    throw new Error('Invalid PayPal email');
  }
  console.log(`Charging $${amount} via PayPal to ${details.email}`);
  return { success: true, transactionId: 'PP-' + Date.now() };
}

// strategies/crypto.js
function cryptoStrategy(amount, details) {
  if (!details.walletAddress || details.walletAddress.length < 26) {
    throw new Error('Invalid wallet address');
  }
  console.log(`Sending $${amount} to wallet ${details.walletAddress.slice(0, 8)}...`);
  return { success: true, transactionId: 'CR-' + Date.now() };
}
```

Now the processor is tiny:

```javascript
// payment-processor.js -- the clean version

// Line 1: A map that connects method names to their strategy functions
const paymentStrategies = {
  'credit-card': creditCardStrategy,
  'paypal': paypalStrategy,
  'crypto': cryptoStrategy,
};

// Line 2: The processor just looks up and calls the right strategy
function processPayment(method, amount, details) {
  // Line 3: Find the strategy for this payment method
  const strategy = paymentStrategies[method];

  // Line 4: If no strategy exists, throw a clear error
  if (!strategy) {
    throw new Error(`Unknown payment method: ${method}`);
  }

  // Line 5: Delegate to the strategy -- the processor does not care HOW it works
  return strategy(amount, details);
}
```

Output is identical:

```
processPayment('credit-card', 49.99, { cardNumber: '4111111111111234', expiry: '12/25' });
// Charging $49.99 to card ending in 1234
// Returns: { success: true, transactionId: 'CC-1698765432100' }

processPayment('paypal', 29.99, { email: 'user@example.com' });
// Charging $29.99 via PayPal to user@example.com
// Returns: { success: true, transactionId: 'PP-1698765432200' }
```

Adding a New Payment Method

Now the magic. Your boss says "Add Apple Pay." Watch how easy this is:

```javascript
// strategies/apple-pay.js -- a brand new file, no existing code touched

function applePayStrategy(amount, details) {
  if (!details.deviceToken) {
    throw new Error('Apple Pay device token required');
  }
  console.log(`Processing $${amount} via Apple Pay`);
  return { success: true, transactionId: 'AP-' + Date.now() };
}

// Just register it:
paymentStrategies['apple-pay'] = applePayStrategy;
```

That is it. No modifying the processor. No touching credit card logic. No risk of breaking PayPal. You extended the system without modifying it.

Before vs After: Side by Side

```
  BEFORE (if/else)                    AFTER (Strategy Pattern)
  ────────────────                    ────────────────────────

  processPayment()                    processPayment()
  ┌──────────────────┐                ┌──────────────────┐
  │ if credit-card   │                │ strategy = lookup │
  │   ... 15 lines   │                │ return strategy() │
  │ else if paypal   │                └──────────────────┘
  │   ... 12 lines   │                        │
  │ else if crypto   │                ┌───────┼────────┐
```

```
  │   ... 13 lines   │                 │       │       │
  │ else if new...   │                 ▼       ▼       ▼
  │   ... more lines │               CC()   PayPal() Crypto()
  └──────────────────┘               (own    (own     (own
                                     file)   file)    file)
  Adding new method:
  - Edit the big function             Adding new method:
  - Risk breaking others              - Create new file
  - Hard to test                      - Register it
                                      - Nothing else changes
```

Function-Based Strategies in JavaScript

JavaScript makes the Strategy Pattern incredibly natural because functions are first-class citizens. You can store them in variables, pass them as arguments, and put them in objects.

Pattern 1: Strategy Map (Object Lookup)

This is the most common approach.

```javascript
// Define strategies as an object map
const sortStrategies = {
  // Each key is a strategy name, each value is a comparison function
  'price-low-high': (a, b) => a.price - b.price,
  'price-high-low': (a, b) => b.price - a.price,
  'name-a-z': (a, b) => a.name.localeCompare(b.name),
  'name-z-a': (a, b) => b.name.localeCompare(a.name),
  'newest': (a, b) => new Date(b.createdAt) - new Date(a.createdAt),
};

// The context function that uses a strategy
function sortProducts(products, strategyName) {
  const strategy = sortStrategies[strategyName];
  if (!strategy) {
    throw new Error(`Unknown sort strategy: ${strategyName}`);
  }
  // .slice() creates a copy so we don't mutate the original array
  return products.slice().sort(strategy);
}

// Usage
const products = [
  { name: 'Laptop', price: 999, createdAt: '2024-01-15' },
  { name: 'Mouse', price: 29, createdAt: '2024-03-01' },
  { name: 'Keyboard', price: 79, createdAt: '2024-02-10' },
```

```
];

console.log(sortProducts(products, 'price-low-high'));
// Output:
// [
//   { name: 'Mouse', price: 29, createdAt: '2024-03-01' },
//   { name: 'Keyboard', price: 79, createdAt: '2024-02-10' },
//   { name: 'Laptop', price: 999, createdAt: '2024-01-15' }
// ]

console.log(sortProducts(products, 'name-a-z'));
// Output:
// [
//   { name: 'Keyboard', price: 79, createdAt: '2024-02-10' },
//   { name: 'Laptop', price: 999, createdAt: '2024-01-15' },
//   { name: 'Mouse', price: 29, createdAt: '2024-03-01' }
// ]
```

Line-by-line breakdown:

- Line 1-7: Each sorting strategy is a comparison function. JavaScript's `.sort()` method expects a function that takes two items and returns a number (negative, zero, or positive).
- `price-low-high`: Subtracting `a.price - b.price` puts smaller prices first.
- `name-a-z`: `localeCompare` compares strings alphabetically, respecting locale rules.
- `sortProducts` function: It looks up the strategy by name, copies the array with `.slice()`, and sorts the copy.

Pattern 2: Strategy as a Function Argument

Sometimes you do not need a map. Just pass the strategy directly.

```javascript
// Instead of looking up from a map, accept the function directly
function processData(data, transformStrategy) {
  return data.map(transformStrategy);
}

// Define strategies
const toUpperCase = (item) => ({ ...item, name: item.name.toUpperCase() });
const addTax = (item) => ({ ...item, price: item.price * 1.1 });
const formatCurrency = (item) => ({ ...item, display: `$${item.price.toFixed(2)}` });

// Use whichever strategy you need
const items = [
  { name: 'Widget', price: 10 },
  { name: 'Gadget', price: 25 },
];
```

```
console.log(processData(items, toUpperCase));
// Output:
// [
//   { name: 'WIDGET', price: 10 },
//   { name: 'GADGET', price: 25 }
// ]

console.log(processData(items, addTax));
// Output:
// [
//   { name: 'Widget', price: 11 },
//   { name: 'Gadget', price: 27.5 }
// ]
```

Pattern 3: Class-Based Strategies

When strategies need internal state or multiple methods, classes work well.

```javascript
// Base "interface" -- not enforced in JS, but documents the contract
// Every strategy must have: validate(details), process(amount, details)

class CreditCardStrategy {
  constructor() {
    this.name = 'Credit Card';
    this.fee = 0.029; // 2.9% processing fee
  }

  validate(details) {
    if (!details.cardNumber || details.cardNumber.length !== 16) {
      return { valid: false, error: 'Card number must be 16 digits' };
    }
    return { valid: true };
  }

  process(amount, details) {
    const totalWithFee = amount + (amount * this.fee);
    console.log(`Charging $${totalWithFee.toFixed(2)} (includes $${(amount * this.fee)....
    return { success: true, charged: totalWithFee };
  }
}

class PayPalStrategy {
  constructor() {
    this.name = 'PayPal';
    this.fee = 0.035; // 3.5% processing fee
  }

  validate(details) {
    if (!details.email || !details.email.includes('@')) {
```

```
      return { valid: false, error: 'Valid email required' };
    }
    return { valid: true };
  }

  process(amount, details) {
    const totalWithFee = amount + (amount * this.fee);
    console.log(`Sending $${totalWithFee.toFixed(2)} via PayPal to ${details.email}`);
    return { success: true, charged: totalWithFee };
  }
}

// The context class
class PaymentProcessor {
  constructor(strategy) {
    this.strategy = strategy; // Store the current strategy
  }

  setStrategy(strategy) {
    this.strategy = strategy; // Swap strategy at runtime
  }

  checkout(amount, details) {
    // Step 1: Validate using the strategy's own rules
    const validation = this.strategy.validate(details);
    if (!validation.valid) {
      throw new Error(validation.error);
    }

    // Step 2: Process using the strategy
    return this.strategy.process(amount, details);
  }
}

// Usage
const processor = new PaymentProcessor(new CreditCardStrategy());
processor.checkout(100, { cardNumber: '4111111111111234', expiry: '12/25' });
// Output: Charging $102.90 (includes $2.90 fee)

// Swap strategy at runtime -- no new processor needed
processor.setStrategy(new PayPalStrategy());
processor.checkout(100, { email: 'user@example.com' });
// Output: Sending $103.50 via PayPal to user@example.com
```

Real-World Use Case 1: Form Validation Strategies

Forms have different validation rules for different fields. An email field needs an `@` sign. A password needs a minimum length. A phone number needs only digits.

```javascript
// validation-strategies.js

// Each strategy takes a value and returns { valid, error? }
const validationStrategies = {
  required: (value) => ({
    valid: value !== null && value !== undefined && value.trim() !== '',
    error: 'This field is required',
  }),

  email: (value) => ({
    valid: /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value),
    error: 'Please enter a valid email address',
  }),

  minLength: (min) => (value) => ({
    // This is a factory that returns a strategy
    valid: value.length >= min,
    error: `Must be at least ${min} characters`,
  }),

  maxLength: (max) => (value) => ({
    valid: value.length <= max,
    error: `Must be no more than ${max} characters`,
  }),

  pattern: (regex, message) => (value) => ({
    valid: regex.test(value),
    error: message,
  }),

  matchField: (fieldName, formValues) => (value) => ({
    valid: value === formValues[fieldName],
    error: `Must match ${fieldName}`,
  }),
};

// A validator that runs multiple strategies on a single field
function validateField(value, strategies) {
  for (const strategy of strategies) {
    const result = strategy(value);
    if (!result.valid) {
      return result; // Return the first error found
    }
  }
  return { valid: true };
}

// Define validation rules for a signup form
const signupRules = {
  username: [
    validationStrategies.required,
    validationStrategies.minLength(3),
    validationStrategies.maxLength(20),
    validationStrategies.pattern(/^[a-zA-Z0-9_]+$/, 'Only letters, numbers, and undersc...
```

```
  ],
  email: [
    validationStrategies.required,
    validationStrategies.email,
  ],
  password: [
    validationStrategies.required,
    validationStrategies.minLength(8),
    validationStrategies.pattern(/[A-Z]/, 'Must contain at least one uppercase letter'),
    validationStrategies.pattern(/[0-9]/, 'Must contain at least one number'),
  ],
};

// Validate the form
function validateForm(formValues, rules) {
  const errors = {};

  for (const [field, strategies] of Object.entries(rules)) {
    const result = validateField(formValues[field] || '', strategies);
    if (!result.valid) {
      errors[field] = result.error;
    }
  }

  return {
    valid: Object.keys(errors).length === 0,
    errors,
  };
}

// Usage
const formData = {
  username: 'jo',
  email: 'not-an-email',
  password: 'short',
};

console.log(validateForm(formData, signupRules));
// Output:
// {
//   valid: false,
//   errors: {
//     username: 'Must be at least 3 characters',
//     email: 'Please enter a valid email address',
//     password: 'Must be at least 8 characters'
//   }
// }

const goodData = {
  username: 'john_doe',
  email: 'john@example.com',
  password: 'SecurePass1',
};

console.log(validateForm(goodData, signupRules));
```

```
// Output:
// { valid: true, errors: {} }
```

Using Validation Strategies in React

```jsx
// useFormValidation.js -- a custom hook
import { useState, useCallback } from 'react';

function useFormValidation(rules) {
  const [errors, setErrors] = useState({});

  const validate = useCallback((fieldName, value) => {
    const fieldRules = rules[fieldName];
    if (!fieldRules) return true;

    for (const strategy of fieldRules) {
      const result = strategy(value);
      if (!result.valid) {
        setErrors(prev => ({ ...prev, [fieldName]: result.error }));
        return false;
      }
    }

    // Clear error if validation passes
    setErrors(prev => {
      const next = { ...prev };
      delete next[fieldName];
      return next;
    });
    return true;
  }, [rules]);

  const validateAll = useCallback((formValues) => {
    const newErrors = {};
    let isValid = true;

    for (const [field, strategies] of Object.entries(rules)) {
      for (const strategy of strategies) {
        const result = strategy(formValues[field] || '');
        if (!result.valid) {
          newErrors[field] = result.error;
          isValid = false;
          break; // Stop at first error per field
        }
      }
    }

    setErrors(newErrors);
    return isValid;
  }, [rules]);
```

```
  return { errors, validate, validateAll };
}

// SignupForm.jsx
function SignupForm() {
  const [values, setValues] = useState({
    username: '',
    email: '',
    password: '',
  });

  const { errors, validate, validateAll } = useFormValidation(signupRules);

  const handleChange = (e) => {
    const { name, value } = e.target;
    setValues(prev => ({ ...prev, [name]: value }));
    validate(name, value); // Validate on change using the strategies
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (validateAll(values)) {
      console.log('Form is valid! Submitting...', values);
    }
  };

  return (
    <form onSubmit={handleSubmit}>
      <div>
        <input
          name="username"
          value={values.username}
          onChange={handleChange}
          placeholder="Username"
        />
        {errors.username && <span className="error">{errors.username}</span>}
      </div>
      <div>
        <input
          name="email"
          value={values.email}
          onChange={handleChange}
          placeholder="Email"
        />
        {errors.email && <span className="error">{errors.email}</span>}
      </div>
      <div>
        <input
          name="password"
          type="password"
          value={values.password}
          onChange={handleChange}
          placeholder="Password"
        />
```

```
        {errors.password && <span className="error">{errors.password}</span>}
      </div>
      <button type="submit">Sign Up</button>
    </form>
  );
}
```

Real-World Use Case 2: Pricing Calculator

Different pricing models need different calculation strategies.

```javascript
// pricing-strategies.js

const pricingStrategies = {
  // Fixed price: everyone pays the same
  fixed: (basePrice, quantity) => {
    return basePrice * quantity;
  },

  // Tiered pricing: price drops as quantity increases
  tiered: (basePrice, quantity) => {
    if (quantity >= 100) return basePrice * 0.7 * quantity;  // 30% off
    if (quantity >= 50) return basePrice * 0.8 * quantity;   // 20% off
    if (quantity >= 10) return basePrice * 0.9 * quantity;   // 10% off
    return basePrice * quantity;                              // Full price
  },

  // Per-seat pricing: common in SaaS
  perSeat: (basePrice, quantity) => {
    const monthlyPerSeat = basePrice;
    return monthlyPerSeat * quantity;
  },

  // Usage-based: pay for what you use (like AWS)
  usageBased: (basePrice, quantity) => {
    const freeAllowance = 100;
    const billableUnits = Math.max(0, quantity - freeAllowance);
    return billableUnits * basePrice;
  },

  // Freemium: free up to a limit, then flat fee
  freemium: (basePrice, quantity) => {
    const freeLimit = 5;
    if (quantity <= freeLimit) return 0;
    return basePrice; // Flat fee for premium
  },
```

```
};

function calculatePrice(strategyName, basePrice, quantity) {
  const strategy = pricingStrategies[strategyName];
  if (!strategy) throw new Error(`Unknown pricing strategy: ${strategyName}`);
  return strategy(basePrice, quantity);
}

// Usage
console.log(calculatePrice('fixed', 10, 5));       // Output: 50
console.log(calculatePrice('tiered', 10, 50));      // Output: 400 (20% off)
console.log(calculatePrice('usageBased', 0.05, 250)); // Output: 7.5 (150 billable units)
console.log(calculatePrice('freemium', 29, 3));     // Output: 0 (within free limit)
console.log(calculatePrice('freemium', 29, 10));    // Output: 29 (premium)
```

Real-World Use Case 3: API Request Strategies

Your app might talk to different backends or use different protocols.

```javascript
// api-strategies.js

const apiStrategies = {
  rest: async (endpoint, data) => {
    const response = await fetch(`/api/rest/${endpoint}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data),
    });
    return response.json();
  },

  graphql: async (endpoint, data) => {
    const query = data.query;
    const variables = data.variables || {};
    const response = await fetch('/api/graphql', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ query, variables }),
    });
    const result = await response.json();
    return result.data;
  },

  mock: async (endpoint, data) => {
    // For development and testing -- returns fake data
    console.log(`[MOCK] ${endpoint}`, data);
```

```
    return { id: 1, status: 'mock-response', ...data };
  },
};

class ApiClient {
  constructor(strategyName = 'rest') {
    this.strategy = apiStrategies[strategyName];
  }

  setStrategy(strategyName) {
    this.strategy = apiStrategies[strategyName];
  }

  async request(endpoint, data) {
    return this.strategy(endpoint, data);
  }
}

// Usage
const client = new ApiClient('rest');

// In production, use REST
await client.request('users', { name: 'Alice' });

// During development, switch to mock
client.setStrategy('mock');
await client.request('users', { name: 'Alice' });
// Output: [MOCK] users { name: 'Alice' }
// Returns: { id: 1, status: 'mock-response', name: 'Alice' }
```

Real-World Use Case 4: Sorting Options in React

A product listing page with a sort dropdown.

```jsx
// SortableProductList.jsx
import { useState, useMemo } from 'react';

// Define sort strategies
const sortStrategies = {
  'relevance': (a, b) => b.relevanceScore - a.relevanceScore,
  'price-asc': (a, b) => a.price - b.price,
  'price-desc': (a, b) => b.price - a.price,
  'rating': (a, b) => b.rating - a.rating,
  'newest': (a, b) => new Date(b.createdAt) - new Date(a.createdAt),
  'name': (a, b) => a.name.localeCompare(b.name),
};
```

```
function SortableProductList({ products }) {
  const [sortBy, setSortBy] = useState('relevance');

  // useMemo so we only re-sort when products or sortBy changes
  const sortedProducts = useMemo(() => {
    const strategy = sortStrategies[sortBy];
    return [...products].sort(strategy);
  }, [products, sortBy]);

  return (
    <div>
      <div className="sort-controls">
        <label htmlFor="sort-select">Sort by: </label>
        <select
          id="sort-select"
          value={sortBy}
          onChange={(e) => setSortBy(e.target.value)}
        >
          <option value="relevance">Relevance</option>
          <option value="price-asc">Price: Low to High</option>
          <option value="price-desc">Price: High to Low</option>
          <option value="rating">Highest Rated</option>
          <option value="newest">Newest First</option>
          <option value="name">Name (A-Z)</option>
        </select>
      </div>

      <ul className="product-list">
        {sortedProducts.map(product => (
          <li key={product.id}>
            {product.name} - ${product.price} ({product.rating} stars)
          </li>
        ))}
      </ul>
    </div>
  );
}
```

How it works:

1. The `sortStrategies` object holds all sorting algorithms.
2. The `sortBy` state tracks which strategy is active.
3. When the user picks a different sort option, React re-renders with the new strategy.
4. `useMemo` prevents unnecessary re-sorting if nothing changed.

When to Use the Strategy Pattern

Situation	Use Strategy Pattern?
Multiple algorithms that do the same type of job	Yes
if/else or switch chains that pick behavior based on a type	Yes
Behavior needs to change at runtime (user picks a sort option)	Yes
You want to test each algorithm independently	Yes
You need to add new behaviors without modifying existing code	Yes
You only have one algorithm and it will never change	No
The logic is simple enough for a ternary or one-line if	No
The variations share almost all their code	Maybe -- consider template method

---

When NOT to Use the Strategy Pattern

1. Only two simple cases: If you just have `if (premium) { ... } else { ... }`, a strategy pattern is overkill.
2. Strategies that are identical except for one value: Use a parameter instead.

```javascript
// DON'T create separate strategies for this:
const add5 = (x) => x + 5;
const add10 = (x) => x + 10;
const add15 = (x) => x + 15;

// DO use a parameter:
const addN = (n) => (x) => x + n;
```

3. Strategies need deep access to internal state: If the strategy needs to reach inside the context object and poke around, you have a coupling problem that strategy alone will not fix.

Common Mistakes

Mistake 1: Strategies with Different Interfaces

```javascript
// WRONG -- these strategies have different signatures
const strategies = {
  email: (value) => { /* validates email */ },
  password: (value, minLength) => { /* needs extra param */ },
  phone: (value, countryCode, format) => { /* even more params */ },
};

// RIGHT -- use a consistent interface
const strategies = {
  email: (value, options) => { /* uses options if needed */ },
  password: (value, options) => { /* options.minLength */ },
  phone: (value, options) => { /* options.countryCode, options.format */ },
};
```

Mistake 2: Forgetting the Fallback

```javascript
// WRONG -- crashes if strategy doesn't exist
function sort(items, strategyName) {
  return items.sort(strategies[strategyName]); // undefined if not found!
}

// RIGHT -- handle missing strategies
function sort(items, strategyName) {
  const strategy = strategies[strategyName];
  if (!strategy) {
    console.warn(`Unknown strategy "${strategyName}", using default`);
    return items.sort(strategies['default']);
  }
  return items.sort(strategy);
}
```

Mistake 3: Too Many Strategies for Simple Differences

```javascript
// WRONG -- three strategies that only differ by one number
const smallDiscount = (price) => price * 0.95;
const mediumDiscount = (price) => price * 0.90;
const largeDiscount = (price) => price * 0.85;

// RIGHT -- one function with a parameter
const applyDiscount = (percentage) => (price) => price * (1 - percentage / 100);
// Usage: applyDiscount(5)(100) => 95
```

Best Practices

1. Keep strategies pure when possible. Strategies that take input and return output (no side effects) are easiest to test and reason about.
2. Give strategies a consistent interface. Every strategy for the same job should accept the same arguments and return the same shape.
3. Use a default strategy. Always have a fallback so your code does not crash when an unknown strategy name appears.
4. Name strategies clearly. `priceLowToHigh` is better than `sort1`. `creditCardStrategy` is better than `strategy1`.
5. Store strategies in a map for runtime selection. This lets users (or configs) pick a strategy by name without if/else chains.
6. Use factories for configurable strategies. If a strategy needs configuration, return a function from a factory function.

```javascript
// Factory that creates a configured strategy
function createRetryStrategy(maxRetries, delayMs) {
  return async function retryStrategy(fn) {
    for (let attempt = 1; attempt <= maxRetries; attempt++) {
      try {
        return await fn();
      } catch (error) {
        if (attempt === maxRetries) throw error;
        await new Promise(r => setTimeout(r, delayMs * attempt));
      }
    }
  };
}

// Use the factory to create different retry behaviors
const aggressiveRetry = createRetryStrategy(5, 100);
const gentleRetry = createRetryStrategy(2, 1000);
```

Quick Summary

The Strategy Pattern replaces conditional logic with interchangeable functions. Instead of a big if/else chain that picks behavior based on a type, you define each behavior as a separate function and let the caller choose which one to use. This makes your code easier to extend (add new strategies without changing existing code), easier to test (test each strategy in isolation), and easier to read (each strategy is a small, focused function).

```
┌──────────────┐        ┌──────────────┐
│   Context    │        │  Strategy    │
│              │──────>│  Interface   │
│ Uses a       │        │              │
│ strategy     │        │ execute()    │
└──────────────┘        └──────────────┘
                              │
                  ┌───────────┼──────────────┐
                  │           │              │
            ┌──────────┐  ┌──────────┐  ┌──────────┐
            │Strategy A│  │Strategy B│  │Strategy C│
            │          │  │          │  │          │
            │execute() │  │execute() │  │execute() │
            └──────────┘  └──────────┘  └──────────┘
```

Key Points

- The Strategy Pattern encapsulates a family of algorithms and makes them interchangeable.
- In JavaScript, strategies are often plain functions stored in an object map.
- The "context" (the code that uses a strategy) does not know the details of how each strategy works.
- You can swap strategies at runtime -- the user picks a sort order, the system picks a pricing model.
- Strategies should share a consistent interface (same inputs, same output shape).
- The pattern follows the Open/Closed Principle: open for extension, closed for modification.
- Function-based strategies are preferred in JavaScript over class-based ones for simple cases.

Practice Questions

1. You have a notification system that sends alerts via email, SMS, and push notification. Each channel has different formatting rules. How would you apply the Strategy Pattern? What would your strategy interface look like?
2. A colleague suggests using the Strategy Pattern for a function that calculates shipping cost, but there is only one shipping provider and the rules never change. Is this a good

use of the pattern? Why or why not?

3. Look at this code. What problem does it have, and how would you fix it using the Strategy Pattern?

```javascript
function formatDate(date, format) {
  if (format === 'US') {
    return `${date.getMonth()+1}/${date.getDate()}/${date.getFullYear()}`;
  } else if (format === 'EU') {
    return `${date.getDate()}/${date.getMonth()+1}/${date.getFullYear()}`;
  } else if (format === 'ISO') {
    return date.toISOString().split('T')[0];
  } else if (format === 'relative') {
    const diff = Date.now() - date.getTime();
    const days = Math.floor(diff / 86400000);
    if (days === 0) return 'today';
    if (days === 1) return 'yesterday';
    return `${days} days ago`;
  }
}
```

4. What is the difference between the Strategy Pattern and simply passing a callback function? When does a callback become a strategy?
5. How does `Array.prototype.sort()` in JavaScript relate to the Strategy Pattern?

Exercises

Exercise 1: Text Formatter Strategies

Build a text formatting system with at least four strategies: `uppercase`, `lowercase`, `titleCase`, and `snakeCase`. Create a `formatText(text, strategyName)` function. Then add a fifth strategy (`camelCase`) without modifying any existing code.

Hints:

- Title case capitalizes the first letter of each word.
- Snake case replaces spaces with underscores and lowercases everything.
- Camel case removes spaces and capitalizes the first letter of each word except the first.

Exercise 2: React Filtering Component

Create a React component that displays a list of movies. Add filter strategies: `all` (show everything), `highRated` (rating above 8), `recent` (released after 2020), and `classic` (released before 2000). Use a dropdown to switch between filters. The filter strategies should be defined

outside the component.

Exercise 3: Composable Strategies

Build a discount calculator that supports combining multiple strategies. For example, a customer might get a "loyalty discount" (10% off) AND a "bulk discount" (5% off for 10+ items) applied together. Create a `composeStrategies` function that takes multiple discount strategies and applies them in sequence.

Hint: Think about function composition. Each strategy takes a price and returns a new price. Composing them means piping the output of one into the input of the next.

What Is Next?

You have learned how to swap algorithms at runtime with the Strategy Pattern. But what about traversing data? In the next chapter, we will explore the Iterator Pattern -- a way to walk through any collection (arrays, trees, paginated API results) with a consistent interface. You will learn about JavaScript's built-in iteration protocol, generators, and how to build custom iterables that make complex data structures feel as simple as a `for...of` loop.

Chapter 7: Iterator Pattern

What You Will Learn

- What the Iterator Pattern is and why uniform traversal matters
- How JavaScript's `Symbol.iterator` protocol works under the hood
- How to make any object iterable with `for...of`
- How generators (`function*`) simplify iterator creation
- How to iterate over paginated API results transparently
- Lazy evaluation and why it saves memory and time
- Real-world iterators in React: virtualized lists and infinite scroll

Why This Chapter Matters

Think about a remote control for your TV. You press "Next Channel" and the TV moves to the next channel. You do not need to know how channels are stored internally -- maybe in a list, maybe in a tree, maybe in a hash table. You do not need to know the total number of channels. You just press "Next" and get the next one.

That is the Iterator Pattern. It provides a standard way to access elements of a collection one by one, without exposing how the collection is structured internally.

In frontend development, you deal with collections constantly. Arrays from API responses. Paginated search results that span many pages. Tree structures of nested comments or file systems. Infinite feeds of social media posts. Each of these has a different internal structure, but you want to process their items the same way: one at a time, in order.

JavaScript has this pattern built directly into the language with the iteration protocol and generators. Understanding how they work gives you the power to create clean, memory-efficient data pipelines and handle complex data sources with elegant code.

The Problem: Different Collections, Different Traversal Code

Imagine you have data stored in different structures and you need to process each item.

The Painful Way (Before the Pattern)

```javascript
// Three different data structures, three different traversal methods

// 1. Array -- use index
const arrayItems = ['Apple', 'Banana', 'Cherry'];
for (let i = 0; i < arrayItems.length; i++) {
  console.log(arrayItems[i]);
}

// 2. Set -- cannot use index!
const setItems = new Set(['Dog', 'Cat', 'Bird']);
// setItems[0] does not work!
setItems.forEach(item => {
  console.log(item);
});

// 3. Map -- use entries
const mapItems = new Map([
  ['name', 'Alice'],
  ['age', '30'],
  ['city', 'Portland']
]);
for (const [key, value] of mapItems.entries()) {
  console.log(`${key}: ${value}`);
}

// 4. Custom tree structure -- completely different
const tree = {
  value: 'root',
  children: [
    {
      value: 'A',
      children: [
        { value: 'A1', children: [] },
        { value: 'A2', children: [] }
      ]
    },
    {
      value: 'B',
      children: [
        { value: 'B1', children: [] }
      ]
    }
```

```
  ]
};

// Need a recursive function just to visit every node
function traverseTree(node) {
  console.log(node.value);
  node.children.forEach(child => traverseTree(child));
}
traverseTree(tree);
```

Output:

```
Apple
Banana
Cherry
Dog
Cat
Bird
name: Alice
age: 30
city: Portland
root
A
A1
A2
B
B1
```

Each data structure requires completely different code to traverse. What if you had a function that processes items regardless of where they come from?

```
┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
│  Array   │  │   Set    │  │   Map    │  │  Tree    │
│          │  │          │  │          │  │          │
│ for(i=0; │  │ .forEach │  │ .entries │  │ recursive│
│  i<len;  │  │          │  │          │  │ function │
│  i++)    │  │          │  │          │  │          │
└──────────┘  └──────────┘  └──────────┘  └──────────┘

  Four different structures = four different traversal methods
  Every new structure = another custom traversal

  What if they all worked the same way?
```

The Solution: JavaScript's Iteration Protocol

JavaScript solved this with the Iterable Protocol and the Iterator Protocol. Any object that follows these protocols can be used with `for...of`, the spread operator (`...`), destructuring, and more.

How It Works

```
Iterable Protocol:
──────────────────

An object is ITERABLE if it has a [Symbol.iterator]() method
that returns an ITERATOR.

Iterator Protocol:
──────────────────

An ITERATOR is an object with a next() method that returns:
{ value: <nextValue>, done: false }   ← still has items
{ value: undefined, done: true }      ← no more items

┌──────────────────────────────┐
│     Iterable Object          │
│                              │
│  [Symbol.iterator]() {       │
│    return {                  │
│      next() {                │
│        return {              │
│          value: ...,         │
│          done: false/true    │
│        };                    │
│      }                       │
│    };                        │
│  }                           │
└──────────────────────────────┘
         │
         ▼
for (const item of iterable) {
  // Gets each value automatically
}
```

Built-In Iterables

Arrays, Strings, Maps, Sets, and TypedArrays are all iterable by default.

```javascript
// All of these work with for...of because they implement Symbol.iterator

// Array
for (const item of ['a', 'b', 'c']) {
```

```
  process.stdout.write(item + ' ');
}
console.log();

// String (iterates over characters)
for (const char of 'Hello') {
  process.stdout.write(char + ' ');
}
console.log();

// Set
for (const item of new Set([1, 2, 3])) {
  process.stdout.write(item + ' ');
}
console.log();

// Map
for (const [key, value] of new Map([['x', 1], ['y', 2]])) {
  process.stdout.write(`${key}=${value} `);
}
console.log();
```

Output:

```
a b c
H e l l o
1 2 3
x=1 y=2
```

The Spread Operator and Destructuring Use Iterators Too

```javascript
const mySet = new Set([10, 20, 30]);

// Spread: converts iterable to array
const arr = [...mySet];
console.log(arr); // [10, 20, 30]

// Destructuring: pulls values from iterable
const [first, second] = mySet;
console.log(first, second); // 10 20

// Array.from: creates array from iterable
const arr2 = Array.from(mySet);
console.log(arr2); // [10, 20, 30]
```

Output:

```
[ 10, 20, 30 ]
10 20
[ 10, 20, 30 ]
```

Building a Custom Iterator with Symbol.iterator

Let us make the tree structure from earlier iterable.

```javascript
// tree-node.js

class TreeNode {
  constructor(value, children = []) {
    this.value = value;
    this.children = children;
  }

  // Make TreeNode iterable using depth-first traversal
  [Symbol.iterator]() {
    let stack = [this]; // Start with the root node

    return {
      next() {
        if (stack.length === 0) {
          return { value: undefined, done: true };
        }

        const current = stack.pop();
        // Add children in reverse order so left children are processed first
        for (let i = current.children.length - 1; i >= 0; i--) {
          stack.push(current.children[i]);
        }

        return { value: current.value, done: false };
      }
    };
  }
}

// Build the same tree from before
const tree = new TreeNode('root', [
  new TreeNode('A', [
    new TreeNode('A1'),
    new TreeNode('A2')
  ]),
  new TreeNode('B', [
    new TreeNode('B1')
```

```
  ])
]);

// Now we can use for...of!
console.log('Depth-first traversal:');
for (const value of tree) {
  console.log(' ', value);
}

// And spread!
const allValues = [...tree];
console.log('\nAll values:', allValues);

// And destructuring!
const [root, firstChild, ...rest] = tree;
console.log('\nRoot:', root);
console.log('First child:', firstChild);
console.log('Rest:', rest);
```

Output:

```
Depth-first traversal:
  root
  A
  A1
  A2
  B
  B1

All values: [ 'root', 'A', 'A1', 'A2', 'B', 'B1' ]

Root: root
First child: A
Rest: [ 'A1', 'A2', 'B', 'B1' ]
```

The tree now works exactly like an array with `for...of`, spread, and destructuring. The consumer does not need to know anything about the tree's internal structure.

```
  BEFORE (custom traversal):        AFTER (Symbol.iterator):

  function traverseTree(node) {     for (const value of tree) {
    console.log(node.value);          console.log(value);
    node.children.forEach(child =>  }
      traverseTree(child)
    );                                // Same code works for
  }                                   // arrays, sets, maps,
  traverseTree(tree);               // trees, and anything
                                      // that implements
  // Only works for this              // Symbol.iterator
  // specific tree structure
```

Generators: The Easy Way to Build Iterators

Writing `Symbol.iterator` by hand with a `next()` method and stack management is tedious. JavaScript generators make it dramatically simpler.

Generator Basics

A generator is a function declared with `function*` that can pause and resume using `yield`.

```javascript
function* countToThree() {
  console.log('About to yield 1');
  yield 1;
  console.log('About to yield 2');
  yield 2;
  console.log('About to yield 3');
  yield 3;
  console.log('Generator done');
}

const gen = countToThree();

console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log(gen.next()); // { value: 3, done: false }
console.log(gen.next()); // { value: undefined, done: true }
```

Output:

```
About to yield 1
{ value: 1, done: false }
About to yield 2
{ value: 2, done: false }
About to yield 3
{ value: 3, done: false }
Generator done
{ value: undefined, done: true }
```

Notice: the generator pauses at each `yield` and only resumes when `next()` is called again. This is lazy evaluation -- nothing runs until you ask for the next value.

Generators Are Iterators

Generators automatically implement the iteration protocol. You can use them with `for...of`.

```javascript
function* fibonacci() {
  let a = 0, b = 1;
  while (true) {  // Infinite sequence!
    yield a;
    [a, b] = [b, a + b];
  }
}

// Take the first 10 Fibonacci numbers
const fib = fibonacci();
for (let i = 0; i < 10; i++) {
  console.log(fib.next().value);
}
```

Output:

```
0
1
1
2
3
5
8
13
21
34
```

An infinite sequence is possible because generators are lazy. The `while (true)` loop does not run forever -- it pauses at each `yield` and only continues when asked.

Tree Traversal with Generators (Much Simpler!)

```javascript
class TreeNode {
  constructor(value, children = []) {
    this.value = value;
    this.children = children;
  }

  // Generator-based iterator -- so much simpler!
  *[Symbol.iterator]() {
    yield this.value;
    for (const child of this.children) {
      yield* child; // yield* delegates to child's iterator
    }
  }
}
```

```
const tree = new TreeNode('root', [
  new TreeNode('A', [
    new TreeNode('A1'),
    new TreeNode('A2')
  ]),
  new TreeNode('B', [
    new TreeNode('B1')
  ])
]);

for (const value of tree) {
  console.log(value);
}
```

Output:

```
root
A
A1
A2
B
B1
```

Compare the generator version to the manual `Symbol.iterator` version from before. The generator is three lines instead of fifteen. The `yield*` keyword delegates to the child node's own iterator, handling the recursion elegantly.

```
  Manual Iterator:                Generator:

  [Symbol.iterator]() {           *[Symbol.iterator]() {
    let stack = [this];             yield this.value;
    return {                        for (const child of this.children) {
      next() {                        yield* child;
        if (stack.length === 0)     }
          return { done: true };  }
        const current = stack.pop();
        for (let i = ...)           3 lines.
          stack.push(child);         Readable.
        return { value, done: f };   Clean.
      }
    };
  }

  15+ lines.
  Manual stack management.
  Error-prone.
```

Iterating Over Paginated API Results

One of the most practical uses of iterators in frontend development is transparently handling paginated API data.

The Problem: Manual Pagination

```javascript
// Without iterators: manual, repetitive pagination code

async function getAllUsers() {
  const allUsers = [];
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    const response = await fetch(`/api/users?page=${page}&limit=20`);
    const data = await response.json();
    allUsers.push(...data.users);
    hasMore = data.hasNextPage;
    page++;
  }

  return allUsers; // Could be thousands of users loaded in memory at once!
}
```

Problems:

1. Loads all data into memory at once
2. Pagination logic is mixed with business logic
3. Every paginated endpoint needs similar boilerplate

The Solution: Async Generator for Pagination

```javascript
// paginated-fetcher.js

async function* fetchPaginated(baseUrl, pageSize = 20) {
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    console.log(`Fetching page ${page}...`);
    const response = await fetch(`${baseUrl}?page=${page}&limit=${pageSize}`);
    const data = await response.json();

    // Yield each item individually
```

```
    for (const item of data.items) {
      yield item;
    }

    hasMore = data.hasNextPage;
    page++;
  }
}

// Usage -- processes items one at a time, page by page
async function processUsers() {
  const users = fetchPaginated('/api/users', 3);

  for await (const user of users) {
    console.log(`Processing user: ${user.name}`);

    // Can stop early! Only fetches pages we need
    if (user.name === 'Target User') {
      console.log('Found target user, stopping.');
      break; // No more pages are fetched!
    }
  }
}
```

Let us simulate this with a mock API:

```javascript
// Mock API for demonstration
function createMockApi() {
  const allUsers = [
    { id: 1, name: 'Alice' },
    { id: 2, name: 'Bob' },
    { id: 3, name: 'Charlie' },
    { id: 4, name: 'Diana' },
    { id: 5, name: 'Eve' },
    { id: 6, name: 'Frank' },
    { id: 7, name: 'Grace' },
    { id: 8, name: 'Hank' }
  ];

  return function mockFetch(url) {
    const params = new URL(url, 'http://localhost').searchParams;
    const page = parseInt(params.get('page'));
    const limit = parseInt(params.get('limit'));
    const start = (page - 1) * limit;
    const end = start + limit;

    return Promise.resolve({
      json: () => Promise.resolve({
        items: allUsers.slice(start, end),
        hasNextPage: end < allUsers.length,
        totalPages: Math.ceil(allUsers.length / limit)
      })
```

```
    });
  };
}

// Replace global fetch with mock
const originalFetch = globalThis.fetch;
globalThis.fetch = createMockApi();

async function* fetchPaginated(baseUrl, pageSize = 3) {
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    console.log(`\n--- Fetching page ${page} ---`);
    const response = await fetch(`${baseUrl}?page=${page}&limit=${pageSize}`);
    const data = await response.json();

    for (const item of data.items) {
      yield item;
    }

    hasMore = data.hasNextPage;
    page++;
  }
}

// Process users lazily
async function main() {
  console.log('=== Processing all users ===');
  for await (const user of fetchPaginated('/api/users', 3)) {
    console.log(`  User: ${user.name} (id: ${user.id})`);
  }

  console.log('\n=== Stopping early ===');
  for await (const user of fetchPaginated('/api/users', 3)) {
    console.log(`  Checking: ${user.name}`);
    if (user.name === 'Diana') {
      console.log('  Found Diana! Stopping.');
      break; // Page 3 is never fetched!
    }
  }
}

main();
```

Output:

```
=== Processing all users ===

--- Fetching page 1 ---
  User: Alice (id: 1)
  User: Bob (id: 2)
  User: Charlie (id: 3)
```

```
--- Fetching page 2 ---
  User: Diana (id: 4)
  User: Eve (id: 5)
  User: Frank (id: 6)

--- Fetching page 3 ---
  User: Grace (id: 7)
  User: Hank (id: 8)

=== Stopping early ===

--- Fetching page 1 ---
  Checking: Alice
  Checking: Bob
  Checking: Charlie

--- Fetching page 2 ---
  Checking: Diana
  Found Diana! Stopping.
```

Notice in the "Stopping early" run: page 3 was never fetched. The generator paused, and when we broke out of the loop, it stopped. No wasted network requests.

```
  Without Generator:                With Generator:

  Fetch ALL pages                    Fetch page 1 ──► yield items
  Store ALL items in memory          Fetch page 2 ──► yield items
  Then process them                  break! ──► STOP
                                     Page 3 never fetched!
  Memory: O(all items)
  Network: ALL pages                 Memory: O(page size)
                                     Network: only needed pages
```

Lazy Evaluation: Process Only What You Need

Lazy evaluation means values are computed only when requested. Generators enable this naturally.

Lazy Range

```javascript
// Eager: creates the entire array in memory
function eagerRange(start, end) {
  const result = [];
  for (let i = start; i <= end; i++) {
    result.push(i);
  }
  return result;
}

// Lazy: generates values one at a time
function* lazyRange(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}

// Eager: allocates an array of 1,000,000 numbers
console.log('Eager range created'); // Uses ~8 MB of memory
const eager = eagerRange(1, 1000000);
console.log('First 3 eager:', eager[0], eager[1], eager[2]);

// Lazy: allocates almost nothing
console.log('Lazy range created'); // Uses almost no memory
const lazy = lazyRange(1, 1000000);
console.log('First 3 lazy:');
console.log(' ', lazy.next().value);
console.log(' ', lazy.next().value);
console.log(' ', lazy.next().value);
// The remaining 999,997 values are never computed!
```

Output:

```
Eager range created
First 3 eager: 1 2 3
Lazy range created
First 3 lazy:
  1
  2
  3
```

Composing Lazy Transformations

You can chain generators to create data processing pipelines where no intermediate arrays are created.

```javascript
// Generator utilities for lazy data processing
```

```
function* map(iterable, fn) {
  for (const item of iterable) {
    yield fn(item);
  }
}

function* filter(iterable, predicate) {
  for (const item of iterable) {
    if (predicate(item)) {
      yield item;
    }
  }
}

function* take(iterable, count) {
  let taken = 0;
  for (const item of iterable) {
    if (taken >= count) return;
    yield item;
    taken++;
  }
}

function* skip(iterable, count) {
  let skipped = 0;
  for (const item of iterable) {
    if (skipped < count) {
      skipped++;
      continue;
    }
    yield item;
  }
}

// Build a lazy pipeline
function* range(start, end) {
  for (let i = start; i <= end; i++) {
    yield i;
  }
}

// Pipeline: range 1-100 -> keep even -> square -> take first 5
const pipeline = take(
  map(
    filter(range(1, 100), n => n % 2 === 0),
    n => n * n
  ),
  5
);

console.log('Pipeline results:');
for (const value of pipeline) {
  console.log(' ', value);
}
// Only the first 10 numbers from range(1, 100) are ever evaluated!
```

```
// The rest are never generated.
```

Output:

```
Pipeline results:
  4
  16
  36
  64
  100
```

```
  Lazy Pipeline Visualization:
  ____________________________

  range(1, 100)     filter(even)     map(n*n)        take(5)
  _____________     ____________     ________        _______

  1 ──────────────► skip
  2 ──────────────► pass ──────────► 4 ──────────► output (1/5)
  3 ──────────────► skip
  4 ──────────────► pass ──────────► 16 ─────────► output (2/5)
  5 ──────────────► skip
  6 ──────────────► pass ──────────► 36 ─────────► output (3/5)
  7 ──────────────► skip
  8 ──────────────► pass ──────────► 64 ─────────► output (4/5)
  9 ──────────────► skip
  10 ─────────────► pass ──────────► 100 ────────► output (5/5)
  11-100 ─────────► NEVER GENERATED (take is satisfied)

  Each item flows through the entire pipeline before the next one starts.
  No intermediate arrays are created.
```

Making Custom Collections Iterable

Example: A Ring Buffer

A ring buffer is a fixed-size collection that overwrites the oldest items when full. Let us make it iterable.

```javascript
class RingBuffer {
  constructor(capacity) {
```

```
    this.capacity = capacity;
    this.buffer = new Array(capacity);
    this.head = 0;    // Where next item goes
    this.count = 0;   // Current number of items
  }

  push(item) {
    this.buffer[this.head] = item;
    this.head = (this.head + 1) % this.capacity;
    if (this.count < this.capacity) {
      this.count++;
    }
  }

  get size() {
    return this.count;
  }

  // Make it iterable -- yields items from oldest to newest
  *[Symbol.iterator]() {
    if (this.count === 0) return;

    const start = this.count < this.capacity
      ? 0
      : this.head; // head points to the oldest item when full

    for (let i = 0; i < this.count; i++) {
      const index = (start + i) % this.capacity;
      yield this.buffer[index];
    }
  }
}

// A ring buffer that holds the last 5 log messages
const recentLogs = new RingBuffer(5);

// Add 8 messages (only last 5 will remain)
recentLogs.push('Log 1: App started');
recentLogs.push('Log 2: User logged in');
recentLogs.push('Log 3: Fetched data');
recentLogs.push('Log 4: Rendered dashboard');
recentLogs.push('Log 5: User clicked button');
recentLogs.push('Log 6: API request sent');   // Overwrites Log 1
recentLogs.push('Log 7: Response received');   // Overwrites Log 2
recentLogs.push('Log 8: Cache updated');       // Overwrites Log 3

console.log(`Recent logs (${recentLogs.size} items):`);
for (const log of recentLogs) {
  console.log(' ', log);
}

// Works with spread too
console.log('\nAs array:', [...recentLogs]);
```

Output:

```
Recent logs (5 items):
  Log 4: Rendered dashboard
  Log 5: User clicked button
  Log 6: API request sent
  Log 7: Response received
  Log 8: Cache updated

As array: [
  'Log 4: Rendered dashboard',
  'Log 5: User clicked button',
  'Log 6: API request sent',
  'Log 7: Response received',
  'Log 8: Cache updated'
]
```

Example: A Date Range Iterator

```javascript
class DateRange {
  constructor(start, end) {
    this.start = new Date(start);
    this.end = new Date(end);
  }

  *[Symbol.iterator]() {
    const current = new Date(this.start);
    while (current <= this.end) {
      yield new Date(current); // Yield a copy
      current.setDate(current.getDate() + 1);
    }
  }
}

const vacation = new DateRange('2025-07-01', '2025-07-05');

console.log('Vacation days:');
for (const day of vacation) {
  const formatted = day.toLocaleDateString('en-US', {
    weekday: 'short',
    month: 'short',
    day: 'numeric'
  });
  console.log(' ', formatted);
}

// Destructure the first and last days
const days = [...vacation];
console.log(`\n${days.length} days total`);
```

Output:

```
Vacation days:
  Tue, Jul 1
  Wed, Jul 2
  Thu, Jul 3
  Fri, Jul 4
  Sat, Jul 5

5 days total
```

Iterators in React: Infinite Scroll with Async Generators

```javascript
// hooks/usePaginatedData.js
import { useState, useCallback, useRef } from 'react';

/**
 * Custom hook that wraps an async generator for use in React components
 */
export function usePaginatedData(fetchGeneratorFn) {
  const [items, setItems] = useState([]);
  const [loading, setLoading] = useState(false);
  const [hasMore, setHasMore] = useState(true);
  const generatorRef = useRef(null);

  const loadMore = useCallback(async () => {
    if (loading || !hasMore) return;

    // Create generator on first call
    if (!generatorRef.current) {
      generatorRef.current = fetchGeneratorFn();
    }

    setLoading(true);

    // Pull the next batch from the generator
    const batch = [];
    const BATCH_SIZE = 10;

    for (let i = 0; i < BATCH_SIZE; i++) {
      const result = await generatorRef.current.next();
      if (result.done) {
        setHasMore(false);
        break;
      }
      batch.push(result.value);
```

```
    }

    setItems(prev => [...prev, ...batch]);
    setLoading(false);
  }, [fetchGeneratorFn, loading, hasMore]);

  const reset = useCallback(() => {
    generatorRef.current = null;
    setItems([]);
    setHasMore(true);
    setLoading(false);
  }, []);

  return { items, loading, hasMore, loadMore, reset };
}
```

```javascript
// components/InfiniteUserList.jsx
import { useEffect, useRef, useCallback } from 'react';
import { usePaginatedData } from '../hooks/usePaginatedData';

// Async generator that fetches users page by page
async function* fetchAllUsers() {
  let page = 1;
  while (true) {
    const response = await fetch(`/api/users?page=${page}&limit=20`);
    const data = await response.json();

    for (const user of data.users) {
      yield user;
    }

    if (!data.hasNextPage) return; // Generator ends
    page++;
  }
}

export default function InfiniteUserList() {
  const { items: users, loading, hasMore, loadMore } = usePaginatedData(
    useCallback(() => fetchAllUsers(), [])
  );

  // Intersection Observer for infinite scroll
  const sentinelRef = useRef(null);

  useEffect(() => {
    const sentinel = sentinelRef.current;
    if (!sentinel) return;

    const observer = new IntersectionObserver(
      (entries) => {
        if (entries[0].isIntersecting && hasMore && !loading) {
          loadMore();
```

```
        }
      },
      { threshold: 0.1 }
    );

    observer.observe(sentinel);
    return () => observer.disconnect();
  }, [hasMore, loading, loadMore]);

  // Load initial data
  useEffect(() => { loadMore(); }, []);

  return (
    <div>
      <h2>Users ({users.length})</h2>
      <ul>
        {users.map(user => (
          <li key={user.id}>{user.name} - {user.email}</li>
        ))}
      </ul>

      {/* Sentinel element -- when it becomes visible, load more */}
      <div ref={sentinelRef} style={{ height: 1 }} />

      {loading && <p>Loading more...</p>}
      {!hasMore && <p>No more users to load.</p>}
    </div>
  );
}
```

```
Infinite Scroll with Async Generator:
────────────────────────────────────


┌──────────────────────────────────┐
│  Async Generator                 │
│  fetchAllUsers()                 │
│                                  │
│  Page 1: yield user1..user20     │──► React State: [user1..user20]
│          (pause)                 │    Render list
│                                  │
│  Page 2: yield user21..user40    │──► React State: [..., user21..user40]
│          (pause)                 │    Render more
│                                  │
│  User scrolls to bottom          │
│  IntersectionObserver fires      │
│  loadMore() calls next()         │
│                                  │
│  Page 3: yield user41..user60    │──► React State: [..., user41..user60]
│          (pause)                 │    Render even more
│                                  │
│  No more pages: return           │──► hasMore = false
└──────────────────────────────────┘    Show "No more users"
```

When to Use / When NOT to Use

When to Use the Iterator Pattern

- Paginated API data: Fetch pages lazily, process items one at a time
- Large datasets: When you cannot fit all data in memory at once
- Custom data structures: Trees, graphs, linked lists -- make them work with `for...of`
- Data processing pipelines: Chain transformations without creating intermediate arrays
- Infinite sequences: Fibonacci, random numbers, sensor readings -- things with no fixed end
- Streaming data: Processing CSV files, log lines, or WebSocket messages line by line
- Abstraction: Hide how data is stored while exposing how it is accessed

When NOT to Use the Iterator Pattern

- Small arrays: If your data fits comfortably in memory and is already an array, just use `.map()`, `.filter()`, and `.reduce()`. Do not add complexity with generators
- When you need random access: Iterators are sequential (forward-only). If you need to access item #500 directly, use an array
- Performance-critical inner loops: Generator function calls have overhead compared to a plain `for` loop. In tight loops running millions of times, this overhead adds up
- When the whole dataset is needed at once: If your algorithm needs all items in memory simultaneously (like sorting), lazy evaluation does not help

---

Common Mistakes

Mistake 1: Trying to Reuse an Exhausted Iterator

```javascript
function* numbers() {
  yield 1;
  yield 2;
  yield 3;
}
```

```
const gen = numbers();

// First loop: works
for (const n of gen) {
  console.log(n); // 1, 2, 3
}

// Second loop: NOTHING! Generator is exhausted
for (const n of gen) {
  console.log(n); // Never runs
}
```

Fix: Create a new generator for each loop, or use an iterable object that creates a fresh iterator each time `[Symbol.iterator]` is called.

```javascript
// Iterable object -- fresh iterator every time
const numbers = {
  *[Symbol.iterator]() {
    yield 1;
    yield 2;
    yield 3;
  }
};

for (const n of numbers) { console.log(n); } // 1, 2, 3
for (const n of numbers) { console.log(n); } // 1, 2, 3 (works again!)
```

Mistake 2: Forgetting `yield*` for Delegation

```javascript
function* inner() {
  yield 'a';
  yield 'b';
}

// WRONG -- yields the generator OBJECT, not its values
function* outerWrong() {
  yield inner(); // yields Generator {}, not 'a' and 'b'
}

// RIGHT -- delegates to inner generator
function* outerRight() {
  yield* inner(); // yields 'a', then 'b'
}

console.log([...outerWrong()]); // [ Generator {} ]
console.log([...outerRight()]); // [ 'a', 'b' ]
```

Output:

```
[ Object [Generator] {} ]
[ 'a', 'b' ]
```

Mistake 3: Not Using `for await...of` with Async Generators

```javascript
async function* asyncNumbers() {
  yield 1;
  yield 2;
  yield 3;
}

// WRONG -- regular for...of with async generator
// for (const n of asyncNumbers()) { } // Fails or gives Promise objects

// RIGHT -- use for await...of
for await (const n of asyncNumbers()) {
  console.log(n); // 1, 2, 3
}
```

Best Practices

1. Use generators over manual iterators. They are shorter, clearer, and less error-prone than manually implementing `next()`.
2. Make your iterable objects return fresh iterators. Implement `[Symbol.iterator]` on the object (not the prototype) so each `for...of` loop gets a new traversal.
3. Use `yield*` for delegation. When one generator needs to yield values from another, use `yield*` to delegate rather than manually iterating.
4. Prefer `for await...of` for async generators instead of manually calling `.next()`. It handles the Promise unwrapping and done-checking for you.
5. Use lazy pipelines for large data. Chain `filter`, `map`, and `take` generators to process large datasets without creating intermediate arrays.
6. Clean up generators with `return()`. When you break out of a `for...of` loop, the generator's `return()` method is called automatically. If you are iterating manually, call `gen.return()` when done.
7. Keep generators pure when possible. Generators that do not depend on external mutable state are easier to test and reason about.

Quick Summary

The Iterator Pattern provides a uniform way to traverse any collection without knowing its internal structure.

```
JavaScript Iteration Ecosystem:
═══════════════════════════════

Iterable Protocol         for...of, spread(...), destructuring
[Symbol.iterator]()       Array.from(), Promise.all()
      │
      ▼
Iterator Protocol         { next() → { value, done } }
      │
      ▼
Generators               function* + yield
(easiest way to          yield* (delegation)
 create iterators)       Lazy evaluation built-in
      │
      ▼
Async Generators         async function* + yield
for await...of           Perfect for paginated APIs
```

Key Points

- The Iterator Pattern provides a standard way to traverse collections one element at a time
- JavaScript's Symbol.iterator protocol makes any object work with `for...of`, spread, and destructuring
- Generators (`function*` + `yield`) are the simplest way to create custom iterators
- `yield*` delegates to another iterable, essential for recursive structures like trees
- Async generators (`async function*`) combined with `for await...of` are perfect for paginated API data
- Lazy evaluation means values are computed only when needed, saving memory and enabling infinite sequences
- Iterators are forward-only and single-use by default -- make objects return fresh iterators for reuse

- Lazy pipelines (chained generators for map, filter, take) process data without intermediate arrays

---

Practice Questions

1. What is the difference between the Iterable Protocol and the Iterator Protocol? What method does each require?
2. Why can you only iterate through a generator once? How do you create an object that can be iterated multiple times?
3. Explain how `yield*` works. What happens if you use `yield` instead of `yield*` when delegating to another generator?
4. How do async generators solve the paginated API problem? What advantage does `break` inside `for await...of` give you?
5. What is lazy evaluation? Give a concrete example where lazy evaluation saves significant memory or computation.

Exercises

Exercise 1: Build a Paginated Table Component

Create an async generator that fetches product data from a mock API in pages of 10. Build a React component that:

- Shows a table of products
- Has "Load More" and "Load All" buttons
- Displays the current count vs total
- Uses the generator to fetch pages on demand

Exercise 2: Lazy Data Pipeline

Build a set of composable generator functions:

- `range(start, end)` -- generates numbers in range
- `map(iterable, fn)` -- transforms each item
- `filter(iterable, fn)` -- keeps items matching predicate

- `take(iterable, n)` -- takes first n items
- `chunk(iterable, size)` -- groups items into arrays of given size

Use these to build a pipeline that:

1. Generates numbers 1 through 1000
2. Keeps only multiples of 3
3. Squares each number
4. Groups them into chunks of 5
5. Takes the first 3 chunks

Verify that only about 45 numbers from the range are actually generated.

Exercise 3: Tree Iterator with Multiple Traversal Orders

Create a `TreeNode` class that supports three different iteration strategies:

- Depth-first pre-order: parent, then children (default `[Symbol.iterator]`)
- Depth-first post-order: children, then parent
- Breadth-first (level order): level by level, left to right

Implement each as a generator method:

```javascript
tree[Symbol.iterator]()    // pre-order (default)
tree.postOrder()           // post-order
tree.breadthFirst()        // level order
```

Test with a tree that has at least 3 levels and verify the output order for each traversal.

What Is Next?

You now have a powerful tool for traversing any data structure uniformly and efficiently. The Iterator Pattern, combined with generators and lazy evaluation, lets you handle paginated APIs, infinite data streams, and custom collections with clean, readable code.

The next chapter covers the Proxy Pattern, where you learn how to intercept and customize operations on objects -- reading properties, writing values, calling functions, and more. This is the foundation for reactivity systems like Vue.js and for building powerful debugging and validation tools.

Chapter 8: The Proxy Pattern

What You Will Learn

- What the Proxy pattern is and why it exists
- How to use the ES6 `Proxy` object and its traps
- How to build validation proxies that protect your data
- How to build logging proxies for debugging
- How to build caching proxies for performance
- How Vue.js uses proxies for reactivity
- When the Proxy pattern helps and when it hurts

Why This Chapter Matters

Imagine you live in an apartment building with a doorman. You do not interact with every delivery person, solicitor, or visitor directly. The doorman stands between you and the outside world. He checks IDs, accepts packages, turns away salespeople, and logs who comes and goes.

You still get everything you need. But the doorman adds a layer of control between you and the world.

The Proxy pattern works exactly the same way. It places an object between the caller and the real object. This "middleman" can validate input, log access, cache results, control permissions, or add any behavior you want -- all without changing the original object.

In frontend development, proxies are everywhere. Vue 3 is built on them. State management libraries use them. Form validation, API wrappers, and lazy-loading systems all benefit from proxies.

This chapter teaches you to build and use proxies confidently.

The Problem

You have an object. You want to control what happens when someone reads from it, writes to it, or interacts with it in any way. But you do not want to modify the object itself.

```javascript
// A simple user object
const user = {
  name: "Alice",
  age: 28,
  email: "alice@example.com",
  role: "admin"
};

// Anyone can do anything to this object
user.age = -5;        // Nonsense, but JavaScript allows it
user.name = "";       // Empty name? Sure, why not
user.role = "superadmin"; // Security? What security?
delete user.email;    // Gone forever

console.log(user);
// { name: '', age: -5, role: 'superadmin' }
```

You need a gatekeeper.

The Solution: Proxy Pattern

The Proxy pattern creates a stand-in for another object. The proxy intercepts operations on the target object and can modify, validate, or extend the behavior.

```
+--------+     +-----------+     +--------+
| Caller | --> |   Proxy   | --> | Target |
+--------+     +-----------+     +--------+
                    |
              Intercepts:
              - get (reading)
              - set (writing)
              - delete
              - has (in operator)
              - apply (function call)
              - and more...
```

The caller talks to the proxy as if it were the real object. The proxy decides what actually happens.

ES6 Proxy: The Built-in Gatekeeper

JavaScript has a built-in `Proxy` constructor since ES6. It takes two arguments:

1. target -- the real object you want to proxy
2. handler -- an object containing "traps" (interceptor functions)

Your First Proxy

```javascript
const target = {
  greeting: "Hello",
  farewell: "Goodbye"
};

const handler = {
  get(target, property, receiver) {
    console.log(`Someone read "${property}"`);
    return target[property];
  }
};

const proxy = new Proxy(target, handler);

console.log(proxy.greeting);
// Output:
// Someone read "greeting"
// "Hello"

console.log(proxy.farewell);
// Output:
// Someone read "farewell"
// "Goodbye"

// The proxy looks and feels like the real object
console.log(proxy.greeting === target.greeting);
// Output: true
```

The `get` trap fires every time someone reads a property. The proxy is invisible to the caller -- it behaves like the real object.

Common Proxy Traps

The `Proxy` handler supports many traps. Here are the ones you will use most:

```
+------------------+----------------------------+---------------------------+
| Trap             | Intercepts                 | Example Use               |
+------------------+----------------------------+---------------------------+
| get              | Reading a property         | Logging, defaults, caching |
| set              | Writing a property         | Validation, reactivity     |
| has              | The "in" operator          | Hiding private properties  |
| deleteProperty   | The "delete" operator      | Preventing deletion        |
| apply            | Function calls             | Timing, memoization        |
| construct        | The "new" operator         | Singletons, pooling        |
| ownKeys          | Object.keys(), for...in    | Filtering properties       |
+------------------+----------------------------+---------------------------+
```

Let us explore the most important ones.

The `get` Trap

Intercepts property reads.

```javascript
const defaults = {
  theme: "light",
  language: "en",
  fontSize: 16
};

const settings = {};

const settingsWithDefaults = new Proxy(settings, {
  get(target, property) {
    if (property in target) {
      return target[property];
    }
    return defaults[property];
  }
});

console.log(settingsWithDefaults.theme);
// Output: "light" (from defaults)

settingsWithDefaults.theme = "dark";
console.log(settingsWithDefaults.theme);
// Output: "dark" (from settings, overrides default)

console.log(settingsWithDefaults.language);
// Output: "en" (still from defaults)
```

The `set` Trap

Intercepts property writes. Must return `true` for success or `false` (which throws a `TypeError` in strict mode).

```javascript
const user = { name: "Alice", age: 28 };

const validatedUser = new Proxy(user, {
  set(target, property, value) {
    if (property === "age") {
      if (typeof value !== "number") {
        throw new TypeError("Age must be a number");
      }
      if (value < 0 || value > 150) {
        throw new RangeError("Age must be between 0 and 150");
      }
    }

    if (property === "name") {
      if (typeof value !== "string" || value.trim().length === 0) {
        throw new TypeError("Name must be a non-empty string");
      }
    }

    target[property] = value;
    return true;
  }
});

validatedUser.name = "Bob";       // Works fine
console.log(validatedUser.name);
// Output: "Bob"

try {
  validatedUser.age = -5;
} catch (error) {
  console.log(error.message);
  // Output: "Age must be between 0 and 150"
}

try {
  validatedUser.name = "";
} catch (error) {
  console.log(error.message);
  // Output: "Name must be a non-empty string"
}
```

The `has` Trap

Intercepts the `in` operator. Useful for hiding "private" properties.

```javascript
const user = {
  name: "Alice",
  _password: "secret123",
  _token: "abc-xyz"
};

const safeUser = new Proxy(user, {
  has(target, property) {
    if (property.startsWith("_")) {
      return false; // Hide private properties
    }
    return property in target;
  }
});

console.log("name" in safeUser);
// Output: true

console.log("_password" in safeUser);
// Output: false (hidden!)

console.log("_token" in safeUser);
// Output: false (hidden!)
```

The `deleteProperty` Trap

Prevents properties from being deleted.

```javascript
const config = {
  apiUrl: "https://api.example.com",
  timeout: 5000,
  retries: 3
};

const protectedConfig = new Proxy(config, {
  deleteProperty(target, property) {
    throw new Error(
      `Cannot delete config property "${property}". ` +
      `Configuration is immutable.`
    );
  }
});

try {
  delete protectedConfig.apiUrl;
} catch (error) {
  console.log(error.message);
  // Output: Cannot delete config property "apiUrl".
  //         Configuration is immutable.
}
```

Pattern 1: Validation Proxy

Problem

You receive data from user forms, API responses, or third-party libraries. You need to ensure the data meets your requirements before it enters your system.

Before: Validation Scattered Everywhere

```javascript
// Validation logic is repeated everywhere
function updateUserName(user, newName) {
  if (typeof newName !== "string" || newName.trim() === "") {
    throw new Error("Invalid name");
  }
  user.name = newName;
}

function updateUserAge(user, newAge) {
  if (typeof newAge !== "number" || newAge < 0 || newAge > 150) {
    throw new Error("Invalid age");
  }
  user.age = newAge;
}

function updateUserEmail(user, newEmail) {
  if (!newEmail.includes("@")) {
    throw new Error("Invalid email");
  }
  user.email = newEmail;
}
```

After: Centralized Validation Proxy

```javascript
function createValidatedObject(target, schema) {
  return new Proxy(target, {
    set(target, property, value) {
      const validator = schema[property];

      if (validator) {
        const result = validator(value);
        if (!result.valid) {
          throw new Error(
```

```
            `Invalid value for "${property}": ${result.message}`
          );
        }
      }

      target[property] = value;
      return true;
    }
  });
}

// Define validation rules in one place
const userSchema = {
  name: (value) => {
    if (typeof value !== "string" || value.trim() === "") {
      return { valid: false, message: "Must be a non-empty string" };
    }
    return { valid: true };
  },
  age: (value) => {
    if (typeof value !== "number" || value < 0 || value > 150) {
      return { valid: false, message: "Must be a number between 0 and 150" };
    }
    return { valid: true };
  },
  email: (value) => {
    if (typeof value !== "string" || !value.includes("@")) {
      return { valid: false, message: "Must be a valid email address" };
    }
    return { valid: true };
  }
};

const user = createValidatedObject({}, userSchema);

user.name = "Alice";   // Works
user.age = 28;         // Works
user.email = "a@b.com"; // Works

console.log(user);
// Output: { name: 'Alice', age: 28, email: 'a@b.com' }

try {
  user.age = -5;
} catch (e) {
  console.log(e.message);
  // Output: Invalid value for "age": Must be a number between 0 and 150
}
```

Real-World Use Case: Form State Validation

```javascript
function createFormState(initialValues, validators) {
  const errors = {};
  const touched = {};

  const state = new Proxy(
    { ...initialValues },
    {
      set(target, field, value) {
        target[field] = value;
        touched[field] = true;

        // Run validation on every change
        if (validators[field]) {
          const error = validators[field](value);
          if (error) {
            errors[field] = error;
          } else {
            delete errors[field];
          }
        }

        return true;
      },
      get(target, property) {
        if (property === "$errors") return { ...errors };
        if (property === "$touched") return { ...touched };
        if (property === "$isValid") return Object.keys(errors).length === 0;
        return target[property];
      }
    }
  );

  return state;
}

// Usage
const form = createFormState(
  { username: "", password: "" },
  {
    username: (val) =>
      val.length < 3 ? "Username must be at least 3 characters" : null,
    password: (val) =>
      val.length < 8 ? "Password must be at least 8 characters" : null
  }
);

form.username = "ab";
console.log(form.$errors);
// Output: { username: 'Username must be at least 3 characters' }
console.log(form.$isValid);
// Output: false

form.username = "alice";
form.password = "secure1234";
```

```
console.log(form.$errors);
// Output: {}
console.log(form.$isValid);
// Output: true
```

Pattern 2: Logging Proxy

Problem

You need to debug which properties are being read or written, and when. Adding `console.log` everywhere clutters your code and is hard to clean up.

Before: Manual Logging

```javascript
class UserService {
  getUser(id) {
    console.log(`[LOG] getUser called with id: ${id}`);
    const result = database.findUser(id);
    console.log(`[LOG] getUser returned:`, result);
    return result;
  }

  updateUser(id, data) {
    console.log(`[LOG] updateUser called with:`, id, data);
    const result = database.updateUser(id, data);
    console.log(`[LOG] updateUser returned:`, result);
    return result;
  }

  // Every method has the same logging boilerplate...
}
```

After: Logging Proxy

```javascript
function createLoggingProxy(target, label = "Object") {
  return new Proxy(target, {
    get(target, property, receiver) {
      const value = Reflect.get(target, property, receiver);

      if (typeof value === "function") {
```

```
        return function (...args) {
          console.log(
            `[${label}] Called ${String(property)}(${
              args.map(a => JSON.stringify(a)).join(", ")
            })`
          );

          const result = value.apply(target, args);

          // Handle promises
          if (result instanceof Promise) {
            return result.then((resolved) => {
              console.log(
                `[${label}] ${String(property)} resolved:`,
                resolved
              );
              return resolved;
            });
          }

          console.log(`[${label}] ${String(property)} returned:`, result);
          return result;
        };
      }

      console.log(`[${label}] Read ${String(property)}:`, value);
      return value;
    },

    set(target, property, value) {
      console.log(
        `[${label}] Set ${String(property)}:`,
        target[property],
        "->",
        value
      );
      target[property] = value;
      return true;
    }
  });
}

// Usage
const calculator = {
  total: 0,
  add(n) {
    this.total += n;
    return this.total;
  },
  multiply(n) {
    this.total *= n;
    return this.total;
  }
};
```

```
const logged = createLoggingProxy(calculator, "Calc");

logged.add(5);
// Output: [Calc] Called add(5)
// Output: [Calc] add returned: 5

logged.multiply(3);
// Output: [Calc] Called multiply(3)
// Output: [Calc] multiply returned: 15

console.log(logged.total);
// Output: [Calc] Read total: 15
// Output: 15
```

You can turn logging on and off simply by swapping the proxy in and out. Zero changes to your actual business logic.

Pattern 3: Caching Proxy

Problem

You have expensive function calls -- API requests, heavy computations, database queries. The same inputs often produce the same outputs. You want to cache results without polluting the original function.

Before: Caching Mixed with Logic

```javascript
const cache = {};

async function fetchUserProfile(userId) {
  // Caching logic mixed with business logic
  if (cache[userId] && Date.now() - cache[userId].timestamp < 60000) {
    return cache[userId].data;
  }

  const response = await fetch(`/api/users/${userId}`);
  const data = await response.json();

  cache[userId] = { data, timestamp: Date.now() };
  return data;
}
```

After: Caching Proxy

```javascript
function createCachingProxy(fn, ttl = 60000) {
  const cache = new Map();

  return new Proxy(fn, {
    apply(target, thisArg, args) {
      const key = JSON.stringify(args);
      const cached = cache.get(key);

      if (cached && Date.now() - cached.timestamp < ttl) {
        console.log(`Cache HIT for args: ${key}`);
        return cached.result;
      }

      console.log(`Cache MISS for args: ${key}`);
      const result = target.apply(thisArg, args);

      // Handle promises
      if (result instanceof Promise) {
        return result.then((resolved) => {
          cache.set(key, { result: resolved, timestamp: Date.now() });
          return resolved;
        });
      }

      cache.set(key, { result, timestamp: Date.now() });
      return result;
    }
  });
}

// Usage with a pure function
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

const cachedFib = createCachingProxy(fibonacci);

console.log(cachedFib(10));
// Output: Cache MISS for args: [10]
// Output: 55

console.log(cachedFib(10));
// Output: Cache HIT for args: [10]
// Output: 55

// Usage with an API function
async function fetchUser(id) {
  const res = await fetch(`/api/users/${id}`);
  return res.json();
}
```

```
const cachedFetchUser = createCachingProxy(fetchUser, 30000);

// First call hits the network
await cachedFetchUser(42);
// Output: Cache MISS for args: [42]

// Second call (within 30 seconds) uses cache
await cachedFetchUser(42);
// Output: Cache HIT for args: [42]
```

Pattern 4: Access Control Proxy

Problem

Different users have different permissions. You need to restrict what they can do with certain objects.

```javascript
function createAccessControlProxy(target, currentUser) {
  const permissions = {
    admin: ["read", "write", "delete"],
    editor: ["read", "write"],
    viewer: ["read"]
  };

  return new Proxy(target, {
    get(target, property) {
      const userPerms = permissions[currentUser.role] || [];

      if (!userPerms.includes("read")) {
        throw new Error(
          `User "${currentUser.name}" does not have read access`
        );
      }

      return target[property];
    },

    set(target, property, value) {
      const userPerms = permissions[currentUser.role] || [];

      if (!userPerms.includes("write")) {
        throw new Error(
          `User "${currentUser.name}" does not have write access`
        );
```

```
      }

      target[property] = value;
      return true;
    },

    deleteProperty(target, property) {
      const userPerms = permissions[currentUser.role] || [];

      if (!userPerms.includes("delete")) {
        throw new Error(
          `User "${currentUser.name}" does not have delete access`
        );
      }

      delete target[property];
      return true;
    }
  });
}

// Usage
const document = { title: "Secret Report", content: "Top secret..." };

const adminView = createAccessControlProxy(
  document,
  { name: "Alice", role: "admin" }
);

const viewerView = createAccessControlProxy(
  document,
  { name: "Bob", role: "viewer" }
);

console.log(viewerView.title);
// Output: "Secret Report" (read is allowed)

try {
  viewerView.title = "Changed!";
} catch (e) {
  console.log(e.message);
  // Output: User "Bob" does not have write access
}

adminView.title = "Updated Report";
// Works fine -- admin has write access
```

How Vue 3 Uses Proxies for Reactivity

Vue 3's reactivity system is built entirely on ES6 Proxies. When you call `reactive()`, Vue wraps your object in a Proxy that tracks which properties are read (dependencies) and triggers re-renders when they change.

Here is a simplified version of how it works:

```javascript
// Simplified Vue 3 reactivity (conceptual)
let activeEffect = null;

function reactive(target) {
  const deps = new Map(); // property -> Set of effects

  return new Proxy(target, {
    get(target, property, receiver) {
      // Track: record which effect depends on this property
      if (activeEffect) {
        if (!deps.has(property)) {
          deps.set(property, new Set());
        }
        deps.get(property).add(activeEffect);
      }

      return Reflect.get(target, property, receiver);
    },

    set(target, property, value, receiver) {
      const oldValue = target[property];
      const result = Reflect.set(target, property, value, receiver);

      // Trigger: notify all effects that depend on this property
      if (oldValue !== value && deps.has(property)) {
        deps.get(property).forEach((effect) => effect());
      }

      return result;
    }
  });
}

function watchEffect(fn) {
  activeEffect = fn;
  fn(); // Run once to collect dependencies
  activeEffect = null;
}

// Usage (similar to Vue 3)
const state = reactive({ count: 0, message: "Hello" });

watchEffect(() => {
  console.log(`Count is: ${state.count}`);
});
// Output: Count is: 0
```

```
state.count = 1;
// Output: Count is: 1

state.count = 2;
// Output: Count is: 2

state.message = "World";
// No output! The effect only depends on "count"
```

```
How Vue 3 Reactivity Works (Simplified):

1. SETUP: Component uses reactive({count: 0})
              |
              v
2. PROXY: Wraps the object with get/set traps
              |
              v
3. RENDER: Component renders, reading state.count
              |
              v
4. TRACK: get trap records "render depends on count"
              |
              v
5. UPDATE: User clicks button, state.count = 1
              |
              v
6. TRIGGER: set trap sees count changed,
            re-runs the render function
              |
              v
7. RE-RENDER: Component updates in the DOM
```

This is why Vue 3 requires you to use `reactive()` or `ref()` -- without the Proxy wrapper, Vue cannot track changes.

Proxy Pattern in React

React does not use proxies internally, but you can use them in your own code:

```javascript
import { useState, useCallback } from "react";

function useValidatedState(initialValue, validator) {
  const [value, setValue] = useState(initialValue);
```

```
  const [error, setError] = useState(null);

  const setValidatedValue = useCallback(
    (newValue) => {
      const validationError = validator(newValue);
      if (validationError) {
        setError(validationError);
        return false;
      }
      setError(null);
      setValue(newValue);
      return true;
    },
    [validator]
  );

  return [value, setValidatedValue, error];
}

// Usage in a component
function AgeInput() {
  const [age, setAge, error] = useValidatedState(0, (value) => {
    if (typeof value !== "number") return "Must be a number";
    if (value < 0 || value > 150) return "Must be between 0 and 150";
    return null;
  });

  return (
    <div>
      <input
        type="number"
        value={age}
        onChange={(e) => setAge(Number(e.target.value))}
      />
      {error && <span className="error">{error}</span>}
    </div>
  );
}
```

Using Reflect with Proxy

The `Reflect` object is the companion to `Proxy`. It provides default behavior for each trap. Always use `Reflect` inside your traps to preserve the original behavior:

```javascript
const proxy = new Proxy(target, {
  get(target, property, receiver) {
```

```
    // Do your custom thing
    console.log(`Reading ${String(property)}`);

    // Then use Reflect for the default behavior
    return Reflect.get(target, property, receiver);
  },

  set(target, property, value, receiver) {
    // Do your custom thing
    console.log(`Writing ${String(property)} = ${value}`);

    // Then use Reflect for the default behavior
    return Reflect.set(target, property, value, receiver);
  }
});
```

Why `Reflect` instead of `target[property]`? Because `Reflect` correctly handles:

- Inherited properties
- Getters and setters
- The `receiver` (the correct `this` value)

When to Use the Proxy Pattern

```
+-----------------------------------------------+
|           USE PROXY WHEN:                     |
+-----------------------------------------------+
|                                               |
|  * You need to validate data on assignment    |
|  * You need to log or monitor object access   |
|  * You need to cache expensive operations     |
|  * You need to control access permissions     |
|  * You need to add "virtual" properties       |
|  * You need to make objects reactive           |
|  * You want to intercept without modifying    |
|    the original object                        |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|       DO NOT USE PROXY WHEN:                  |
+-----------------------------------------------+
|                                               |
|  * Simple if/else checks would suffice        |
|  * Performance is critical (proxies add       |
|    overhead to every property access)         |
```

```
|  * You need to support IE11 (no polyfill)     |
|  * The object is only used in one place       |
|  * A simple wrapper function would work       |
|  * You are proxying objects in a tight loop   |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Forgetting to Return `true` from `set`

```javascript
// WRONG -- causes TypeError in strict mode
const proxy = new Proxy({}, {
  set(target, prop, value) {
    target[prop] = value;
    // Missing return true!
  }
});

// CORRECT
const proxy = new Proxy({}, {
  set(target, prop, value) {
    target[prop] = value;
    return true; // Always return true from set
  }
});
```

Mistake 2: Infinite Loops with `set`

```javascript
// WRONG -- infinite loop!
const proxy = new Proxy({}, {
  set(target, prop, value) {
    proxy[prop] = value; // Writing to proxy triggers set again!
    return true;
  }
});

// CORRECT
const proxy = new Proxy({}, {
  set(target, prop, value) {
    target[prop] = value; // Write to target, not proxy
    return true;
```

```
  }
});
```

Mistake 3: Not Using Reflect

```javascript
// FRAGILE -- does not handle getters/setters correctly
const proxy = new Proxy(target, {
  get(target, prop) {
    return target[prop];
  }
});

// ROBUST -- handles all cases correctly
const proxy = new Proxy(target, {
  get(target, prop, receiver) {
    return Reflect.get(target, prop, receiver);
  }
});
```

Mistake 4: Proxying Everything

```javascript
// OVER-ENGINEERED -- a simple function would be better
const addProxy = new Proxy(
  function add(a, b) { return a + b; },
  {
    apply(target, thisArg, args) {
      return target.apply(thisArg, args);
    }
  }
);

// Just use the function directly!
function add(a, b) { return a + b; }
```

Best Practices

1. Use `Reflect` in your traps -- it preserves correct behavior for getters, setters, and inheritance.
2. Keep traps focused -- each proxy should do one thing well (validate OR log OR cache, not all three).

3. Compose proxies if needed -- wrap a proxy in another proxy for layered behavior.
4. Be aware of performance -- every property access through a proxy has overhead. Do not use proxies in performance-critical loops.
5. Document proxy behavior -- since proxies are invisible, other developers may not realize an object is proxied. Add comments or use clear naming conventions.
6. Test proxy edge cases -- test with `undefined` values, prototype chain lookups, `Symbol` properties, and `JSON.stringify`.

Quick Summary

The Proxy pattern places a middleman between the caller and the real object. This middleman intercepts operations and can add validation, logging, caching, access control, or any other behavior without modifying the original object.

```
                            +-- Validation Proxy
                            |
Object  -->  Proxy  -->  --+-- Logging Proxy
                            |
                            +-- Caching Proxy
                            |
                            +-- Access Control Proxy
```

ES6 gave us a native `Proxy` constructor with traps for every operation. Vue 3 uses this for its entire reactivity system. The `Reflect` object provides default implementations for each trap.

Key Points

- The Proxy pattern creates a stand-in that intercepts operations on a target object
- ES6 `Proxy` provides built-in support with traps for `get`, `set`, `has`, `deleteProperty`, `apply`, and more
- Validation proxies centralize data validation in one place
- Logging proxies add debugging without touching business logic
- Caching proxies store results of expensive operations
- Vue 3 reactivity is built on ES6 Proxies
- Always use `Reflect` inside trap handlers for correct behavior

- Proxies add overhead -- do not use them in performance-critical paths

---

Practice Questions

1. What is the difference between a Proxy's `target` and `handler`? What role does each play?
2. Why should you return `true` from the `set` trap? What happens if you forget?
3. Explain how Vue 3 uses Proxies for reactivity. What happens in the `get` and `set` traps?
4. What is `Reflect` and why should you use it inside proxy traps instead of directly accessing `target[property]`?
5. A colleague suggests using a Proxy to validate a single form field that is only used in one component. Is this a good idea? Why or why not?

Exercises

Exercise 1: Negative Array Indexing

Create a proxy that allows negative indexing for arrays, like Python. `arr[-1]` should return the last element, `arr[-2]` the second-to-last, and so on.

```javascript
function createNegativeArray(arr) {
  // Your code here
  // Hint: Use the get trap
  // Hint: Check if the property is a negative number
}

const arr = createNegativeArray([10, 20, 30, 40, 50]);
console.log(arr[-1]); // Should output: 50
console.log(arr[-2]); // Should output: 40
console.log(arr[0]);  // Should output: 10
```

Exercise 2: Read-Only Proxy

Create a proxy that makes any object completely read-only. It should throw an error if someone tries to write, delete, or define properties.

```javascript
function createReadOnly(target) {
  // Your code here
  // Hint: Use set, deleteProperty, and defineProperty traps
}

const config = createReadOnly({ apiKey: "abc123", timeout: 5000 });
console.log(config.apiKey);     // Should work: "abc123"
config.apiKey = "new";          // Should throw: "Cannot modify..."
delete config.timeout;          // Should throw: "Cannot delete..."
```

Exercise 3: Observable Object

Create a proxy that lets you register "watchers" on specific properties. When those properties change, the watchers are called with the old and new values.

```javascript
function createObservable(target) {
  // Your code here
  // Hint: Store watchers in a Map
  // Hint: Add a $watch method via the get trap
}

const user = createObservable({ name: "Alice", age: 28 });

user.$watch("name", (oldVal, newVal) => {
  console.log(`Name changed from "${oldVal}" to "${newVal}"`);
});

user.name = "Bob";
// Should output: Name changed from "Alice" to "Bob"
```

What Is Next?

Now that you know how to intercept and control access to objects with the Proxy pattern, the next chapter introduces the Decorator pattern. While the Proxy controls access to an existing object, the Decorator adds new behavior by wrapping functions or objects with additional functionality. You will learn how to build composable decorators like `withLogging`, `withRetry`, and `withCache` that stack on top of each other like layers of armor.

Chapter 9: The Decorator Pattern

What You Will Learn

- What the Decorator pattern is and how it differs from the Proxy pattern
- How higher-order functions act as decorators in JavaScript
- How to build practical decorators: `withLogging`, `withRetry`, `withCache`
- How to compose multiple decorators together
- How decorators apply to React components and hooks
- When decorators help and when they add unnecessary complexity

Why This Chapter Matters

Think of a plain donut. It tastes fine on its own. Now add chocolate glaze. Still a donut, but with extra behavior (chocolate). Add sprinkles on top of the glaze. Still a donut, still has chocolate, now with even more decoration.

You never changed the original donut. You wrapped it with additional layers. Each layer adds something new, and you can mix and match layers however you like.

The Decorator pattern works exactly this way. You take an existing function or object and wrap it with new behavior. The original stays unchanged. The wrapping is reusable. And you can stack decorators like layers on a donut.

This is one of the most practical patterns in frontend development. Need to add logging to a function? Decorate it. Need retry logic for an API call? Decorate it. Need caching? Decorate it. Need all three? Stack them.

The Problem

You have a function that works well. Now you need to add cross-cutting concerns -- logging, error handling, caching, timing, authentication checks -- without modifying the original function.

```javascript
// A simple API function
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  return response.json();
}

// Now you need:
// - Logging of every call
// - Retry on failure
// - Caching of results
// - Timing of how long it takes

// Do you add all of this inside fetchUser?
// That turns 3 lines into 30 lines.
// And every other API function needs the same treatment.
```

The Solution: Decorator Pattern

The Decorator pattern wraps an existing function or object with additional behavior. The wrapper has the same interface as the original, so callers do not know (or care) about the decoration.

```
Without decorators:
+--------------------+
| fetchUser          |
| - fetch logic      |
| - logging logic    |
| - retry logic      |
| - caching logic    |
| - timing logic     |
+--------------------+
Everything jammed into one function.

With decorators:
+---------------------------+
| withTiming                |
|  +----------------------+ |
|  | withCache            | |
|  |  +-----------------+  | |
|  |  | withRetry       |  | |
|  |  |  +------------+ |  | |
```

```
|  |  |  | withLogging| |  | |
|  |  |  |  +-------+ | |  | |
|  |  |  |  | fetch  | | |  | |
|  |  |  |  | User   | | |  | |
|  |  |  |  +-------+ | |  | |
|  |  |  +------------+ |  | |
|  |  +-----------------+  | |
|  +-----------------------+ |
+----------------------------+
Each concern is a separate, reusable layer.
```

Higher-Order Functions as Decorators

In JavaScript, functions are first-class citizens. A higher-order function (HOF) takes a function as an argument and returns a new function. This is the foundation of the Decorator pattern in JavaScript.

```javascript
// The simplest decorator
function shout(fn) {
  return function (...args) {
    const result = fn(...args);
    return typeof result === "string" ? result.toUpperCase() : result;
  };
}

function greet(name) {
  return `Hello, ${name}`;
}

const shoutGreet = shout(greet);

console.log(greet("Alice"));
// Output: Hello, Alice

console.log(shoutGreet("Alice"));
// Output: HELLO, ALICE
```

The original `greet` function is unchanged. `shoutGreet` is a decorated version. Same input, enhanced output.

Decorator 1: withLogging

Problem

You want to log every function call for debugging, but you do not want logging code inside your business logic.

Before: Logging Inside Business Logic

```javascript
function calculateTotal(items) {
  console.log("[calculateTotal] called with:", items);
  const start = Date.now();

  const total = items.reduce((sum, item) => sum + item.price * item.qty, 0);

  console.log("[calculateTotal] returned:", total);
  console.log("[calculateTotal] took:", Date.now() - start, "ms");
  return total;
}
```

After: Logging Decorator

```javascript
function withLogging(fn, label) {
  const name = label || fn.name || "anonymous";

  return function (...args) {
    console.log(`[${name}] called with:`, ...args);

    const result = fn.apply(this, args);

    if (result instanceof Promise) {
      return result.then((resolved) => {
        console.log(`[${name}] resolved:`, resolved);
        return resolved;
      }).catch((error) => {
        console.log(`[${name}] rejected:`, error.message);
        throw error;
      });
    }

    console.log(`[${name}] returned:`, result);
    return result;
  };
}

// Usage
function calculateTotal(items) {
  return items.reduce((sum, item) => sum + item.price * item.qty, 0);
}
```

```
const loggedCalculate = withLogging(calculateTotal);

loggedCalculate([
  { price: 10, qty: 2 },
  { price: 5, qty: 3 }
]);
// Output:
// [calculateTotal] called with: [{price: 10, qty: 2}, {price: 5, qty: 3}]
// [calculateTotal] returned: 35
```

The business logic stays clean. Logging is added from the outside.

Real-World Use Case: Debugging API Calls

```javascript
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error(`User ${id} not found`);
  return response.json();
}

async function fetchPosts(userId) {
  const response = await fetch(`/api/users/${userId}/posts`);
  return response.json();
}

// During development, add logging to all API calls
const api = {
  fetchUser: withLogging(fetchUser),
  fetchPosts: withLogging(fetchPosts)
};

// In production, use the originals
const api = {
  fetchUser,
  fetchPosts
};
```

Decorator 2: withRetry

Problem

Network requests fail. Servers return 500 errors. Connections drop. You need to retry failed operations, but retry logic is complex and should not be mixed into every function.

Before: Retry Inside Business Logic

```javascript
async function fetchData(url) {
  let lastError;
  for (let attempt = 1; attempt <= 3; attempt++) {
    try {
      const response = await fetch(url);
      if (!response.ok) throw new Error(`HTTP ${response.status}`);
      return await response.json();
    } catch (error) {
      lastError = error;
      console.log(`Attempt ${attempt} failed: ${error.message}`);
      if (attempt < 3) {
        await new Promise((r) => setTimeout(r, 1000 * attempt));
      }
    }
  }
  throw lastError;
}
```

After: Retry Decorator

```javascript
function withRetry(fn, options = {}) {
  const {
    maxAttempts = 3,
    delay = 1000,
    backoff = "exponential", // "fixed" or "exponential"
    onRetry = () => {}
  } = options;

  return async function (...args) {
    let lastError;

    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
      try {
        return await fn.apply(this, args);
      } catch (error) {
        lastError = error;

        if (attempt === maxAttempts) break;

        const waitTime =
          backoff === "exponential"
            ? delay * Math.pow(2, attempt - 1)
            : delay;

        onRetry({
          attempt,
          maxAttempts,
```

```
          error,
          waitTime
        });

        await new Promise((r) => setTimeout(r, waitTime));
      }
    }

    throw lastError;
  };
}

// Usage
async function fetchUser(id) {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  return response.json();
}

const resilientFetchUser = withRetry(fetchUser, {
  maxAttempts: 3,
  delay: 1000,
  backoff: "exponential",
  onRetry: ({ attempt, maxAttempts, error, waitTime }) => {
    console.log(
      `Attempt ${attempt}/${maxAttempts} failed: ${error.message}. ` +
      `Retrying in ${waitTime}ms...`
    );
  }
});

// This will automatically retry up to 3 times
const user = await resilientFetchUser(42);
```

```
Retry with exponential backoff:

Attempt 1 --> FAIL --> wait 1s
Attempt 2 --> FAIL --> wait 2s
Attempt 3 --> FAIL --> throw error

                  time -->
    |-----|
    Att 1  1s wait
               |-----|
               Att 2  2s wait
                          |-----|
                          Att 3  give up
```

Decorator 3: withCache

Problem

The same function is called with the same arguments multiple times. Each call is expensive (network request, heavy computation). You want to cache results.

Before: Caching Inside Function

```javascript
const cache = new Map();

async function fetchUserProfile(userId) {
  if (cache.has(userId)) {
    const entry = cache.get(userId);
    if (Date.now() - entry.timestamp < 60000) {
      return entry.data;
    }
  }

  const response = await fetch(`/api/users/${userId}`);
  const data = await response.json();
  cache.set(userId, { data, timestamp: Date.now() });
  return data;
}
```

After: Caching Decorator

```javascript
function withCache(fn, options = {}) {
  const {
    ttl = 60000,      // Time to live in milliseconds
    maxSize = 100,    // Maximum cache entries
    keyFn = (...args) => JSON.stringify(args)
  } = options;

  const cache = new Map();

  function decorated(...args) {
    const key = keyFn(...args);

    // Check cache
    if (cache.has(key)) {
      const entry = cache.get(key);
      if (Date.now() - entry.timestamp < ttl) {
        return entry.value;
      }
```

```
      cache.delete(key); // Expired
    }

    // Call original function
    const result = fn.apply(this, args);

    // Handle async results
    if (result instanceof Promise) {
      return result.then((resolved) => {
        storeInCache(key, resolved);
        return resolved;
      });
    }

    storeInCache(key, result);
    return result;
  }

  function storeInCache(key, value) {
    // Evict oldest if at max size
    if (cache.size >= maxSize) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
    }
    cache.set(key, { value, timestamp: Date.now() });
  }

  // Expose cache controls
  decorated.clearCache = () => cache.clear();
  decorated.cacheSize = () => cache.size;

  return decorated;
}

// Usage
async function fetchUser(id) {
  console.log(`Fetching user ${id} from API...`);
  const response = await fetch(`/api/users/${id}`);
  return response.json();
}

const cachedFetchUser = withCache(fetchUser, { ttl: 30000 });

await cachedFetchUser(1);
// Output: Fetching user 1 from API...

await cachedFetchUser(1);
// No output -- served from cache!

await cachedFetchUser(2);
// Output: Fetching user 2 from API...

console.log(cachedFetchUser.cacheSize());
// Output: 2
```

```
cachedFetchUser.clearCache();
```

Composing Decorators

The real power of decorators is composition. You can stack multiple decorators to build complex behavior from simple pieces.

```javascript
// Each decorator does one thing
const resilientCachedLoggedFetchUser =
  withLogging(
    withRetry(
      withCache(fetchUser, { ttl: 30000 }),
      { maxAttempts: 3 }
    ),
    "fetchUser"
  );

// When called:
// 1. withLogging logs the call
// 2. withRetry handles failures
// 3. withCache checks the cache first
// 4. fetchUser makes the actual request (if not cached)
```

A Compose Utility

Reading nested function calls is hard. A `compose` utility makes it cleaner:

```javascript
function compose(...decorators) {
  return function (fn) {
    return decorators.reduceRight(
      (decorated, decorator) => decorator(decorated),
      fn
    );
  };
}

// Usage
const enhance = compose(
  (fn) => withLogging(fn, "fetchUser"),
  (fn) => withRetry(fn, { maxAttempts: 3 }),
  (fn) => withCache(fn, { ttl: 30000 })
);
```

```
const enhancedFetchUser = enhance(fetchUser);
```

A Pipeline Utility (Left to Right)

Some people prefer reading left to right:

```javascript
function pipe(...decorators) {
  return function (fn) {
    return decorators.reduce(
      (decorated, decorator) => decorator(decorated),
      fn
    );
  };
}

const enhance = pipe(
  (fn) => withCache(fn, { ttl: 30000 }),
  (fn) => withRetry(fn, { maxAttempts: 3 }),
  (fn) => withLogging(fn, "fetchUser")
);

const enhancedFetchUser = enhance(fetchUser);
```

```
Decorator composition (reading from inside out):

  withLogging( withRetry( withCache( fetchUser ) ) )
       |            |          |         |
       |            |          |    Original function
       |            |          |
       |            |    Adds caching layer
       |            |
       |      Adds retry layer
       |
  Adds logging layer

Call flow:
  caller --> logging --> retry --> cache --> fetchUser
  caller <-- logging <-- retry <-- cache <-- fetchUser
```

Decorator 4: withTiming

A quick and useful decorator for performance monitoring:

```javascript
function withTiming(fn, label) {
  const name = label || fn.name || "anonymous";

  return async function (...args) {
    const start = performance.now();

    try {
      const result = await fn.apply(this, args);
      const duration = (performance.now() - start).toFixed(2);
      console.log(`[Timer] ${name} completed in ${duration}ms`);
      return result;
    } catch (error) {
      const duration = (performance.now() - start).toFixed(2);
      console.log(`[Timer] ${name} failed after ${duration}ms`);
      throw error;
    }
  };
}

// Usage
const timedFetch = withTiming(fetch, "fetch");
await timedFetch("https://api.example.com/data");
// Output: [Timer] fetch completed in 234.56ms
```

Decorator 5: withDebounce

Prevent a function from being called too frequently:

```javascript
function withDebounce(fn, delay = 300) {
  let timeoutId;

  return function (...args) {
    clearTimeout(timeoutId);

    return new Promise((resolve) => {
      timeoutId = setTimeout(() => {
        resolve(fn.apply(this, args));
      }, delay);
    });
  };
}

// Usage: Search input that only fires after user stops typing
async function searchAPI(query) {
  const response = await fetch(`/api/search?q=${query}`);
  return response.json();
```

```
}

const debouncedSearch = withDebounce(searchAPI, 300);

// Called rapidly (e.g., user typing "react")
debouncedSearch("r");
debouncedSearch("re");
debouncedSearch("rea");
debouncedSearch("reac");
debouncedSearch("react");
// Only the last call ("react") actually fires, after 300ms of silence
```

Decorators in React

Decorating Event Handlers

```javascript
function preventDoubleClick(handler, delay = 1000) {
  let lastClick = 0;

  return function (...args) {
    const now = Date.now();
    if (now - lastClick < delay) return;
    lastClick = now;
    return handler.apply(this, args);
  };
}

function SubmitButton({ onSubmit }) {
  const safeSubmit = preventDoubleClick(onSubmit);

  return <button onClick={safeSubmit}>Submit Order</button>;
}
```

Decorating Custom Hooks

```javascript
// A decorator for any async hook to add loading/error states
function withAsyncState(useDataHook) {
  return function (...args) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);
```

```
    useEffect(() => {
      let cancelled = false;
      setLoading(true);

      useDataHook(...args)
        .then((result) => {
          if (!cancelled) {
            setData(result);
            setLoading(false);
          }
        })
        .catch((err) => {
          if (!cancelled) {
            setError(err);
            setLoading(false);
          }
        });

      return () => { cancelled = true; };
    }, args);

    return { data, loading, error };
  };
}
```

Decorator vs Proxy: What Is the Difference?

These two patterns look similar but serve different purposes:

```
+------------------+-----------------------------------+----------------------------+
| Aspect           | Proxy                             | Decorator                  |
+------------------+-----------------------------------+----------------------------+
| Purpose          | Control ACCESS to an object       | Add BEHAVIOR to a function |
| Mechanism        | Intercepts operations             | Wraps with new function    |
| Interface        | Same as target (transparent)      | Same as original function  |
| Focus            | The object and its properties     | The function and its result|
| JS Support       | ES6 Proxy constructor             | Higher-order functions     |
| Use case         | Validation, access control        | Logging, retry, caching    |
+------------------+-----------------------------------+----------------------------+
```

Simple rule: Proxies control objects. Decorators enhance functions.

When to Use the Decorator Pattern

```
+-----------------------------------------------+
|           USE DECORATORS WHEN:                |
+-----------------------------------------------+
|                                               |
|  * You need to add cross-cutting concerns     |
|    (logging, timing, retry, caching)          |
|  * The same concern applies to many functions |
|  * You want to compose behaviors flexibly     |
|  * You want to toggle behavior on/off easily  |
|  * You need to keep business logic clean      |
|  * Different environments need different      |
|    behavior (dev vs production)               |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|       DO NOT USE DECORATORS WHEN:             |
+-----------------------------------------------+
|                                               |
|  * The function is only used in one place     |
|  * Adding the behavior inline is clearer      |
|  * The decorator stack gets too deep (3+)     |
|    and becomes hard to debug                  |
|  * The function signature changes with        |
|    decoration (breaks the pattern)            |
|  * Simple conditional logic would suffice     |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Losing `this` Context

```javascript
// WRONG -- "this" is lost
function withLogging(fn) {
  return function (...args) {
    console.log("called");
    return fn(...args); // "this" is undefined!
  };
}

// CORRECT -- preserve "this"
```

```
function withLogging(fn) {
  return function (...args) {
    console.log("called");
    return fn.apply(this, args); // "this" is preserved
  };
}
```

Mistake 2: Hiding the Original Function Name

```javascript
// WRONG -- debugger shows "anonymous"
function withLogging(fn) {
  return function (...args) {
    console.log(`${fn.name} called`);
    return fn.apply(this, args);
  };
}

// BETTER -- preserve the function name
function withLogging(fn) {
  const wrapper = function (...args) {
    console.log(`${fn.name} called`);
    return fn.apply(this, args);
  };
  Object.defineProperty(wrapper, "name", { value: fn.name });
  return wrapper;
}
```

Mistake 3: Too Many Layers

```javascript
// TOO MANY DECORATORS -- hard to debug
const fn = withAuth(
  withLogging(
    withTiming(
      withRetry(
        withCache(
          withValidation(
            withThrottle(fetchUser)
          )
        )
      )
    )
  )
);

// When something goes wrong, which layer caused it?
// Keep decorator stacks to 2-3 layers maximum
```

Mistake 4: Not Handling Both Sync and Async

```javascript
// WRONG -- only works for sync functions
function withLogging(fn) {
  return function (...args) {
    const result = fn.apply(this, args);
    console.log("Result:", result); // Logs a Promise object!
    return result;
  };
}

// CORRECT -- handles both sync and async
function withLogging(fn) {
  return function (...args) {
    const result = fn.apply(this, args);
    if (result instanceof Promise) {
      return result.then((val) => {
        console.log("Result:", val);
        return val;
      });
    }
    console.log("Result:", result);
    return result;
  };
}
```

Best Practices

1. Preserve the function interface -- the decorated function should accept the same arguments and return the same type as the original.
2. Preserve `this` context -- always use `fn.apply(this, args)` or `fn.call(this, ...args)` inside decorators.
3. Handle both sync and async -- check if the result is a Promise and handle both cases.
4. Keep decorators focused -- one decorator, one concern. Do not build a "super decorator" that does everything.
5. Limit decorator depth -- stacking more than 3 decorators makes debugging difficult. If you need more, consider a different approach.
6. Make decorators configurable -- accept options objects so users can customize behavior.

Quick Summary

The Decorator pattern wraps functions with additional behavior without modifying the original. Higher-order functions are the natural way to implement decorators in JavaScript. Common decorators include `withLogging`, `withRetry`, `withCache`, `withTiming`, and `withDebounce`. Decorators can be composed together, and the order of composition matters.

```
Original:     fetchUser(id)
Decorated:    withLogging(withRetry(withCache(fetchUser)))(id)

Same input, same output, extra behavior.
```

Key Points

- The Decorator pattern adds behavior to functions without modifying them
- Higher-order functions are the JavaScript way to implement decorators
- Common decorators: `withLogging`, `withRetry`, `withCache`, `withTiming`, `withDebounce`
- Decorators can be composed (stacked) for complex behavior
- Always preserve `this` context with `fn.apply(this, args)`
- Handle both synchronous and asynchronous functions
- Keep decorator stacks shallow (2-3 layers) for debuggability
- Proxy controls access to objects; Decorator adds behavior to functions

---

Practice Questions

1. What is the difference between the Decorator pattern and the Proxy pattern? When would you choose one over the other?
2. Why is it important to use `fn.apply(this, args)` inside a decorator instead of `fn(...args)`?
3. You have a `withRetry` decorator with exponential backoff. Explain the delay sequence for `maxAttempts: 4` with `delay: 1000`.
4. How would you compose `withLogging`, `withCache`, and `withRetry` so that logging happens on the outermost layer and caching on the innermost layer? Why does this order matter?

5. A colleague wants to add a decorator that changes the return type of a function (e.g., wraps the result in an object). Is this a good use of the Decorator pattern? Why or why not?

Exercises

Exercise 1: withThrottle Decorator

Create a `withThrottle` decorator that ensures a function is called at most once every N milliseconds. Unlike debounce (which waits for silence), throttle runs immediately and then ignores calls for the next N ms.

```javascript
function withThrottle(fn, interval = 300) {
  // Your code here
}

const throttledLog = withThrottle(console.log, 1000);
throttledLog("first");  // Logs immediately
throttledLog("second"); // Ignored (within 1000ms)
throttledLog("third");  // Ignored (within 1000ms)
// After 1000ms...
throttledLog("fourth"); // Logs
```

Exercise 2: withAuth Decorator

Create a `withAuth` decorator that checks if a user is authenticated before calling the function. If not authenticated, it should throw an error or redirect.

```javascript
function withAuth(fn, getUser) {
  // Your code here
  // Hint: getUser() returns the current user or null
}

const getUser = () => ({ name: "Alice", role: "admin" }); // or null
const protectedFetch = withAuth(fetchSensitiveData, getUser);
```

Exercise 3: Composing Decorators

Build a `createAPI` function that takes a base fetcher function and an options object, then applies the appropriate decorators based on the options.

```javascript
function createAPI(baseFetcher, options = {}) {
  // Your code here
  // Hint: Conditionally apply decorators based on options
}

const api = createAPI(fetchUser, {
  cache: { ttl: 30000 },
  retry: { maxAttempts: 3 },
  logging: true,
  timing: true
});
```

What Is Next?

You now know how to add behavior to functions by wrapping them in decorators. The next chapter introduces the Facade pattern, which takes the opposite approach. Instead of adding complexity layer by layer, the Facade hides complexity behind a simple interface. You will learn how to build clean, simple APIs that shield callers from the messy details underneath.

Chapter 10: The Facade Pattern

What You Will Learn

- What the Facade pattern is and why simplicity matters
- How to hide complex subsystems behind simple interfaces
- How to build an API facade that unifies multiple endpoints
- How to build SDK wrappers that tame third-party complexity
- How to build a form submission facade that handles validation, formatting, and API calls
- When to use a facade and when direct access is better

Why This Chapter Matters

Think about driving a car. You turn the steering wheel, press the gas pedal, and hit the brakes. Three simple controls.

Behind those controls, hundreds of systems work together: the engine management computer, fuel injection, transmission, power steering pump, brake hydraulics, ABS sensors, traction control. You do not need to know about any of them. The steering wheel, gas pedal, and brake pedal form a facade -- a simple interface that hides enormous complexity.

In frontend development, complexity hides everywhere. A "simple" form submission might involve validation, data transformation, file uploads, API calls, error handling, retry logic, and analytics tracking. A "simple" analytics event might require initializing three different SDKs, formatting data differently for each, and handling failures silently.

The Facade pattern gives your code a steering wheel. It provides one simple function that handles all the complexity underneath so callers do not need to think about it.

The Problem

You need to interact with a complex system -- multiple APIs, third-party libraries, or intricate business logic -- and the complexity is leaking into every part of your codebase.

```javascript
// A developer wants to submit a contact form.
// Here is what they have to deal with:

async function handleSubmit(formData) {
  // 1. Validate every field manually
  if (!formData.name || formData.name.trim().length < 2) {
    throw new Error("Name is required");
  }
  if (!formData.email || !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) {
    throw new Error("Valid email is required");
  }
  if (!formData.message || formData.message.trim().length < 10) {
    throw new Error("Message must be at least 10 characters");
  }

  // 2. Sanitize the data
  const sanitized = {
    name: formData.name.trim(),
    email: formData.email.toLowerCase().trim(),
    message: formData.message.trim(),
    submittedAt: new Date().toISOString()
  };

  // 3. Get auth token
  const token = localStorage.getItem("auth_token");
  if (!token) {
    throw new Error("Not authenticated");
  }

  // 4. Make the API call with proper headers
  const response = await fetch("https://api.example.com/contact", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`,
      "X-Request-ID": crypto.randomUUID()
    },
    body: JSON.stringify(sanitized)
  });

  // 5. Handle different error codes
  if (response.status === 401) {
    localStorage.removeItem("auth_token");
    throw new Error("Session expired. Please log in again.");
  }
  if (response.status === 429) {
```

```
    throw new Error("Too many requests. Please wait.");
  }
  if (!response.ok) {
    const error = await response.json();
    throw new Error(error.message || "Submission failed");
  }

  // 6. Track analytics
  if (window.gtag) {
    window.gtag("event", "form_submit", {
      event_category: "contact",
      event_label: "success"
    });
  }

  return response.json();
}
```

Every form in the app repeats similar patterns. Different developers do it differently. Bugs hide in the details.

The Solution: Facade Pattern

The Facade pattern provides a unified, simple interface to a complex subsystem. It does not eliminate the complexity -- it organizes and hides it.

```
Without a Facade:

+--------+     +-- Validator
|        | --> +-- Sanitizer
| Caller | --> +-- Auth Service
|        | --> +-- HTTP Client
+--------+     +-- Error Handler
               +-- Analytics

The caller must know about EVERY subsystem.

With a Facade:

+--------+     +-----------+     +-- Validator
|        |     |           | --> +-- Sanitizer
| Caller | --> |  Facade   | --> +-- Auth Service
|        |     |           | --> +-- HTTP Client
+--------+     +-----------+     +-- Error Handler
                                 +-- Analytics
```

```
The caller only knows about the Facade.
```

Facade 1: API Facade

Problem

Your app talks to a REST API with many endpoints. Each endpoint has different URL patterns, HTTP methods, headers, and data formats. This complexity is spread across your entire codebase.

Before: Raw API Calls Everywhere

```javascript
// In UserProfile component
const response = await fetch("https://api.example.com/v2/users/42", {
  headers: { "Authorization": `Bearer ${getToken()}` }
});
const user = await response.json();

// In Settings component
const response = await fetch("https://api.example.com/v2/users/42", {
  method: "PATCH",
  headers: {
    "Authorization": `Bearer ${getToken()}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ theme: "dark" })
});

// In Admin component
const response = await fetch("https://api.example.com/v2/users?role=admin", {
  headers: { "Authorization": `Bearer ${getToken()}` }
});
const admins = await response.json();
```

After: API Facade

```javascript
// api.js -- The Facade

const BASE_URL = "https://api.example.com/v2";
```

```
function getHeaders() {
  const headers = { "Content-Type": "application/json" };
  const token = localStorage.getItem("auth_token");
  if (token) {
    headers["Authorization"] = `Bearer ${token}`;
  }
  return headers;
}

async function handleResponse(response) {
  if (response.status === 401) {
    localStorage.removeItem("auth_token");
    window.location.href = "/login";
    throw new Error("Session expired");
  }

  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || `HTTP Error ${response.status}`);
  }

  // Handle 204 No Content
  if (response.status === 204) return null;

  return response.json();
}

async function request(endpoint, options = {}) {
  const { method = "GET", body, params } = options;

  let url = `${BASE_URL}${endpoint}`;
  if (params) {
    const searchParams = new URLSearchParams(params);
    url += `?${searchParams}`;
  }

  const response = await fetch(url, {
    method,
    headers: getHeaders(),
    body: body ? JSON.stringify(body) : undefined
  });

  return handleResponse(response);
}

// Public API -- clean, simple, consistent
const api = {
  users: {
    getById: (id) => request(`/users/${id}`),

    list: (filters) => request("/users", { params: filters }),

    update: (id, data) =>
      request(`/users/${id}`, { method: "PATCH", body: data }),
```

```
    delete: (id) =>
      request(`/users/${id}`, { method: "DELETE" }),

    create: (data) =>
      request("/users", { method: "POST", body: data })
  },

  posts: {
    getByUser: (userId) => request(`/users/${userId}/posts`),

    create: (userId, data) =>
      request(`/users/${userId}/posts`, { method: "POST", body: data }),

    delete: (userId, postId) =>
      request(`/users/${userId}/posts/${postId}`, { method: "DELETE" })
  }
};

export default api;
```

Now every component uses the same clean interface:

```javascript
// In UserProfile component
const user = await api.users.getById(42);

// In Settings component
await api.users.update(42, { theme: "dark" });

// In Admin component
const admins = await api.users.list({ role: "admin" });
```

No more remembering URLs, headers, methods, or error handling. The facade handles all of it.

Facade 2: SDK Wrapper

Problem

You use a third-party analytics SDK. Its API is verbose and complex. You also want the ability to swap it out later without rewriting your entire app.

Before: SDK Calls Scattered Everywhere

```javascript
// In CheckoutPage.jsx
import amplitude from "amplitude-js";

function handlePurchase(cart) {
  amplitude.getInstance().logEvent("purchase_completed", {
    total: cart.total,
    itemCount: cart.items.length,
    currency: "USD",
    items: cart.items.map((i) => ({
      id: i.id,
      name: i.name,
      price: i.price,
      quantity: i.quantity
    }))
  });
}

// In SearchPage.jsx
import amplitude from "amplitude-js";

function handleSearch(query, results) {
  amplitude.getInstance().logEvent("search_performed", {
    query: query,
    resultCount: results.length,
    timestamp: Date.now()
  });
}

// Amplitude is referenced directly in 50+ files.
// Want to switch to Mixpanel? Rewrite everything.
```

After: Analytics Facade

```javascript
// analytics.js -- The Facade

class Analytics {
  constructor() {
    this.providers = [];
    this.initialized = false;
  }

  init(config = {}) {
    // Initialize all providers behind the facade
    if (config.amplitude) {
      const amplitude = require("amplitude-js");
      amplitude.getInstance().init(config.amplitude.apiKey);
      this.providers.push({
        name: "amplitude",
        track: (event, props) => {
          amplitude.getInstance().logEvent(event, props);
```

```
        },
        identify: (userId, traits) => {
          amplitude.getInstance().setUserId(userId);
          amplitude.getInstance().setUserProperties(traits);
        }
      });
    }

    if (config.mixpanel) {
      const mixpanel = require("mixpanel-browser");
      mixpanel.init(config.mixpanel.token);
      this.providers.push({
        name: "mixpanel",
        track: (event, props) => mixpanel.track(event, props),
        identify: (userId, traits) => {
          mixpanel.identify(userId);
          mixpanel.people.set(traits);
        }
      });
    }

    this.initialized = true;
  }

  track(event, properties = {}) {
    if (!this.initialized) {
      console.warn("Analytics not initialized");
      return;
    }

    const enrichedProps = {
      ...properties,
      timestamp: Date.now(),
      page: window.location.pathname
    };

    this.providers.forEach((provider) => {
      try {
        provider.track(event, enrichedProps);
      } catch (error) {
        console.error(
          `Analytics error (${provider.name}):`,
          error.message
        );
      }
    });
  }

  identify(userId, traits = {}) {
    this.providers.forEach((provider) => {
      try {
        provider.identify(userId, traits);
      } catch (error) {
        console.error(
          `Analytics identify error (${provider.name}):`,
```

```
          error.message
        );
      }
    });
  }
}

// Export a singleton instance
const analytics = new Analytics();
export default analytics;
```

Now the entire app uses one simple interface:

```javascript
// Initialize once at app startup
analytics.init({
  amplitude: { apiKey: "amp-key-123" },
  mixpanel: { token: "mp-token-456" }
});

// In CheckoutPage.jsx
analytics.track("purchase_completed", {
  total: cart.total,
  itemCount: cart.items.length
});

// In SearchPage.jsx
analytics.track("search_performed", {
  query: query,
  resultCount: results.length
});

// Want to switch from Amplitude to Mixpanel?
// Change ONE file: analytics.js
// Zero changes in your 50+ components.
```

Facade 3: Complex Form Submission

Problem

Submitting a form involves many steps: validation, sanitization, file uploads, API calls, optimistic UI updates, error handling, and analytics. You do not want every form to handle all of this.

Before: Every Step Handled Manually

```javascript
async function handleContactSubmit(formData) {
  // 30+ lines of validation, sanitization,
  // API calls, error handling...
  // (See the "Problem" section above)
}

async function handleProfileSubmit(formData) {
  // Another 30+ lines of similar logic
  // with slight differences...
}

async function handleFeedbackSubmit(formData) {
  // And another 30+ lines...
}
```

After: Form Submission Facade

```javascript
// formFacade.js

const validators = {
  required: (value, fieldName) =>
    !value || (typeof value === "string" && !value.trim())
      ? `${fieldName} is required`
      : null,

  email: (value) =>
    value && !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)
      ? "Invalid email address"
      : null,

  minLength: (min) => (value, fieldName) =>
    value && value.length < min
      ? `${fieldName} must be at least ${min} characters`
      : null,

  maxLength: (max) => (value, fieldName) =>
    value && value.length > max
      ? `${fieldName} must be at most ${max} characters`
      : null
};

function createFormFacade(config) {
  const {
    endpoint,
    method = "POST",
    fieldRules = {},
    transform = (data) => data,
    onSuccess = () => {},
    onError = () => {},
```

```
    analyticsEvent = null
  } = config;

  return async function submitForm(formData) {
    // Step 1: Validate
    const errors = {};
    for (const [field, rules] of Object.entries(fieldRules)) {
      for (const rule of rules) {
        const error = rule(formData[field], field);
        if (error) {
          errors[field] = error;
          break;
        }
      }
    }

    if (Object.keys(errors).length > 0) {
      const error = new Error("Validation failed");
      error.fieldErrors = errors;
      onError(error);
      throw error;
    }

    // Step 2: Transform data
    const transformed = transform(formData);

    // Step 3: Submit
    try {
      const response = await api.request(endpoint, {
        method,
        body: transformed
      });

      // Step 4: Track analytics
      if (analyticsEvent) {
        analytics.track(analyticsEvent, {
          success: true,
          fields: Object.keys(formData)
        });
      }

      // Step 5: Call success handler
      onSuccess(response);
      return response;
    } catch (error) {
      if (analyticsEvent) {
        analytics.track(analyticsEvent, {
          success: false,
          error: error.message
        });
      }

      onError(error);
      throw error;
    }
```

```
  };
}

// Usage -- define forms declaratively
const submitContactForm = createFormFacade({
  endpoint: "/contact",
  fieldRules: {
    name: [validators.required, validators.minLength(2)],
    email: [validators.required, validators.email],
    message: [validators.required, validators.minLength(10)]
  },
  transform: (data) => ({
    ...data,
    name: data.name.trim(),
    email: data.email.toLowerCase().trim(),
    message: data.message.trim()
  }),
  analyticsEvent: "contact_form_submitted",
  onSuccess: () => showToast("Message sent!"),
  onError: (err) => showToast(err.message, "error")
});

const submitFeedbackForm = createFormFacade({
  endpoint: "/feedback",
  fieldRules: {
    rating: [validators.required],
    comment: [validators.minLength(5)]
  },
  analyticsEvent: "feedback_submitted"
});
```

Now each form component is clean:

```javascript
function ContactForm() {
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    message: ""
  });
  const [errors, setErrors] = useState({});

  async function handleSubmit(e) {
    e.preventDefault();
    try {
      await submitContactForm(formData);
    } catch (error) {
      if (error.fieldErrors) {
        setErrors(error.fieldErrors);
      }
    }
  }
```

```
  return (
    <form onSubmit={handleSubmit}>
      <input
        value={formData.name}
        onChange={(e) => setFormData({ ...formData, name: e.target.value })}
      />
      {errors.name && <span className="error">{errors.name}</span>}
      {/* ... other fields ... */}
      <button type="submit">Send</button>
    </form>
  );
}
```

Facade 4: Storage Facade

Abstract away the differences between `localStorage`, `sessionStorage`, and cookies:

```javascript
const storage = {
  get(key, defaultValue = null) {
    try {
      const item = localStorage.getItem(key);
      if (item === null) return defaultValue;
      return JSON.parse(item);
    } catch {
      return defaultValue;
    }
  },

  set(key, value, options = {}) {
    try {
      const serialized = JSON.stringify(value);
      localStorage.setItem(key, serialized);

      // Optionally set expiry
      if (options.ttl) {
        const expiresAt = Date.now() + options.ttl;
        localStorage.setItem(`${key}__expires`, expiresAt.toString());
      }
    } catch (error) {
      console.warn("Storage write failed:", error.message);
    }
  },

  remove(key) {
    localStorage.removeItem(key);
    localStorage.removeItem(`${key}__expires`);
```

```
  },

  isExpired(key) {
    const expiresAt = localStorage.getItem(`${key}__expires`);
    if (!expiresAt) return false;
    return Date.now() > Number(expiresAt);
  }
};

// Usage -- simple and consistent
storage.set("user", { name: "Alice", role: "admin" }, { ttl: 3600000 });

const user = storage.get("user");
console.log(user);
// Output: { name: 'Alice', role: 'admin' }

storage.get("nonexistent", "fallback");
// Output: "fallback"
```

Facade in React: Custom Hooks as Facades

In React, custom hooks often serve as facades. They hide complexity behind a simple interface:

```javascript
// This hook is a facade over:
// - fetch API
// - loading state
// - error handling
// - caching
// - retry logic
// - abort controller

function useAPI(endpoint) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controller = new AbortController();
    let retries = 0;

    async function fetchData() {
      setLoading(true);
      setError(null);

      while (retries < 3) {
```

```
        try {
          const response = await fetch(endpoint, {
            signal: controller.signal,
            headers: { Authorization: `Bearer ${getToken()}` }
          });

          if (!response.ok) throw new Error(`HTTP ${response.status}`);
          const json = await response.json();
          setData(json);
          setLoading(false);
          return;
        } catch (err) {
          if (err.name === "AbortError") return;
          retries++;
          if (retries >= 3) {
            setError(err);
            setLoading(false);
          } else {
            await new Promise((r) => setTimeout(r, 1000 * retries));
          }
        }
      }
    }

    fetchData();
    return () => controller.abort();
  }, [endpoint]);

  return { data, loading, error };
}

// Components see only the simple interface:
function UserProfile({ userId }) {
  const { data: user, loading, error } = useAPI(`/api/users/${userId}`);

  if (loading) return <Spinner />;
  if (error) return <ErrorMessage error={error} />;
  return <div>{user.name}</div>;
}
```

```
What the component sees:        What the hook hides:

  { data, loading, error }       +-- fetch API calls
                                 +-- AbortController
                                 +-- Authentication headers
                                 +-- Retry logic (3 attempts)
                                 +-- Exponential backoff
                                 +-- Error handling
                                 +-- Loading state management
                                 +-- Cleanup on unmount
```

When to Use the Facade Pattern

```
+-----------------------------------------------+
|           USE FACADE WHEN:                    |
+-----------------------------------------------+
|                                               |
|  * A subsystem has many parts and callers     |
|    only need a subset of functionality        |
|  * You want to decouple your code from a      |
|    third-party library (so you can swap it)   |
|  * The same complex operation is repeated     |
|    across many parts of the codebase          |
|  * You want to provide different interfaces   |
|    for different audiences (simple vs advanced)|
|  * Complex initialization or configuration    |
|    needs to happen before using a system      |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|       DO NOT USE FACADE WHEN:                 |
+-----------------------------------------------+
|                                               |
|  * The subsystem is already simple enough     |
|  * Callers genuinely need fine-grained        |
|    control over the subsystem                 |
|  * The facade would just be a pass-through    |
|    with no added value                        |
|  * You are building a facade "just in case"   |
|    you might swap libraries (YAGNI)           |
|  * The facade hides important behavior that   |
|    callers need to understand                 |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: The "God Facade"

```javascript
// WRONG -- facade does too much
const app = {
  login() { /* ... */ },
  logout() { /* ... */ },
  fetchUser() { /* ... */ },
```

```
  createPost() { /* ... */ },
  sendEmail() { /* ... */ },
  processPayment() { /* ... */ },
  generateReport() { /* ... */ },
  uploadFile() { /* ... */ },
  // 50 more methods...
};

// CORRECT -- separate facades for separate concerns
const auth = { login, logout, refreshToken };
const users = { get, list, update, create };
const posts = { get, list, create, delete: remove };
```

Mistake 2: Hiding Errors

```javascript
// WRONG -- swallowing errors silently
const api = {
  async fetchUser(id) {
    try {
      const res = await fetch(`/api/users/${id}`);
      return res.json();
    } catch {
      return null; // Caller has no idea something went wrong
    }
  }
};

// CORRECT -- let errors propagate or surface them clearly
const api = {
  async fetchUser(id) {
    const res = await fetch(`/api/users/${id}`);
    if (!response.ok) {
      throw new APIError(`Failed to fetch user ${id}`, res.status);
    }
    return res.json();
  }
};
```

Mistake 3: Premature Facade

```javascript
// WRONG -- creating a facade for one use
function sendEmail(to, subject, body) {
  // This just wraps one function call
  return emailService.send({ to, subject, body });
}
```

```
// If the facade adds no value, skip it.
// Use emailService.send() directly.
```

Mistake 4: Leaking Abstractions

```javascript
// WRONG -- facade exposes internal details
const analytics = {
  // Callers should not need to know about Amplitude
  getAmplitudeInstance() {
    return amplitude.getInstance();
  },
  track(event, props) {
    this.getAmplitudeInstance().logEvent(event, props);
  }
};

// CORRECT -- keep internals hidden
const analytics = {
  track(event, props) {
    amplitude.getInstance().logEvent(event, props);
  }
};
```

Best Practices

1. Keep the facade interface minimal -- expose only what callers actually need. You can always add methods later.
2. Do not replace access to the subsystem -- facades should simplify access, not prevent direct access when it is genuinely needed.
3. Handle errors appropriately -- facades should translate low-level errors into meaningful errors, not swallow them.
4. Make facades testable -- accept dependencies through constructor or configuration rather than hardcoding them.
5. One facade per concern -- do not build a "mega facade" that covers everything. Have separate facades for auth, analytics, API, etc.
6. Document what is hidden -- since the facade hides complexity, document what it does under the hood so future developers can understand it.

Quick Summary

The Facade pattern provides a simple interface to a complex subsystem. It does not change how the subsystem works -- it just provides an easier way to use it. Common uses include API facades, SDK wrappers, form submission handlers, and storage abstractions. In React, custom hooks often serve as facades that hide complex state management, side effects, and API interactions behind a simple return value.

```
Complex subsystem:        Facade:
  Validator                submitForm(data)
  Sanitizer                  --> validates
  Auth Service               --> sanitizes
  HTTP Client                --> authenticates
  Error Handler              --> submits
  Analytics                  --> tracks
                             --> handles errors
```

Key Points

- The Facade pattern hides complexity behind a simple interface
- Facades do not add new behavior -- they organize existing behavior
- API facades centralize endpoint URLs, headers, auth, and error handling
- SDK wrappers let you swap third-party libraries without rewriting your app
- Form submission facades unify validation, transformation, and API calls
- In React, custom hooks are natural facades
- Keep facades minimal and focused on one concern
- Do not swallow errors inside facades -- translate and surface them

---

Practice Questions

1. How does the Facade pattern differ from the Decorator pattern? Both involve wrapping -- what is the key distinction?
2. Your team uses three different analytics providers. Explain how a facade would help, and what interface you would expose.

3. A colleague argues that facades create an unnecessary layer of abstraction. When is this argument valid? When is it wrong?
4. You are building a facade for a payment processing system. What methods would you expose? What would you hide?
5. How do React custom hooks act as facades? Give an example of a hook that hides at least three sources of complexity.

Exercises

Exercise 1: Notification Facade

Build a facade that unifies email, SMS, and push notifications behind a single `notify()` function. The facade should route to the correct service based on the notification type.

```javascript
function createNotificationFacade(services) {
  // Your code here
}

const notify = createNotificationFacade({
  email: emailService,
  sms: smsService,
  push: pushService
});

await notify({
  type: "email",
  to: "alice@example.com",
  subject: "Welcome!",
  body: "Thanks for signing up."
});
```

Exercise 2: Media Player Facade

Create a facade for a media player that supports audio and video. The facade should handle initialization, playback controls, volume, and cleanup behind simple methods like `play()`, `pause()`, `setVolume()`, and `destroy()`.

```javascript
function createMediaPlayer(element, source) {
  // Your code here
  // Hint: Handle different source types (mp3, mp4, stream URL)
```

```
  // Hint: Normalize the browser APIs
}

const player = createMediaPlayer("#player", "song.mp3");
player.play();
player.setVolume(0.5);
player.pause();
player.destroy();
```

Exercise 3: React Data Fetching Facade Hook

Build a `useResource` hook that serves as a facade over data fetching. It should handle loading, error, refetching, pagination, and caching behind a simple API.

```javascript
function useResource(endpoint, options = {}) {
  // Your code here
  // Return: { data, loading, error, refetch, nextPage, prevPage }
}

function UserList() {
  const { data, loading, error, refetch, nextPage } =
    useResource("/api/users", { pageSize: 10 });

  // Clean component, all complexity hidden in the hook
}
```

What Is Next?

The Facade pattern simplifies how you use complex systems. The next chapter introduces the Factory pattern, which simplifies how you create objects and components. Instead of using `new` everywhere and knowing which class to instantiate, a factory function decides for you based on input. You will learn how to build component factories, notification factories, and more.

Chapter 11: The Factory Pattern

What You Will Learn

- What the Factory pattern is and why centralized creation matters
- How factory functions simplify object and component creation
- How to build a component factory for dynamic UIs
- How to build a notification factory that handles different channels
- How to build a chart factory that creates the right visualization
- How to build a form field factory for dynamic forms
- When to use factories and when simpler approaches work better

Why This Chapter Matters

Imagine a pizza restaurant. You, the customer, do not walk into the kitchen, find the flour, knead the dough, prepare the sauce, grate the cheese, and fire up the oven. You just say "margherita" and the kitchen does the rest.

The kitchen is a factory. You tell it what you want. It knows how to make it. The creation details are hidden from you.

In frontend development, you constantly create things: UI components, form fields, notification toasts, chart visualizations, API response handlers. Without the Factory pattern, the creation logic is scattered everywhere. Every part of the codebase needs to know which class to use, what options to set, and how to configure the result.

The Factory pattern centralizes this. You say "give me a bar chart" and the factory handles everything -- choosing the right library, setting the right defaults, configuring the right options. Change how charts are created? Change one factory, not fifty components.

The Problem

You need to create different types of objects based on some input, and the creation logic is complex or varies by type.

```javascript
// Creating notifications -- the hard way
function showNotification(type, message, options) {
  if (type === "success") {
    const el = document.createElement("div");
    el.className = "toast toast-success";
    el.innerHTML = `<span class="icon">✓</span><span>${message}</span>`;
    el.style.backgroundColor = "#10b981";
    el.style.color = "white";
    document.body.appendChild(el);
    setTimeout(() => el.remove(), options.duration || 3000);
  } else if (type === "error") {
    const el = document.createElement("div");
    el.className = "toast toast-error";
    el.innerHTML = `<span class="icon">✗</span><span>${message}</span>`;
    el.style.backgroundColor = "#ef4444";
    el.style.color = "white";
    document.body.appendChild(el);
    // Errors should stay longer
    setTimeout(() => el.remove(), options.duration || 5000);
  } else if (type === "warning") {
    // More copy-pasted code...
  } else if (type === "info") {
    // Even more copy-pasted code...
  }
}
```

Every notification type has similar code with slight variations. Adding a new type means adding another `if` branch. The creation logic is tangled with the rendering logic.

The Solution: Factory Pattern

The Factory pattern encapsulates object creation. A factory function takes input (usually a type or configuration) and returns the right object, fully constructed and ready to use.

```
Without a Factory:

  Component A --> new BarChart(...)
  Component B --> new LineChart(...)
  Component C --> new PieChart(...)
```

```
  Each component knows about every chart type.

With a Factory:

  Component A --> chartFactory("bar")  --> BarChart
  Component B --> chartFactory("line") --> LineChart
  Component C --> chartFactory("pie")  --> PieChart

  Components only know about the factory.
  The factory knows about the chart types.
```

Basic Factory Function

The simplest factory is just a function that returns different objects based on input:

```javascript
function createButton(type) {
  const base = {
    padding: "8px 16px",
    border: "none",
    borderRadius: "4px",
    cursor: "pointer",
    fontSize: "14px"
  };

  switch (type) {
    case "primary":
      return {
        ...base,
        backgroundColor: "#3b82f6",
        color: "white"
      };
    case "secondary":
      return {
        ...base,
        backgroundColor: "transparent",
        color: "#3b82f6",
        border: "1px solid #3b82f6"
      };
    case "danger":
      return {
        ...base,
        backgroundColor: "#ef4444",
        color: "white"
      };
    default:
```

```
      return {
        ...base,
        backgroundColor: "#e5e7eb",
        color: "#374151"
      };
  }
}

console.log(createButton("primary"));
// Output: { padding: '8px 16px', border: 'none', borderRadius: '4px',
//           cursor: 'pointer', fontSize: '14px',
//           backgroundColor: '#3b82f6', color: 'white' }

console.log(createButton("danger"));
// Output: { padding: '8px 16px', border: 'none', borderRadius: '4px',
//           cursor: 'pointer', fontSize: '14px',
//           backgroundColor: '#ef4444', color: 'white' }
```

Factory 1: Component Factory

Problem

You have a dashboard where users can add widgets. Each widget type is a different React component. You need to render the right component based on the widget type stored in a configuration.

Before: Giant Switch Statement in JSX

```javascript
function Dashboard({ widgets }) {
  return (
    <div className="dashboard">
      {widgets.map((widget) => {
        switch (widget.type) {
          case "chart":
            return (
              <ChartWidget
                key={widget.id}
                data={widget.data}
                chartType={widget.chartType}
                title={widget.title}
              />
            );
          case "table":
```

```
            return (
              <TableWidget
                key={widget.id}
                columns={widget.columns}
                data={widget.data}
                sortable={widget.sortable}
              />
            );
          case "metric":
            return (
              <MetricWidget
                key={widget.id}
                value={widget.value}
                label={widget.label}
                trend={widget.trend}
              />
            );
          case "text":
            return (
              <TextWidget
                key={widget.id}
                content={widget.content}
                format={widget.format}
              />
            );
          default:
            return <div key={widget.id}>Unknown widget type</div>;
        }
      })}
    </div>
  );
}
```

After: Component Factory

```javascript
// widgetFactory.js

import ChartWidget from "./widgets/ChartWidget";
import TableWidget from "./widgets/TableWidget";
import MetricWidget from "./widgets/MetricWidget";
import TextWidget from "./widgets/TextWidget";
import UnknownWidget from "./widgets/UnknownWidget";

// Registry of widget types
const widgetRegistry = {
  chart: ChartWidget,
  table: TableWidget,
  metric: MetricWidget,
  text: TextWidget
};
```

```
function createWidget(config) {
  const Component = widgetRegistry[config.type] || UnknownWidget;
  return <Component key={config.id} {...config} />;
}

// Allow registering new widget types at runtime
function registerWidget(type, component) {
  widgetRegistry[type] = component;
}

export { createWidget, registerWidget };
```

Now the dashboard is clean:

```javascript
function Dashboard({ widgets }) {
  return (
    <div className="dashboard">
      {widgets.map((widget) => createWidget(widget))}
    </div>
  );
}
```

Adding a new widget type requires zero changes to the Dashboard:

```javascript
// Some other module adds a new widget type
import { registerWidget } from "./widgetFactory";
import MapWidget from "./widgets/MapWidget";

registerWidget("map", MapWidget);

// Now any dashboard config with type: "map" will work automatically
```

```
Before (tight coupling):
  Dashboard knows about:
    ChartWidget, TableWidget, MetricWidget,
    TextWidget, MapWidget, ImageWidget...

After (factory):
  Dashboard knows about:
    createWidget(config)

  Factory knows about:
    ChartWidget, TableWidget, MetricWidget,
    TextWidget, MapWidget, ImageWidget...

Adding a new type:
  Before: Edit Dashboard + import new component
  After:  registerWidget("new", NewComponent) -- done
```

Factory 2: Notification Factory

Problem

Your app sends notifications through multiple channels: toast popups, browser notifications, email, and in-app messages. Each channel has a different API and different options.

Before: Channel-Specific Code Everywhere

```javascript
// In CheckoutPage
if (Notification.permission === "granted") {
  new Notification("Order placed!", {
    body: "Your order #1234 has been confirmed.",
    icon: "/icons/success.png"
  });
}

// In SettingsPage
const toast = document.createElement("div");
toast.className = "toast toast-info";
toast.textContent = "Settings saved";
document.body.appendChild(toast);
setTimeout(() => toast.remove(), 3000);

// In ErrorBoundary
fetch("/api/notify", {
  method: "POST",
  body: JSON.stringify({
    channel: "email",
    to: adminEmail,
    subject: "App Error",
    body: errorDetails
  })
});
```

After: Notification Factory

```javascript
// notificationFactory.js

const notificationCreators = {
  toast(config) {
```

```
    return {
      show() {
        const el = document.createElement("div");
        el.className = `toast toast-${config.variant || "info"}`;
        el.setAttribute("role", "alert");

        el.innerHTML = `
          <div class="toast-content">
            <span class="toast-icon">${getIcon(config.variant)}</span>
            <div>
              <strong>${config.title || ""}</strong>
              <p>${config.message}</p>
            </div>
            <button class="toast-close" onclick="this.parentElement.parentElement.remov...
              x
            </button>
          </div>
        `;

        document.getElementById("toast-container").appendChild(el);

        const duration = config.duration || (config.variant === "error" ? 8000 : 4000);
        setTimeout(() => el.remove(), duration);
      }
    };
  },

  browser(config) {
    return {
      async show() {
        if (Notification.permission !== "granted") {
          const permission = await Notification.requestPermission();
          if (permission !== "granted") {
            // Fallback to toast
            return notificationCreators.toast(config).show();
          }
        }
        new Notification(config.title || "Notification", {
          body: config.message,
          icon: config.icon || "/icons/default.png"
        });
      }
    };
  },

  inApp(config) {
    return {
      show() {
        // Dispatch to in-app notification center
        window.dispatchEvent(
          new CustomEvent("app:notification", {
            detail: {
              id: Date.now(),
              title: config.title,
              message: config.message,
```

```
              variant: config.variant,
              read: false,
              timestamp: new Date().toISOString()
            }
          })
        );
      }
    };
  }
};

function getIcon(variant) {
  const icons = {
    success: "&#10003;",
    error: "&#10007;",
    warning: "&#9888;",
    info: "&#8505;"
  };
  return icons[variant] || icons.info;
}

function createNotification(channel, config) {
  const creator = notificationCreators[channel];
  if (!creator) {
    throw new Error(`Unknown notification channel: ${channel}`);
  }
  return creator(config);
}

// Convenience function: create and show immediately
function notify(channel, config) {
  const notification = createNotification(channel, config);
  notification.show();
  return notification;
}

export { createNotification, notify };
```

Usage is now clean and consistent:

```javascript
// Show a toast
notify("toast", {
  variant: "success",
  title: "Order placed!",
  message: "Your order #1234 has been confirmed."
});

// Show a browser notification
notify("browser", {
  title: "New message",
  message: "Alice sent you a message."
});
```

```
// Add to in-app notification center
notify("inApp", {
  variant: "info",
  title: "System update",
  message: "A new version is available."
});
```

Factory 3: Chart Factory

Problem

Your analytics dashboard shows different chart types. Each chart type needs different libraries, options, and data transformations.

```javascript
// chartFactory.js

const chartDefaults = {
  bar: {
    responsive: true,
    borderRadius: 4,
    categoryPercentage: 0.8
  },
  line: {
    responsive: true,
    tension: 0.4,
    fill: false,
    pointRadius: 4
  },
  pie: {
    responsive: true,
    cutout: "0%"
  },
  doughnut: {
    responsive: true,
    cutout: "60%"
  }
};

const colorPalettes = {
  default: ["#3b82f6", "#ef4444", "#10b981", "#f59e0b", "#8b5cf6"],
  pastel: ["#93c5fd", "#fca5a5", "#6ee7b7", "#fcd34d", "#c4b5fd"],
  monochrome: ["#111827", "#374151", "#6b7280", "#9ca3af", "#d1d5db"]
};
```

```
function createChartConfig(type, data, options = {}) {
  const {
    title,
    palette = "default",
    animate = true,
    legend = true,
    ...restOptions
  } = options;

  // Validate type
  if (!chartDefaults[type]) {
    throw new Error(
      `Unknown chart type: "${type}". ` +
      `Available: ${Object.keys(chartDefaults).join(", ")}`
    );
  }

  // Apply colors from palette
  const colors = colorPalettes[palette] || colorPalettes.default;
  const coloredDatasets = data.datasets.map((dataset, i) => ({
    ...dataset,
    backgroundColor: dataset.backgroundColor || colors[i % colors.length],
    borderColor: dataset.borderColor || colors[i % colors.length]
  }));

  return {
    type,
    data: {
      ...data,
      datasets: coloredDatasets
    },
    options: {
      ...chartDefaults[type],
      ...restOptions,
      animation: animate ? { duration: 750 } : false,
      plugins: {
        legend: { display: legend },
        title: title
          ? { display: true, text: title, font: { size: 16 } }
          : { display: false }
      }
    }
  };
}

export { createChartConfig };
```

Usage:

```javascript
// Simple bar chart
const barConfig = createChartConfig("bar", {
  labels: ["Jan", "Feb", "Mar", "Apr"],
```

```
  datasets: [{ label: "Revenue", data: [12, 19, 3, 5] }]
}, {
  title: "Monthly Revenue",
  palette: "default"
});

// Line chart with custom options
const lineConfig = createChartConfig("line", {
  labels: ["Mon", "Tue", "Wed", "Thu", "Fri"],
  datasets: [
    { label: "This week", data: [65, 59, 80, 81, 56] },
    { label: "Last week", data: [28, 48, 40, 19, 86] }
  ]
}, {
  title: "Daily Active Users",
  palette: "pastel",
  legend: true
});

// Use with Chart.js
new Chart(canvasElement, barConfig);
new Chart(canvasElement, lineConfig);
```

Factory 4: Form Field Factory

Problem

Your app has dynamic forms where the fields are defined by a configuration (from a CMS, API, or admin panel). You need to render the right input component for each field type.

```javascript
// formFieldFactory.jsx

import React from "react";

const fieldComponents = {
  text({ name, label, value, onChange, placeholder, required }) {
    return (
      <div className="form-field">
        <label htmlFor={name}>
          {label} {required && <span className="required">*</span>}
        </label>
        <input
          id={name}
          name={name}
          type="text"
```

```
          value={value}
          onChange={onChange}
          placeholder={placeholder}
        />
      </div>
    );
  },

  textarea({ name, label, value, onChange, placeholder, rows = 4 }) {
    return (
      <div className="form-field">
        <label htmlFor={name}>{label}</label>
        <textarea
          id={name}
          name={name}
          value={value}
          onChange={onChange}
          placeholder={placeholder}
          rows={rows}
        />
      </div>
    );
  },

  select({ name, label, value, onChange, options = [] }) {
    return (
      <div className="form-field">
        <label htmlFor={name}>{label}</label>
        <select id={name} name={name} value={value} onChange={onChange}>
          <option value="">Select...</option>
          {options.map((opt) => (
            <option key={opt.value} value={opt.value}>
              {opt.label}
            </option>
          ))}
        </select>
      </div>
    );
  },

  checkbox({ name, label, value, onChange }) {
    return (
      <div className="form-field form-field-checkbox">
        <label>
          <input
            name={name}
            type="checkbox"
            checked={value}
            onChange={onChange}
          />
          {label}
        </label>
      </div>
    );
  },
```

```
  date({ name, label, value, onChange, min, max }) {
    return (
      <div className="form-field">
        <label htmlFor={name}>{label}</label>
        <input
          id={name}
          name={name}
          type="date"
          value={value}
          onChange={onChange}
          min={min}
          max={max}
        />
      </div>
    );
  }
};

function createFormField(fieldConfig, value, onChange) {
  const Component = fieldComponents[fieldConfig.type];

  if (!Component) {
    console.warn(`Unknown field type: "${fieldConfig.type}"`);
    return null;
  }

  return (
    <Component
      key={fieldConfig.name}
      {...fieldConfig}
      value={value}
      onChange={onChange}
    />
  );
}

export { createFormField };
```

Usage with a dynamic form:

```javascript
function DynamicForm({ fields, onSubmit }) {
  const [values, setValues] = useState({});

  function handleChange(e) {
    const { name, value, type, checked } = e.target;
    setValues((prev) => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value
    }));
  }
```

```
  return (
    <form onSubmit={(e) => { e.preventDefault(); onSubmit(values); }}>
      {fields.map((field) =>
        createFormField(field, values[field.name] || "", handleChange)
      )}
      <button type="submit">Submit</button>
    </form>
  );
}

// The fields come from configuration
const contactFields = [
  { type: "text", name: "name", label: "Full Name", required: true },
  { type: "text", name: "email", label: "Email", required: true },
  {
    type: "select",
    name: "subject",
    label: "Subject",
    options: [
      { value: "general", label: "General Inquiry" },
      { value: "support", label: "Technical Support" },
      { value: "billing", label: "Billing Question" }
    ]
  },
  { type: "textarea", name: "message", label: "Message", rows: 6 },
  { type: "checkbox", name: "subscribe", label: "Subscribe to newsletter" }
];

// Render the form
<DynamicForm fields={contactFields} onSubmit={handleSubmit} />
```

```
How the Form Field Factory works:

  Configuration (JSON):               Output (React components):

  { type: "text",                -->   <input type="text" ... />
    name: "email" }

  { type: "select",              -->   <select>
    name: "country",                     <option>US</option>
    options: [...] }                     <option>UK</option>
                                       </select>

  { type: "textarea",           -->   <textarea rows="4" ... />
    name: "bio" }

  { type: "checkbox",           -->   <input type="checkbox" ... />
    name: "agree" }

The factory picks the right component for each type.
```

Factory with Registration Pattern

The most flexible factories allow runtime registration of new types:

```javascript
function createFactory(defaultTypes = {}) {
  const registry = { ...defaultTypes };

  function create(type, ...args) {
    const creator = registry[type];
    if (!creator) {
      throw new Error(
        `Unknown type: "${type}". ` +
        `Registered types: ${Object.keys(registry).join(", ")}`
      );
    }
    return creator(...args);
  }

  function register(type, creator) {
    if (registry[type]) {
      console.warn(`Overwriting existing type: "${type}"`);
    }
    registry[type] = creator;
  }

  function unregister(type) {
    delete registry[type];
  }

  function getTypes() {
    return Object.keys(registry);
  }

  return { create, register, unregister, getTypes };
}

// Usage
const buttonFactory = createFactory({
  primary: (props) => ({ ...props, variant: "primary", color: "blue" }),
  secondary: (props) => ({ ...props, variant: "secondary", color: "gray" }),
  danger: (props) => ({ ...props, variant: "danger", color: "red" })
});

console.log(buttonFactory.getTypes());
// Output: ["primary", "secondary", "danger"]

const btn = buttonFactory.create("primary", { label: "Click me" });
console.log(btn);
// Output: { label: 'Click me', variant: 'primary', color: 'blue' }

// Register a new type at runtime
buttonFactory.register("ghost", (props) => ({
```

```
  ...props,
  variant: "ghost",
  color: "transparent",
  border: "none"
}));

const ghostBtn = buttonFactory.create("ghost", { label: "Ghost" });
console.log(ghostBtn);
// Output: { label: 'Ghost', variant: 'ghost',
//           color: 'transparent', border: 'none' }
```

Factory vs Constructor: When to Use Which

```
+------------------+-----------------------------------+----------------------------+
| Aspect           | Constructor (new Class)           | Factory Function           |
+------------------+-----------------------------------+----------------------------+
| Return type      | Always the same class             | Can return different types |
| Flexibility      | Fixed construction logic          | Dynamic based on input     |
| Naming           | Must be class name                | Can be descriptive name    |
| Testing          | Harder to mock                    | Easy to mock               |
| When to use      | You always want the same type     | You choose type at runtime |
+------------------+-----------------------------------+----------------------------+
```

When to Use the Factory Pattern

```
+-------------------------------------------------+
|           USE FACTORY WHEN:                     |
+-------------------------------------------------+
|                                                 |
|  * You create different types based on input    |
|  * Creation logic is complex or has many        |
|    configuration options                        |
|  * You want to centralize creation so changes   |
|    happen in one place                          |
|  * You need runtime registration of new types   |
|  * You want to decouple components from the     |
|    specific classes they use                    |
|  * Configuration-driven UIs (CMS, admin         |
|    panels, form builders)                       |
|                                                 |
```

```
+-----------------------------------------------+

+-----------------------------------------------+
|       DO NOT USE FACTORY WHEN:                |
+-----------------------------------------------+
|                                               |
|  * You always create the same type of object  |
|  * Creation logic is simple (just "new X()")  |
|  * There are only 1-2 types and they rarely   |
|    change                                     |
|  * A simple if/else or ternary would be       |
|    clearer                                    |
|  * The factory just wraps a constructor with  |
|    no added value                             |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Factory Returns Inconsistent Interfaces

```javascript
// WRONG -- different return shapes
function createShape(type) {
  if (type === "circle") {
    return { radius: 10, getArea: () => Math.PI * 100 };
  }
  if (type === "square") {
    return { sideLength: 10, calculateArea: () => 100 }; // Different method name!
  }
}

// CORRECT -- consistent interface
function createShape(type) {
  if (type === "circle") {
    return { type: "circle", dimensions: { radius: 10 }, getArea: () => Math.PI * 100 };
  }
  if (type === "square") {
    return { type: "square", dimensions: { side: 10 }, getArea: () => 100 };
  }
}
```

Mistake 2: Factory Does Too Much

```javascript
// WRONG -- factory creates, validates, saves, and notifies
function createUser(data) {
  // Validate
  if (!data.name) throw new Error("Name required");
  // Create
  const user = { id: uuid(), ...data, createdAt: new Date() };
  // Save to database
  database.save("users", user);
  // Send welcome email
  emailService.send(user.email, "Welcome!");
  return user;
}

// CORRECT -- factory only creates
function createUser(data) {
  return {
    id: uuid(),
    name: data.name,
    email: data.email,
    createdAt: new Date(),
    role: data.role || "user"
  };
}

// Other functions handle other concerns
const user = createUser(data);
await userRepository.save(user);
await emailService.sendWelcome(user);
```

Mistake 3: Hardcoding Types Instead of Using a Registry

```javascript
// WRONG -- adding a type requires editing the factory
function createChart(type, data) {
  switch (type) {
    case "bar": return new BarChart(data);
    case "line": return new LineChart(data);
    case "pie": return new PieChart(data);
    // Adding "radar" means editing this function
  }
}

// BETTER -- registry-based factory
const chartRegistry = {};

function registerChart(type, ChartClass) {
  chartRegistry[type] = ChartClass;
}

function createChart(type, data) {
  const ChartClass = chartRegistry[type];
```

```
  if (!ChartClass) throw new Error(`Unknown chart: ${type}`);
  return new ChartClass(data);
}

// Types can be registered from anywhere
registerChart("bar", BarChart);
registerChart("line", LineChart);
registerChart("pie", PieChart);
registerChart("radar", RadarChart); // No factory edit needed
```

Best Practices

1. Return consistent interfaces -- all objects from a factory should have the same methods and properties, regardless of type.
2. Keep factories focused on creation -- a factory creates objects. It should not save, validate, or send notifications.
3. Use a registry for extensibility -- instead of a switch statement, use a map/object so new types can be registered without editing the factory.
4. Provide sensible defaults -- factories should fill in reasonable default values so callers do not need to specify everything.
5. Validate input early -- throw clear errors if the requested type does not exist or required configuration is missing.
6. Make factories testable -- accept dependencies as parameters rather than importing them internally.

Quick Summary

The Factory pattern centralizes object creation. Instead of each part of the codebase knowing how to construct different types, a factory function takes a type or configuration and returns the right object. This makes adding new types easy, keeps creation logic consistent, and decouples the rest of the code from specific implementations.

```
Caller: "I need a bar chart"
          |
Factory:  (looks up "bar" in registry)
          (creates BarChart with defaults)
```

```
          (applies user config)
          (returns ready-to-use chart)
          |
Caller:  (uses chart -- does not care how it was made)
```

Key Points

- The Factory pattern centralizes and simplifies object creation
- Factories return different objects based on input type or configuration
- Component factories are ideal for configuration-driven UIs (dashboards, form builders)
- Registry-based factories allow runtime registration of new types
- All factory products should share a consistent interface
- Factories should focus only on creation -- not validation, saving, or side effects
- Use factories when types are dynamic; skip them when you always create the same thing

---

Practice Questions

1. What is the main difference between using a factory function and calling `new ClassName()` directly?
2. A dashboard supports 5 widget types. A new requirement adds 3 more. How does a factory-based approach make this easier than a switch statement in the component?
3. Why should all objects returned by a factory share a consistent interface? What problems arise if they do not?
4. Your form field factory currently supports `text`, `select`, and `checkbox`. How would you add a new `dateRange` field type using a registry pattern?
5. A colleague suggests creating a factory for user objects that also validates data and saves to the database. Why is this problematic?

Exercises

Exercise 1: HTTP Client Factory

Create a factory that builds pre-configured HTTP clients for different APIs. Each client should have `get`, `post`, `put`, and `delete` methods with the base URL, headers, and error handling baked in.

```javascript
function createAPIClient(config) {
  // Your code here
}

const githubAPI = createAPIClient({
  baseURL: "https://api.github.com",
  headers: { Accept: "application/vnd.github.v3+json" }
});

const internalAPI = createAPIClient({
  baseURL: "https://api.myapp.com/v2",
  headers: { Authorization: `Bearer ${token}` }
});

const repos = await githubAPI.get("/users/octocat/repos");
const users = await internalAPI.get("/users");
```

Exercise 2: Theme Factory

Build a factory that creates complete theme objects from a minimal configuration (just a primary color and mode). The factory should generate complementary colors, typography scales, spacing, and shadows.

```javascript
function createTheme(config) {
  // Your code here
  // Hint: Derive secondary/accent colors from primary
  // Hint: Generate a type scale (12px, 14px, 16px, ...)
}

const lightTheme = createTheme({ primary: "#3b82f6", mode: "light" });
const darkTheme = createTheme({ primary: "#3b82f6", mode: "dark" });
```

Exercise 3: Validation Rule Factory

Create a factory that builds validation functions from a declarative schema. The factory should support common rules (required, minLength, maxLength, email, pattern) and allow custom rules.

```javascript
function createValidator(schema) {
  // Your code here
}

const validateUser = createValidator({
  name: [{ rule: "required" }, { rule: "minLength", value: 2 }],
  email: [{ rule: "required" }, { rule: "email" }],
  age: [{ rule: "required" }, { rule: "min", value: 18 }]
});

const errors = validateUser({ name: "A", email: "bad", age: 15 });
// Output: { name: "Must be at least 2 characters",
//           email: "Invalid email",
//           age: "Must be at least 18" }
```

What Is Next?

The Factory pattern centralizes how you create things. The next chapter covers the Mediator pattern, which centralizes how things communicate. Instead of objects talking directly to each other (creating a tangled web of dependencies), a mediator sits in the middle and coordinates all communication. You will see how this pattern simplifies complex component interactions.

Chapter 12: Mediator Pattern

What You Will Learn

- What the Mediator pattern is and why it exists
- How many-to-many communication creates tangled code
- How to build a central mediator that coordinates communication
- How to implement a chat room using the Mediator pattern
- How to build a form mediator where fields react to each other
- The differences between Mediator and Pub/Sub
- When the mediator itself becomes too complex and what to do about it

Why This Chapter Matters

Imagine a busy airport with dozens of planes in the sky. What if every pilot had to talk directly to every other pilot to avoid collisions? That would be chaos. Instead, airports use air traffic controllers. Every pilot talks to the controller, and the controller coordinates everyone.

This is exactly the problem you face in frontend development. When you have a chat room with multiple users, a form where fields depend on each other, or a dialog with multiple interactive sections, direct communication between every component becomes a nightmare to maintain.

The Mediator pattern gives you that air traffic controller for your code.

The Problem: Many-to-Many Communication

Let us start with a real problem. You are building a chat room. You have multiple users who need to send messages to each other.

The Naive Approach (Direct Communication)

```javascript
// Without Mediator - every user knows about every other user

class User {
  constructor(name) {
    this.name = name;
    this.otherUsers = []; // Must track ALL other users
  }

  addContact(user) {
    this.otherUsers.push(user);
  }

  sendMessage(message) {
    // Must loop through ALL contacts and send individually
    this.otherUsers.forEach(user => {
      user.receiveMessage(message, this.name);
    });
  }

  receiveMessage(message, fromUser) {
    console.log(`${this.name} received from ${fromUser}: ${message}`);
  }
}

// Setting up connections is painful
const alice = new User('Alice');
const bob = new User('Bob');
const charlie = new User('Charlie');

// Every user must know about every other user
alice.addContact(bob);
alice.addContact(charlie);
bob.addContact(alice);
bob.addContact(charlie);
charlie.addContact(alice);
charlie.addContact(bob);

alice.sendMessage('Hello everyone!');
```

Output:

```
Bob received from Alice: Hello everyone!
Charlie received from Alice: Hello everyone!
```

What Is Wrong With This?

Look at the connections required:

```
Direct Communication (3 users):

    Alice <-------> Bob
      ^              ^
      |              |
      v              v
    Charlie <------> (needs connection to both)

    Connections needed: 3

Direct Communication (5 users):

    Alice <-> Bob
    Alice <-> Charlie
    Alice <-> Diana
    Alice <-> Eve
    Bob <-> Charlie
    Bob <-> Diana
    Bob <-> Eve
    Charlie <-> Diana
    Charlie <-> Eve
    Diana <-> Eve

    Connections needed: 10
```

The formula for connections is `n * (n - 1) / 2`. With 10 users, you need 45 connections. With 20 users, you need 190. This grows very fast.

Every time you add a new user, you must update every existing user. Every user depends on every other user. Testing one user means setting up all the others.

The Solution: Central Mediator

Instead of everyone talking to everyone, everyone talks to one central point: the mediator.

```
With Mediator:

    Alice ----\
               \
    Bob -------> Mediator (Chat Room)
               /
    Charlie --/

    Connections needed: 3 (always equals number of users)
```

Think of it like this:

Situation	Without Mediator	With Mediator
Airport	Pilots talk to each other	Pilots talk to control tower
Chat	Users message each user	Users message chat room
Form	Fields check every other field	Fields report to form manager
Game	Players interact directly	Players interact through game engine

---

Building a Chat Room Mediator

Let us build this step by step.

Step 1: Create the Mediator (Chat Room)

```javascript
// The mediator - coordinates all communication
class ChatRoom {
  constructor() {
    this.users = new Map(); // Track all users by name
  }

  // Register a user with the chat room
  register(user) {
    this.users.set(user.name, user);
    user.chatRoom = this; // Give user a reference to the mediator
    console.log(`${user.name} joined the chat room`);
  }

  // The core mediator method - routes messages
  sendMessage(message, fromUser, toUser) {
    if (toUser) {
      // Private message - send to specific user
      const recipient = this.users.get(toUser);
      if (recipient) {
        recipient.receiveMessage(message, fromUser);
      } else {
        console.log(`User ${toUser} not found`);
      }
    } else {
      // Broadcast - send to everyone except sender
```

```
      this.users.forEach((user, name) => {
        if (name !== fromUser) {
          user.receiveMessage(message, fromUser);
        }
      });
    }
  }
}
```

Line-by-line explanation:

- `this.users = new Map()` - The mediator keeps track of all participants. A Map gives us fast lookup by name.
- `register(user)` - When a user joins, we store them and give them a reference back to the mediator. This is the only connection each user needs.
- `sendMessage(message, fromUser, toUser)` - This is the heart of the mediator. It decides how to route messages. If `toUser` is specified, it sends a private message. Otherwise, it broadcasts to everyone except the sender.

Step 2: Create the User (Colleague)

```javascript
// The colleague - only knows about the mediator, not other users
class ChatUser {
  constructor(name) {
    this.name = name;
    this.chatRoom = null; // Will be set when registered
    this.messageLog = [];
  }

  // Send a message through the mediator
  send(message, toUser) {
    console.log(`${this.name} sends: ${message}`);
    this.chatRoom.sendMessage(message, this.name, toUser);
  }

  // Receive a message (called by the mediator)
  receiveMessage(message, fromUser) {
    const entry = `${fromUser}: ${message}`;
    this.messageLog.push(entry);
    console.log(`[${this.name}'s screen] ${entry}`);
  }
}
```

Line-by-line explanation:

- `this.chatRoom = null` - The user does not know about other users. It only has a reference to the mediator (chat room).

- `send(message, toUser)` - Instead of finding and calling other users directly, the user asks the mediator to deliver the message.
- `receiveMessage(message, fromUser)` - The mediator calls this method when a message arrives. The user does not know or care who else is in the chat.

Step 3: Use It

```javascript
// Create the mediator
const chatRoom = new ChatRoom();

// Create users
const alice = new ChatUser('Alice');
const bob = new ChatUser('Bob');
const charlie = new ChatUser('Charlie');

// Register users with the mediator
chatRoom.register(alice);
chatRoom.register(bob);
chatRoom.register(charlie);

// Alice broadcasts to everyone
alice.send('Good morning, team!');

console.log('---');

// Bob sends a private message to Alice
bob.send('Hey Alice, got a minute?', 'Alice');

console.log('---');

// Charlie broadcasts
charlie.send('Meeting starts in 5 minutes');
```

Output:

```
Alice joined the chat room
Bob joined the chat room
Charlie joined the chat room
Alice sends: Good morning, team!
[Bob's screen] Alice: Good morning, team!
[Charlie's screen] Alice: Good morning, team!
---
Bob sends: Hey Alice, got a minute?
[Alice's screen] Bob: Hey Alice, got a minute?
---
Charlie sends: Meeting starts in 5 minutes
[Alice's screen] Charlie: Meeting starts in 5 minutes
[Bob's screen] Charlie: Meeting starts in 5 minutes
```

The Key Difference

```
BEFORE (without mediator):
- Alice must know about Bob AND Charlie
- Bob must know about Alice AND Charlie
- Adding Diana means updating Alice, Bob, AND Charlie

AFTER (with mediator):
- Alice only knows about ChatRoom
- Bob only knows about ChatRoom
- Adding Diana means registering her with ChatRoom (one change)
```

Real-World Example: Form Mediator

Forms are one of the most common places where the Mediator pattern shines. Imagine a shipping form where:

- Selecting "International" shipping shows a customs field
- Entering a country updates the tax rate
- Selecting "Express" shipping disables certain payment methods
- The submit button enables only when all required fields are valid

Without a mediator, every field would need to know about every other field. With a mediator, each field reports to the form mediator, and the mediator decides what to update.

Building the Form Mediator

```javascript
// Form Mediator - coordinates all field interactions
class FormMediator {
  constructor() {
    this.fields = {};
    this.rules = [];
  }

  // Register a form field
  registerField(name, field) {
    this.fields[name] = field;
    field.mediator = this;
  }

  // Add a coordination rule
  addRule(rule) {
```

```
    this.rules.push(rule);
  }

  // Called when any field changes - this is the mediator logic
  fieldChanged(fieldName, value) {
    console.log(`Field "${fieldName}" changed to: ${value}`);

    // Run all rules to determine what else should change
    this.rules.forEach(rule => {
      rule(fieldName, value, this.fields);
    });

    // Check if form is valid
    this.validateForm();
  }

  validateForm() {
    const allValid = Object.values(this.fields)
      .filter(f => f.required && f.visible)
      .every(f => f.isValid());

    if (this.fields.submitButton) {
      this.fields.submitButton.setEnabled(allValid);
    }

    console.log(`Form valid: ${allValid}`);
  }
}
```

Line-by-line explanation:

- `this.fields = {}` - Stores all form fields by name, so rules can reference any field.
- `this.rules = []` - Stores functions that define how fields interact. Each rule runs when any field changes.
- `fieldChanged(fieldName, value)` - The core method. When a field changes, the mediator runs all rules and revalidates the form. No field needs to know about any other field.

The Form Fields

```javascript
// A form field that communicates through the mediator
class FormField {
  constructor(name, options = {}) {
    this.name = name;
    this.value = options.defaultValue || '';
    this.required = options.required || false;
    this.visible = options.visible !== false;
    this.enabled = options.enabled !== false;
```

```
    this.mediator = null;
  }

  setValue(value) {
    this.value = value;
    // Tell the mediator about the change - not other fields
    this.mediator.fieldChanged(this.name, value);
  }

  setVisible(visible) {
    this.visible = visible;
    console.log(`  "${this.name}" is now ${visible ? 'visible' : 'hidden'}`);
  }

  setEnabled(enabled) {
    this.enabled = enabled;
    console.log(`  "${this.name}" is now ${enabled ? 'enabled' : 'disabled'}`);
  }

  isValid() {
    if (!this.required || !this.visible) return true;
    return this.value !== '' && this.value !== null;
  }
}
```

Putting It Together

```javascript
// Create the mediator
const formMediator = new FormMediator();

// Create form fields
const shippingType = new FormField('shippingType', { required: true });
const country = new FormField('country', { required: true });
const customsNumber = new FormField('customsNumber', {
  required: true,
  visible: false
});
const expressOption = new FormField('expressOption');
const paypalOption = new FormField('paypalOption');
const submitButton = new FormField('submitButton');

// Register all fields with the mediator
formMediator.registerField('shippingType', shippingType);
formMediator.registerField('country', country);
formMediator.registerField('customsNumber', customsNumber);
formMediator.registerField('expressOption', expressOption);
formMediator.registerField('paypalOption', paypalOption);
formMediator.registerField('submitButton', submitButton);

// Define rules - the coordination logic lives in the mediator
```

```
formMediator.addRule((fieldName, value, fields) => {
  // Rule 1: Show customs field for international shipping
  if (fieldName === 'shippingType') {
    const isInternational = value === 'international';
    fields.customsNumber.setVisible(isInternational);
  }
});

formMediator.addRule((fieldName, value, fields) => {
  // Rule 2: Disable PayPal for express shipping
  if (fieldName === 'expressOption') {
    const isExpress = value === 'express';
    fields.paypalOption.setEnabled(!isExpress);
  }
});

// Simulate user filling out the form
console.log('=== User selects International shipping ===');
shippingType.setValue('international');

console.log('\n=== User enters country ===');
country.setValue('Canada');

console.log('\n=== User selects Express shipping ===');
expressOption.setValue('express');
```

Output:

```
=== User selects International shipping ===
Field "shippingType" changed to: international
  "customsNumber" is now visible
Form valid: false

=== User enters country ===
Field "country" changed to: Canada
Form valid: false

=== User selects Express shipping ===
Field "expressOption" changed to: express
  "paypalOption" is now disabled
Form valid: false
```

The form stays invalid because `customsNumber` is now visible and required but empty. Once the user fills it in, the form becomes valid.

Form Mediator in React

Let us see how this pattern looks in a React application.

```jsx
import React, { useState, useCallback } from 'react';

// The mediator logic lives in a custom hook
function useFormMediator() {
  const [formState, setFormState] = useState({
    shippingType: '',
    country: '',
    customsNumber: '',
    expressShipping: false,
    paymentMethod: 'credit-card',
  });

  const [fieldVisibility, setFieldVisibility] = useState({
    customsNumber: false,
  });

  const [fieldEnabled, setFieldEnabled] = useState({
    paypalOption: true,
  });

  // The mediator function - coordinates all changes
  const updateField = useCallback((fieldName, value) => {
    // Update the field value
    setFormState(prev => ({ ...prev, [fieldName]: value }));

    // Apply coordination rules
    if (fieldName === 'shippingType') {
      // Rule: Show customs field for international shipping
      setFieldVisibility(prev => ({
        ...prev,
        customsNumber: value === 'international',
      }));
    }

    if (fieldName === 'expressShipping') {
      // Rule: Disable PayPal for express shipping
      setFieldEnabled(prev => ({
        ...prev,
        paypalOption: !value,
      }));
    }
  }, []);

  // Check form validity
  const isFormValid = formState.shippingType !== ''
    && formState.country !== ''
    && (!fieldVisibility.customsNumber || formState.customsNumber !== '');

  return {
    formState,
    fieldVisibility,
```

```
    fieldEnabled,
    updateField,
    isFormValid,
  };
}

// The form component - fields do NOT talk to each other
function ShippingForm() {
  const {
    formState,
    fieldVisibility,
    fieldEnabled,
    updateField,
    isFormValid,
  } = useFormMediator();

  return (
    <form>
      <h2>Shipping Details</h2>

      <label>Shipping Type:</label>
      <select
        value={formState.shippingType}
        onChange={(e) => updateField('shippingType', e.target.value)}
      >
        <option value="">Select...</option>
        <option value="domestic">Domestic</option>
        <option value="international">International</option>
      </select>

      <label>Country:</label>
      <input
        type="text"
        value={formState.country}
        onChange={(e) => updateField('country', e.target.value)}
      />

      {fieldVisibility.customsNumber && (
        <>
          <label>Customs Number:</label>
          <input
            type="text"
            value={formState.customsNumber}
            onChange={(e) => updateField('customsNumber', e.target.value)}
          />
        </>
      )}

      <label>
        <input
          type="checkbox"
          checked={formState.expressShipping}
          onChange={(e) => updateField('expressShipping', e.target.checked)}
        />
        Express Shipping
```

```
      </label>

      <label>Payment Method:</label>
      <select
        value={formState.paymentMethod}
        onChange={(e) => updateField('paymentMethod', e.target.value)}
      >
        <option value="credit-card">Credit Card</option>
        <option value="paypal" disabled={!fieldEnabled.paypalOption}>
          PayPal {!fieldEnabled.paypalOption && '(not available with Express)'}
        </option>
      </select>

      <button type="submit" disabled={!isFormValid}>
        Place Order
      </button>
    </form>
  );
}
```

How the mediator works in React:

```
Field Change Flow:

  User changes       updateField()         State updates
  "shippingType" --> (mediator logic) --> fieldVisibility changes
       |                                  formState changes
       |                                       |
       v                                       v
  Only this field      React re-renders    Other fields
  calls updateField    the component        show/hide/
                                            enable/disable
```

Each field only calls `updateField`. It does not know about any other field. The `useFormMediator` hook is the mediator that decides what else should change.

Real-World Use Case: Dialog Coordination

Dialogs (modals) with multiple sections are another common use case. Imagine a settings dialog where changing one option affects what other options are available.

```javascript
// Dialog Mediator
class DialogMediator {
```

```
  constructor() {
    this.sections = {};
    this.state = {};
  }

  registerSection(name, section) {
    this.sections[name] = section;
    section.mediator = this;
  }

  notify(sectionName, event, data) {
    console.log(`[Dialog] ${sectionName} triggered: ${event}`);

    switch (event) {
      case 'theme-changed':
        // When theme changes, update preview and reset custom colors
        if (this.sections.preview) {
          this.sections.preview.updateTheme(data.theme);
        }
        if (data.theme !== 'custom' && this.sections.colors) {
          this.sections.colors.disable();
        }
        if (data.theme === 'custom' && this.sections.colors) {
          this.sections.colors.enable();
        }
        break;

      case 'font-size-changed':
        // When font size changes, warn if too large for mobile
        if (this.sections.preview) {
          this.sections.preview.updateFontSize(data.size);
        }
        if (data.size > 24 && this.sections.warnings) {
          this.sections.warnings.show('Large fonts may not display well on mobile');
        }
        break;

      case 'section-collapsed':
        // When a section collapses, expand the next one
        const sectionNames = Object.keys(this.sections);
        const currentIndex = sectionNames.indexOf(sectionName);
        const nextSection = this.sections[sectionNames[currentIndex + 1]];
        if (nextSection && nextSection.expand) {
          nextSection.expand();
        }
        break;
    }
  }
}
```

Real-World Use Case: Wizard Steps

Multi-step wizards are a perfect fit for the Mediator pattern. Each step needs to know about the state of other steps, but with a mediator, each step only talks to the wizard controller.

```jsx
import React, { useState, useCallback } from 'react';

function useWizardMediator(steps) {
  const [currentStep, setCurrentStep] = useState(0);
  const [stepData, setStepData] = useState({});
  const [stepValidity, setStepValidity] = useState(
    steps.reduce((acc, step) => ({ ...acc, [step.id]: false }), {})
  );

  // Mediator: handle step completion
  const completeStep = useCallback((stepId, data) => {
    setStepData(prev => ({ ...prev, [stepId]: data }));
    setStepValidity(prev => ({ ...prev, [stepId]: true }));

    // Automatically advance to next step
    setCurrentStep(prev => Math.min(prev + 1, steps.length - 1));
  }, [steps.length]);

  // Mediator: check if user can skip to a step
  const canGoToStep = useCallback((targetStep) => {
    // Can only go to steps where all previous steps are valid
    for (let i = 0; i < targetStep; i++) {
      if (!stepValidity[steps[i].id]) return false;
    }
    return true;
  }, [stepValidity, steps]);

  // Mediator: get data from a previous step
  const getStepData = useCallback((stepId) => {
    return stepData[stepId] || {};
  }, [stepData]);

  return {
    currentStep,
    setCurrentStep,
    completeStep,
    canGoToStep,
    getStepData,
    isComplete: Object.values(stepValidity).every(Boolean),
  };
}

// Usage
function CheckoutWizard() {
  const steps = [
    { id: 'address', title: 'Shipping Address' },
```

```
    { id: 'shipping', title: 'Shipping Method' },
    { id: 'payment', title: 'Payment' },
    { id: 'review', title: 'Review Order' },
  ];

  const wizard = useWizardMediator(steps);

  // Each step component only talks to the wizard mediator
  // It does not know about other steps
  return (
    <div className="wizard">
      <div className="wizard-steps">
        {steps.map((step, index) => (
          <button
            key={step.id}
            onClick={() => wizard.canGoToStep(index) && wizard.setCurrentStep(index)}
            disabled={!wizard.canGoToStep(index)}
            className={index === wizard.currentStep ? 'active' : ''}
          >
            {step.title}
          </button>
        ))}
      </div>

      <div className="wizard-content">
        {wizard.currentStep === 0 && (
          <AddressStep
            onComplete={(data) => wizard.completeStep('address', data)}
          />
        )}
        {wizard.currentStep === 1 && (
          <ShippingStep
            address={wizard.getStepData('address')}
            onComplete={(data) => wizard.completeStep('shipping', data)}
          />
        )}
        {wizard.currentStep === 2 && (
          <PaymentStep
            onComplete={(data) => wizard.completeStep('payment', data)}
          />
        )}
        {wizard.currentStep === 3 && (
          <ReviewStep
            allData={{
              ...wizard.getStepData('address'),
              ...wizard.getStepData('shipping'),
              ...wizard.getStepData('payment'),
            }}
          />
        )}
      </div>
    </div>
  );
}
```

How the wizard mediator coordinates steps:

```
Step Flow:

  AddressStep               WizardMediator            ShippingStep
      |                          |                          |
      |-- completeStep(data) --> |                          |
      |                          |-- setCurrentStep(1) --> |
      |                          |-- passes address   --> |
      |                          |   data as props          |
      |                          |                          |
      |                          | <-- completeStep(data) -|
      |                          |-- setCurrentStep(2) -->
```

Each step only knows about the mediator. The `AddressStep` does not import or reference `ShippingStep`. The mediator handles all the coordination.

Mediator vs Pub/Sub: What Is the Difference?

These patterns look similar but have an important difference.

```
Mediator:
- Has LOGIC about how to coordinate
- Knows about specific participants
- Makes decisions about what happens

    Component A ---> Mediator ---> Component B
                    (decides      (mediator calls
                     what to do)   B directly)

Pub/Sub:
- Is a DUMB message bus
- Does NOT know about specific participants
- Just passes messages along

    Component A ---> Event Bus ---> Component B
                    (just routes    (subscribed to
                     messages)      that event)
```

Feature	Mediator	Pub/Sub
Intelligence	Contains business logic	No logic, just routing
Participant awareness	Knows its participants	Does not know subscribers

Control flow	Mediator decides what happens	Publishers do not know who listens
Coupling	Participants coupled to mediator	Loose coupling through events
Complexity	Logic centralized in mediator	Logic distributed in handlers

Concrete Example

```javascript
// MEDIATOR approach - the mediator contains the logic
class LoginMediator {
  notify(sender, event) {
    if (event === 'login-success') {
      // Mediator DECIDES what happens next
      this.userPanel.showWelcome(sender.username);
      this.navbar.showUserMenu();
      this.chatWidget.connect(sender.userId);
      this.notificationService.fetchUnread(sender.userId);
    }
  }
}

// PUB/SUB approach - no central logic
loginService.publish('login-success', { userId: 123, username: 'Alice' });

// Each subscriber decides independently what to do
eventBus.subscribe('login-success', (data) => {
  userPanel.showWelcome(data.username);
});
eventBus.subscribe('login-success', (data) => {
  navbar.showUserMenu();
});
eventBus.subscribe('login-success', (data) => {
  chatWidget.connect(data.userId);
});
```

With the Mediator, the coordination logic is in one place. With Pub/Sub, it is spread across subscribers. Neither is always better; they solve different problems.

When the Mediator Becomes Too Complex

One danger of the Mediator pattern is that the mediator can grow into a "god object" that knows too much and does too much.

Signs Your Mediator Is Too Complex

```javascript
// BAD: Mediator doing too much
class GodMediator {
  notify(sender, event, data) {
    // 500+ lines of switch/case statements
    switch (event) {
      case 'user-login':
        // 30 lines of logic
        break;
      case 'user-logout':
        // 25 lines of logic
        break;
      case 'item-added':
        // 40 lines of logic
        break;
      // ... 50 more cases
    }
  }
}
```

How to Fix It: Split the Mediator

```javascript
// GOOD: Split into focused mediators

class AuthMediator {
  // Only handles authentication-related coordination
  notify(sender, event, data) {
    switch (event) {
      case 'login':
        this.userPanel.showWelcome(data.username);
        this.navbar.showUserMenu();
        break;
      case 'logout':
        this.userPanel.showLogin();
        this.navbar.showGuestMenu();
        break;
    }
  }
}

class CartMediator {
  // Only handles shopping cart coordination
  notify(sender, event, data) {
    switch (event) {
      case 'item-added':
        this.cartIcon.updateCount(data.count);
        this.priceDisplay.recalculate();
        break;
```

```
      case 'item-removed':
        this.cartIcon.updateCount(data.count);
        this.priceDisplay.recalculate();
        if (data.count === 0) {
          this.checkoutButton.disable();
        }
        break;
    }
  }
}
```

Rule of Thumb

If your mediator has more than 5 to 7 event types, consider splitting it. Each mediator should handle one area of coordination.

```
BEFORE (one giant mediator):

  Auth ----\
  Cart -----> God Mediator (handles everything)
  Chat ----/
  Nav  ---/

AFTER (focused mediators):

  Auth -------> AuthMediator
  Cart -------> CartMediator
  Chat -------> ChatMediator
  Nav  -------> NavMediator
```

When to Use / When NOT to Use

Use the Mediator Pattern When

1. Multiple components need to communicate - Chat rooms, collaborative editors, game state
2. Form fields depend on each other - Show/hide, enable/disable based on other fields
3. Dialog sections coordinate - Settings panels, wizard steps
4. You want to centralize complex coordination logic - Instead of spreading it across many components
5. Adding new participants should be easy - Just register them with the mediator

Do NOT Use the Mediator Pattern When

1. Only two components communicate - Direct communication is simpler
2. Communication is one-way - Use simple props or callbacks instead
3. There is no coordination logic - If you just need event routing, use Pub/Sub
4. The mediator would have no logic - It would just be a pass-through (use Pub/Sub instead)
5. Components are already well-isolated - Do not add complexity where it is not needed

---

Common Mistakes

Mistake 1: Making Everything Go Through the Mediator

```javascript
// BAD: Simple parent-child communication through mediator
class ButtonMediator {
  notify(sender, event) {
    if (event === 'click') {
      this.label.setText('Clicked!');
    }
  }
}

// GOOD: Just use a callback
<Button onClick={() => setLabel('Clicked!')} />
```

If two components have a simple, direct relationship, a mediator adds unnecessary complexity.

Mistake 2: Letting Participants Know About Each Other

```javascript
// BAD: User still references other users
class ChatUser {
  send(message) {
    // This defeats the purpose of having a mediator
    this.chatRoom.users.forEach(user => {
      if (user !== this) user.receive(message);
    });
  }
}
```

```
// GOOD: User only talks to mediator
class ChatUser {
  send(message) {
    this.chatRoom.sendMessage(message, this.name);
  }
}
```

The whole point is that participants only know about the mediator, not about each other.

Mistake 3: Not Defining Clear Events

```javascript
// BAD: Passing raw data without structure
mediator.notify(this, { type: 'something', data: someData, extra: moreData });

// GOOD: Define clear event types
mediator.notify(this, 'FIELD_CHANGED', { fieldName: 'email', value: 'a@b.com' });
mediator.notify(this, 'STEP_COMPLETED', { stepId: 'address', data: addressData });
```

Best Practices

1. Keep the mediator focused - One mediator per area of coordination (auth, cart, chat)
2. Define clear event types - Use constants or enums for event names
3. Participants should not access each other - They should only know about the mediator
4. Keep coordination logic in the mediator - Not in the participants
5. In React, custom hooks make great mediators - `useFormMediator`, `useWizardMediator`
6. Document the rules - Since all logic is centralized, document what triggers what
7. Test the mediator independently - Since all logic is in one place, it is easy to test

---

Quick Summary

The Mediator pattern replaces many-to-many communication with a central coordinator. Instead of every component talking to every other component, each component only talks to the mediator. The mediator contains the logic for how to coordinate the components. This reduces coupling, makes it easy to add new participants, and centralizes complex interaction logic. In React, custom hooks often serve as mediators for form coordination, wizard steps,

and dialog management.

Key Points

- The Mediator pattern introduces a central object that controls communication between other objects
- Without a mediator, N components need N*(N-1)/2 connections; with a mediator, they need only N
- The mediator contains business logic about how to coordinate (unlike Pub/Sub which is a dumb message bus)
- Air traffic controller is the classic analogy: pilots talk to the tower, not to each other
- In React, custom hooks serve as mediators, centralizing state coordination logic
- A mediator that grows too large should be split into focused sub-mediators
- Each participant should only know about the mediator, never about other participants

---

Practice Questions

1. You have a dashboard with 5 widgets that all need to update when the user changes a date filter. Would you use a Mediator or Pub/Sub pattern? Why?
2. A form has 3 fields: country, state, and city. Selecting a country populates states, and selecting a state populates cities. Draw the communication flow using a mediator.
3. Your mediator class has grown to 800 lines with 20 event types. What would you do to improve this?
4. What is the key difference between a Mediator and an Event Bus (Pub/Sub)?
5. In the chat room example, what would you need to change if you wanted to add a "typing indicator" feature? How does the mediator make this easier?

Exercises

Exercise 1: Game State Mediator

Build a simple tic-tac-toe game mediator that:

- Tracks whose turn it is
- Validates moves (is the cell empty?)
- Checks for a winner after each move
- Notifies the display to update
- Handles game reset

Hint: The mediator should coordinate between the board, the players, and the display.

Exercise 2: Notification Center Mediator

Build a notification center mediator in React that:

- Receives notifications from different sources (email, chat, system)
- Applies rules: if "Do Not Disturb" is on, only show urgent notifications
- Groups notifications by source
- Auto-dismisses notifications after a timeout
- Updates the notification badge count

Exercise 3: Multi-Panel Editor

Build a mediator for a code editor with three panels:

- File tree panel (selecting a file opens it in the editor)
- Code editor panel (editing code updates the preview)
- Preview panel (shows the output)
- When a file is saved, the file tree should show a "saved" indicator

Each panel should only communicate through the mediator.

What Is Next?

Now that you understand how the Mediator pattern centralizes communication between many components, we will explore the Mixin pattern. Mixins solve a different problem: how do you share behavior across unrelated classes without creating deep inheritance chains? You will learn why mixins were popular, why they fell out of favor, and what replaced them in modern React.

Chapter 13: The Mixin Pattern

What You Will Learn

- What the Mixin pattern is and what problem it solves
- How `Object.assign` enables property sharing across unrelated objects
- How mixins were used in class-based JavaScript and frameworks like React
- Why mixins fell out of favor and what problems they cause
- How React Hooks replaced mixins as the modern way to share behavior
- When (if ever) mixins are still appropriate

Why This Chapter Matters

Imagine you are building with LEGO bricks. You have a car. You want to add wings so it can fly. You do not want to rebuild the car from scratch -- you just want to snap on the wings. Later you decide to add a boat hull so it can float. Snap it on. Now your creation can drive, fly, and float.

This is the idea behind mixins: take behaviors from different sources and "mix" them into an object. Your car object gets flying behavior from one mixin and floating behavior from another. No inheritance hierarchy needed.

For years, mixins were a popular way to share code in JavaScript. React used them heavily before version 0.13. Vue had them until Vue 3. Backbone, Ember, and many other frameworks relied on them.

Then the community realized that mixins cause more problems than they solve. This chapter teaches you what mixins are, why they seemed like a good idea, what went wrong, and what replaced them. Understanding this history will help you recognize mixin-like patterns in legacy code and choose better alternatives in new code.

The Problem

You have multiple unrelated objects or classes that need to share the same behavior. JavaScript's single inheritance model means a class can only extend one parent.

```javascript
// You want to add event handling to different objects
class User {
  constructor(name) { this.name = name; }
}

class Product {
  constructor(title) { this.title = title; }
}

class Order {
  constructor(id) { this.id = id; }
}

// All three need event handling: on(), off(), emit()
// They have no common parent class.
// You cannot make them all extend "EventEmitter"
// because User might already extend "Model".
```

Inheritance does not work here because:

- These classes are unrelated
- JavaScript only supports single inheritance
- Creating a deep inheritance chain (EventEmitter -> Model -> User) is fragile

---

The Solution: Mixins

A mixin is an object (or function) that provides methods which can be "mixed into" other objects. The key idea: instead of inheriting from a parent, you copy properties from a mixin.

```
Inheritance:                    Mixins:

     EventEmitter               +-- EventMixin  (on, off, emit)
          |                     |
        Model                   +-- Serializable (toJSON, fromJSON)
          |                     |
        User                    User.mixIn(EventMixin, Serializable)

     One parent allowed.        Multiple mixins allowed.
```

Object.assign: The Mixin Mechanism

The simplest way to mix properties into an object is `Object.assign`:

```javascript
const canFly = {
  fly() {
    console.log(`${this.name} is flying!`);
  },
  land() {
    console.log(`${this.name} has landed.`);
  }
};

const canSwim = {
  swim() {
    console.log(`${this.name} is swimming!`);
  },
  dive(depth) {
    console.log(`${this.name} is diving ${depth}m deep!`);
  }
};

const canRun = {
  run(speed) {
    console.log(`${this.name} is running at ${speed} km/h!`);
  }
};

// Create a duck that can do all three
const duck = { name: "Donald" };
Object.assign(duck, canFly, canSwim, canRun);

duck.fly();
// Output: Donald is flying!

duck.swim();
// Output: Donald is swimming!

duck.run(15);
// Output: Donald is running at 15 km/h!
```

`Object.assign` copies all enumerable own properties from the source objects to the target. If two sources have the same property name, the last one wins.

Mixins with Classes

You can also mix behavior into class prototypes:

```javascript
// Mixin: Event handling
const EventMixin = {
  _ensureListeners() {
    if (!this._listeners) {
      this._listeners = {};
    }
  },

  on(event, callback) {
    this._ensureListeners();
    if (!this._listeners[event]) {
      this._listeners[event] = [];
    }
    this._listeners[event].push(callback);
  },

  off(event, callback) {
    this._ensureListeners();
    if (!this._listeners[event]) return;
    this._listeners[event] = this._listeners[event]
      .filter((cb) => cb !== callback);
  },

  emit(event, ...args) {
    this._ensureListeners();
    if (!this._listeners[event]) return;
    this._listeners[event].forEach((cb) => cb(...args));
  }
};

// Mixin: Serialization
const SerializableMixin = {
  toJSON() {
    const obj = {};
    for (const key of Object.keys(this)) {
      if (!key.startsWith("_")) {
        obj[key] = this[key];
      }
    }
    return JSON.stringify(obj);
  }
};

// Apply mixins to a class
class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
```

```
  }

  greet() {
    return `Hi, I'm ${this.name}`;
  }
}

// Mix in behaviors
Object.assign(User.prototype, EventMixin, SerializableMixin);

// Usage
const user = new User("Alice", "alice@example.com");

user.on("nameChange", (oldName, newName) => {
  console.log(`Name changed from ${oldName} to ${newName}`);
});

console.log(user.greet());
// Output: Hi, I'm Alice

console.log(user.toJSON());
// Output: {"name":"Alice","email":"alice@example.com"}
```

Functional Mixins (A Better Approach)

Instead of plain objects, you can use functions that apply behavior to a target. This gives you more control:

```javascript
function withTimestamps(target) {
  target.createdAt = new Date();
  target.updatedAt = new Date();

  const originalSet = Object.getOwnPropertyDescriptor(
    Object.getPrototypeOf(target) || target,
    "set"
  );

  target.touch = function () {
    this.updatedAt = new Date();
  };

  return target;
}

function withValidation(target, rules) {
  target.validate = function () {
```

```
    const errors = {};
    for (const [field, rule] of Object.entries(rules)) {
      const error = rule(this[field]);
      if (error) errors[field] = error;
    }
    return {
      valid: Object.keys(errors).length === 0,
      errors
    };
  };
  return target;
}

function withHistory(target, trackedFields) {
  target._history = [];

  target.saveSnapshot = function () {
    const snapshot = {};
    for (const field of trackedFields) {
      snapshot[field] = this[field];
    }
    snapshot._timestamp = new Date();
    this._history.push(snapshot);
  };

  target.getHistory = function () {
    return [...this._history];
  };

  return target;
}

// Usage: compose multiple functional mixins
let user = { name: "Alice", email: "alice@example.com" };

user = withTimestamps(user);
user = withValidation(user, {
  name: (val) => (!val ? "Name required" : null),
  email: (val) => (!val || !val.includes("@") ? "Valid email required" : null)
});
user = withHistory(user, ["name", "email"]);

console.log(user.validate());
// Output: { valid: true, errors: {} }

user.saveSnapshot();
user.name = "Bob";
user.touch();
user.saveSnapshot();

console.log(user.getHistory());
// Output: [
//   { name: 'Alice', email: 'alice@example.com', _timestamp: ... },
//   { name: 'Bob', email: 'alice@example.com', _timestamp: ... }
// ]
```

Mixins in Legacy React (Pre-Hooks)

Before React Hooks, React supported mixins through `createReactClass`. Understanding this helps you work with legacy codebases:

```javascript
// React with mixins (pre-2015 -- DO NOT USE in new code)
var SetIntervalMixin = {
  componentDidMount: function () {
    this.intervals = [];
  },
  setInterval: function (fn, delay) {
    this.intervals.push(window.setInterval(fn, delay));
  },
  componentWillUnmount: function () {
    this.intervals.forEach(clearInterval);
  }
};

var TickingClock = React.createClass({
  mixins: [SetIntervalMixin], // Use the mixin

  getInitialState: function () {
    return { seconds: 0 };
  },

  componentDidMount: function () {
    this.setInterval(this.tick, 1000);
  },

  tick: function () {
    this.setState({ seconds: this.state.seconds + 1 });
  },

  render: function () {
    return <p>Running for {this.state.seconds} seconds.</p>;
  }
});
```

This looks clean for simple cases. But it did not scale.

Why Mixins Fell Out of Favor

The React team wrote a famous blog post titled "Mixins Considered Harmful" in 2016. Here are the core problems:

Problem 1: Name Collisions

Two mixins can define the same method name, and the last one wins silently:

```javascript
const TimerMixin = {
  start() {
    console.log("Starting timer...");
  }
};

const AnimationMixin = {
  start() {
    console.log("Starting animation...");
  }
};

const widget = {};
Object.assign(widget, TimerMixin, AnimationMixin);

widget.start();
// Output: Starting animation...
// The timer's start() was silently overwritten!
```

```
Name collision:

  TimerMixin:      { start(), stop(), reset() }
  AnimationMixin:  { start(), pause(), resume() }
                       ^
                   COLLISION!

  Object.assign(widget, TimerMixin, AnimationMixin)

  widget.start() --> calls AnimationMixin.start()
  TimerMixin.start() is GONE. Silently.
```

Problem 2: Implicit Dependencies

Mixins often depend on properties they expect to exist on the target, but this dependency is invisible:

```javascript
const FormValidationMixin = {
  validate() {
```

```
    // This mixin assumes this.rules exists.
    // Where does this.rules come from?
    // Nobody knows without reading all the code.
    for (const [field, rule] of Object.entries(this.rules)) {
      if (!rule(this.state[field])) {
        return false;
      }
    }
    return true;
  }
};

// If the component does not define this.rules, it breaks silently
// or throws a cryptic error at runtime.
```

Problem 3: Snowballing Complexity

Over time, mixins depend on each other. Removing one mixin breaks another. You cannot understand a component without reading every mixin it uses:

```javascript
// Real-world horror show:
var MyComponent = React.createClass({
  mixins: [
    AuthMixin,
    PermissionMixin,
    FormMixin,
    ValidationMixin,
    AnalyticsMixin,
    TooltipMixin,
    ScrollMixin,
    ResizeMixin
  ],
  // Good luck understanding what "this" has on it
});
```

Problem 4: No Clear Data Flow

With mixins, data comes from unknown sources. You cannot trace where a method or property was defined:

```javascript
// Where does this.trackEvent come from?
// Where does this.currentUser come from?
// Where does this.showModal come from?
// Answer: ¯\_(ツ)_/¯ check the mixins
```

The Modern Alternative: React Hooks

React Hooks solve every problem that mixins had. Here is a side-by-side comparison:

Mixin: setInterval

```javascript
// OLD: Mixin approach
var SetIntervalMixin = {
  intervals: [],
  componentDidMount() {
    // implicit dependency on component lifecycle
  },
  setInterval(fn, delay) {
    this.intervals.push(window.setInterval(fn, delay));
  },
  componentWillUnmount() {
    this.intervals.forEach(clearInterval);
  }
};
```

Hook: useInterval

```javascript
// NEW: Hook approach
import { useEffect, useRef } from "react";

function useInterval(callback, delay) {
  const savedCallback = useRef(callback);

  useEffect(() => {
    savedCallback.current = callback;
  }, [callback]);

  useEffect(() => {
    if (delay === null) return;

    const id = setInterval(() => savedCallback.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}

// Usage
function TickingClock() {
  const [seconds, setSeconds] = useState(0);
```

```
  useInterval(() => {
    setSeconds((s) => s + 1);
  }, 1000);

  return <p>Running for {seconds} seconds.</p>;
}
```

Why Hooks Are Better

```
+-------------------------+----------------------------+---------------------------+
| Problem                 | Mixins                     | Hooks                     |
+-------------------------+----------------------------+---------------------------+
| Name collisions         | Silent overwrites          | No collisions -- each     |
|                         |                            | hook has its own scope    |
+-------------------------+----------------------------+---------------------------+
| Implicit dependencies   | Depends on "this" having   | Dependencies are explicit |
|                         | certain properties         | (function arguments)      |
+-------------------------+----------------------------+---------------------------+
| Data flow               | Unclear where data comes   | Clear: hook returns data  |
|                         | from                       |                           |
+-------------------------+----------------------------+---------------------------+
| Composability           | Mixins interact through    | Hooks compose through     |
|                         | shared "this"              | returned values           |
+-------------------------+----------------------------+---------------------------+
| Testability             | Need to mock "this"        | Just call the function    |
+-------------------------+----------------------------+---------------------------+
```

More Hook Examples (Replacing Common Mixins)

```javascript
// Hook: useWindowSize (replaces ResizeMixin)
function useWindowSize() {
  const [size, setSize] = useState({
    width: window.innerWidth,
    height: window.innerHeight
  });

  useEffect(() => {
    function handleResize() {
      setSize({
        width: window.innerWidth,
        height: window.innerHeight
      });
    }
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);
```

```
  return size;
}

// Hook: useLocalStorage (replaces PersistenceMixin)
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];
}

// Hook: useOnClickOutside (replaces ClickOutsideMixin)
function useOnClickOutside(ref, handler) {
  useEffect(() => {
    function listener(event) {
      if (!ref.current || ref.current.contains(event.target)) return;
      handler(event);
    }

    document.addEventListener("mousedown", listener);
    document.addEventListener("touchstart", listener);
    return () => {
      document.removeEventListener("mousedown", listener);
      document.removeEventListener("touchstart", listener);
    };
  }, [ref, handler]);
}
```

Composing Hooks (No Collision Risk)

```javascript
function useAutosaveForm(formId) {
  // Compose multiple hooks -- no collisions possible!
  const [values, setValues] = useLocalStorage(`form-${formId}`, {});
  const [isDirty, setIsDirty] = useState(false);
  const windowSize = useWindowSize();

  useInterval(() => {
    if (isDirty) {
      saveToServer(formId, values);
```

```
      setIsDirty(false);
    }
  }, 30000);

  function updateField(name, value) {
    setValues((prev) => ({ ...prev, [name]: value }));
    setIsDirty(true);
  }

  return {
    values,
    updateField,
    isDirty,
    isMobile: windowSize.width < 768
  };
}
```

When Mixins Might Still Be Appropriate

Despite the problems, object mixins (not class/component mixins) can be useful in specific cases:

```javascript
// Plain objects with no lifecycle or state complexity
const Printable = {
  print() {
    console.log(JSON.stringify(this, null, 2));
  }
};

const Comparable = {
  equals(other) {
    return JSON.stringify(this) === JSON.stringify(other);
  }
};

// Simple data objects where collisions are manageable
function createRecord(data) {
  return Object.assign({}, data, Printable, Comparable);
}

const record = createRecord({ name: "Alice", age: 28 });
record.print();
// Output: { "name": "Alice", "age": 28 }
```

This works because:

- The objects are simple (no lifecycle, no state)

- The mixin methods have unique, descriptive names
- There is no implicit dependency
- The code is small enough to understand at a glance

When to Use / When NOT to Use Mixins

```
+-----------------------------------------------+
|      STILL ACCEPTABLE (plain objects):        |
+-----------------------------------------------+
|                                               |
|  * Adding utility methods to plain data       |
|    objects (serialization, comparison)         |
|  * Small, well-defined behavior sets with     |
|    unique method names                        |
|  * Prototyping or quick scripts               |
|  * When you control all the mixins and can    |
|    guarantee no collisions                    |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|        DO NOT USE MIXINS:                     |
+-----------------------------------------------+
|                                               |
|  * In React components (use Hooks instead)    |
|  * When mixins depend on each other           |
|  * When mixins depend on the target object's  |
|    internal state                             |
|  * In large codebases with many contributors  |
|  * When methods might collide                 |
|  * For anything involving lifecycle,           |
|    state management, or side effects          |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Using Mixins for React Components

```javascript
// WRONG -- mixins in modern React
const MyComponent = React.createClass({
  mixins: [SomeMixin]
  // This API was removed in React 16
});

// CORRECT -- use hooks
function MyComponent() {
  const something = useSomething();
  // ...
}
```

Mistake 2: Mutating Shared Mixin Objects

```javascript
// WRONG -- all users share the same object
const CounterMixin = {
  count: 0, // Shared across ALL objects!
  increment() { this.count++; }
};

const a = Object.assign({}, CounterMixin);
const b = Object.assign({}, CounterMixin);
// Wait -- count is copied by value (0), so this actually works.

// But with reference types:
const ListMixin = {
  items: [], // This IS shared! Danger!
  addItem(item) { this.items.push(item); }
};

const a = Object.assign({}, ListMixin);
const b = Object.assign({}, ListMixin);
a.addItem("hello");
console.log(b.items);
// Output: ["hello"] -- Both objects share the same array!

// CORRECT -- use a function that creates fresh state
function createListMixin() {
  return {
    items: [],
    addItem(item) { this.items.push(item); }
  };
}

const a = Object.assign({}, createListMixin());
const b = Object.assign({}, createListMixin());
a.addItem("hello");
console.log(b.items);
// Output: [] -- Separate arrays!
```

Mistake 3: Deep Mixin Chains

```javascript
// WRONG -- mixins that depend on other mixins
const BaseMixin = { /* ... */ };
const NetworkMixin = { /* depends on BaseMixin */ };
const AuthMixin = { /* depends on NetworkMixin */ };
const DataMixin = { /* depends on AuthMixin */ };

Object.assign(obj, BaseMixin, NetworkMixin, AuthMixin, DataMixin);
// Fragile chain -- remove one and everything breaks

// CORRECT -- use composition with explicit dependencies
function createAuthenticatedClient(config) {
  const network = createNetworkClient(config);
  const auth = createAuthHandler(config.credentials);
  // Explicit composition, not implicit mixing
}
```

Best Practices

1. Prefer hooks over mixins for React components -- there is no good reason to use mixins in modern React.
2. If you must use mixins, keep them small, independent, and use unique method names to avoid collisions.
3. Use functional mixins (functions that modify a target) over plain object mixins -- they can create fresh state for each target.
4. Never let mixins depend on each other -- each mixin should work independently.
5. Document expected properties -- if a mixin needs `this.name` to exist, state that clearly.
6. Consider alternatives first: hooks, composition, utility functions, or the Decorator pattern.

Quick Summary

The Mixin pattern lets you add behavior from multiple sources to a single object using `Object.assign` or similar copying mechanisms. Mixins were popular in early JavaScript frameworks because they solved the single-inheritance limitation. However, they cause serious problems at scale: name collisions, implicit dependencies, snowballing complexity, and unclear data flow. React Hooks are the modern replacement for component-level mixins,

solving all of these problems with explicit dependencies, isolated state, and composability.

```
Evolution of shared behavior:

  Mixins (2010s)           Hooks (2019+)
  +-----------+            +-----------+
  | Mixin A   |            | useA()    | --> returns { data, methods }
  | Mixin B   | all dump   | useB()    | --> returns { data, methods }
  | Mixin C   | into       | useC()    | --> returns { data, methods }
  +-----------+ "this"     +-----------+
       |                        |
  Collision risk           No collisions
  Implicit deps            Explicit deps
  Hard to trace            Easy to trace
```

Key Points

- Mixins copy properties from source objects into a target using `Object.assign`
- They solve the single-inheritance problem by allowing multiple behavior sources
- Mixins were widely used in React (createClass), Vue, Backbone, and other frameworks
- They cause name collisions, implicit dependencies, and snowballing complexity
- React Hooks replaced mixins for component behavior sharing
- Hooks have isolated scope, explicit dependencies, and composability
- Plain object mixins are still acceptable for simple utility methods on data objects
- Functional mixins (functions that modify targets) are safer than plain object mixins

---

Practice Questions

1. What is a mixin and how does `Object.assign` make it possible in JavaScript?
2. Explain the name collision problem with mixins. How do React Hooks avoid this problem?
3. What are "implicit dependencies" in the context of mixins? Give an example.
4. A legacy React codebase uses `createReactClass` with mixins. You are tasked with modernizing it. What is your approach?
5. When might a plain object mixin still be a reasonable choice? What safeguards would you put in place?

Exercises

Exercise 1: Mixin to Hook Conversion

Convert this mixin-style code to a React Hook:

```javascript
// Convert this mixin to a hook
const MouseTrackerMixin = {
  getInitialState() {
    return { mouseX: 0, mouseY: 0 };
  },
  componentDidMount() {
    window.addEventListener("mousemove", this.handleMouseMove);
  },
  componentWillUnmount() {
    window.removeEventListener("mousemove", this.handleMouseMove);
  },
  handleMouseMove(event) {
    this.setState({ mouseX: event.clientX, mouseY: event.clientY });
  }
};

// Write: function useMousePosition() { ... }
```

Exercise 2: Safe Object Mixin

Create a `safeMixin` function that applies mixins to an object but throws an error if any property name collisions are detected, instead of silently overwriting.

```javascript
function safeMixin(target, ...mixins) {
  // Your code here
  // Hint: Check for property name conflicts before applying
}

const a = { greet() {} };
const b = { greet() {} }; // Collision!

safeMixin({}, a, b); // Should throw: "Property collision: greet"
```

Exercise 3: Compose Hooks

Build a `useFormField` hook that composes `useLocalStorage`, validation logic, and debounced onChange handling into a single reusable hook. It should replace what would have been a "FormFieldMixin" in the old world.

```javascript
function useFormField(name, options = {}) {
  // Your code here
  // Return: { value, onChange, error, isDirty, isTouched }
}

function SignupForm() {
  const email = useFormField("email", {
    validate: (v) => (!v.includes("@") ? "Invalid email" : null),
    persist: true,
    debounce: 300
  });

  return (
    <input value={email.value} onChange={email.onChange} />
    {email.error && <span>{email.error}</span>}
  );
}
```

What Is Next?

You have seen how mixins share behavior -- and the problems they cause. The next chapter introduces Component Composition, React's preferred way to build complex UIs from simple pieces. Instead of mixing behavior into components, you compose components together using `children`, slot patterns, and render functions. This is the pattern the React team recommends over both inheritance and mixins.

Chapter 14: Component Composition

What You Will Learn

- What component composition is and why React favors it over inheritance
- How the `children` prop enables flexible, reusable components
- How the slot pattern gives you named insertion points
- Why composition beats inheritance for building UIs
- How specialization and containment patterns work
- How to build layout components that compose cleanly
- When composition is the right choice and when it is not

Why This Chapter Matters

Think about building a house. You do not forge a single giant piece of metal in the shape of a house. Instead, you compose the house from smaller pieces: bricks, beams, windows, doors, pipes. Each piece does one thing well. The combination creates something complex.

React components work the same way. The React team explicitly recommends composition over inheritance. In fact, the React documentation states that they have not found any use cases where they would recommend creating component inheritance hierarchies.

Yet many developers coming from object-oriented backgrounds instinctively reach for inheritance. They create `BaseButton` and then extend it to make `PrimaryButton`, `DangerButton`, and `IconButton`. This leads to rigid, fragile hierarchies.

Composition is the alternative. Instead of extending components, you combine them. A button contains an icon. A card contains a header, body, and footer. A page contains a sidebar and a main area. Each piece is independent. The combination is flexible.

This chapter teaches you the composition patterns that make React powerful.

The Problem

You need to build a variety of related components. They share some behavior or structure but differ in specific ways.

```javascript
// The inheritance temptation:
class BaseDialog extends React.Component {
  render() {
    return (
      <div className="dialog-overlay">
        <div className="dialog">
          <div className="dialog-header">{this.renderHeader()}</div>
          <div className="dialog-body">{this.renderBody()}</div>
          <div className="dialog-footer">{this.renderFooter()}</div>
        </div>
      </div>
    );
  }
}

class ConfirmDialog extends BaseDialog {
  renderHeader() { return <h2>Are you sure?</h2>; }
  renderBody() { return <p>{this.props.message}</p>; }
  renderFooter() {
    return (
      <>
        <button onClick={this.props.onCancel}>Cancel</button>
        <button onClick={this.props.onConfirm}>Confirm</button>
      </>
    );
  }
}

class AlertDialog extends BaseDialog {
  renderHeader() { return <h2>Alert</h2>; }
  renderBody() { return <p>{this.props.message}</p>; }
  renderFooter() {
    return <button onClick={this.props.onClose}>OK</button>;
  }
}

// Problems:
// 1. Deep coupling to BaseDialog
// 2. What if you need a dialog without a header?
// 3. What if you need a dialog with two body sections?
// 4. Every variation requires a new subclass
```

The Solution: Composition

Instead of inheriting structure, you compose it from independent pieces:

```javascript
function Dialog({ children }) {
  return (
    <div className="dialog-overlay">
      <div className="dialog">
        {children}
      </div>
    </div>
  );
}

function DialogHeader({ children }) {
  return <div className="dialog-header">{children}</div>;
}

function DialogBody({ children }) {
  return <div className="dialog-body">{children}</div>;
}

function DialogFooter({ children }) {
  return <div className="dialog-footer">{children}</div>;
}
```

Now you compose any dialog you need:

```javascript
// Confirm dialog
function ConfirmDialog({ message, onConfirm, onCancel }) {
  return (
    <Dialog>
      <DialogHeader>
        <h2>Are you sure?</h2>
      </DialogHeader>
      <DialogBody>
        <p>{message}</p>
      </DialogBody>
      <DialogFooter>
        <button onClick={onCancel}>Cancel</button>
        <button onClick={onConfirm}>Confirm</button>
      </DialogFooter>
    </Dialog>
  );
}
```

```
// Alert dialog
function AlertDialog({ message, onClose }) {
  return (
    <Dialog>
      <DialogHeader>
        <h2>Alert</h2>
      </DialogHeader>
      <DialogBody>
        <p>{message}</p>
      </DialogBody>
      <DialogFooter>
        <button onClick={onClose}>OK</button>
      </DialogFooter>
    </Dialog>
  );
}

// Dialog without a header -- no problem!
function SimpleDialog({ children, onClose }) {
  return (
    <Dialog>
      <DialogBody>{children}</DialogBody>
      <DialogFooter>
        <button onClick={onClose}>Close</button>
      </DialogFooter>
    </Dialog>
  );
}
```

```
Inheritance:                    Composition:

     BaseDialog                 Dialog
     /    |    \                  +-- DialogHeader (optional)
  Confirm Alert Form             +-- DialogBody
                                 +-- DialogFooter (optional)

  Fixed structure.              Flexible structure.
  New type = new subclass.      New type = new arrangement.
```

Pattern 1: The `children` Prop

The `children` prop is the foundation of composition in React. Any JSX placed between a component's opening and closing tags becomes `props.children`.

```javascript
// A Card component that accepts any content
function Card({ children }) {
  return <div className="card">{children}</div>;
}

// Use it with anything
<Card>
  <h2>Simple text</h2>
  <p>Just a paragraph inside a card.</p>
</Card>

<Card>
  <img src="photo.jpg" alt="A photo" />
  <p>A card with an image.</p>
</Card>

<Card>
  <UserProfile user={currentUser} />
  <ActivityFeed userId={currentUser.id} />
</Card>
```

The `Card` component does not know or care what its children are. It provides the wrapper. The caller provides the content. This is containment.

Children Can Be Anything

```javascript
// String
<Card>Hello World</Card>

// Elements
<Card><h1>Title</h1></Card>

// Components
<Card><UserAvatar user={user} /></Card>

// Multiple items
<Card>
  <Header />
  <Body />
  <Footer />
</Card>

// Even functions (render props -- covered in a later chapter)
<Card>{(cardState) => <p>Card is {cardState}</p>}</Card>

// Nothing
<Card />
// children is undefined
```

Pattern 2: Named Slots

Sometimes `children` is not enough. You need to pass different content to different parts of a component. This is the slot pattern.

Before: Props for Everything

```javascript
// Messy -- too many content props
function Page({
  headerContent,
  sidebarContent,
  mainContent,
  footerContent
}) {
  return (
    <div className="page">
      <header>{headerContent}</header>
      <aside>{sidebarContent}</aside>
      <main>{mainContent}</main>
      <footer>{footerContent}</footer>
    </div>
  );
}

// Usage is clunky
<Page
  headerContent={<NavBar />}
  sidebarContent={<SideMenu items={menuItems} />}
  mainContent={<ArticleList articles={articles} />}
  footerContent={<Copyright />}
/>
```

After: Named Slot Props

```javascript
function PageLayout({ header, sidebar, children, footer }) {
  return (
    <div className="page-layout">
      {header && <header className="page-header">{header}</header>}
      <div className="page-content">
        {sidebar && <aside className="page-sidebar">{sidebar}</aside>}
        <main className="page-main">{children}</main>
      </div>
      {footer && <footer className="page-footer">{footer}</footer>}
    </div>
  );
}
```

```
// Usage -- clean and readable
<PageLayout
  header={<NavBar />}
  sidebar={<SideMenu items={menuItems} />}
  footer={<Copyright />}
>
  <ArticleList articles={articles} />
</PageLayout>
```

The `children` prop is used for the "main" content. Named props (`header`, `sidebar`, `footer`) serve as named slots for secondary content.

A More Complete Example: Card with Slots

```javascript
function Card({ header, footer, children, variant = "default" }) {
  return (
    <div className={`card card-${variant}`}>
      {header && (
        <div className="card-header">{header}</div>
      )}
      <div className="card-body">{children}</div>
      {footer && (
        <div className="card-footer">{footer}</div>
      )}
    </div>
  );
}

// Product card
<Card
  variant="product"
  header={<img src={product.image} alt={product.name} />}
  footer={
    <div className="card-actions">
      <span className="price">${product.price}</span>
      <button>Add to Cart</button>
    </div>
  }
>
  <h3>{product.name}</h3>
  <p>{product.description}</p>
  <StarRating rating={product.rating} />
</Card>

// User profile card
<Card
  header={<Avatar user={user} size="large" />}
  footer={<button>Send Message</button>}
>
```

```
  <h3>{user.name}</h3>
  <p>{user.bio}</p>
</Card>

// Simple card (no header or footer)
<Card>
  <p>Just some text in a card.</p>
</Card>
```

```
Named slots:

  +-----------------------------+
  |  header slot (optional)     |  <-- props.header
  +-----------------------------+
  |                             |
  |  children (main content)    |  <-- props.children
  |                             |
  +-----------------------------+
  |  footer slot (optional)     |  <-- props.footer
  +-----------------------------+

  Each slot is independent.
  You can fill any combination.
```

Pattern 3: Composition vs Inheritance

React explicitly recommends composition over inheritance. Here is why, with concrete examples.

Inheritance Approach (Avoid This)

```javascript
// Base component
class Button extends React.Component {
  render() {
    return (
      <button
        className={`btn btn-${this.getVariant()}`}
        onClick={this.props.onClick}
      >
        {this.renderIcon()}
        {this.props.label}
      </button>
    );
```

```
  }

  getVariant() { return "default"; }
  renderIcon() { return null; }
}

// Subclasses for variations
class PrimaryButton extends Button {
  getVariant() { return "primary"; }
}

class DangerButton extends Button {
  getVariant() { return "danger"; }
}

class IconButton extends Button {
  renderIcon() {
    return <span className="icon">{this.props.icon}</span>;
  }
}

// What about a primary icon button?
// PrimaryIconButton extends... what? PrimaryButton or IconButton?
// JavaScript only allows single inheritance!
```

Composition Approach (Do This)

```javascript
function Button({ variant = "default", icon, children, onClick }) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {icon && <span className="btn-icon">{icon}</span>}
      {children}
    </button>
  );
}

// Every variation is just a different set of props:
<Button variant="primary">Save</Button>
<Button variant="danger">Delete</Button>
<Button variant="primary" icon={<SaveIcon />}>Save</Button>
<Button variant="danger" icon={<TrashIcon />}>Delete</Button>
```

Need a specialized button? Compose, do not extend:

```javascript
// Specialization through composition
function SubmitButton({ children = "Submit", ...props }) {
  return (
    <Button variant="primary" type="submit" {...props}>
```

```
      {children}
    </Button>
  );
}

function DeleteButton({ onDelete, itemName, ...props }) {
  return (
    <Button
      variant="danger"
      icon={<TrashIcon />}
      onClick={() => {
        if (confirm(`Delete ${itemName}?`)) {
          onDelete();
        }
      }}
      {...props}
    >
      Delete {itemName}
    </Button>
  );
}

// Usage
<SubmitButton onClick={handleSubmit} />
<DeleteButton itemName="this post" onDelete={handleDelete} />
```

Pattern 4: Specialization

Specialization is when a "specific" component renders a "general" component with specific props. It is composition for creating variations.

```javascript
// General component
function Alert({ type, title, children, onDismiss }) {
  const icons = {
    success: "✓",
    error: "✗",
    warning: "⚠",
    info: "ℹ"
  };

  return (
    <div className={`alert alert-${type}`} role="alert">
      <span className="alert-icon">{icons[type]}</span>
      <div className="alert-content">
        {title && <strong className="alert-title">{title}</strong>}
        <div className="alert-message">{children}</div>
```

```
      </div>
      {onDismiss && (
        <button className="alert-dismiss" onClick={onDismiss}>
          x
        </button>
      )}
    </div>
  );
}

// Specialized versions
function SuccessAlert({ title = "Success", children, ...props }) {
  return (
    <Alert type="success" title={title} {...props}>
      {children}
    </Alert>
  );
}

function ErrorAlert({ title = "Error", children, ...props }) {
  return (
    <Alert type="error" title={title} {...props}>
      {children}
    </Alert>
  );
}

function ValidationError({ errors }) {
  if (errors.length === 0) return null;
  return (
    <ErrorAlert title="Please fix the following errors:">
      <ul>
        {errors.map((error, i) => (
          <li key={i}>{error}</li>
        ))}
      </ul>
    </ErrorAlert>
  );
}

// Usage
<SuccessAlert>Your profile has been updated.</SuccessAlert>

<ValidationError errors={["Name is required", "Email is invalid"]} />
```

```
Specialization:

  Alert (general)
    |
    +-- SuccessAlert (Alert with type="success")
    |
    +-- ErrorAlert (Alert with type="error")
    |     |
```

```
    |     +-- ValidationError (ErrorAlert with error list)
    |
    +-- WarningAlert (Alert with type="warning")

  Each specialized version WRAPS the general version.
  No inheritance involved.
```

Pattern 5: Containment

Containment is when a component does not know its children ahead of time. It provides a visual container, and the caller fills it with whatever content they need.

```javascript
// Pure containment: the component is just a box
function Panel({ title, children }) {
  return (
    <section className="panel">
      <div className="panel-header">
        <h3>{title}</h3>
      </div>
      <div className="panel-content">{children}</div>
    </section>
  );
}

// The Panel knows nothing about what goes inside it
<Panel title="User Settings">
  <ToggleSetting label="Dark Mode" value={darkMode} onChange={setDarkMode} />
  <ToggleSetting label="Notifications" value={notifs} onChange={setNotifs} />
  <SelectSetting label="Language" options={languages} value={lang} onChange={setLang} />
</Panel>

<Panel title="Recent Orders">
  <OrderTable orders={recentOrders} />
</Panel>

<Panel title="Quick Stats">
  <StatGrid stats={dashboardStats} />
</Panel>
```

Containment with Multiple Slots

```javascript
function SplitPane({ left, right, ratio = "1:1" }) {
  const [leftFlex, rightFlex] = ratio.split(":").map(Number);

  return (
    <div className="split-pane">
      <div className="split-pane-left" style={{ flex: leftFlex }}>
        {left}
      </div>
      <div className="split-pane-right" style={{ flex: rightFlex }}>
        {right}
      </div>
    </div>
  );
}

// Usage
<SplitPane
  ratio="1:3"
  left={<Navigation items={navItems} />}
  right={<MainContent />}
/>
```

Pattern 6: Layout Components

Layout components are a specific application of containment. They handle positioning and spacing, while content components handle what is displayed.

```javascript
// Stack: vertical layout with consistent spacing
function Stack({ gap = 16, children }) {
  return (
    <div
      style={{
        display: "flex",
        flexDirection: "column",
        gap: `${gap}px`
      }}
    >
      {children}
    </div>
  );
}

// Row: horizontal layout
function Row({ gap = 16, align = "center", justify = "flex-start", children }) {
  return (
    <div
```

```
      style={{
        display: "flex",
        flexDirection: "row",
        gap: `${gap}px`,
        alignItems: align,
        justifyContent: justify
      }}
    >
      {children}
    </div>
  );
}

// Center: center content both ways
function Center({ children }) {
  return (
    <div
      style={{
        display: "flex",
        alignItems: "center",
        justifyContent: "center",
        height: "100%"
      }}
    >
      {children}
    </div>
  );
}

// Container: constrain width and center horizontally
function Container({ maxWidth = 1200, children }) {
  return (
    <div
      style={{
        maxWidth: `${maxWidth}px`,
        margin: "0 auto",
        padding: "0 16px"
      }}
    >
      {children}
    </div>
  );
}
```

Composing layout components:

```javascript
function SettingsPage() {
  return (
    <Container maxWidth={800}>
      <Stack gap={32}>
        <h1>Settings</h1>
```

```
        <Stack gap={16}>
          <h2>Profile</h2>
          <Row gap={16} align="center">
            <Avatar user={currentUser} size={64} />
            <Stack gap={4}>
              <span className="name">{currentUser.name}</span>
              <span className="email">{currentUser.email}</span>
            </Stack>
            <button style={{ marginLeft: "auto" }}>Edit</button>
          </Row>
        </Stack>

        <Stack gap={16}>
          <h2>Preferences</h2>
          <Row justify="space-between">
            <span>Dark Mode</span>
            <Toggle value={darkMode} onChange={setDarkMode} />
          </Row>
          <Row justify="space-between">
            <span>Notifications</span>
            <Toggle value={notifications} onChange={setNotifications} />
          </Row>
        </Stack>
      </Stack>
    </Container>
  );
}
```

```
Layout composition:

  Container (max-width: 800px, centered)
    Stack (gap: 32px, vertical)
      h1
      Stack (gap: 16px)
        h2
        Row (gap: 16px, aligned center)
          Avatar
          Stack (gap: 4px)
            Name
            Email
          Button
      Stack (gap: 16px)
        h2
        Row (space-between)
          Label | Toggle
        Row (space-between)
          Label | Toggle

  Layout = composition of layout primitives.
  Content fills the layout slots.
```

Composing Components with React.Children

Sometimes you need to modify or inspect children. React provides utilities for this:

```javascript
import React from "react";

function Toolbar({ children }) {
  // Add spacing between all children
  return (
    <div className="toolbar">
      {React.Children.map(children, (child, index) => (
        <div
          className="toolbar-item"
          style={{ marginLeft: index > 0 ? "8px" : "0" }}
        >
          {child}
        </div>
      ))}
    </div>
  );
}

// Usage
<Toolbar>
  <Button>Save</Button>
  <Button>Edit</Button>
  <Button variant="danger">Delete</Button>
</Toolbar>
```

Note: Manipulating children directly is an advanced technique. Prefer explicit props (like the slot pattern) when possible. `React.Children` is useful for truly generic components like toolbars, lists, and grids.

When to Use Component Composition

```
+-----------------------------------------------+
|           USE COMPOSITION WHEN:               |
+-----------------------------------------------+
|                                               |
|  * Building UI components with variable       |
|    content (cards, dialogs, layouts)           |
|  * Creating specialized versions of a         |
|    general component                          |
|  * Building layout primitives (Stack, Row)    |
```

```
|  * You need flexibility in what goes inside   |
|    a component                                |
|  * You are tempted to use inheritance to       |
|    create component variations                |
|  * Components need to be independently        |
|    testable and reusable                      |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|    COMPOSITION MIGHT NOT BE ENOUGH WHEN:      |
+-----------------------------------------------+
|                                               |
|  * Child components need to share state with  |
|    the parent (consider Context or Compound   |
|    Components instead)                        |
|  * You need to intercept and modify child     |
|    behavior (consider Render Props)           |
|  * The composition tree gets very deep with   |
|    many levels of nesting                     |
|  * You need to pass the same prop through     |
|    many levels ("prop drilling")              |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Using Inheritance Instead of Composition

```javascript
// WRONG -- inheritance
class PrimaryButton extends Button { /* ... */ }
class LargeButton extends Button { /* ... */ }
class LargePrimaryButton extends ???  // Impossible!

// CORRECT -- composition via props
<Button variant="primary" size="large">Click Me</Button>
```

Mistake 2: Prop Drilling Through Composition

```javascript
// WRONG -- passing theme through every level
<App theme={theme}>
  <Layout theme={theme}>
```

```
    <Sidebar theme={theme}>
      <NavItem theme={theme}>Home</NavItem>
    </Sidebar>
  </Layout>
</App>

// CORRECT -- use Context for cross-cutting concerns
const ThemeContext = React.createContext("light");

function App() {
  return (
    <ThemeContext.Provider value={theme}>
      <Layout>
        <Sidebar>
          <NavItem>Home</NavItem>
        </Sidebar>
      </Layout>
    </ThemeContext.Provider>
  );
}
```

Mistake 3: Over-Composing Simple Things

```javascript
// OVER-COMPOSED -- too many tiny components
<Card>
  <CardBorder>
    <CardPadding>
      <CardShadow>
        <CardBackground>
          <CardContent>Hello</CardContent>
        </CardBackground>
      </CardShadow>
    </CardPadding>
  </CardBorder>
</Card>

// BETTER -- one component with props
<Card border shadow padding="16px" background="white">
  Hello
</Card>
```

Mistake 4: Not Using children

```javascript
// WRONG -- passing content as a prop
<Card content={<p>Hello world</p>} />
```

```
// CORRECT -- use children for primary content
<Card>
  <p>Hello world</p>
</Card>
```

Best Practices

1. Use `children` for primary content -- it is the most natural and readable pattern.
2. Use named props for secondary slots -- `header`, `footer`, `sidebar`, `icon` are common slot names.
3. Prefer composition over inheritance -- always. The React team recommends this, and years of practice have proven it.
4. Keep layout and content separate -- layout components handle positioning; content components handle data.
5. Make components flexible by default -- accept `children` and optional slots rather than hardcoding content.
6. Use specialization for common variants -- create `SuccessAlert`, `ErrorAlert` wrappers instead of requiring callers to remember `type="error"` every time.

Quick Summary

Component composition is React's primary strategy for building complex UIs from simple pieces. Instead of inheriting behavior, components are combined using `children`, named slots, specialization, and containment. Layout components handle positioning while content components fill the slots. This approach is more flexible, more testable, and more maintainable than class inheritance.

```
Composition recipe:

  1. Build small, focused components
  2. Use children for primary content
  3. Use named props for secondary slots
  4. Specialize general components by wrapping them
  5. Compose layout with Stack, Row, Container
  6. Let callers decide what goes inside
```

Key Points

- Component composition combines simple components to build complex UIs
- The `children` prop is the foundation -- it lets callers put anything inside a component
- Named slot props (`header`, `footer`, `sidebar`) provide multiple insertion points
- Specialization creates specific versions by wrapping general components with preset props
- Containment creates visual containers that accept any content
- Layout components (Stack, Row, Container) handle positioning separately from content
- React explicitly recommends composition over inheritance
- Avoid prop drilling through composition -- use Context for cross-cutting data

---

Practice Questions

1. What is the difference between containment and specialization? Give an example of each.
2. Why does React recommend composition over inheritance? What problems does inheritance cause?
3. A colleague has built `BaseInput`, `TextInput extends BaseInput`, and `EmailInput extends TextInput`. How would you refactor this to use composition?
4. When would you use named slot props instead of just `children`? Give a specific example.
5. What is the difference between a layout component and a content component? Why is this separation useful?

Exercises

Exercise 1: Composable Modal System

Build a modal system using composition. Create `Modal`, `ModalHeader`, `ModalBody`, and `ModalFooter` components. Then compose three different modals: a confirmation modal, a form

modal, and a success modal.

```javascript
// Build these components:
function Modal({ isOpen, onClose, size, children }) { /* ... */ }
function ModalHeader({ children }) { /* ... */ }
function ModalBody({ children }) { /* ... */ }
function ModalFooter({ children }) { /* ... */ }

// Then compose:
// 1. ConfirmDeleteModal
// 2. EditProfileModal (with a form)
// 3. SuccessModal (just a message and close button)
```

Exercise 2: Layout System

Build a complete layout system with `PageLayout`, `Stack`, `Row`, `Grid`, `Container`, and `Spacer` components. Then use them to build a dashboard page layout.

```javascript
// Build these layout primitives:
function Stack({ gap, children }) { /* ... */ }
function Row({ gap, align, justify, children }) { /* ... */ }
function Grid({ columns, gap, children }) { /* ... */ }
function Container({ maxWidth, children }) { /* ... */ }
function Spacer({ size }) { /* ... */ }

// Then compose a dashboard layout
function DashboardPage() {
  // Use your layout primitives to create a
  // responsive dashboard with sidebar, header,
  // stat cards, and a data table
}
```

Exercise 3: Composable Form

Build a form system where `Form`, `FormSection`, `FormField`, and `FormActions` are composable. Different pages can create different forms by composing these pieces differently.

```javascript
function Form({ onSubmit, children }) { /* ... */ }
function FormSection({ title, description, children }) { /* ... */ }
function FormField({ label, error, required, children }) { /* ... */ }
function FormActions({ children }) { /* ... */ }

// Compose a registration form and a settings form
// using the same building blocks
```

What Is Next?

You have learned how to compose components to build flexible UIs. The next chapter introduces the Container/Presentational pattern, which takes composition a step further by separating components into two categories: containers (which handle data and logic) and presentational components (which handle rendering). This separation makes components more reusable and easier to test.

Chapter 15: The Container/Presentational Pattern

What You Will Learn

- What the Container/Presentational pattern is and what problem it solves
- How to separate data-fetching logic from rendering logic
- How container components fetch, transform, and manage state
- How presentational components focus purely on displaying data
- How React Hooks changed this pattern and when the separation is still valuable
- When to use this pattern and when it adds unnecessary complexity

Why This Chapter Matters

Think about a restaurant. The kitchen prepares the food. The waiter presents it to you. These are two different roles with different responsibilities.

The kitchen (container) handles all the complex work: sourcing ingredients, cooking, managing orders, handling timing. The waiter (presentational) handles presentation: carrying the plate, describing the dish, refilling your water. Neither does the other's job.

Now imagine a restaurant where the waiter also cooks your food at your table. Chaotic, right? That is what happens when a React component both fetches data AND renders complex UI. It becomes hard to test, hard to reuse, and hard to understand.

The Container/Presentational pattern (also called Smart/Dumb components) splits components into two categories:

- Container components handle data, state, and logic
- Presentational components handle rendering and styling

This pattern was introduced by Dan Abramov (co-creator of Redux) in 2015. It became one of the most influential patterns in React. Years later, Dan himself updated his guidance, noting that Hooks can replace much of the need for this separation. This chapter covers both the original pattern and the modern perspective.

The Problem

Components that mix data logic and rendering become messy, untestable, and unreusable.

```javascript
// This component does EVERYTHING:
function UserDashboard() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [sortBy, setSortBy] = useState("name");
  const [filterRole, setFilterRole] = useState("all");
  const [searchQuery, setSearchQuery] = useState("");

  useEffect(() => {
    async function fetchUsers() {
      try {
        setLoading(true);
        const response = await fetch("/api/users");
        if (!response.ok) throw new Error("Failed to fetch");
        const data = await response.json();
        setUsers(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }
    fetchUsers();
  }, []);

  const filteredUsers = users
    .filter((u) => filterRole === "all" || u.role === filterRole)
    .filter((u) =>
      u.name.toLowerCase().includes(searchQuery.toLowerCase())
    )
    .sort((a, b) => a[sortBy].localeCompare(b[sortBy]));

  if (loading) {
    return (
      <div className="dashboard">
        <div className="spinner-container">
```

```
          <div className="spinner" />
          <p>Loading users...</p>
        </div>
      </div>
    );
  }

  if (error) {
    return (
      <div className="dashboard">
        <div className="error-container">
          <h2>Something went wrong</h2>
          <p>{error}</p>
          <button onClick={() => window.location.reload()}>
            Retry
          </button>
        </div>
      </div>
    );
  }

  return (
    <div className="dashboard">
      <div className="dashboard-header">
        <h1>Users ({filteredUsers.length})</h1>
        <input
          type="search"
          placeholder="Search users..."
          value={searchQuery}
          onChange={(e) => setSearchQuery(e.target.value)}
        />
      </div>

      <div className="dashboard-filters">
        <select
          value={filterRole}
          onChange={(e) => setFilterRole(e.target.value)}
        >
          <option value="all">All Roles</option>
          <option value="admin">Admin</option>
          <option value="editor">Editor</option>
          <option value="viewer">Viewer</option>
        </select>
        <select
          value={sortBy}
          onChange={(e) => setSortBy(e.target.value)}
        >
          <option value="name">Sort by Name</option>
          <option value="email">Sort by Email</option>
          <option value="role">Sort by Role</option>
        </select>
      </div>

      <table className="user-table">
        <thead>
```

```
          <tr>
            <th>Name</th>
            <th>Email</th>
            <th>Role</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {filteredUsers.map((user) => (
            <tr key={user.id}>
              <td>{user.name}</td>
              <td>{user.email}</td>
              <td>
                <span className={`badge badge-${user.role}`}>
                  {user.role}
                </span>
              </td>
              <td>
                <button onClick={() => handleEdit(user.id)}>Edit</button>
                <button onClick={() => handleDelete(user.id)}>Delete</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );
}
```

This 100+ line component:

- Fetches data from an API
- Manages loading and error states
- Filters, searches, and sorts data
- Renders the entire UI including loading and error states
- Cannot be reused with different data sources
- Cannot be tested without mocking the API
- Cannot be displayed in a design system (Storybook) without a running API

The Solution: Container/Presentational Split

Split the monolith into two components:

```
+--------------------+         +----------------------+
| Container          |         | Presentational       |
| (UserDashboard     |  props  | (UserDashboard       |
|  Container)        | ------> |  View)               |
|                    |         |                      |
| - Fetches data     |         | - Receives data      |
| - Manages state    |         | - Renders UI         |
| - Handles errors   |         | - Handles styling    |
| - Transforms data  |         | - Calls callbacks    |
| - Business logic   |         | - No side effects    |
+--------------------+         +----------------------+
```

Presentational Component

The presentational component receives all data through props and renders the UI. It has no side effects, no API calls, no state management (except UI state like "is dropdown open").

```javascript
// UserDashboardView.jsx -- Presentational

function UserDashboardView({
  users,
  loading,
  error,
  searchQuery,
  onSearchChange,
  filterRole,
  onFilterChange,
  sortBy,
  onSortChange,
  onEdit,
  onDelete,
  onRetry
}) {
  if (loading) {
    return (
      <div className="dashboard">
        <LoadingSpinner message="Loading users..." />
      </div>
    );
  }

  if (error) {
    return (
      <div className="dashboard">
        <ErrorMessage message={error} onRetry={onRetry} />
      </div>
    );
  }
```

```
  return (
    <div className="dashboard">
      <DashboardHeader
        count={users.length}
        searchQuery={searchQuery}
        onSearchChange={onSearchChange}
      />
      <DashboardFilters
        filterRole={filterRole}
        onFilterChange={onFilterChange}
        sortBy={sortBy}
        onSortChange={onSortChange}
      />
      <UserTable
        users={users}
        onEdit={onEdit}
        onDelete={onDelete}
      />
    </div>
  );
}

function DashboardHeader({ count, searchQuery, onSearchChange }) {
  return (
    <div className="dashboard-header">
      <h1>Users ({count})</h1>
      <input
        type="search"
        placeholder="Search users..."
        value={searchQuery}
        onChange={(e) => onSearchChange(e.target.value)}
      />
    </div>
  );
}

function DashboardFilters({ filterRole, onFilterChange, sortBy, onSortChange }) {
  return (
    <div className="dashboard-filters">
      <select value={filterRole} onChange={(e) => onFilterChange(e.target.value)}>
        <option value="all">All Roles</option>
        <option value="admin">Admin</option>
        <option value="editor">Editor</option>
        <option value="viewer">Viewer</option>
      </select>
      <select value={sortBy} onChange={(e) => onSortChange(e.target.value)}>
        <option value="name">Sort by Name</option>
        <option value="email">Sort by Email</option>
        <option value="role">Sort by Role</option>
      </select>
    </div>
  );
}

function UserTable({ users, onEdit, onDelete }) {
```

```
  return (
    <table className="user-table">
      <thead>
        <tr>
          <th>Name</th>
          <th>Email</th>
          <th>Role</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        {users.map((user) => (
          <UserRow
            key={user.id}
            user={user}
            onEdit={onEdit}
            onDelete={onDelete}
          />
        ))}
      </tbody>
    </table>
  );
}

function UserRow({ user, onEdit, onDelete }) {
  return (
    <tr>
      <td>{user.name}</td>
      <td>{user.email}</td>
      <td>
        <span className={`badge badge-${user.role}`}>{user.role}</span>
      </td>
      <td>
        <button onClick={() => onEdit(user.id)}>Edit</button>
        <button onClick={() => onDelete(user.id)}>Delete</button>
      </td>
    </tr>
  );
}
```

Container Component

The container component handles all logic and passes data down to the presentational component:

```javascript
// UserDashboardContainer.jsx -- Container

function UserDashboardContainer() {
  const [users, setUsers] = useState([]);
```

```
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [sortBy, setSortBy] = useState("name");
  const [filterRole, setFilterRole] = useState("all");
  const [searchQuery, setSearchQuery] = useState("");

  const fetchUsers = useCallback(async () => {
    try {
      setLoading(true);
      setError(null);
      const response = await fetch("/api/users");
      if (!response.ok) throw new Error("Failed to fetch users");
      const data = await response.json();
      setUsers(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => {
    fetchUsers();
  }, [fetchUsers]);

  // Data transformation
  const processedUsers = useMemo(() => {
    return users
      .filter((u) => filterRole === "all" || u.role === filterRole)
      .filter((u) =>
        u.name.toLowerCase().includes(searchQuery.toLowerCase())
      )
      .sort((a, b) => {
        const aVal = a[sortBy] || "";
        const bVal = b[sortBy] || "";
        return aVal.localeCompare(bVal);
      });
  }, [users, filterRole, searchQuery, sortBy]);

  // Event handlers
  const handleEdit = useCallback((userId) => {
    // Navigate to edit page or open modal
    console.log("Edit user:", userId);
  }, []);

  const handleDelete = useCallback(async (userId) => {
    if (!window.confirm("Delete this user?")) return;
    try {
      await fetch(`/api/users/${userId}`, { method: "DELETE" });
      setUsers((prev) => prev.filter((u) => u.id !== userId));
    } catch (err) {
      setError("Failed to delete user");
    }
  }, []);
```

```
  // Pass everything to the presentational component
  return (
    <UserDashboardView
      users={processedUsers}
      loading={loading}
      error={error}
      searchQuery={searchQuery}
      onSearchChange={setSearchQuery}
      filterRole={filterRole}
      onFilterChange={setFilterRole}
      sortBy={sortBy}
      onSortChange={setSortBy}
      onEdit={handleEdit}
      onDelete={handleDelete}
      onRetry={fetchUsers}
    />
  );
}
```

The Benefits

```
+-----------------------------------------------+
|  BENEFIT                 | HOW                 |
+-----------------------------------------------+
|                          |                     |
|  Reusability             | Presentational      |
|                          | component works     |
|                          | with any data       |
|                          | source              |
|                          |                     |
|  Testability             | Test UI without     |
|                          | API. Test logic     |
|                          | without rendering.  |
|                          |                     |
|  Storybook/Design System | Presentational      |
|                          | components work in  |
|                          | Storybook with      |
|                          | mock data           |
|                          |                     |
|  Team Collaboration      | Designer works on   |
|                          | presentational.     |
|                          | Backend dev works   |
|                          | on container.       |
|                          |                     |
|  Separation of Concerns  | Each file has one   |
|                          | job                 |
+-----------------------------------------------+
```

Testing Becomes Easy

```javascript
// Testing the presentational component -- no API needed!
describe("UserDashboardView", () => {
  it("renders users in a table", () => {
    const users = [
      { id: 1, name: "Alice", email: "alice@test.com", role: "admin" },
      { id: 2, name: "Bob", email: "bob@test.com", role: "editor" }
    ];

    render(
      <UserDashboardView
        users={users}
        loading={false}
        error={null}
        searchQuery=""
        onSearchChange={jest.fn()}
        filterRole="all"
        onFilterChange={jest.fn()}
        sortBy="name"
        onSortChange={jest.fn()}
        onEdit={jest.fn()}
        onDelete={jest.fn()}
        onRetry={jest.fn()}
      />
    );

    expect(screen.getByText("Alice")).toBeInTheDocument();
    expect(screen.getByText("Bob")).toBeInTheDocument();
  });

  it("shows loading spinner when loading", () => {
    render(
      <UserDashboardView loading={true} users={[]} /* ... */ />
    );

    expect(screen.getByText("Loading users...")).toBeInTheDocument();
  });

  it("shows error message with retry button", () => {
    const onRetry = jest.fn();
    render(
      <UserDashboardView
        loading={false}
        error="Network error"
        users={[]}
        onRetry={onRetry}
        /* ... */
      />
    );

    expect(screen.getByText("Network error")).toBeInTheDocument();
    fireEvent.click(screen.getByText("Retry"));
```

```
    expect(onRetry).toHaveBeenCalled();
  });
});
```

Reuse with Different Data Sources

```javascript
// Same presentational component, different data source!

// From REST API
function UserDashboardFromAPI() {
  const { users, loading, error } = useUsersAPI();
  return <UserDashboardView users={users} loading={loading} error={error} /* ... */ />;
}

// From GraphQL
function UserDashboardFromGraphQL() {
  const { data, loading, error } = useQuery(GET_USERS);
  return <UserDashboardView users={data?.users} loading={loading} error={error} /* ... ...
}

// From local state (Storybook, testing)
function UserDashboardStory() {
  return (
    <UserDashboardView
      users={mockUsers}
      loading={false}
      error={null}
      /* ... */
    />
  );
}
```

How Hooks Changed This Pattern

In 2019, Dan Abramov updated his original article to say he no longer recommends splitting components into containers and presentational components as a strict rule. Why? Because Hooks can extract the logic without needing a separate container component.

Before Hooks: Container Component Required

```javascript
// Container component exists just to provide data
function UserListContainer() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/users")
      .then((r) => r.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  return <UserList users={users} loading={loading} />;
}
```

After Hooks: Custom Hook Replaces Container

```javascript
// Custom hook extracts the logic
function useUsers() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("/api/users")
      .then((r) => r.json())
      .then(setUsers)
      .catch((e) => setError(e.message))
      .finally(() => setLoading(false));
  }, []);

  return { users, loading, error };
}

// Component uses the hook directly -- no container needed
function UserList() {
  const { users, loading, error } = useUsers();

  if (loading) return <Spinner />;
  if (error) return <ErrorMessage message={error} />;

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

```
Evolution:

  2015: Container + Presentational (two components)

    UserListContainer --> UserList
         (data)            (UI)

  2019+: Hook + Component (one component + one hook)

    useUsers() hook --> UserList
       (data)            (UI + uses hook)

  The separation still exists,
  but it moved from components to hooks.
```

When Hooks Are Enough

For simple cases, a custom hook is cleaner than a separate container:

```javascript
// Simple case -- hook is enough
function ProductCard({ productId }) {
  const { product, loading } = useProduct(productId);

  if (loading) return <CardSkeleton />;
  return (
    <div className="product-card">
      <img src={product.image} alt={product.name} />
      <h3>{product.name}</h3>
      <p>${product.price}</p>
    </div>
  );
}
```

When You Still Want the Full Split

For complex cases, the container/presentational split is still valuable:

```javascript
// Complex case -- full split still helps
// When:
// 1. The presentational component is reused with different data sources
// 2. You need the component in Storybook without a real backend
// 3. The data logic is complex (filters, sorting, pagination, caching)
// 4. Different team members work on logic vs UI

function DataTable({ columns, rows, loading, error, onSort, onFilter, onPageChange }) {
```

```
  // This presentational component is used by:
  // - UserListContainer (fetches from /api/users)
  // - ProductListContainer (fetches from /api/products)
  // - OrderListContainer (fetches from /api/orders)
  // - Storybook stories (uses mock data)
  // - Tests (uses fixture data)
}
```

The Modern Approach: Hooks + Presentational When Needed

The modern recommendation is:

1. Start with hooks to extract data logic
2. Split into presentational only when you need reuse across different data sources
3. Do not split just because "the pattern says so"

```javascript
// Step 1: Start simple with a hook
function useUserDashboard() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [filters, setFilters] = useState({
    role: "all",
    search: "",
    sortBy: "name"
  });

  useEffect(() => {
    fetchUsers().then(setUsers).finally(() => setLoading(false));
  }, []);

  const filteredUsers = useMemo(() => {
    return users
      .filter((u) => filters.role === "all" || u.role === filters.role)
      .filter((u) => u.name.toLowerCase().includes(filters.search.toLowerCase()))
      .sort((a, b) => a[filters.sortBy].localeCompare(b[filters.sortBy]));
  }, [users, filters]);

  return {
    users: filteredUsers,
    loading,
    error,
    filters,
```

```
    setFilters,
    refetch: () => fetchUsers().then(setUsers)
  };
}

// Step 2: Use the hook in your component
function UserDashboard() {
  const { users, loading, error, filters, setFilters, refetch } =
    useUserDashboard();

  // Render...
}

// Step 3: ONLY IF NEEDED -- extract a presentational component
// for Storybook, reuse, or testing purposes
```

Identifying Container vs Presentational

Here is a quick reference:

```
+----------------------------+----------------------------+
| Container                  | Presentational             |
+----------------------------+----------------------------+
| Fetches data               | Receives data via props    |
| Manages state              | Minimal state (UI only)    |
| Calls APIs                 | Calls callbacks via props  |
| Handles side effects       | No side effects            |
| Often has no DOM markup    | Full of DOM/JSX            |
| Connects to stores/context | No store/context access    |
| Not reusable (specific)    | Highly reusable (generic)  |
| Tested with integration    | Tested with unit tests     |
+----------------------------+----------------------------+
```

More Examples

```javascript
// CONTAINER: handles all the data logic
function CommentSectionContainer({ postId }) {
  const [comments, setComments] = useState([]);
  const [loading, setLoading] = useState(true);
  const [newComment, setNewComment] = useState("");

  useEffect(() => {
    api.comments.getByPost(postId).then((data) => {
```

```
      setComments(data);
      setLoading(false);
    });
  }, [postId]);

  async function handleSubmit() {
    const comment = await api.comments.create(postId, {
      body: newComment
    });
    setComments((prev) => [...prev, comment]);
    setNewComment("");
  }

  async function handleDelete(commentId) {
    await api.comments.delete(commentId);
    setComments((prev) => prev.filter((c) => c.id !== commentId));
  }

  return (
    <CommentSection
      comments={comments}
      loading={loading}
      newComment={newComment}
      onNewCommentChange={setNewComment}
      onSubmit={handleSubmit}
      onDelete={handleDelete}
    />
  );
}

// PRESENTATIONAL: pure rendering
function CommentSection({
  comments,
  loading,
  newComment,
  onNewCommentChange,
  onSubmit,
  onDelete
}) {
  if (loading) return <Spinner />;

  return (
    <div className="comments">
      <h3>Comments ({comments.length})</h3>

      <div className="comment-list">
        {comments.map((comment) => (
          <Comment
            key={comment.id}
            comment={comment}
            onDelete={onDelete}
          />
        ))}
      </div>
```

```
      <div className="comment-form">
        <textarea
          value={newComment}
          onChange={(e) => onNewCommentChange(e.target.value)}
          placeholder="Write a comment..."
        />
        <button
          onClick={onSubmit}
          disabled={!newComment.trim()}
        >
          Post Comment
        </button>
      </div>
    </div>
  );
}

function Comment({ comment, onDelete }) {
  return (
    <div className="comment">
      <div className="comment-header">
        <strong>{comment.author.name}</strong>
        <span className="comment-date">
          {new Date(comment.createdAt).toLocaleDateString()}
        </span>
      </div>
      <p className="comment-body">{comment.body}</p>
      <button
        className="comment-delete"
        onClick={() => onDelete(comment.id)}
      >
        Delete
      </button>
    </div>
  );
}
```

When to Use the Container/Presentational Pattern

```
+-----------------------------------------------+
|           USE THIS PATTERN WHEN:              |
+-----------------------------------------------+
|                                               |
|  * The presentational component is reused     |
|    across different data sources              |
|  * You need components in Storybook or a      |
|    design system without real backends        |
|  * The data logic is complex enough to        |
```

```
|    warrant isolation                          |
|  * Different team members handle data vs UI   |
|  * You want to test UI rendering separately   |
|    from data fetching                         |
|                                               |
+-----------------------------------------------+

+-----------------------------------------------+
|       DO NOT USE THIS PATTERN WHEN:           |
+-----------------------------------------------+
|                                               |
|  * A custom hook extracts the logic cleanly   |
|  * The component is simple and only used once |
|  * The split would create two trivial files   |
|    that are harder to navigate than one file  |
|  * You are splitting "just because" without   |
|    a concrete benefit                         |
|  * The presentational component would have    |
|    20+ props (too much prop passing)          |
|                                               |
+-----------------------------------------------+
```

Common Mistakes

Mistake 1: Splitting Too Early

```javascript
// WRONG -- premature split for a simple component
// UserNameContainer.jsx
function UserNameContainer({ userId }) {
  const user = useUser(userId);
  return <UserName name={user.name} />;
}

// UserName.jsx
function UserName({ name }) {
  return <span className="user-name">{name}</span>;
}

// CORRECT -- just use a hook, no split needed
function UserName({ userId }) {
  const user = useUser(userId);
  return <span className="user-name">{user.name}</span>;
}
```

Mistake 2: Too Many Props

```javascript
// WRONG -- the presentational component has too many props
<UserDashboardView
  users={users}
  loading={loading}
  error={error}
  searchQuery={searchQuery}
  onSearchChange={onSearchChange}
  filterRole={filterRole}
  onFilterChange={onFilterChange}
  sortBy={sortBy}
  onSortChange={onSortChange}
  selectedUsers={selectedUsers}
  onSelectUser={onSelectUser}
  onSelectAll={onSelectAll}
  page={page}
  pageSize={pageSize}
  totalPages={totalPages}
  onPageChange={onPageChange}
  onPageSizeChange={onPageSizeChange}
  onEdit={onEdit}
  onDelete={onDelete}
  onExport={onExport}
  onRetry={onRetry}
/>

// BETTER -- group related props into objects
<UserDashboardView
  data={{ users, loading, error }}
  filters={{ searchQuery, filterRole, sortBy }}
  pagination={{ page, pageSize, totalPages }}
  selection={{ selectedUsers }}
  onFilterChange={handleFilterChange}
  onPageChange={handlePageChange}
  onSelectionChange={handleSelectionChange}
  actions={{ onEdit, onDelete, onExport, onRetry }}
/>
```

Mistake 3: Presentational Components with Side Effects

```javascript
// WRONG -- presentational component fetches data
function UserCard({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`).then(r => r.json()).then(setUser);
  }, [userId]);

  return <div>{user?.name}</div>;
}
```

```
// CORRECT -- presentational receives data
function UserCard({ user }) {
  return <div>{user.name}</div>;
}
```

Mistake 4: Container Components with Markup

```javascript
// WRONG -- container has styling and layout
function UserListContainer() {
  const users = useUsers();

  return (
    <div className="fancy-layout with-gradient">
      <div className="animated-border">
        <UserList users={users} />
      </div>
    </div>
  );
}

// CORRECT -- container only passes data, no styling
function UserListContainer() {
  const users = useUsers();
  return <UserList users={users} />;
}
```

Best Practices

1. Start with hooks -- extract data logic into custom hooks first. Only split into container/presentational if you have a concrete reason (reuse, Storybook, team split).
2. Presentational components should be pure -- given the same props, they render the same output. No API calls, no side effects.
3. Container components should have minimal markup -- ideally just a single presentational component. Layout and styling belong in the presentational component.
4. Use TypeScript or PropTypes -- the props interface between container and presentational is a contract. Type it clearly.
5. Group related props -- if you have 15+ props, group them into objects (data, filters, actions, etc.).
6. Name clearly -- use suffixes like `Container`, `View`, or `Presentation` to make the role obvious. Or use a `hooks/` folder for the data logic and keep the component file for

rendering.

Quick Summary

The Container/Presentational pattern separates data logic from rendering. Container components fetch data, manage state, and handle side effects. Presentational components receive data through props and render the UI. React Hooks shifted the pattern: instead of a separate container component, a custom hook can extract the data logic while the component handles both the hook call and the rendering. The full split is still valuable when presentational components need to be reused across different data sources, displayed in Storybook, or tested in isolation.

```
2015 approach:                    2019+ approach:

  Container Component             Custom Hook
    (fetches data)                  (fetches data)
          |                               |
          v                               v
  Presentational Component         Component
    (renders UI)                    (calls hook + renders UI)

                    OR when reuse is needed:

                    Custom Hook
                        |
                        v
                  Container Component
                        |
                        v
                  Presentational Component
                    (reusable across data sources)
```

Key Points

- Container components handle data, state, and side effects
- Presentational components handle rendering and receive everything via props
- This separation improves testability, reusability, and Storybook support
- React Hooks can replace container components with custom hooks

- The full container/presentational split is still useful when presentational components need reuse
- Do not split prematurely -- start with hooks and split only when needed
- Keep presentational components pure (no side effects, no API calls)
- Keep container components minimal (no styling, no layout markup)

Practice Questions

1. What is the main difference between a container component and a presentational component? Give one example of each.
2. How did React Hooks change the Container/Presentational pattern? When is a custom hook sufficient, and when do you still need the full split?
3. A presentational component has 18 props. What are two strategies to reduce this complexity?
4. Your team uses Storybook for UI development. How does the Container/Presentational pattern help with this?
5. A junior developer suggests that every component should be split into container and presentational. Why is this advice too rigid?

Exercises

Exercise 1: Refactor a Monolith Component

Take this monolith component and refactor it into a custom hook and a presentational component:

```javascript
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [newTodo, setNewTodo] = useState("");
  const [filter, setFilter] = useState("all");

  useEffect(() => {
    fetch("/api/todos")
      .then((r) => r.json())
      .then(setTodos);
```

```
  }, []);

  function addTodo() {
    fetch("/api/todos", {
      method: "POST",
      body: JSON.stringify({ text: newTodo, done: false })
    })
      .then((r) => r.json())
      .then((todo) => {
        setTodos([...todos, todo]);
        setNewTodo("");
      });
  }

  function toggleTodo(id) {
    const todo = todos.find((t) => t.id === id);
    fetch(`/api/todos/${id}`, {
      method: "PATCH",
      body: JSON.stringify({ done: !todo.done })
    }).then(() => {
      setTodos(todos.map((t) =>
        t.id === id ? { ...t, done: !t.done } : t
      ));
    });
  }

  const filtered = todos.filter((t) => {
    if (filter === "active") return !t.done;
    if (filter === "done") return t.done;
    return true;
  });

  return (
    <div>
      <h1>Todos</h1>
      <input value={newTodo} onChange={(e) => setNewTodo(e.target.value)} />
      <button onClick={addTodo}>Add</button>
      <div>
        <button onClick={() => setFilter("all")}>All</button>
        <button onClick={() => setFilter("active")}>Active</button>
        <button onClick={() => setFilter("done")}>Done</button>
      </div>
      <ul>
        {filtered.map((todo) => (
          <li
            key={todo.id}
            onClick={() => toggleTodo(todo.id)}
            style={{ textDecoration: todo.done ? "line-through" : "none" }}
          >
            {todo.text}
          </li>
        ))}
      </ul>
    </div>
  );
```

```
}
```

Exercise 2: Reusable Data Table

Build a presentational `DataTable` component that can display any tabular data. Then create two different container components (or hooks) that feed it user data and product data respectively.

```javascript
// Presentational -- works with any data
function DataTable({ columns, rows, onRowClick, sortBy, onSort }) {
  // Your code here
}

// Container 1: Users
function UserTable() {
  // Fetch users, transform into rows, render DataTable
}

// Container 2: Products
function ProductTable() {
  // Fetch products, transform into rows, render DataTable
}
```

Exercise 3: Storybook-Ready Components

Take a component you have built previously (or the UserDashboard from this chapter) and create a version that works in Storybook. Write three stories: default state, loading state, and error state.

```javascript
// UserDashboard.stories.jsx

export default { title: "UserDashboard" };

export const Default = () => (
  <UserDashboardView
    users={mockUsers}
    loading={false}
    error={null}
    /* ... mock all props */
  />
);

export const Loading = () => (
  <UserDashboardView loading={true} users={[]} /* ... */ />
```

```
);

export const Error = () => (
  <UserDashboardView
    loading={false}
    error="Failed to fetch users"
    users={[]}
    /* ... */
  />
);
```

What Is Next?

You have learned how to separate data logic from UI rendering. The next chapter introduces Higher-Order Components (HOCs), another way to share behavior across components. HOCs wrap a component with additional functionality -- similar to how decorators wrap functions. You will learn when HOCs are useful, how they compare to hooks, and the common pitfalls to avoid.

Chapter 16: Higher-Order Components

What You Will Learn

- What a Higher-Order Component (HOC) is and how it works mechanically
- How to build practical HOCs like `withAuth`, `withLoading`, and `withLogger`
- How to forward refs through HOCs correctly
- How to compose multiple HOCs together
- Why React hooks have replaced most HOC use cases
- When HOCs still make sense in modern codebases

Why This Chapter Matters

Imagine you run a delivery company. Every driver needs a GPS tracker, a fuel card, and a uniform before they can start working. You could equip each driver individually, filling out paperwork from scratch every time. Or you could create an onboarding process -- a function that takes a new driver and returns an equipped, ready-to-go driver.

Higher-Order Components work the same way. They are functions that take a component and return a new, enhanced component -- one that has been "equipped" with extra behavior like authentication checks, loading states, or logging. The original component does not need to know about any of this. It just does its job while the HOC handles the cross-cutting concern.

HOCs were the dominant pattern for code reuse in React before hooks arrived. Understanding them matters for three reasons:

- Many production codebases still use them heavily
- Some problems (like decorating class components) are still best solved with HOCs
- The concept of wrapping and enhancing functions is fundamental to programming beyond React

The Core Concept: Functions That Return Components

A Higher-Order Component is a function that takes a component as an argument and returns a new component.

```
+-------------------------------------------------------+
|                  Higher-Order Component               |
|                                                       |
|   Input:              Output:                         |
|   +------------+      +-------------------------+    |
|   | Component  | ---> | Enhanced Component      |    |
|   | (original) |      | (original + new powers) |    |
|   +------------+      +-------------------------+    |
|                                                       |
+-------------------------------------------------------+
```

Here is the simplest possible HOC:

```jsx
// A HOC is just a function
function withGreeting(WrappedComponent) {
  // It returns a new component
  return function EnhancedComponent(props) {
    return (
      <div>
        <p>Hello! Here is the wrapped component:</p>
        <WrappedComponent {...props} />
      </div>
    );
  };
}

// Original component
function UserProfile({ name }) {
  return <h2>{name}</h2>;
}

// Enhanced component
const UserProfileWithGreeting = withGreeting(UserProfile);

// Usage
function App() {
  return <UserProfileWithGreeting name="Alice" />;
}
```

```
// Output:
// Hello! Here is the wrapped component:
// Alice
```

The key insight: `withGreeting` does not modify `UserProfile`. It creates a brand new component that renders `UserProfile` inside it and passes all props through.

Pattern 1: withAuth -- Protecting Routes

The Problem

Multiple pages need authentication checks. Without a HOC, every component repeats the same logic:

```jsx
// Dashboard.jsx
function Dashboard({ user }) {
  if (!user) {
    return <Navigate to="/login" />;
  }
  return <h1>Welcome to Dashboard</h1>;
}

// Settings.jsx
function Settings({ user }) {
  if (!user) {
    return <Navigate to="/login" />;
  }
  return <h1>Settings Page</h1>;
}

// Profile.jsx
function Profile({ user }) {
  if (!user) {
    return <Navigate to="/login" />;
  }
  return <h1>Profile Page</h1>;
}
```

Three components, three identical auth checks. If you change the redirect path, you need to update all three.

The Solution

```jsx
function withAuth(WrappedComponent) {
  return function AuthenticatedComponent(props) {
    const { user } = useContext(AuthContext);

    if (!user) {
      return <Navigate to="/login" />;
    }

    return <WrappedComponent {...props} user={user} />;
  };
}
```

Now the original components are clean:

```jsx
// Dashboard.jsx -- no auth logic here
function Dashboard({ user }) {
  return <h1>Welcome, {user.name}!</h1>;
}

export default withAuth(Dashboard);

// Settings.jsx -- no auth logic here
function Settings({ user }) {
  return <h1>Settings for {user.email}</h1>;
}

export default withAuth(Settings);
```

```
+-------------------------------------------+
|  withAuth(Dashboard)                      |
|                                           |
|  Is user logged in?                       |
|    NO  --> <Navigate to="/login" />       |
|    YES --> +-------------------------+    |
|            |  Dashboard              |    |
|            |  (receives user prop)   |    |
|            +-------------------------+    |
+-------------------------------------------+
```

Before vs After

Before: Every protected page has its own auth check. Bug fixes require updating N files.

After: Auth logic lives in one place. Adding protection to a new page is a single line: `export default withAuth(NewPage)`.

Real-World Use Case

In a SaaS application with 40+ routes, 30 of them need authentication. Without `withAuth`, that is 30 files with identical if-not-authenticated-redirect logic. With the HOC, each file just wraps its export.

Pattern 2: withLoading -- Handling Async States

The Problem

Components that fetch data all need loading and error states:

```jsx
function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch('/api/users')
      .then(res => res.json())
      .then(data => { setUsers(data); setLoading(false); })
      .catch(err => { setError(err); setLoading(false); });
  }, []);

  if (loading) return <Spinner />;
  if (error) return <ErrorMessage error={error} />;

  return (
    <ul>
      {users.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

The Solution

```jsx
function withLoading(WrappedComponent, fetchFn) {
  return function LoadingComponent(props) {
    const [data, setData] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
      setLoading(true);
```

```
      fetchFn(props)
        .then(result => {
          setData(result);
          setLoading(false);
        })
        .catch(err => {
          setError(err);
          setLoading(false);
        });
    }, []);  // eslint-disable-line react-hooks/exhaustive-deps

    if (loading) return <Spinner />;
    if (error) return <ErrorMessage error={error} />;

    return <WrappedComponent {...props} data={data} />;
  };
}

// Usage
function UserList({ data }) {
  return (
    <ul>
      {data.map(u => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}

const UserListWithLoading = withLoading(
  UserList,
  () => fetch('/api/users').then(r => r.json())
);
```

Now `UserList` only cares about rendering users. It receives `data` and renders it. Loading, error handling, and fetching are all handled by the HOC.

Pattern 3: withLogger -- Cross-Cutting Concerns

This HOC logs when a component mounts, updates, or unmounts:

```jsx
function withLogger(WrappedComponent) {
  const componentName =
    WrappedComponent.displayName ||
    WrappedComponent.name ||
    'Component';
```

```
  return function LoggedComponent(props) {
    useEffect(() => {
      console.log(`[${componentName}] Mounted`);
      return () => console.log(`[${componentName}] Unmounted`);
    }, []);

    useEffect(() => {
      console.log(`[${componentName}] Updated`, props);
    });

    return <WrappedComponent {...props} />;
  };
}

// Usage
const LoggedDashboard = withLogger(Dashboard);

// Console output when LoggedDashboard renders:
// [Dashboard] Mounted
// [Dashboard] Updated { user: { name: "Alice" } }
```

Composing Multiple HOCs

You can stack HOCs to combine behaviors:

```jsx
// Apply auth + logging to Dashboard
const EnhancedDashboard = withLogger(withAuth(Dashboard));
```

This reads inside-out: first wrap with auth, then wrap the result with logging.

```
+-----------------------------------------+
|  withLogger                             |
|  +-------------------------------------+  |
|  |  withAuth                           |  |
|  |  +-----------------------------+ |  |
|  |  |  Dashboard                  | |  |
|  |  |  (the actual component)     | |  |
|  |  +-----------------------------+ |  |
|  +-------------------------------------+  |
+-----------------------------------------+
```

For better readability, you can use a `compose` utility:

```jsx
// Simple compose function
function compose(...fns) {
  return function (component) {
    return fns.reduceRight(
      (wrapped, fn) => fn(wrapped),
      component
    );
  };
}

const enhance = compose(
  withLogger,
  withAuth,
  withTheme
);

const EnhancedDashboard = enhance(Dashboard);
```

This is equivalent to `withLogger(withAuth(withTheme(Dashboard)))` but much easier to read.

Forwarding Refs Through HOCs

One critical problem with HOCs: the `ref` prop does not pass through automatically. React treats `ref` specially -- it is not a regular prop.

The Problem

```jsx
function withTooltip(WrappedComponent) {
  return function TooltipComponent(props) {
    return (
      <div className="tooltip-wrapper">
        <WrappedComponent {...props} />
      </div>
    );
  };
}

const FancyButton = withTooltip(
  function Button(props) {
    return <button {...props}>Click me</button>;
  }
);

// This ref points to TooltipComponent, NOT the inner button!
```

```
const ref = useRef();
<FancyButton ref={ref} />
```

The Solution: React.forwardRef

```jsx
function withTooltip(WrappedComponent) {
  const WithTooltip = React.forwardRef(function TooltipComponent(props, ref) {
    return (
      <div className="tooltip-wrapper">
        <WrappedComponent {...props} ref={ref} />
      </div>
    );
  });

  // Preserve the display name for DevTools
  const name = WrappedComponent.displayName || WrappedComponent.name;
  WithTooltip.displayName = `withTooltip(${name})`;

  return WithTooltip;
}

// Now the ref correctly points to the inner button
const FancyButton = withTooltip(
  React.forwardRef(function Button(props, ref) {
    return <button ref={ref} {...props}>Click me</button>;
  })
);

const ref = useRef();
<FancyButton ref={ref} />
// ref.current === the <button> DOM node
```

```
Without forwardRef:          With forwardRef:
ref --> TooltipComponent      ref --> <button> DOM node
        |                             ^
        v                             |
        <div>                         forwarded through
          <button> (lost!)            TooltipComponent
```

Setting displayName

Always set `displayName` on HOC-returned components. Without it, React DevTools shows a tree of anonymous components that is impossible to debug:

```
// Bad DevTools tree:
<Anonymous>
  <Anonymous>
    <Anonymous>

// Good DevTools tree (with displayName):
<withLogger(withAuth(Dashboard))>
  <withAuth(Dashboard)>
    <Dashboard>
```

Why Hooks Replaced Most HOCs

React 16.8 introduced hooks, and most HOC use cases can now be handled more cleanly with custom hooks.

HOC Approach (Before)

```jsx
function withWindowWidth(WrappedComponent) {
  return function WithWindowWidth(props) {
    const [width, setWidth] = useState(window.innerWidth);

    useEffect(() => {
      const handle = () => setWidth(window.innerWidth);
      window.addEventListener('resize', handle);
      return () => window.removeEventListener('resize', handle);
    }, []);

    return <WrappedComponent {...props} windowWidth={width} />;
  };
}

const ResponsiveNav = withWindowWidth(function Nav({ windowWidth }) {
  return windowWidth > 768 ? <DesktopNav /> : <MobileNav />;
});
```

Hook Approach (After)

```jsx
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);

  useEffect(() => {
```

```
    const handle = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handle);
    return () => window.removeEventListener('resize', handle);
  }, []);

  return width;
}

function Nav() {
  const windowWidth = useWindowWidth();
  return windowWidth > 768 ? <DesktopNav /> : <MobileNav />;
}
```

Why Hooks Win

Problem with HOCs	How Hooks Solve It
Wrapper hell -- stacking 5 HOCs creates 5 extra layers in the component tree	Hooks add zero extra components
Prop collisions -- two HOCs might inject a prop with the same name	Hooks return values you name yourself
Indirection -- hard to trace where a prop comes from	Hooks are called directly in the component
Static composition -- HOCs are applied at definition time, not render time	Hooks run at render time and can use conditions (via early returns)

```
HOC wrapper hell:                    Hooks -- flat tree:
<withAuth>                           <Dashboard>
  <withTheme>                          (uses useAuth hook)
    <withLogger>                       (uses useTheme hook)
      <withWindowWidth>                (uses useLogger hook)
        <Dashboard />                  (uses useWindowWidth hook)
      </withWindowWidth>
    </withLogger>
  </withTheme>
</withAuth>
```

When HOCs Still Make Sense

Hooks solved most problems, but HOCs remain useful in specific situations:

1. Wrapping class components -- class components cannot use hooks
2. Adding wrapper DOM elements -- a HOC can insert a `<div>` with specific styles around any component

3. Route-level decorators -- frameworks like Next.js sometimes use HOC-like patterns for layout wrapping
4. Third-party library integration -- some libraries (like Redux's legacy `connect`) are built on HOCs

---

When to Use / When NOT to Use

Use HOCs When

- You need to add behavior to class components that cannot use hooks
- You need to wrap a component with additional DOM elements
- You are working in a codebase that already uses HOCs consistently
- You need to enhance components from a third-party library you cannot modify

Do NOT Use HOCs When

- You can use a custom hook instead (most cases in modern React)
- The logic does not need to wrap the component tree (use hooks)
- You are creating a new project with only function components
- The HOC would only pass down a single prop (a hook returning that value is simpler)

Common Mistakes

Mistake 1: Creating HOCs Inside render

```jsx
// WRONG -- creates a new component type every render
function App() {
  // This causes the entire subtree to unmount/remount!
  const Enhanced = withAuth(Dashboard);
  return <Enhanced />;
}

// CORRECT -- create HOC outside the component
const EnhancedDashboard = withAuth(Dashboard);
```

```
function App() {
  return <EnhancedDashboard />;
}
```

When you create a HOC inside render, React sees a different component type on each render and destroys the entire subtree, losing all state.

Mistake 2: Not Passing Props Through

```jsx
// WRONG -- swallows all original props
function withAuth(WrappedComponent) {
  return function(props) {
    const user = useAuth();
    // Forgot {...props}!
    return <WrappedComponent user={user} />;
  };
}

// CORRECT -- spread all props
function withAuth(WrappedComponent) {
  return function(props) {
    const user = useAuth();
    return <WrappedComponent {...props} user={user} />;
  };
}
```

Mistake 3: Not Copying Static Methods

```jsx
// Original component has static methods
Dashboard.navigationOptions = { title: 'Home' };

// HOC loses them!
const Enhanced = withAuth(Dashboard);
console.log(Enhanced.navigationOptions); // undefined

// Fix: copy static methods
function withAuth(WrappedComponent) {
  function Enhanced(props) { /* ... */ }

  // Manually copy statics
  Enhanced.navigationOptions = WrappedComponent.navigationOptions;

  // Or use hoist-non-react-statics library
  // hoistNonReactStatics(Enhanced, WrappedComponent);

  return Enhanced;
}
```

Mistake 4: Mutating the Original Component

```jsx
// WRONG -- mutates the input
function withAuth(WrappedComponent) {
  WrappedComponent.prototype.checkAuth = function() { /* ... */ };
  return WrappedComponent;
}

// CORRECT -- composition, not mutation
function withAuth(WrappedComponent) {
  return function Enhanced(props) {
    // Add behavior through composition
    return <WrappedComponent {...props} />;
  };
}
```

Best Practices

1. Name your HOCs with the `with` prefix -- `withAuth`, `withLoading`, `withTheme`. This convention immediately signals what the function does.
2. Always set displayName -- makes debugging in React DevTools possible.
3. Pass through all unrelated props -- use `{...props}` to ensure the wrapped component receives everything it expects.
4. Use forwardRef when the wrapped component needs a ref -- otherwise consumers cannot get a reference to the inner component.
5. Create HOCs outside of components -- never inside render methods or function component bodies.
6. Prefer hooks for new code -- only reach for HOCs when hooks cannot solve the problem.
7. Keep HOCs focused -- each HOC should do one thing. Compose them for complex behavior rather than building one giant HOC.

Quick Summary

A Higher-Order Component is a function that takes a component and returns a new, enhanced component. HOCs inject props, wrap rendering, and add behavior without modifying the original component. They were the primary code-reuse pattern before hooks

and remain relevant for class components, DOM wrapping, and legacy codebases. In modern React, prefer custom hooks for most cross-cutting concerns.

Key Points

- A HOC is a function: `(Component) => EnhancedComponent`
- HOCs do not modify the input component -- they compose it
- Always spread props through: `<WrappedComponent {...props} />`
- Use `React.forwardRef` to pass refs through HOCs
- Set `displayName` for debuggability
- Never create HOCs inside render -- it destroys component state
- Hooks have replaced most HOC use cases since React 16.8
- HOCs still shine for class components, DOM wrappers, and third-party library integration
- Compose multiple HOCs with a `compose` utility for readability

Practice Questions

1. What is the difference between a Higher-Order Component and a Higher-Order Function? How does the concept from functional programming apply to React components?
2. You have a HOC `withTheme` that injects a `theme` prop and a HOC `withUser` that also injects a prop called `theme` (containing user theme preferences). What happens when you compose them? How would you fix it?
3. Explain why creating a HOC inside a component's render method causes all state in the wrapped component tree to be lost on every render.
4. A teammate argues that all HOCs should be rewritten as hooks immediately. Give two concrete examples where a HOC is a better choice than a hook.
5. How does `React.forwardRef` solve the ref-forwarding problem in HOCs? What would happen if you tried to pass `ref` as a regular prop named `innerRef` instead?

Exercises

Exercise 1: Build withPermission

Create a `withPermission` HOC that checks if the current user has a specific permission before rendering a component. If the user lacks the permission, show an "Access Denied" message.

```jsx
// Your HOC should work like this:
const AdminPanel = withPermission('admin')(AdminPanelBase);
const EditorPanel = withPermission('editor')(EditorPanelBase);
```

Hint: This is a HOC factory -- a function that returns a HOC. The pattern is `withPermission(config)(Component)`.

Exercise 2: Build withRetry

Create a `withRetry` HOC that wraps a component performing data fetching. If the fetch fails, show a "Retry" button. When clicked, it should attempt the fetch again up to 3 times.

Exercise 3: Convert a HOC to a Hook

Take the `withWindowWidth` HOC from this chapter and convert it to a `useWindowWidth` hook. Then refactor a component that used the HOC to use the hook instead. Compare the before and after code in terms of readability, component tree depth, and flexibility.

What Is Next?

Higher-Order Components wrap a component from the outside. But what if you want to share behavior by passing a function into a component that controls what gets rendered? That is the Render Props pattern -- a technique where a component receives a function as a prop and calls it to determine its output. In Chapter 17, you will learn how render props offer an alternative to HOCs and why hooks ultimately simplified both approaches.

Chapter 17: Render Props

What You Will Learn

- What the render props pattern is and why it exists
- How to build practical render prop components: mouse tracker, toggle, fetch wrapper
- The "children as a function" variation and when to prefer it
- How render props compare to HOCs and hooks
- Performance considerations and how to avoid unnecessary re-renders
- When render props remain useful in a hooks-first world

Why This Chapter Matters

Picture a vending machine. You put in your money (input), and the machine gives you a snack (output). Now imagine a more flexible machine: you put in your money, and instead of giving you a fixed snack, it gives you a token that you can exchange at any counter for whatever you want -- a sandwich, a drink, a salad. The machine handles the payment logic, but you decide what you get.

Render props work the same way. A component handles the logic (tracking the mouse, managing a toggle, fetching data), but instead of deciding what to render, it calls a function you provide and lets you decide. The component supplies the data; you supply the UI.

This pattern was the second major approach to code reuse in React (after HOCs) and solved several of the problems that HOCs created. While hooks have since simplified most of these use cases, render props remain the clearest way to understand the principle of inversion of control -- letting consumers decide how shared logic manifests visually.

The Core Concept

A render prop is a prop whose value is a function that returns JSX. The component calls this function, passing it some data, and renders whatever the function returns.

```jsx
// The component with shared logic
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (event) => {
      setPosition({ x: event.clientX, y: event.clientY });
    };
    window.addEventListener('mousemove', handleMouseMove);
    return () => window.removeEventListener('mousemove', handleMouseMove);
  }, []);

  // Call the render prop with the data
  return render(position);
}

// Consumer decides what to render
function App() {
  return (
    <MouseTracker
      render={(position) => (
        <p>
          Mouse is at ({position.x}, {position.y})
        </p>
      )}
    />
  );
}

// Output (as mouse moves):
// Mouse is at (245, 182)
```

```
+-----------------------------------------------------+
|  MouseTracker Component                             |
|                                                     |
|  State: { x: 245, y: 182 }                         |
|                                                     |
|  Logic: addEventListener('mousemove', ...)          |
|                                                     |
|  Rendering:                                         |
|    return render({ x: 245, y: 182 })                |
|            |                                        |
|            v                                        |
|    Consumer's function decides                      |
|    what JSX to produce                              |
+-----------------------------------------------------+
```

The same `MouseTracker` can render completely different UIs:

```jsx
// As a tooltip that follows the cursor
<MouseTracker
  render={({ x, y }) => (
    <div style={{ position: 'fixed', left: x + 10, top: y + 10 }}>
      Tooltip content here
    </div>
  )}
/>

// As a coordinate display in a debug panel
<MouseTracker
  render={({ x, y }) => (
    <div className="debug-panel">
      <strong>X:</strong> {x} | <strong>Y:</strong> {y}
    </div>
  )}
/>

// As a heat map dot
<MouseTracker
  render={({ x, y }) => (
    <svg width="100%" height="100%">
      <circle cx={x} cy={y} r={20} fill="rgba(255,0,0,0.3)" />
    </svg>
  )}
/>
```

One component, three completely different visual outputs. The logic is reused; the presentation is flexible.

Children as a Function

Instead of passing the function through a prop called `render`, you can pass it as the component's `children`. This reads more naturally in JSX:

```jsx
// Component accepts children as a function
function MouseTracker({ children }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handleMouseMove = (event) => {
      setPosition({ x: event.clientX, y: event.clientY });
```

```
    };
    window.addEventListener('mousemove', handleMouseMove);
    return () => window.removeEventListener('mousemove', handleMouseMove);
  }, []);

  return children(position);
}

// Usage -- looks cleaner
function App() {
  return (
    <MouseTracker>
      {(position) => (
        <p>Mouse is at ({position.x}, {position.y})</p>
      )}
    </MouseTracker>
  );
}
```

Both approaches are functionally identical. The community generally prefers `children` as a function because:

- It avoids inventing a prop name (`render`, `renderItem`, `renderHeader`)
- It visually nests content inside the component tags, which feels natural
- It is consistent with how `children` works elsewhere in React

However, if the component needs multiple render functions, named props are better:

```jsx
<DataTable
  data={users}
  renderHeader={(columns) => (
    <tr>{columns.map(col => <th key={col}>{col}</th>)}</tr>
  )}
  renderRow={(user) => (
    <tr key={user.id}>
      <td>{user.name}</td>
      <td>{user.email}</td>
    </tr>
  )}
  renderEmpty={() => <p>No users found.</p>}
/>
```

Pattern 1: Toggle with Render Props

The Problem

You need toggle behavior (show/hide, expand/collapse, on/off) in many places, each with different UI:

```jsx
// A modal toggle
function ModalExample() {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <>
      <button onClick={() => setIsOpen(true)}>Open Modal</button>
      {isOpen && <Modal onClose={() => setIsOpen(false)} />}
    </>
  );
}

// A dropdown toggle -- same logic, different UI
function DropdownExample() {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <>
      <button onClick={() => setIsOpen(!isOpen)}>Menu</button>
      {isOpen && <DropdownMenu />}
    </>
  );
}
```

The Solution

```jsx
function Toggle({ children, initialOn = false }) {
  const [on, setOn] = useState(initialOn);

  const toggle = () => setOn(prev => !prev);
  const setOff = () => setOn(false);
  const setOnTrue = () => setOn(true);

  return children({ on, toggle, setOn: setOnTrue, setOff });
}

// Modal usage
function ModalExample() {
  return (
    <Toggle>
      {({ on, setOn, setOff }) => (
        <>
          <button onClick={setOn}>Open Modal</button>
          {on && <Modal onClose={setOff} />}
        </>
      )}
    </Toggle>
  );
```

```
}

// Dropdown usage
function DropdownExample() {
  return (
    <Toggle>
      {({ on, toggle }) => (
        <>
          <button onClick={toggle}>Menu</button>
          {on && <DropdownMenu />}
        </>
      )}
    </Toggle>
  );
}

// Accordion usage
function AccordionItem({ title, content }) {
  return (
    <Toggle>
      {({ on, toggle }) => (
        <div className="accordion-item">
          <button onClick={toggle}>
            {title} {on ? '▼' : '▶'}
          </button>
          {on && <div className="accordion-content">{content}</div>}
        </div>
      )}
    </Toggle>
  );
}
```

Pattern 2: Fetch with Render Props

```jsx
function Fetch({ url, children }) {
  const [state, setState] = useState({
    data: null,
    loading: true,
    error: null,
  });

  useEffect(() => {
    setState({ data: null, loading: true, error: null });

    fetch(url)
      .then(res => {
```

```
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then(data => setState({ data, loading: false, error: null }))
      .catch(error => setState({ data: null, loading: false, error }));
  }, [url]);

  return children(state);
}

// Usage
function UserProfile({ userId }) {
  return (
    <Fetch url={`/api/users/${userId}`}>
      {({ data, loading, error }) => {
        if (loading) return <Spinner />;
        if (error) return <ErrorMessage error={error} />;
        return (
          <div>
            <h2>{data.name}</h2>
            <p>{data.email}</p>
          </div>
        );
      }}
    </Fetch>
  );
}
```

You can even compose multiple `Fetch` components:

```jsx
function Dashboard({ userId }) {
  return (
    <Fetch url={`/api/users/${userId}`}>
      {({ data: user, loading: loadingUser }) => (
        <Fetch url={`/api/users/${userId}/posts`}>
          {({ data: posts, loading: loadingPosts }) => {
            if (loadingUser || loadingPosts) return <Spinner />;
            return (
              <div>
                <h1>{user.name}'s Dashboard</h1>
                <PostList posts={posts} />
              </div>
            );
          }}
        </Fetch>
      )}
    </Fetch>
  );
}
```

This nesting illustrates both the power and the main weakness of render props. The logic composition works, but deep nesting hurts readability. This problem is sometimes called the

"callback pyramid" or "render prop hell."

Render Props vs HOCs vs Hooks

All three patterns solve the same problem: sharing stateful logic between components. Here is how they compare:

```
+-------------------------------------------------------------+
|                    Code Reuse Approaches                    |
+-------------------------------------------------------------+
|                                                             |
|  HOC:              Wraps from outside, injects props        |
|  withMouse(Comp)   Static composition at definition time    |
|                                                             |
|  Render Prop:      Consumer provides rendering function     |
|  <Mouse>{fn}</Mouse>  Dynamic composition at render time    |
|                                                             |
|  Hook:             Directly call in function body           |
|  useMouse()        No wrapper, no nesting, cleanest API     |
|                                                             |
+-------------------------------------------------------------+
```

Aspect	HOC	Render Props	Hooks
Extra components in tree	Yes (wrapper)	Yes (provider)	No
Prop collision risk	Yes	No	No
Nesting problem	Wrapper hell	Callback pyramid	None
Works with class components	Yes	Yes	No
Dynamic composition	No (definition-time)	Yes (render-time)	Yes
Readability	Moderate	Can degrade with nesting	Best

The Same Logic in All Three Styles

HOC:

```jsx
function withMouse(Component) {
  return function WithMouse(props) {
    const [pos, setPos] = useState({ x: 0, y: 0 });
    useEffect(() => { /* ... listener setup ... */ }, []);
    return <Component {...props} mouse={pos} />;
  };
}

const App = withMouse(function App({ mouse }) {
  return <p>{mouse.x}, {mouse.y}</p>;
});
```

Render Props:

```jsx
function Mouse({ children }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => { /* ... listener setup ... */ }, []);
  return children(pos);
}

function App() {
  return (
    <Mouse>
      {(mouse) => <p>{mouse.x}, {mouse.y}</p>}
    </Mouse>
  );
}
```

Hook:

```jsx
function useMouse() {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  useEffect(() => { /* ... listener setup ... */ }, []);
  return pos;
}

function App() {
  const mouse = useMouse();
  return <p>{mouse.x}, {mouse.y}</p>;
}
```

The hook version is the simplest. No extra components, no nesting, no indirection.

Performance Considerations

The Inline Function Problem

Every render creates a new inline function, which means the render prop component's children are always "different":

```jsx
// This creates a new function every render
<MouseTracker>
  {(pos) => <Cursor x={pos.x} y={pos.y} />}
</MouseTracker>
```

If `MouseTracker` uses `React.memo` or `shouldComponentUpdate`, the new function reference defeats the optimization.

The Fix: Extract the Render Function

```jsx
// Define the function outside the render
function renderCursor(pos) {
  return <Cursor x={pos.x} y={pos.y} />;
}

function App() {
  return <MouseTracker>{renderCursor}</MouseTracker>;
}
```

Or use `useCallback`:

```jsx
function App() {
  const renderCursor = useCallback(
    (pos) => <Cursor x={pos.x} y={pos.y} />,
    []
  );
  return <MouseTracker>{renderCursor}</MouseTracker>;
}
```

When to Use / When NOT to Use

Use Render Props When

- You need to share behavior with class components that cannot use hooks

- A component library exposes render prop APIs (like React Router's older versions, Formik, Downshift)
- You need multiple render customization points (`renderHeader`, `renderRow`, `renderFooter`)
- You want consumers to have full control over what gets rendered with shared data

Do NOT Use Render Props When

- A custom hook can handle the logic (most cases in modern React)
- The nesting would go deeper than two levels -- switch to hooks
- You are using it just to pass down a single value -- a prop or context is simpler
- Performance is critical and you cannot extract the render function

---

Common Mistakes

Mistake 1: Forgetting That children Can Be a Function

```jsx
// WRONG -- treating children as JSX when it is a function
function DataProvider({ children }) {
  const data = useFetchData();
  return <div>{children}</div>;  // children is a function, not JSX!
}

// CORRECT -- call children as a function
function DataProvider({ children }) {
  const data = useFetchData();
  return children(data);
}
```

Mistake 2: Excessive Nesting

```jsx
// AVOID -- "render prop hell"
<Auth>
  {(user) => (
    <Theme>
      {(theme) => (
        <Locale>
          {(locale) => (
            <Fetch url="/api/data">
              {({ data }) => (
```

```
                <Page user={user} theme={theme} locale={locale} data={data} />
              )}
            </Fetch>
          )}
        </Locale>
      )}
    </Theme>
  )}
</Auth>

// BETTER -- use hooks
function Page() {
  const user = useAuth();
  const theme = useTheme();
  const locale = useLocale();
  const { data } = useFetch('/api/data');
  return <div>...</div>;
}
```

Mistake 3: Not Handling the Case Where children Is Not a Function

```jsx
// Defensive approach
function Toggle({ children }) {
  const [on, setOn] = useState(false);

  if (typeof children !== 'function') {
    throw new Error(
      'Toggle expects children to be a function, ' +
      `but received ${typeof children}`
    );
  }

  return children({ on, toggle: () => setOn(prev => !prev) });
}
```

Best Practices

1. Prefer children as a function over a named `render` prop for single-render-function components. It reads more naturally.
2. Use named render props when a component needs multiple render customization points.
3. Extract render functions to avoid creating new function references on every render.

4. Type your render props with TypeScript or PropTypes so consumers know what data they will receive.
5. Provide sensible defaults -- if the render prop is optional, render something reasonable when it is not provided.
6. Consider hooks first -- if you are writing a new render prop component in a modern codebase, ask whether a hook would be simpler.
7. Keep the callback data minimal -- pass only the data the consumer needs, not your entire internal state.

Quick Summary

The render props pattern gives a component its logic while letting consumers control the visual output. A component calls a function (passed as a prop or as children) with its internal data, and the function returns JSX. This inverts control of rendering and enables flexible code reuse. While hooks have become the preferred approach for most logic-sharing, render props remain valuable for multi-slot rendering APIs and class component compatibility.

Key Points

- A render prop is a function passed to a component that returns JSX
- "Children as a function" is the most common variation: `<Comp>{(data) => <UI />}</Comp>`
- Render props solve the same problem as HOCs but with dynamic composition at render time
- Unlike HOCs, render props have no prop collision risk -- consumers name their own variables
- Deep nesting of render props creates readability problems ("callback pyramid")
- Extract render functions or use `useCallback` to prevent unnecessary re-renders
- Hooks have replaced most render prop use cases since React 16.8
- Render props still shine for multi-slot rendering APIs like `renderHeader`, `renderRow`, `renderFooter`

Practice Questions

1. Explain the difference between passing a render prop via a named prop (`render={fn}`) versus via `children`. When would you choose one over the other?
2. A component uses three render prop providers nested inside each other. What readability problem does this create, and what two alternative patterns could flatten it?
3. Why does creating an inline function for a render prop cause performance issues with `React.memo`? How do you fix it?
4. You are building a sortable list component. The sorting logic is reusable, but different consumers need different row layouts. Would you use a render prop, a HOC, or a hook? Justify your choice.
5. How does the render props pattern implement "inversion of control"? Explain using the `MouseTracker` example.

Exercises

Exercise 1: Build a WindowSize Render Prop Component

Create a `WindowSize` component that tracks the browser window's width and height and exposes them via a children-as-a-function pattern. Use it to build two different consumers: one that shows a responsive layout label ("mobile" / "tablet" / "desktop") and one that shows exact pixel dimensions.

Exercise 2: Build a Form Field Validator

Create a `FieldValidator` render prop component that accepts a `value` and an array of `rules` (like `required`, `minLength`, `email`). It should expose `{ isValid, errors }` to its children function. Use it to validate a signup form with name, email, and password fields.

Exercise 3: Convert Render Props to Hooks

Take the `Toggle` and `Fetch` render prop components from this chapter. Convert each into a custom hook (`useToggle`, `useFetch`). Then refactor a consumer component from render props to hooks. Write a short comparison of the two approaches noting lines of code, readability, and nesting depth.

What Is Next?

You have now seen two patterns for sharing logic: HOCs wrap from the outside, and render props inject from within. But what about components that naturally belong together -- like a `<Select>` with its `<Option>` children, or `<Tabs>` with `<Tab>` panels? These related sub-components need to share state implicitly without forcing the consumer to wire everything up manually. That is the Compound Components pattern, which you will learn in Chapter 19.

Chapter 18: Custom Hooks Pattern

What You Will Learn

- Why duplicated stateful logic is a problem and how custom hooks solve it
- The rules of hooks and why they exist
- How to build practical custom hooks: `useToggle`, `useLocalStorage`, `useFetch`, `useDebounce`, `useMediaQuery`, and `useOnClickOutside`
- How to compose hooks together to build more powerful abstractions
- How to test custom hooks
- When to extract a hook and when not to bother
- The critical insight: hooks share logic, NOT state

Why This Chapter Matters

Imagine you are a chef in a busy restaurant. Every time you make a salad, you wash the lettuce, chop the vegetables, mix the dressing, and plate it. Now imagine doing that exact same process for every single salad order, writing out the full recipe from scratch each time. That would be madness. Instead, you develop a routine -- a repeatable process you can follow without thinking about the steps.

Custom hooks are your reusable recipes for React components. When you find yourself writing the same `useState` + `useEffect` combination in three different components, you are copy-pasting a recipe. Custom hooks let you extract that recipe into a single function you can call anywhere.

Without custom hooks, teams end up with:

- Copy-pasted logic that drifts apart over time
- Bug fixes applied in one place but forgotten in others
- Components bloated with logic that hides the actual UI concerns
- Testing nightmares because logic is tangled with rendering

This chapter teaches you to spot duplicated logic, extract it cleanly, and build a toolkit of hooks that make your entire codebase simpler.

The Problem: Duplicated Stateful Logic

Consider two components that both need to track window width for responsive behavior:

```jsx
// Component A: Navigation
function Navigation() {
  const [windowWidth, setWindowWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWindowWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return (
    <nav>
      {windowWidth > 768 ? <DesktopMenu /> : <MobileMenu />}
    </nav>
  );
}

// Component B: Sidebar
function Sidebar() {
  const [windowWidth, setWindowWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWindowWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return (
    <aside>
      {windowWidth > 1024 ? <FullSidebar /> : <CollapsedSidebar />}
    </aside>
  );
}
```

The window-tracking logic is identical in both components. If you discover a bug (say, you need to debounce the resize handler for performance), you need to fix it in every single component that uses this pattern. With five components doing this, you have five places to update.

The Solution: Extract a Custom Hook

A custom hook is simply a JavaScript function whose name starts with `use`. It can call other hooks inside it.

```jsx
// useWindowWidth.js
function useWindowWidth() {
  const [windowWidth, setWindowWidth] = useState(window.innerWidth);

  useEffect(() => {
    const handleResize = () => setWindowWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return windowWidth;
}
```

Now both components become clean:

```jsx
function Navigation() {
  const windowWidth = useWindowWidth();
  return (
    <nav>
      {windowWidth > 768 ? <DesktopMenu /> : <MobileMenu />}
    </nav>
  );
}

function Sidebar() {
  const windowWidth = useWindowWidth();
  return (
    <aside>
      {windowWidth > 1024 ? <FullSidebar /> : <CollapsedSidebar />}
    </aside>
  );
}
```

Bug fix in one place. Clean components. Testable logic.

Rules of Hooks

Before building custom hooks, you must understand the rules that all hooks follow. These are not suggestions -- React will break if you violate them.

Rule 1: Only Call Hooks at the Top Level

Hooks must always be called in the same order. Never put them inside conditions, loops, or nested functions.

```jsx
// WRONG - hook inside a condition
function Profile({ userId }) {
  if (userId) {
    const [user, setUser] = useState(null);  // React loses track!
  }
}

// CORRECT - always call, conditionally use
function Profile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    if (userId) {
      fetchUser(userId).then(setUser);
    }
  }, [userId]);
}
```

Why? React identifies hooks by their call order. If a hook is sometimes called and sometimes skipped, the order shifts and React pairs the wrong hook with the wrong state.

Think of it like a row of lockers numbered 1, 2, 3. React opens locker 1 for the first `useState`, locker 2 for the second, and so on. If you skip locker 1 sometimes, everything shifts and locker 2 gets locker 1's data.

```
First render:              Second render (if skipped):
+--------+--------+       +--------+--------+
|  Hook  | Locker |       |  Hook  | Locker |
+--------+--------+       +--------+--------+
| state1 |   1    |       | state2 |   1    |  <-- WRONG! state2 gets
| state2 |   2    |       | effect |   2    |      state1's value
| effect |   3    |       +--------+--------+
+--------+--------+
```

Rule 2: Only Call Hooks from React Functions

Call hooks from React function components or from other custom hooks. Never from regular JavaScript functions, class components, or event handlers.

```jsx
// WRONG - regular function
function getUser() {
  const [user, setUser] = useState(null);  // Not a component or hook!
}

// CORRECT - custom hook (starts with "use")
function useUser() {
  const [user, setUser] = useState(null);
  return user;
}
```

Rule 3: Custom Hooks Must Start with "use"

This is not just a naming convention. React's linter uses this prefix to check that you follow the other rules.

```jsx
// React linter will NOT check rules for this:
function getToggle() { ... }

// React linter WILL check rules for this:
function useToggle() { ... }
```

Building Practical Custom Hooks

Let us build six hooks that solve real problems you will encounter in production applications.

Hook 1: useToggle

Problem: Many components need a boolean that flips between true and false -- modals, dropdowns, dark mode, show/hide sections.

Before:

```jsx
function FAQ() {
  const [isOpen1, setIsOpen1] = useState(false);
  const [isOpen2, setIsOpen2] = useState(false);
  const [isOpen3, setIsOpen3] = useState(false);

  return (
    <div>
```

```
      <button onClick={() => setIsOpen1(!isOpen1)}>Question 1</button>
      {isOpen1 && <p>Answer 1</p>}

      <button onClick={() => setIsOpen2(!isOpen2)}>Question 2</button>
      {isOpen2 && <p>Answer 2</p>}

      <button onClick={() => setIsOpen3(!isOpen3)}>Question 3</button>
      {isOpen3 && <p>Answer 3</p>}
    </div>
  );
}
```

The Hook:

```jsx
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  const toggle = useCallback(() => setValue(v => !v), []);
  const setTrue = useCallback(() => setValue(true), []);
  const setFalse = useCallback(() => setValue(false), []);

  return [value, toggle, { setTrue, setFalse }];
}
```

Line-by-line explanation:

- `function useToggle(initialValue = false)` -- accepts an optional starting value, defaults to false.
- `const [value, setValue] = useState(initialValue)` -- creates the boolean state.
- `const toggle = useCallback(() => setValue(v => !v), [])` -- creates a stable function that flips the value. `useCallback` prevents unnecessary re-renders in child components.
- `const setTrue / setFalse` -- convenience methods for explicit control.
- Returns an array with the value, toggle function, and an object of helpers.

After:

```jsx
function FAQ() {
  const [isOpen1, toggleOpen1] = useToggle();
  const [isOpen2, toggleOpen2] = useToggle();
  const [isOpen3, toggleOpen3] = useToggle();

  return (
    <div>
      <button onClick={toggleOpen1}>Question 1</button>
      {isOpen1 && <p>Answer 1</p>}
```

```
      <button onClick={toggleOpen2}>Question 2</button>
      {isOpen2 && <p>Answer 2</p>}

      <button onClick={toggleOpen3}>Question 3</button>
      {isOpen3 && <p>Answer 3</p>}
    </div>
  );
}
```

Output (when user clicks "Question 1"):

```
Before click:  isOpen1 = false, Question 1 answer hidden
After click:   isOpen1 = true,  Question 1 answer visible
After click:   isOpen1 = false, Question 1 answer hidden again
```

Hook 2: useLocalStorage

Problem: You want state that persists across page refreshes. Think user preferences, saved form drafts, or shopping cart items.

Before:

```jsx
function Settings() {
  const [theme, setTheme] = useState(() => {
    try {
      const saved = localStorage.getItem('theme');
      return saved ? JSON.parse(saved) : 'light';
    } catch {
      return 'light';
    }
  });

  useEffect(() => {
    localStorage.setItem('theme', JSON.stringify(theme));
  }, [theme]);

  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

That is a lot of boilerplate. And you need it for every piece of state you want to persist.

The Hook:

```jsx
function useLocalStorage(key, initialValue) {
  // 1. Initialize state from localStorage or fallback
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = window.localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch (error) {
      console.warn(`Error reading localStorage key "${key}":`, error);
      return initialValue;
    }
  });

  // 2. Update localStorage whenever state changes
  useEffect(() => {
    try {
      window.localStorage.setItem(key, JSON.stringify(storedValue));
    } catch (error) {
      console.warn(`Error setting localStorage key "${key}":`, error);
    }
  }, [key, storedValue]);

  // 3. Listen for changes from other tabs
  useEffect(() => {
    const handleStorageChange = (e) => {
      if (e.key === key && e.newValue !== null) {
        setStoredValue(JSON.parse(e.newValue));
      }
    };
    window.addEventListener('storage', handleStorageChange);
    return () => window.removeEventListener('storage', handleStorageChange);
  }, [key]);

  return [storedValue, setStoredValue];
}
```

Line-by-line explanation:

- `useState(() => {...})` -- lazy initializer reads from localStorage once on mount.
- `try/catch` -- localStorage can fail (private browsing, storage full, etc.).
- `JSON.parse/stringify` -- handles any serializable value, not just strings.
- The first `useEffect` syncs state changes back to localStorage.
- The second `useEffect` listens for the `storage` event, which fires when another tab changes the same key. This keeps tabs in sync.
- Returns the same `[value, setter]` API as `useState`, making it a drop-in replacement.

After:

```jsx
function Settings() {
  const [theme, setTheme] = useLocalStorage('theme', 'light');

  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
  );
}
```

Output:

```
Page load (first time):     theme = "light"    (uses initialValue)
Click button:               theme = "dark"     (saved to localStorage)
Refresh page:               theme = "dark"     (read from localStorage)
```

Hook 3: useFetch

Problem: Almost every component that talks to an API needs loading state, error state, and data state. Writing that trio everywhere is tedious and error-prone.

Before:

```jsx
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    setLoading(true);
    setError(null);

    fetch(`/api/users/${userId}`)
      .then(res => {
        if (!res.ok) throw new Error('Failed to fetch');
        return res.json();
      })
      .then(data => {
        setUser(data);
        setLoading(false);
      })
      .catch(err => {
        setError(err.message);
        setLoading(false);
      });
  }, [userId]);
```

```
  if (loading) return <Spinner />;
  if (error) return <Error message={error} />;
  return <div>{user.name}</div>;
}
```

The Hook:

```jsx
function useFetch(url, options = {}) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Serialize options to detect real changes
  const optionsKey = JSON.stringify(options);

  useEffect(() => {
    const controller = new AbortController();

    async function fetchData() {
      setLoading(true);
      setError(null);

      try {
        const response = await fetch(url, {
          ...options,
          signal: controller.signal,
        });

        if (!response.ok) {
          throw new Error(`HTTP ${response.status}: ${response.statusText}`);
        }

        const json = await response.json();
        setData(json);
      } catch (err) {
        if (err.name !== 'AbortError') {
          setError(err.message);
        }
      } finally {
        if (!controller.signal.aborted) {
          setLoading(false);
        }
      }
    }

    fetchData();

    return () => controller.abort();
  }, [url, optionsKey]);

  return { data, loading, error };
}
```

Line-by-line explanation:

- `AbortController` -- cancels the fetch if the component unmounts or the URL changes before the request finishes. This prevents the "update state on unmounted component" warning.
- `if (err.name !== 'AbortError')` -- when we abort intentionally, we do not treat it as an error.
- `if (!controller.signal.aborted)` -- only set loading to false if the request was not aborted.
- Returns an object `{ data, loading, error }` so callers can destructure what they need.

After:

```jsx
function UserProfile({ userId }) {
  const { data: user, loading, error } = useFetch(`/api/users/${userId}`);

  if (loading) return <Spinner />;
  if (error) return <Error message={error} />;
  return <div>{user.name}</div>;
}
```

Output (happy path):

```
Mount:        { data: null,   loading: true,  error: null }
Fetch done:   { data: {name: "Alice"}, loading: false, error: null }
```

Output (error):

```
Mount:        { data: null,   loading: true,  error: null }
Fetch fails:  { data: null,   loading: false, error: "HTTP 404: Not Found" }
```

```
Data flow through useFetch:

  Component               useFetch               Server
  ---------               --------               ------
      |                       |                      |
      |-- render(url) ------->|                      |
      |                       |-- fetch(url) ------->|
      |<-- {loading: true} ---|                      |
      |                       |<---- JSON data ------|
      |<-- {data, loading: false} ---|               |
      |                       |                      |
      |-- unmount ----------->|                      |
      |                       |-- abort() ---------->|
```

Hook 4: useDebounce

Problem: Some operations should not fire on every keystroke. Search inputs, window resize handlers, and save-as-you-type features all need debouncing -- waiting until the user stops doing something before acting.

Think of an elevator door. It does not close the instant someone presses the button. It waits a moment in case someone else is about to step in. If another person arrives, the timer resets and waits again.

Before:

```jsx
function SearchPage() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);
  const timeoutRef = useRef(null);

  const handleChange = (e) => {
    const value = e.target.value;
    setQuery(value);

    if (timeoutRef.current) clearTimeout(timeoutRef.current);

    timeoutRef.current = setTimeout(() => {
      fetch(`/api/search?q=${value}`)
        .then(res => res.json())
        .then(setResults);
    }, 300);
  };

  useEffect(() => {
    return () => clearTimeout(timeoutRef.current);
  }, []);

  return (
    <div>
      <input value={query} onChange={handleChange} />
      <ResultsList results={results} />
    </div>
  );
}
```

The Hook:

```jsx
function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);
```

```
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}
```

Line-by-line explanation:

- Takes any value and a delay in milliseconds.
- Sets up a timer that updates `debouncedValue` after the delay.
- If `value` changes before the timer fires, the cleanup function runs (`clearTimeout`), canceling the old timer, and a new one starts. This is the "elevator door resetting" behavior.
- Returns the debounced value -- it only changes when the input has been stable for `delay` milliseconds.

After:

```jsx
function SearchPage() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 300);
  const { data: results } = useFetch(
    debouncedQuery ? `/api/search?q=${debouncedQuery}` : null
  );

  return (
    <div>
      <input
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <ResultsList results={results || []} />
    </div>
  );
}
```

Output (user types "react" quickly):

```
Keystroke "r":     query = "r",     debouncedQuery = ""       (no fetch)
Keystroke "e":     query = "re",    debouncedQuery = ""       (timer reset)
Keystroke "a":     query = "rea",   debouncedQuery = ""       (timer reset)
Keystroke "c":     query = "reac",  debouncedQuery = ""       (timer reset)
```

```
Keystroke "t":     query = "react", debouncedQuery = ""       (timer reset)
300ms passes:      query = "react", debouncedQuery = "react"  (fetch fires!)
```

Without debouncing: 5 API calls. With debouncing: 1 API call.

Hook 5: useMediaQuery

Problem: You need to render different content or apply different logic based on screen size, user preferences (dark mode), or other CSS media query conditions.

The Hook:

```jsx
function useMediaQuery(query) {
  const [matches, setMatches] = useState(() => {
    if (typeof window === 'undefined') return false;
    return window.matchMedia(query).matches;
  });

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);
    const handler = (event) => setMatches(event.matches);

    // Modern browsers
    mediaQuery.addEventListener('change', handler);
    // Set initial value
    setMatches(mediaQuery.matches);

    return () => mediaQuery.removeEventListener('change', handler);
  }, [query]);

  return matches;
}
```

Line-by-line explanation:

- `window.matchMedia(query)` -- creates a MediaQueryList object that tracks whether the document matches the given CSS media query string.
- The `change` event fires whenever the match status changes (e.g., user resizes the browser past a breakpoint).
- SSR safety: `typeof window === 'undefined'` returns false during server-side rendering.
- Returns a simple boolean: does the query match right now?

Usage:

```jsx
function App() {
  const isMobile = useMediaQuery('(max-width: 768px)');
  const prefersDark = useMediaQuery('(prefers-color-scheme: dark)');
  const prefersReducedMotion = useMediaQuery('(prefers-reduced-motion: reduce)');

  return (
    <div className={prefersDark ? 'dark' : 'light'}>
      {isMobile ? <MobileLayout /> : <DesktopLayout />}
      {!prefersReducedMotion && <FancyAnimation />}
    </div>
  );
}
```

Output (on a phone):

```
isMobile = true
prefersDark = false  (depends on user's OS setting)
prefersReducedMotion = false

Rendered: <MobileLayout /> with light theme and animations
```

Hook 6: useOnClickOutside

Problem: Dropdown menus, modals, and popovers need to close when the user clicks anywhere outside of them.

The Hook:

```jsx
function useOnClickOutside(ref, handler) {
  useEffect(() => {
    const listener = (event) => {
      // Do nothing if clicking ref's element or its children
      if (!ref.current || ref.current.contains(event.target)) {
        return;
      }
      handler(event);
    };

    document.addEventListener('mousedown', listener);
    document.addEventListener('touchstart', listener);

    return () => {
      document.removeEventListener('mousedown', listener);
      document.removeEventListener('touchstart', listener);
    };
  }, [ref, handler]);
}
```

Line-by-line explanation:

- `ref.current.contains(event.target)` -- checks if the click happened inside the referenced element or any of its children. If it did, we ignore it.
- We listen for both `mousedown` (desktop) and `touchstart` (mobile) to cover all devices.
- We use `mousedown` instead of `click` because `click` fires after `mouseup`, which can cause timing issues where the dropdown opens and immediately closes.

Usage:

```jsx
function Dropdown() {
  const [isOpen, toggle, { setFalse: close }] = useToggle();
  const dropdownRef = useRef(null);

  useOnClickOutside(dropdownRef, close);

  return (
    <div ref={dropdownRef}>
      <button onClick={toggle}>Menu</button>
      {isOpen && (
        <ul className="dropdown-menu">
          <li>Profile</li>
          <li>Settings</li>
          <li>Logout</li>
        </ul>
      )}
    </div>
  );
}
```

```
Click behavior:

  +------------------+
  |  [Menu Button]   |  <-- clicking here toggles menu
  |  +------------+  |
  |  | Profile    |  |
  |  | Settings   |  |  <-- clicking inside: menu stays open
  |  | Logout     |  |
  |  +------------+  |
  +------------------+
                        <-- clicking OUTSIDE: menu closes
  Any click out here
  triggers the handler
```

Composing Hooks: Building Bigger From Smaller

One of the most powerful ideas behind custom hooks is composition. You can build complex hooks by combining simpler ones, just like building with LEGO bricks.

Example: usePaginatedFetch

```jsx
function usePagination(initialPage = 1, pageSize = 10) {
  const [page, setPage] = useState(initialPage);

  const nextPage = useCallback(() => setPage(p => p + 1), []);
  const prevPage = useCallback(() => setPage(p => Math.max(1, p - 1)), []);
  const goToPage = useCallback((p) => setPage(p), []);

  return {
    page,
    pageSize,
    offset: (page - 1) * pageSize,
    nextPage,
    prevPage,
    goToPage,
  };
}
```

Now compose `useFetch` and `usePagination`:

```jsx
function usePaginatedFetch(baseUrl, pageSize = 10) {
  const pagination = usePagination(1, pageSize);
  const url = `${baseUrl}?page=${pagination.page}&limit=${pagination.pageSize}`;
  const { data, loading, error } = useFetch(url);

  return {
    data,
    loading,
    error,
    ...pagination,
  };
}
```

Usage:

```jsx
function UserList() {
  const {
    data: users,
    loading,
    error,
    page,
    nextPage,
    prevPage,
```

```
  } = usePaginatedFetch('/api/users', 20);

  if (loading) return <Spinner />;
  if (error) return <Error message={error} />;

  return (
    <div>
      <ul>
        {users.map(user => <li key={user.id}>{user.name}</li>)}
      </ul>
      <button onClick={prevPage} disabled={page === 1}>Previous</button>
      <span>Page {page}</span>
      <button onClick={nextPage}>Next</button>
    </div>
  );
}
```

```
Hook composition:

  usePaginatedFetch
  +-----------------------------+
  |  usePagination              |
  |  +-------------------+      |
  |  | page, pageSize,   |      |
  |  | nextPage, prevPage|------+--- builds URL
  |  +-------------------+      |
  |                             |
  |  useFetch(url)              |
  |  +-------------------+      |
  |  | data, loading,    |------+--- returns data
  |  | error             |      |
  |  +-------------------+      |
  +-----------------------------+
```

Critical Insight: Hooks Share Logic, NOT State

This is the most common misconception about custom hooks. If two components call `useToggle()`, they each get their own independent state. The hook shares the logic (the code), not the state (the data).

```jsx
function ComponentA() {
  const [isOpen, toggle] = useToggle(false);
  // isOpen is independent -- changing it does NOT affect ComponentB
  return <button onClick={toggle}>{isOpen ? 'Open' : 'Closed'}</button>;
```

```
}

function ComponentB() {
  const [isOpen, toggle] = useToggle(false);
  // This is a COMPLETELY SEPARATE isOpen
  return <button onClick={toggle}>{isOpen ? 'Open' : 'Closed'}</button>;
}
```

```
Two components using useToggle:

  ComponentA                    ComponentB
  +-------------------+        +-------------------+
  | useToggle()       |        | useToggle()       |
  | state: false      |        | state: false      |
  +-------------------+        +-------------------+
        |                             |
  Click toggleA              Click toggleB
        |                             |
  +-------------------+        +-------------------+
  | state: true       |        | state: false      |  <-- independent!
  +-------------------+        +-------------------+
```

If you need shared state between components, you need Context (Chapter 21) or a state management library, not just a custom hook.

Testing Custom Hooks

You cannot call hooks outside of React components. To test them, use `@testing-library/react`'s `renderHook` utility.

```jsx
import { renderHook, act } from '@testing-library/react';

describe('useToggle', () => {
  test('starts with initial value', () => {
    const { result } = renderHook(() => useToggle(false));
    expect(result.current[0]).toBe(false);
  });

  test('toggles value', () => {
    const { result } = renderHook(() => useToggle(false));

    act(() => {
      result.current[1]();  // call toggle
```

```
    });

    expect(result.current[0]).toBe(true);
  });

  test('setTrue forces true', () => {
    const { result } = renderHook(() => useToggle(false));

    act(() => {
      result.current[2].setTrue();
    });

    expect(result.current[0]).toBe(true);
  });
});

describe('useDebounce', () => {
  beforeEach(() => {
    jest.useFakeTimers();
  });

  afterEach(() => {
    jest.useRealTimers();
  });

  test('returns initial value immediately', () => {
    const { result } = renderHook(() => useDebounce('hello', 500));
    expect(result.current).toBe('hello');
  });

  test('updates value after delay', () => {
    const { result, rerender } = renderHook(
      ({ value, delay }) => useDebounce(value, delay),
      { initialProps: { value: 'hello', delay: 500 } }
    );

    // Change the value
    rerender({ value: 'world', delay: 500 });

    // Value should not have changed yet
    expect(result.current).toBe('hello');

    // Fast-forward time
    act(() => {
      jest.advanceTimersByTime(500);
    });

    // Now it should be updated
    expect(result.current).toBe('world');
  });
});
```

Key points about testing hooks:

- `renderHook` creates a tiny component that calls your hook.

- `result.current` always holds the latest return value.
- `act()` wraps any state updates so React processes them.
- Use `jest.useFakeTimers()` for time-dependent hooks.
- `rerender` lets you simulate prop changes.

When to Extract a Custom Hook

Extract when:

- You see the same `useState + useEffect` pattern in 2+ components
- A component has logic that is not related to its rendering
- You want to test stateful logic independently from UI
- The hook has a clear, descriptive name (`useAuth`, `useCart`, `useForm`)

Do NOT extract when:

- The logic is used in only one component and is simple
- You are just wrapping a single `useState` call (that adds no value)
- The "hook" would need 10 parameters to be flexible enough
- You are forcing reuse where none naturally exists

```
Decision flow:

  Is the logic used in multiple components?
  ├── Yes --> Extract a hook
  └── No
       |
       Is the logic complex enough to test separately?
       ├── Yes --> Extract a hook
       └── No
            |
            Is the component getting hard to read?
            ├── Yes --> Extract a hook
            └── No  --> Leave it inline
```

Real-World Use Cases

Form Handling

```jsx
function useForm(initialValues) {
  const [values, setValues] = useState(initialValues);
  const [errors, setErrors] = useState({});
  const [touched, setTouched] = useState({});

  const handleChange = useCallback((e) => {
    const { name, value } = e.target;
    setValues(prev => ({ ...prev, [name]: value }));
  }, []);

  const handleBlur = useCallback((e) => {
    const { name } = e.target;
    setTouched(prev => ({ ...prev, [name]: true }));
  }, []);

  const reset = useCallback(() => {
    setValues(initialValues);
    setErrors({});
    setTouched({});
  }, [initialValues]);

  return { values, errors, touched, handleChange, handleBlur, reset, setErrors };
}

// Usage
function SignupForm() {
  const { values, errors, handleChange, handleBlur } = useForm({
    email: '',
    password: '',
    name: '',
  });

  return (
    <form>
      <input name="email" value={values.email} onChange={handleChange} onBlur={handleBl...
      {errors.email && <span>{errors.email}</span>}
      {/* ... more fields */}
    </form>
  );
}
```

API State Management

```jsx
function useApiState() {
  const [state, dispatch] = useReducer(apiReducer, {
    data: null,
    loading: false,
    error: null,
  });
```

```
  const execute = useCallback(async (apiCall) => {
    dispatch({ type: 'LOADING' });
    try {
      const data = await apiCall();
      dispatch({ type: 'SUCCESS', payload: data });
      return data;
    } catch (error) {
      dispatch({ type: 'ERROR', payload: error.message });
      throw error;
    }
  }, []);

  return { ...state, execute };
}

function apiReducer(state, action) {
  switch (action.type) {
    case 'LOADING':
      return { ...state, loading: true, error: null };
    case 'SUCCESS':
      return { data: action.payload, loading: false, error: null };
    case 'ERROR':
      return { ...state, loading: false, error: action.payload };
    default:
      return state;
  }
}
```

Responsive Design

```jsx
function useBreakpoint() {
  const isMobile = useMediaQuery('(max-width: 639px)');
  const isTablet = useMediaQuery('(min-width: 640px) and (max-width: 1023px)');
  const isDesktop = useMediaQuery('(min-width: 1024px)');

  if (isMobile) return 'mobile';
  if (isTablet) return 'tablet';
  return 'desktop';
}

// Usage
function Navigation() {
  const breakpoint = useBreakpoint();

  switch (breakpoint) {
    case 'mobile':
      return <HamburgerMenu />;
    case 'tablet':
      return <CompactMenu />;
```

```
    case 'desktop':
      return <FullMenu />;
  }
}
```

When to Use / When NOT to Use

Use Custom Hooks When:

- Multiple components share the same stateful logic
- You want to keep components focused on rendering
- You need to test stateful logic in isolation
- The logic involves `useState`, `useEffect`, or other hooks

Do NOT Use Custom Hooks When:

- You just need a pure utility function (no hooks inside)
- The abstraction is used only once and is trivial
- You would need too many parameters to make it flexible
- A simple helper function would do the job

Common Mistakes

1. Not following the naming convention:

```jsx
// WRONG - React's linter will not enforce hook rules
function fetchData() {
  const [data, setData] = useState(null);
  // ...
}

// CORRECT
function useFetchData() {
  const [data, setData] = useState(null);
  // ...
}
```

2. Thinking hooks share state:

```jsx
// WRONG assumption
function ComponentA() {
  const [count, setCount] = useCounter();
  // Changing count here does NOT change it in ComponentB
}
```

3. Creating hooks that are too specific:

```jsx
// TOO SPECIFIC - only works for users
function useFetchUsers() {
  return useFetch('/api/users');
}

// BETTER - works for anything
function useFetch(url) {
  // ...
}
```

4. Not cleaning up effects:

```jsx
// WRONG - event listener leaks
function useWindowScroll() {
  const [scrollY, setScrollY] = useState(0);
  useEffect(() => {
    window.addEventListener('scroll', () => setScrollY(window.scrollY));
    // Missing cleanup!
  }, []);
  return scrollY;
}

// CORRECT
function useWindowScroll() {
  const [scrollY, setScrollY] = useState(0);
  useEffect(() => {
    const handler = () => setScrollY(window.scrollY);
    window.addEventListener('scroll', handler);
    return () => window.removeEventListener('scroll', handler);
  }, []);
  return scrollY;
}
```

5. Using stale closures:

```jsx
// WRONG - handler captures stale value
function useInterval(callback, delay) {
  useEffect(() => {
    const id = setInterval(callback, delay);  // callback is stale!
    return () => clearInterval(id);
  }, [delay]);  // callback not in deps
}

// CORRECT - use a ref for the latest callback
function useInterval(callback, delay) {
  const savedCallback = useRef(callback);

  useEffect(() => {
    savedCallback.current = callback;
  }, [callback]);

  useEffect(() => {
    const id = setInterval(() => savedCallback.current(), delay);
    return () => clearInterval(id);
  }, [delay]);
}
```

Best Practices

1. Start with the API you want. Write how you want to use the hook first, then implement it.
2. Return consistent shapes. If you return an object, always include the same keys. If you return an array, keep the same order.
3. Accept configuration, return state and actions. Hooks take in parameters and return values and functions.
4. Use `useCallback` for returned functions. This prevents unnecessary re-renders in components that receive these functions as props.
5. Handle edge cases. Think about SSR, initial render, unmounting, rapid state changes.
6. Write TypeScript types. Even if you use JavaScript, JSDoc types help.
7. Keep hooks focused. One hook, one responsibility. Compose for complexity.

Quick Summary

Custom hooks extract reusable stateful logic from components into standalone functions. They follow the naming convention `useXxx`, obey the rules of hooks (top-level calls only, React

functions only), and share logic but not state. You can compose simple hooks into complex ones, test them with `renderHook`, and use them to keep components clean and focused on rendering.

Key Points

- A custom hook is a function starting with `use` that calls other hooks
- Hooks must be called at the top level, never inside conditions or loops
- Each component calling a hook gets its own independent state
- Compose hooks by calling simpler hooks inside more complex ones
- Extract a hook when you see the same stateful pattern in multiple components
- Always clean up effects (event listeners, timers, subscriptions)
- Test hooks with `renderHook` and `act` from testing library

Practice Questions

1. You have three components that all track whether the user is online or offline using `navigator.onLine` and the `online`/`offline` window events. How would you extract this into a custom hook called `useOnlineStatus`? What would it return?
2. Why does calling `useToggle()` in two different components give each component its own independent boolean? What would you need to do if you wanted both components to share the same toggle state?
3. A junior developer wrote a custom hook that calls `useState` inside an `if` statement. The app works on the first render but crashes when the condition changes. Explain why this happens using the "lockers" analogy.
4. You have a `useFetch` hook and a `useDebounce` hook. How would you compose them to create a `useSearch` hook that debounces the search query before making an API call?
5. Your `useLocalStorage` hook works fine when the user changes values in one tab, but the other tab does not update. What browser API would you use to fix this, and how does it work?

Exercises

Exercise 1: Build useHover

Create a `useHover` hook that returns `[ref, isHovered]`. The caller attaches the `ref` to an element, and `isHovered` becomes `true` when the mouse is over that element.

Requirements:

- Returns a ref and a boolean
- Works with any HTML element
- Cleans up event listeners on unmount

Hint: Use `useRef` for the element reference and `useState` for the hover state. Set up `mouseenter` and `mouseleave` listeners in a `useEffect`.

Exercise 2: Build useCountdown

Create a `useCountdown(targetDate)` hook that returns the time remaining until a target date.

Requirements:

- Returns `{ days, hours, minutes, seconds }`
- Updates every second
- Returns all zeros when the target date has passed
- Clears the interval when the countdown reaches zero

Exercise 3: Build usePrevious

Create a `usePrevious(value)` hook that returns the value from the previous render.

Requirements:

- Returns `undefined` on the first render
- Returns the previous value on subsequent renders
- Works with any type of value

Hint: Use `useRef` to store the previous value and `useEffect` to update it after each render.

What Is Next?

Now that you know how to extract reusable logic into custom hooks, the next chapter explores Compound Components -- a pattern for building flexible UI components that work together as a team. You will learn how to create components like `Select/Option` and `Tabs/Tab/TabPanel` that share state implicitly, giving users of your components a clean and flexible API.

Chapter 19: Compound Components

What You Will Learn

- What compound components are and the problem they solve
- How to build compound components using `React.Children` and `cloneElement`
- How to build compound components using React Context (the modern approach)
- How to implement real-world patterns: `Select`/`Option`, `Tabs`/`Tab`/`TabPanel`
- How to validate that only allowed children are used
- How to balance flexibility with guardrails in component APIs

Why This Chapter Matters

Think about an HTML `<select>` element. You do not write `<select items={['Red', 'Blue', 'Green']} />`. Instead, you write:

```html
<select>
  <option value="red">Red</option>
  <option value="blue">Blue</option>
  <option value="green">Green</option>
</select>
```

The `<select>` and `<option>` work together. The `<select>` manages which option is selected, and the `<option>` elements know how to render themselves and respond to clicks -- all without you manually wiring up state between them. They share state implicitly.

This is the compound components pattern: a set of components that work together to form a complete UI element, sharing state behind the scenes so the consumer gets a clean, declarative API.

Without this pattern, component APIs tend to become unwieldy. You end up with components that accept dozens of props -- arrays of items, custom render functions for each slot, callback props for every possible interaction. Compound components flip this around: instead of one monolithic component, you provide a family of smaller components that compose naturally.

The Problem: Monolithic Component APIs

Consider building a Tabs component. The monolithic approach might look like this:

```jsx
// Monolithic API -- everything through props
<Tabs
  tabs={[
    { label: 'Profile', content: <ProfilePanel /> },
    { label: 'Settings', content: <SettingsPanel /> },
    { label: 'Billing', content: <BillingPanel />, disabled: true },
  ]}
  defaultActiveIndex={0}
  onChange={(index) => console.log(index)}
  tabClassName="custom-tab"
  panelClassName="custom-panel"
  renderTab={(tab, isActive) => (
    <span className={isActive ? 'active' : ''}>{tab.label}</span>
  )}
/>
```

This has problems:

- The API surface is huge -- many props to learn and document
- Customization requires render props or special prop shapes
- Adding a new feature (like an icon in one tab) requires changing the data structure
- TypeScript types become complex nested objects

The Compound Component Solution

```jsx
// Compound API -- natural, declarative, flexible
<Tabs defaultIndex={0} onChange={(index) => console.log(index)}>
  <TabList>
    <Tab>Profile</Tab>
    <Tab>Settings</Tab>
    <Tab disabled>Billing</Tab>
  </TabList>
```

```
  <TabPanels>
    <TabPanel><ProfilePanel /></TabPanel>
    <TabPanel><SettingsPanel /></TabPanel>
    <TabPanel><BillingPanel /></TabPanel>
  </TabPanels>
</Tabs>
```

This reads like HTML. Each piece is a separate component that you can style, extend, or customize independently. Want an icon in one tab? Just add it:

```jsx
<Tab><Icon name="user" /> Profile</Tab>
```

No API change needed.

```
Monolithic:                      Compound:
+-------------------------+    +-------------------------+
|  <Tabs                  |    |  <Tabs>                 |
|    tabs={[...]}         |    |    <TabList>             |
|    renderTab={fn}       |    |      <Tab>Profile</Tab> |
|    renderPanel={fn}     |    |      <Tab>Settings</Tab>|
|    onChange={fn}        |    |    </TabList>            |
|    tabClassName="..."   |    |    <TabPanels>           |
|    panelClassName="..." |    |      <TabPanel>...</TabPanel>
|    ...20 more props     |    |      <TabPanel>...</TabPanel>
|  />                     |    |    </TabPanels>          |
+-------------------------+    |  </Tabs>                 |
                               +-------------------------+
```

Approach 1: cloneElement (Legacy)

The original way to share state between parent and children was `React.Children` and `React.cloneElement`. The parent iterates over its children and injects props into them.

Simple Select/Option Example

```jsx
function Select({ children, value, onChange }) {
  return (
    <div className="select" role="listbox">
      {React.Children.map(children, (child) => {
        // Clone each child and inject extra props
```

```
        return React.cloneElement(child, {
          isSelected: child.props.value === value,
          onSelect: () => onChange(child.props.value),
        });
      })}
    </div>
  );
}

function Option({ value, children, isSelected, onSelect }) {
  return (
    <div
      className={`option ${isSelected ? 'selected' : ''}`}
      onClick={onSelect}
      role="option"
      aria-selected={isSelected}
    >
      {children}
    </div>
  );
}

// Usage
function ColorPicker() {
  const [color, setColor] = useState('red');

  return (
    <Select value={color} onChange={setColor}>
      <Option value="red">Red</Option>
      <Option value="blue">Blue</Option>
      <Option value="green">Green</Option>
    </Select>
  );
}

// Output (when "red" is selected):
// [Red]  (highlighted)
//  Blue
//  Green
```

How cloneElement Works

```
1. Parent receives children:
   <Select>
     <Option value="red">Red</Option>    <-- original child
     <Option value="blue">Blue</Option>  <-- original child
   </Select>

2. Parent clones each child with extra props:
   React.cloneElement(child, {
     isSelected: true,    <-- injected
```

```
      onSelect: fn         <-- injected
    })

3. Resulting element:
    <Option value="red" isSelected={true} onSelect={fn}>Red</Option>
```

Problems with cloneElement

This approach has significant limitations:

1. Only works with direct children -- if you wrap an `<Option>` in a `<div>`, the parent cannot find it
2. Fragile -- it relies on the children being specific component types
3. Implicit prop injection -- the child receives props it did not declare, which is confusing
4. No nesting flexibility -- adding intermediate wrapper components breaks the pattern

```jsx
// This breaks with cloneElement:
<Select value={color} onChange={setColor}>
  <div className="group">          {/* <-- wrapper breaks it */}
    <Option value="red">Red</Option>
  </div>
  <Option value="blue">Blue</Option>
</Select>
```

Approach 2: Context (Modern)

The modern approach uses React Context to share state. The parent provides state through context, and children consume it. This works regardless of nesting depth.

Select/Option with Context

```jsx
// Create a context for the Select state
const SelectContext = React.createContext();

function Select({ children, value, onChange }) {
  return (
    <SelectContext.Provider value={{ selectedValue: value, onChange }}>
      <div className="select" role="listbox">
        {children}
```

```
      </div>
    </SelectContext.Provider>
  );
}

function Option({ value, children }) {
  const { selectedValue, onChange } = useContext(SelectContext);
  const isSelected = value === selectedValue;

  return (
    <div
      className={`option ${isSelected ? 'selected' : ''}`}
      onClick={() => onChange(value)}
      role="option"
      aria-selected={isSelected}
    >
      {children}
    </div>
  );
}

// Usage -- exactly the same API
function ColorPicker() {
  const [color, setColor] = useState('red');

  return (
    <Select value={color} onChange={setColor}>
      <Option value="red">Red</Option>
      <Option value="blue">Blue</Option>
      <Option value="green">Green</Option>
    </Select>
  );
}
```

Now nesting works perfectly:

```jsx
// This works with Context approach:
<Select value={color} onChange={setColor}>
  <div className="primary-colors">
    <Option value="red">Red</Option>
    <Option value="blue">Blue</Option>
  </div>
  <div className="secondary-colors">
    <Option value="green">Green</Option>
    <Option value="purple">Purple</Option>
  </div>
</Select>
```

```
cloneElement approach:           Context approach:

Select                           Select (provides context)
```

```
  |-- cloneElement(Option)         |-- <div> (any wrapper)
  |-- cloneElement(Option)         |     |-- Option (reads context)
  |-- cloneElement(Option)         |     |-- Option (reads context)
                                   |-- <div>
Can only inject into                    |-- Option (reads context)
DIRECT children
                                 Works at ANY depth
```

Full Example: Tabs Component

Here is a complete Tabs implementation using context-based compound components:

```jsx
// --- Context ---
const TabsContext = React.createContext();

// --- Parent: Tabs ---
function Tabs({ children, defaultIndex = 0, onChange }) {
  const [activeIndex, setActiveIndex] = useState(defaultIndex);

  const selectTab = (index) => {
    setActiveIndex(index);
    onChange?.(index);
  };

  return (
    <TabsContext.Provider value={{ activeIndex, selectTab }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

// --- TabList ---
function TabList({ children }) {
  return (
    <div className="tab-list" role="tablist">
      {React.Children.map(children, (child, index) =>
        React.cloneElement(child, { index })
      )}
    </div>
  );
}

// --- Tab ---
function Tab({ children, index, disabled = false }) {
  const { activeIndex, selectTab } = useContext(TabsContext);
  const isActive = index === activeIndex;
```

```
  return (
    <button
      className={`tab ${isActive ? 'tab--active' : ''}`}
      onClick={() => !disabled && selectTab(index)}
      disabled={disabled}
      role="tab"
      aria-selected={isActive}
    >
      {children}
    </button>
  );
}

// --- TabPanels ---
function TabPanels({ children }) {
  const { activeIndex } = useContext(TabsContext);
  return (
    <div className="tab-panels">
      {React.Children.toArray(children)[activeIndex]}
    </div>
  );
}

// --- TabPanel ---
function TabPanel({ children }) {
  return (
    <div className="tab-panel" role="tabpanel">
      {children}
    </div>
  );
}

// --- Attach sub-components to parent ---
Tabs.TabList = TabList;
Tabs.Tab = Tab;
Tabs.TabPanels = TabPanels;
Tabs.TabPanel = TabPanel;
```

Usage:

```jsx
function SettingsPage() {
  return (
    <Tabs defaultIndex={0} onChange={(i) => console.log('Tab:', i)}>
      <Tabs.TabList>
        <Tabs.Tab>General</Tabs.Tab>
        <Tabs.Tab>Security</Tabs.Tab>
        <Tabs.Tab disabled>Advanced</Tabs.Tab>
      </Tabs.TabList>
      <Tabs.TabPanels>
        <Tabs.TabPanel>
          <GeneralSettings />
```

```
        </Tabs.TabPanel>
        <Tabs.TabPanel>
          <SecuritySettings />
        </Tabs.TabPanel>
        <Tabs.TabPanel>
          <AdvancedSettings />
        </Tabs.TabPanel>
      </Tabs.TabPanels>
    </Tabs>
  );
}

// Output (when "General" tab is active):
// [General]  Security  Advanced(disabled)
// +--------------------------------+
// |  General settings content...    |
// +--------------------------------+
```

Notice the hybrid approach: `TabList` uses `cloneElement` to inject the `index` prop (since tabs must know their position), while `Tab` and `TabPanels` use context for the shared state. This is a common real-world pattern -- use whatever works best for each sub-component.

Attaching Sub-Components as Static Properties

A clean API convention is to attach sub-components as properties of the parent:

```jsx
// Definition
Tabs.TabList = TabList;
Tabs.Tab = Tab;
Tabs.TabPanels = TabPanels;
Tabs.TabPanel = TabPanel;

// Usage -- single import gives you everything
import { Tabs } from './Tabs';

<Tabs>
  <Tabs.TabList>
    <Tabs.Tab>...</Tabs.Tab>
  </Tabs.TabList>
  <Tabs.TabPanels>
    <Tabs.TabPanel>...</Tabs.TabPanel>
  </Tabs.TabPanels>
</Tabs>
```

This pattern has two benefits:

1. Discoverability -- consumers type `Tabs.` and see all available sub-components
2. Single import -- no need to import `Tab`, `TabList`, `TabPanel` separately

Libraries like Radix UI, Reach UI, and Chakra UI use this approach extensively.

Validating Children

You can add guardrails to ensure consumers use the right sub-components:

```jsx
function Select({ children, value, onChange }) {
  // Validate that all direct children are Option components
  React.Children.forEach(children, (child) => {
    if (child.type !== Option) {
      console.warn(
        `Select expects Option children, but received ${
          child.type?.name || child.type || typeof child
        }`
      );
    }
  });

  return (
    <SelectContext.Provider value={{ selectedValue: value, onChange }}>
      <div className="select" role="listbox">
        {children}
      </div>
    </SelectContext.Provider>
  );
}
```

A more robust approach is to use context and throw if the context is missing:

```jsx
function useSelectContext() {
  const context = useContext(SelectContext);
  if (!context) {
    throw new Error(
      'Option must be used within a Select component. ' +
      'Wrap your Option components inside <Select>.'
    );
  }
  return context;
}

function Option({ value, children }) {
  const { selectedValue, onChange } = useSelectContext();
```

```
  // ...
}
```

This gives a clear error message if someone uses `<Option>` outside of a `<Select>`.

Real-World Use Case: Accordion

```jsx
const AccordionContext = React.createContext();

function Accordion({ children, allowMultiple = false }) {
  const [openItems, setOpenItems] = useState(new Set());

  const toggleItem = (id) => {
    setOpenItems((prev) => {
      const next = new Set(prev);
      if (next.has(id)) {
        next.delete(id);
      } else {
        if (!allowMultiple) next.clear();
        next.add(id);
      }
      return next;
    });
  };

  const isOpen = (id) => openItems.has(id);

  return (
    <AccordionContext.Provider value={{ toggleItem, isOpen }}>
      <div className="accordion">{children}</div>
    </AccordionContext.Provider>
  );
}

function AccordionItem({ children, id }) {
  return <div className="accordion-item">{children}</div>;
}

function AccordionHeader({ children, itemId }) {
  const { toggleItem, isOpen } = useContext(AccordionContext);

  return (
    <button
      className="accordion-header"
      onClick={() => toggleItem(itemId)}
      aria-expanded={isOpen(itemId)}
```

```
    >
      {children}
      <span>{isOpen(itemId) ? '▼' : '▶'}</span>
    </button>
  );
}

function AccordionPanel({ children, itemId }) {
  const { isOpen } = useContext(AccordionContext);

  if (!isOpen(itemId)) return null;

  return <div className="accordion-panel">{children}</div>;
}

// Usage
function FAQ() {
  return (
    <Accordion allowMultiple={false}>
      <AccordionItem id="q1">
        <AccordionHeader itemId="q1">
          What is React?
        </AccordionHeader>
        <AccordionPanel itemId="q1">
          React is a JavaScript library for building user interfaces.
        </AccordionPanel>
      </AccordionItem>

      <AccordionItem id="q2">
        <AccordionHeader itemId="q2">
          What are hooks?
        </AccordionHeader>
        <AccordionPanel itemId="q2">
          Hooks let you use state and other features in function components.
        </AccordionPanel>
      </AccordionItem>
    </Accordion>
  );
}

// Output (when "What is React?" is expanded):
// [v] What is React?
//     React is a JavaScript library for building user interfaces.
// [>] What are hooks?
```

When to Use / When NOT to Use

Use Compound Components When

- You have a set of components that only make sense together (tabs, accordions, selects)
- The monolithic prop-based API is becoming unwieldy
- Consumers need flexibility in the structure and layout of sub-components
- You want a declarative, HTML-like API

Do NOT Use Compound Components When

- The component is simple enough for a single component with a few props
- There is only one logical way to compose the children (no flexibility needed)
- The components do not share state -- they are independent
- The overhead of context and multiple components is not justified by the flexibility gained

---

Common Mistakes

Mistake 1: Relying Only on cloneElement

```jsx
// FRAGILE -- breaks with wrapper elements
<Select>
  <div className="group">
    <Option value="a">A</Option>   {/* cloneElement cannot reach this */}
  </div>
</Select>

// ROBUST -- use Context instead
// Option reads from SelectContext regardless of nesting depth
```

Mistake 2: Not Providing Error Messages for Misuse

```jsx
// BAD -- silent failure when used outside parent
function Tab({ index }) {
  const ctx = useContext(TabsContext);
  // ctx is undefined if used outside Tabs -- silent bugs
}

// GOOD -- explicit error
function Tab({ index }) {
  const ctx = useContext(TabsContext);
  if (!ctx) {
    throw new Error('Tab must be rendered inside a Tabs component');
```

```
  }
}
```

Mistake 3: Putting Too Much in Context

```jsx
// BAD -- entire parent state in context causes all children to re-render
<TabsContext.Provider value={{
  activeIndex,
  selectTab,
  tabs,          // array that changes reference
  animating,     // changes frequently
  history,       // grows over time
}}>

// BETTER -- only the minimum needed state
<TabsContext.Provider value={{ activeIndex, selectTab }}>
```

Mistake 4: Forgetting to Memoize Context Value

```jsx
// BAD -- new object every render, all consumers re-render
function Tabs({ children }) {
  const [activeIndex, setActiveIndex] = useState(0);
  return (
    <TabsContext.Provider value={{ activeIndex, setActiveIndex }}>
      {children}
    </TabsContext.Provider>
  );
}

// GOOD -- stable reference when values have not changed
function Tabs({ children }) {
  const [activeIndex, setActiveIndex] = useState(0);
  const value = useMemo(
    () => ({ activeIndex, setActiveIndex }),
    [activeIndex]
  );
  return (
    <TabsContext.Provider value={value}>
      {children}
    </TabsContext.Provider>
  );
}
```

Best Practices

1. Use Context over cloneElement for state sharing. Context works at any nesting depth and does not break with wrapper elements.
2. Attach sub-components as static properties of the parent (`Tabs.Tab`, `Tabs.Panel`) for a clean, discoverable API.
3. Create a custom hook for consuming the context (like `useTabsContext`) that throws a descriptive error when used outside the provider.
4. Memoize the context value to prevent unnecessary re-renders of all consuming children.
5. Keep the context value minimal -- only include the state that sub-components actually need.
6. Add ARIA attributes to make compound components accessible. Tabs need `role="tablist"`, `role="tab"`, `role="tabpanel"`, and `aria-selected`.
7. Consider a hybrid approach -- use `cloneElement` for injecting positional data (like index) and Context for shared state.

Quick Summary

Compound components are a group of related components that share implicit state to form a cohesive UI element. Instead of one component with dozens of props, you provide a family of components that compose declaratively. The modern approach uses React Context so that sub-components can read shared state regardless of nesting depth. This pattern powers tabs, accordions, selects, menus, and many other multi-part UI elements.

Key Points

- Compound components split a complex UI element into collaborating sub-components
- The parent manages state; children consume it implicitly
- `cloneElement` works for direct children but breaks with nesting
- React Context is the modern, robust approach to sharing state
- Attach sub-components as static properties for clean imports: `Tabs.Tab`
- Always create a custom context hook that throws if used outside the provider
- Memoize context values to avoid unnecessary re-renders

- This pattern is used extensively by component libraries: Radix UI, Reach UI, Headless UI, Chakra UI

Practice Questions

1. What are the limitations of using `React.cloneElement` for compound components, and how does React Context solve them?
2. Explain why memoizing the context value in a compound component parent is important for performance. What happens if you do not memoize it?
3. A teammate builds a `<Menu>` compound component but consumers report that `<Menu.Item>` throws errors when they wrap items in a styled `<div>`. What approach was likely used, and how would you fix it?
4. Compare the monolithic API (`<Tabs tabs={[...]} />`) with the compound API (`<Tabs><Tab>...</Tab></Tabs>`). What are two advantages and one disadvantage of the compound approach?
5. How would you prevent a consumer from using `<Option>` outside of a `<Select>` component? Write the code for the error handling.

Exercises

Exercise 1: Build a Compound Select Component

Build a `Select` / `Option` compound component using Context. Requirements:

- `Select` manages which option is selected
- `Option` highlights itself when selected and calls `onChange` when clicked
- Support wrapping options in `<div>` group containers
- Show a clear error if `Option` is used outside `Select`

Exercise 2: Build Compound Accordion

Build an `Accordion` with `AccordionItem`, `AccordionHeader`, and `AccordionPanel` sub-components. Requirements:

- Support single-expand mode (only one item open at a time)

- Support multi-expand mode (multiple items can be open)
- Add keyboard navigation (Enter/Space to toggle)
- Add ARIA attributes for accessibility

Exercise 3: Extend Tabs with Controlled Mode

Take the `Tabs` component from this chapter and add support for controlled mode. The consumer should be able to pass an `activeIndex` prop to control which tab is active externally (like a controlled input). When `activeIndex` is provided, the component should use it instead of internal state.

What Is Next?

The Tabs exercise asked you to support both "controlled" and "uncontrolled" modes. This is a fundamental pattern in React: who owns the state? In Chapter 20, you will explore the Controlled and Uncontrolled Components pattern in depth -- understanding when a component should manage its own state versus deferring to its parent, and how to build components that support both modes gracefully.

Chapter 20: Controlled and Uncontrolled Components

What You Will Learn

- The fundamental question behind this pattern: who owns the state?
- How controlled components work: parent owns state via `value` + `onChange`
- How uncontrolled components work: component owns state internally, accessed via `ref`
- How to build components that support both modes
- When to use each approach and why
- How `useImperativeHandle` lets you expose custom APIs through refs

Why This Chapter Matters

Imagine a thermostat. In one mode, you set the temperature manually -- you have full control, and the system does exactly what you tell it. In another mode, the thermostat runs on a schedule it manages internally -- you can read the current temperature, but you are not actively controlling every change.

This is the core tension in React component design: should the parent control the component's state (like the manual thermostat), or should the component manage its own state internally (like the scheduled one)?

Getting this wrong leads to real bugs:

- A form input that ignores what the user types because `value` is set but `onChange` is missing
- A modal that cannot be closed programmatically because it manages its own open/closed state
- A date picker whose state gets out of sync with the form data because neither side has clear ownership

This chapter teaches you to reason about state ownership, build components that work in either mode, and choose the right approach for each situation.

Controlled Components

A controlled component does not manage its own state. Its parent provides the current value and a callback to change it. The component is a "puppet" -- it displays what it is told and reports user interactions back.

```jsx
function ControlledInput() {
  const [name, setName] = useState('');

  return (
    <input
      type="text"
      value={name}                          // Parent says what to show
      onChange={(e) => setName(e.target.value)}  // Parent decides what to do
    />
  );
}
```

```
+-----------------------------------------------------+
|  Parent Component (owns state)                      |
|                                                     |
|  state: name = "Alice"                              |
|                                                     |
|  +---------------------------------------------+ |
|  |  <input>                                     | |
|  |                                              | |
|  |  value="Alice"  (what to display)            | |
|  |  onChange={fn}   (what to report)             | |
|  |                                              | |
|  |  User types "B" --> calls onChange("AliceB") | |
|  |  Parent updates state to "AliceB"            | |
|  |  Input re-renders with value="AliceB"        | |
|  +---------------------------------------------+ |
+-----------------------------------------------------+
```

The data flows in a single direction:

1. Parent passes `value` down
2. User interacts with the input
3. Input calls `onChange` with the new value
4. Parent updates its state

5. New state flows back down as `value`

Why Controlled?

Controlled components give you complete power over the value at every moment:

```jsx
function PhoneInput() {
  const [phone, setPhone] = useState('');

  const handleChange = (e) => {
    // Only allow digits
    const digits = e.target.value.replace(/\D/g, '');

    // Format as (XXX) XXX-XXXX
    let formatted = digits;
    if (digits.length > 3) {
      formatted = `(${digits.slice(0, 3)}) ${digits.slice(3)}`;
    }
    if (digits.length > 6) {
      formatted = `(${digits.slice(0, 3)}) ${digits.slice(3, 6)}-${digits.slice(6, 10)}`;
    }

    setPhone(formatted);
  };

  return <input value={phone} onChange={handleChange} />;
}

// User types: 5551234567
// Input shows: (555) 123-4567
```

You can validate, transform, or reject input at every keystroke. This is impossible with an uncontrolled component.

Uncontrolled Components

An uncontrolled component manages its own internal state. The parent does not pass `value` -- instead, it reads the value when needed using a `ref`.

```jsx
function UncontrolledInput() {
  const inputRef = useRef();

  const handleSubmit = () => {
```

```
    // Read the value only when needed
    console.log('Name:', inputRef.current.value);
  };

  return (
    <div>
      <input
        type="text"
        defaultValue=""     // Initial value only, not continuously controlled
        ref={inputRef}      // Gives parent access to read the DOM value
      />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}
```

```
+-----------------------------------------------------+
|  Parent Component (does NOT own input state)        |
|                                                     |
|  ref ----> points to the <input> DOM node           |
|                                                     |
|  +-----------------------------------------------+ |
|  |  <input>                                      | |
|  |                                               | |
|  |  defaultValue=""  (initial value only)        | |
|  |  The DOM manages its own current value        | |
|  |                                               | |
|  |  User types "Alice" --> DOM updates itself    | |
|  |  Parent reads ref.current.value when needed   | |
|  +-----------------------------------------------+ |
+-----------------------------------------------------+
```

Notice the key differences:

Controlled	Uncontrolled
`value` prop	`defaultValue` prop
`onChange` handler required	`onChange` optional
React state is the source of truth	DOM is the source of truth
Re-renders on every change	No re-render on input changes
Full control at every keystroke	Read value only when needed

Why Uncontrolled?

Uncontrolled components are simpler for cases where you do not need to intercept or transform every change:

```jsx
function SimpleSearchForm() {
  const searchRef = useRef();

  const handleSubmit = (e) => {
    e.preventDefault();
    const query = searchRef.current.value;
    // Search only happens on submit, not on every keystroke
    performSearch(query);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input ref={searchRef} defaultValue="" placeholder="Search..." />
      <button type="submit">Search</button>
    </form>
  );
}
```

This form does not need to know the value on every keystroke. It only cares about the value when the user submits. Using controlled state here would cause unnecessary re-renders on every character typed.

File Inputs: Always Uncontrolled

Some inputs can only be uncontrolled. File inputs are the classic example -- you cannot programmatically set their value for security reasons:

```jsx
function FileUpload() {
  const fileRef = useRef();

  const handleUpload = () => {
    const file = fileRef.current.files[0];
    if (file) {
      console.log('Uploading:', file.name, file.size, 'bytes');
    }
  };

  return (
    <div>
      {/* File inputs are ALWAYS uncontrolled */}
      <input type="file" ref={fileRef} />
```

```
      <button onClick={handleUpload}>Upload</button>
    </div>
  );
}
```

Building Components That Support Both Modes

The most flexible custom components support both controlled and uncontrolled modes. The consumer decides which mode to use based on whether they pass a `value` prop.

```jsx
function useControllableState({ value, defaultValue, onChange }) {
  // If value is provided, the component is controlled
  const isControlled = value !== undefined;

  // Internal state for uncontrolled mode
  const [internalValue, setInternalValue] = useState(defaultValue);

  // Use the controlled value if provided, otherwise internal
  const currentValue = isControlled ? value : internalValue;

  const setValue = (nextValue) => {
    if (!isControlled) {
      setInternalValue(nextValue);
    }
    onChange?.(nextValue);
  };

  return [currentValue, setValue];
}
```

Now use this hook to build a reusable Toggle:

```jsx
function Toggle({ value, defaultValue = false, onChange, children }) {
  const [on, setOn] = useControllableState({
    value,
    defaultValue,
    onChange,
  });

  return (
    <button onClick={() => setOn(!on)}>
      {children}: {on ? 'ON' : 'OFF'}
    </button>
  );
```

```
}

// Uncontrolled usage -- Toggle manages its own state
function Example1() {
  return <Toggle defaultValue={false}>WiFi</Toggle>;
  // Click toggles internally. No parent state needed.
}

// Controlled usage -- parent owns the state
function Example2() {
  const [wifiOn, setWifiOn] = useState(true);

  return (
    <div>
      <Toggle value={wifiOn} onChange={setWifiOn}>WiFi</Toggle>
      <p>WiFi is {wifiOn ? 'enabled' : 'disabled'}</p>
      <button onClick={() => setWifiOn(false)}>Force Off</button>
    </div>
  );
}
```

```
Uncontrolled mode:                Controlled mode:

<Toggle defaultValue={false}>    <Toggle value={wifiOn} onChange={setWifiOn}>
  Internal state: on = false       No internal state
  Click -> setOn(true)             Click -> onChange(true)
  Internal state: on = true        Parent: setWifiOn(true)
  Re-renders itself                Parent re-renders Toggle
                                   with value={true}
```

---

useImperativeHandle: Custom Ref APIs

Sometimes you want an uncontrolled component but need to expose specific methods to the parent -- not the raw DOM node, but a curated API. `useImperativeHandle` lets you define exactly what a `ref` exposes.

The Problem

```jsx
// Parent wants to focus, clear, and validate the input
// but ref.current gives the raw DOM node with 100+ properties
const inputRef = useRef();
```

```
// This works but exposes everything:
inputRef.current.value = '';      // direct DOM mutation
inputRef.current.focus();
inputRef.current.style.border = 'red';  // dangerous!
```

The Solution

```jsx
const FancyInput = React.forwardRef(function FancyInput(
  { label, defaultValue = '' },
  ref
) {
  const inputRef = useRef();
  const [error, setError] = useState('');

  // Expose only specific methods
  useImperativeHandle(ref, () => ({
    focus() {
      inputRef.current.focus();
    },
    clear() {
      inputRef.current.value = '';
      setError('');
    },
    validate() {
      const value = inputRef.current.value;
      if (!value.trim()) {
        setError('This field is required');
        return false;
      }
      setError('');
      return true;
    },
    getValue() {
      return inputRef.current.value;
    },
  }));

  return (
    <div>
      <label>{label}</label>
      <input ref={inputRef} defaultValue={defaultValue} />
      {error && <span className="error">{error}</span>}
    </div>
  );
});

// Parent usage
function RegistrationForm() {
  const nameRef = useRef();
  const emailRef = useRef();
```

```
  const handleSubmit = () => {
    const nameValid = nameRef.current.validate();
    const emailValid = emailRef.current.validate();

    if (nameValid && emailValid) {
      const data = {
        name: nameRef.current.getValue(),
        email: emailRef.current.getValue(),
      };
      submitForm(data);
    }
  };

  const handleReset = () => {
    nameRef.current.clear();
    emailRef.current.clear();
    nameRef.current.focus();
  };

  return (
    <form>
      <FancyInput ref={nameRef} label="Name" />
      <FancyInput ref={emailRef} label="Email" />
      <button type="button" onClick={handleSubmit}>Submit</button>
      <button type="button" onClick={handleReset}>Reset</button>
    </form>
  );
}
```

```
Without useImperativeHandle:      With useImperativeHandle:

ref.current = DOM <input>         ref.current = {
  .value                            focus(),
  .focus()                          clear(),
  .blur()                           validate(),
  .style                            getValue()
  .className                      }
  .setAttribute()
  .addEventListener()             Clean, intentional API
  ... 100+ DOM properties         Only what consumers need
```

When to Use useImperativeHandle

- When a parent needs to trigger actions on a child (focus, scroll, animate)
- When you want to hide the raw DOM API and expose a cleaner interface
- When building library components where consumers should not access internals
- When you need methods that combine DOM access with state updates (like `validate`)

Choosing Between Controlled and Uncontrolled

```
+-------------------------------------------------------+
|  Decision Guide: Controlled vs Uncontrolled           |
+-------------------------------------------------------+
|                                                       |
|  Do you need to validate/transform on every change?  |
|    YES --> Controlled                                 |
|                                                       |
|  Do you need the value to sync with other UI?         |
|    YES --> Controlled                                 |
|                                                       |
|  Do you need to programmatically set the value?       |
|    YES --> Controlled                                 |
|                                                       |
|  Is it a simple form submitted on button click?       |
|    YES --> Uncontrolled is fine                        |
|                                                       |
|  Is it a file input?                                  |
|    YES --> Must be uncontrolled                       |
|                                                       |
|  Are re-renders on every keystroke a concern?         |
|    YES --> Consider uncontrolled                      |
|                                                       |
|  Not sure?                                            |
|    --> Default to controlled                          |
+-------------------------------------------------------+
```

Controlled is Best For

- Forms with real-time validation ("password must be 8+ characters" as they type)
- Formatted inputs (phone numbers, credit cards, currency)
- Conditional logic that depends on the current value ("show confirm email if email is filled")
- Two-way binding between multiple inputs (start date must be before end date)

Uncontrolled is Best For

- Simple forms that only need values on submit
- File inputs (which must be uncontrolled)
- Integration with non-React code that manages its own DOM state
- Performance-sensitive scenarios where you want to avoid re-renders on every keystroke

When to Use / When NOT to Use

Use Controlled When

- You need to intercept, validate, or transform values in real time
- The value needs to be synchronized with other components or external state
- You need to programmatically reset or set values
- You are building a form with complex validation rules

Use Uncontrolled When

- You only need the value at submission time
- The input is a file upload
- You are integrating with a third-party DOM library
- Performance matters and you want to avoid per-keystroke re-renders

Use Both Modes When

- You are building a reusable component library
- Different consumers have different needs
- You want maximum flexibility (the `useControllableState` pattern)

Common Mistakes

Mistake 1: Controlled Input Without onChange

```jsx
// WRONG -- input is "frozen" and cannot be edited
function Broken() {
  const [name, setName] = useState('Alice');
  return <input value={name} />;
  // React warns: "You provided a `value` prop without an `onChange` handler"
}

// CORRECT
function Working() {
```

```
  const [name, setName] = useState('Alice');
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}

// Also correct -- if you truly want read-only, use readOnly
function ReadOnly() {
  return <input value="Alice" readOnly />;
}
```

Mistake 2: Switching Between Controlled and Uncontrolled

```jsx
// WRONG -- starts uncontrolled, becomes controlled
function Broken() {
  const [name, setName] = useState(undefined); // undefined = uncontrolled

  return (
    <input
      value={name}  // First render: undefined (uncontrolled)
      onChange={(e) => setName(e.target.value)}  // After typing: "A" (controlled!)
    />
  );
  // React warns: "changing from uncontrolled to controlled"
}

// CORRECT -- always controlled, start with empty string
function Working() {
  const [name, setName] = useState('');  // '' is a controlled value
  return <input value={name} onChange={(e) => setName(e.target.value)} />;
}
```

Mistake 3: Using value Instead of defaultValue for Uncontrolled

```jsx
// WRONG -- this is controlled (frozen) not uncontrolled
<input value="initial" ref={inputRef} />

// CORRECT -- defaultValue only sets the initial value
<input defaultValue="initial" ref={inputRef} />
```

Mistake 4: Over-Using useImperativeHandle

```jsx
// BAD -- reimplementing what controlled props already do
useImperativeHandle(ref, () => ({
```

```
  setValue(v) { setInternalValue(v); },
  getValue() { return internalValue; },
  setDisabled(d) { setDisabled(d); },
}));

// This should just be controlled props:
<MyInput value={value} onChange={setValue} disabled={disabled} />
```

`useImperativeHandle` is for imperative actions (focus, scroll, animate) not for state that should be props.

Best Practices

1. Default to controlled components unless you have a specific reason for uncontrolled. Controlled components are more predictable and easier to debug.
2. Initialize controlled state with the correct type -- use `''` for strings, `false` for booleans, `[]` for arrays. Never use `undefined` as initial state for controlled components.
3. Build reusable components to support both modes using the `useControllableState` pattern. This gives consumers flexibility.
4. Use `defaultValue` / `defaultChecked` for uncontrolled components, never `value` / `checked`.
5. Reserve `useImperativeHandle` for imperative actions like `focus()`, `scrollTo()`, and `validate()`. Do not use it to replace controlled props.
6. Warn on controlled/uncontrolled switches -- if your custom component detects a change from one mode to the other, log a warning in development.
7. Use `React.forwardRef` with `useImperativeHandle` so the ref API is available to parent components.

Quick Summary

Controlled components receive their value from props and notify changes via callbacks -- the parent is the single source of truth. Uncontrolled components manage their own internal state, with the parent reading values imperatively through refs when needed. The best reusable components support both modes, letting the consumer choose based on their needs. `useImperativeHandle` extends the uncontrolled pattern by letting you expose a custom, limited API through refs instead of the raw DOM.

Key Points

- Controlled: `value` + `onChange` -- parent owns the state
- Uncontrolled: `defaultValue` + `ref` -- component owns the state
- Controlled gives real-time access and transformation; uncontrolled gives simplicity
- Never switch a component between controlled and uncontrolled modes
- File inputs are always uncontrolled
- `useControllableState` lets you build components that support both modes
- `useImperativeHandle` exposes a curated ref API for imperative actions
- When in doubt, choose controlled

Practice Questions

1. A text input has `value={name}` but no `onChange` handler. What happens when the user tries to type, and why?
2. Explain the difference between `value` and `defaultValue` on an `<input>`. What mode does each one activate?
3. You are building a date picker component for a library. Should it be controlled, uncontrolled, or both? Explain your reasoning.
4. What is the purpose of `useImperativeHandle`, and why would you use it instead of exposing the raw DOM node through a ref?
5. A component's state is initialized with `useState(undefined)` and is passed as `value` to an input. The developer then types in the input, and React logs a warning about switching from uncontrolled to controlled. Explain what happened and how to fix it.

Exercises

Exercise 1: Build a Controllable Rating Component

Build a star rating component (`<Rating />`) that works in both controlled and uncontrolled modes:

- Uncontrolled: `<Rating defaultValue={3} />` -- manages its own state
- Controlled: `<Rating value={rating} onChange={setRating} />` -- parent owns state
- Display 5 clickable stars that highlight based on the current rating

Exercise 2: Build a Form with useImperativeHandle

Create a `FormField` component that wraps an input and exposes these methods via `useImperativeHandle`:

- `focus()` -- focuses the input
- `validate()` -- returns `true`/`false` and displays an error message
- `reset()` -- clears the input and error state
- `getValue()` -- returns the current value

Build a registration form using three `FormField` components (name, email, password) and a submit button that calls `validate()` on all fields.

Exercise 3: Controlled/Uncontrolled Accordion

Build an accordion component that supports both modes:

- Uncontrolled: `<Accordion defaultOpenIndex={0}>` -- manages which panel is open internally
- Controlled: `<Accordion openIndex={idx} onToggle={setIdx}>` -- parent controls which panel is open
- Add a "Expand All" button that only works in controlled mode

---

What Is Next?

Controlled components let a parent manage one component's state. But what about state that needs to be shared across many components at different levels of the tree? Passing props through every intermediate component (prop drilling) becomes painful fast. In Chapter 21, you will learn the Provider pattern -- using React Context to make state available to any component in the tree without drilling props through every level.

Chapter 21: The Provider Pattern

What You Will Learn

- What the Provider pattern is and the problem it solves (prop drilling)
- How React Context works: `createContext`, `Provider`, and `useContext`
- How to compose multiple providers cleanly
- Performance pitfalls of Context and how to avoid them
- When and how to split contexts for better performance
- How to combine Context with useReducer for complex state

Why This Chapter Matters

Imagine a large office building. The CEO makes a decision that every employee needs to know. Without an intercom system, the CEO tells the VP, the VP tells each director, each director tells each manager, each manager tells each team lead, and each team lead tells each employee. The message passes through every level, even if the people in the middle do not care about it at all.

This is prop drilling in React. A piece of state lives at the top of the component tree, and you pass it down through every intermediate component until it reaches the one that actually needs it. The intermediate components receive props they do not use, their signatures become cluttered, and refactoring is painful because renaming a prop requires changes at every level.

The Provider pattern solves this with the equivalent of an intercom system: React Context. You "broadcast" state from a provider, and any component in the tree can tune in and receive it directly -- no matter how deep it sits. No intermediaries needed.

The Problem: Prop Drilling

```jsx
// Top-level App has the theme
function App() {
  const [theme, setTheme] = useState('light');

  return (
    <Layout theme={theme}>
      <Sidebar theme={theme}>
        <Navigation theme={theme}>
          <NavItem theme={theme}>Home</NavItem>  {/* Finally uses theme */}
        </Navigation>
      </Sidebar>
    </Layout>
  );
}

// Layout does NOT use theme -- just passes it down
function Layout({ theme, children }) {
  return <div className="layout">{children}</div>;
  // theme is received but never used!
}

// Sidebar does NOT use theme -- just passes it down
function Sidebar({ theme, children }) {
  return <aside className="sidebar">{children}</aside>;
  // theme is received but never used!
}
```

```
Prop Drilling:                        Provider Pattern:

App (theme)                          App
  |                                    |
  v  passes theme                    ThemeProvider (theme)
Layout (theme) -- does not use!        |
  |                                    |  context "broadcast"
  v  passes theme                      |
Sidebar (theme) -- does not use!       v
  |                                  Layout -- no theme prop
  v  passes theme                      |
Navigation (theme) -- does not use!    v
  |                                  Sidebar -- no theme prop
  v  passes theme                      |
NavItem (theme) -- USES IT             v
                                     Navigation -- no theme prop
                                       |
                                       v
                                     NavItem -- useContext(ThemeContext)
```

With prop drilling, four components receive `theme` but only one uses it. With context, only the consumer accesses it.

The Solution: Context + Provider

Step 1: Create a Context

```jsx
const ThemeContext = React.createContext('light');  // default value
```

The default value is used only when a component reads the context without a provider above it in the tree. Think of it as a fallback.

Step 2: Create a Provider Component

```jsx
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const toggleTheme = () => {
    setTheme(prev => prev === 'light' ? 'dark' : 'light');
  };

  const value = { theme, toggleTheme };

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Step 3: Create a Custom Hook for Consumption

```jsx
function useTheme() {
  const context = useContext(ThemeContext);
  if (context === undefined) {
    throw new Error('useTheme must be used within a ThemeProvider');
  }
  return context;
}
```

Step 4: Use It

```jsx
function App() {
  return (
    <ThemeProvider>
      <Layout>
        <Sidebar>
          <Navigation>
            <NavItem>Home</NavItem>
          </Navigation>
        </Sidebar>
      </Layout>
    </ThemeProvider>
  );
}

// NavItem consumes the context directly -- no drilling
function NavItem({ children }) {
  const { theme, toggleTheme } = useTheme();

  return (
    <button
      className={`nav-item nav-item--${theme}`}
      onClick={toggleTheme}
    >
      {children}
    </button>
  );
}

// Layout, Sidebar, Navigation -- no theme prop at all!
function Layout({ children }) {
  return <div className="layout">{children}</div>;
}
```

Layout, Sidebar, and Navigation are now completely unaware of theme. They do not receive it, pass it, or care about it.

Complete Provider Pattern Implementation

Here is a full, production-quality provider setup:

```jsx
// auth-context.js

const AuthContext = React.createContext();

function AuthProvider({ children }) {
```

```
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Check for existing session on mount
    checkAuthSession()
      .then(setUser)
      .catch(() => setUser(null))
      .finally(() => setLoading(false));
  }, []);

  const login = async (email, password) => {
    const user = await loginAPI(email, password);
    setUser(user);
    return user;
  };

  const logout = async () => {
    await logoutAPI();
    setUser(null);
  };

  const value = useMemo(
    () => ({ user, loading, login, logout }),
    [user, loading]
  );

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
}

export { AuthProvider, useAuth };
```

Usage across the app:

```jsx
// index.jsx
function App() {
  return (
    <AuthProvider>
      <Router>
        <Routes />
```

```
      </Router>
    </AuthProvider>
  );
}

// Any component, at any depth
function UserMenu() {
  const { user, logout } = useAuth();

  if (!user) return <LoginButton />;

  return (
    <div>
      <span>{user.name}</span>
      <button onClick={logout}>Log out</button>
    </div>
  );
}

// Another component, completely different part of the tree
function WelcomeBanner() {
  const { user, loading } = useAuth();

  if (loading) return <Spinner />;
  if (!user) return null;

  return <h1>Welcome back, {user.name}!</h1>;
}
```

Multiple Providers: Composition

Real applications need multiple contexts: auth, theme, locale, notifications, feature flags. You compose them by nesting providers:

```jsx
function App() {
  return (
    <AuthProvider>
      <ThemeProvider>
        <LocaleProvider>
          <NotificationProvider>
            <Router>
              <Routes />
            </Router>
          </NotificationProvider>
        </LocaleProvider>
      </ThemeProvider>
```

```
    </AuthProvider>
  );
}
```

This nesting is purely visual -- it does not affect performance. Each provider is independent. But if the indentation bothers you, extract a utility:

```jsx
// Compose providers to avoid deep nesting
function ComposeProviders({ providers, children }) {
  return providers.reduceRight(
    (kids, Parent) => <Parent>{kids}</Parent>,
    children
  );
}

function App() {
  return (
    <ComposeProviders
      providers={[
        AuthProvider,
        ThemeProvider,
        LocaleProvider,
        NotificationProvider,
      ]}
    >
      <Router>
        <Routes />
      </Router>
    </ComposeProviders>
  );
}
```

```
Provider composition (nesting):

+-------------------------------------------+
|  AuthProvider                             |
|  +----------------------------------------+|
|  |  ThemeProvider                         ||
|  |  +-----------------------------------+||
|  |  |  LocaleProvider                  |||
|  |  |  +-------------------------------+|||
|  |  |  |  NotificationProvider         ||||
|  |  |  |  +---------------------------+|||||
|  |  |  |  |  <App />                  ||||||
|  |  |  |  +---------------------------+|||||
|  |  |  +-------------------------------+|||
|  |  +-----------------------------------+||
|  +----------------------------------------+|
+-------------------------------------------+

Each component can useAuth(), useTheme(),
```

```
useLocale(), or useNotifications() independently.
```

Performance Pitfalls

Context has a critical performance characteristic: when a context value changes, every component that consumes that context re-renders. This is by design, but it can cause problems if you are not careful.

Pitfall 1: Object References Change Every Render

```jsx
// BAD -- new object every render, ALL consumers re-render
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  return (
    <ThemeContext.Provider
      value={{ theme, setTheme }}  // New object every render!
    >
      {children}
    </ThemeContext.Provider>
  );
}
```

Even if `theme` has not changed, every render creates a new `{ theme, setTheme }` object. React sees a different reference and re-renders all consumers.

```jsx
// GOOD -- memoize the value
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');

  const value = useMemo(
    () => ({ theme, setTheme }),
    [theme]  // Only create a new object when theme actually changes
  );

  return (
    <ThemeContext.Provider value={value}>
      {children}
    </ThemeContext.Provider>
  );
}
```

Pitfall 2: One Fat Context for Everything

```jsx
// BAD -- one context with everything
const AppContext = React.createContext();

function AppProvider({ children }) {
  const [user, setUser] = useState(null);
  const [theme, setTheme] = useState('light');
  const [notifications, setNotifications] = useState([]);
  const [locale, setLocale] = useState('en');

  const value = useMemo(() => ({
    user, setUser,
    theme, setTheme,
    notifications, setNotifications,
    locale, setLocale,
  }), [user, theme, notifications, locale]);

  return (
    <AppContext.Provider value={value}>
      {children}
    </AppContext.Provider>
  );
}
```

When a new notification arrives, `notifications` changes, which changes the context value, which re-renders every consumer -- even components that only use `theme` and never touch notifications.

Splitting Contexts for Performance

The solution is to split unrelated state into separate contexts:

```jsx
// Separate contexts for separate concerns
const AuthContext = React.createContext();
const ThemeContext = React.createContext();
const NotificationContext = React.createContext();

// Each provider manages only its own state
function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = useMemo(() => ({ user, setUser }), [user]);
  return (
    <AuthContext.Provider value={value}>{children}</AuthContext.Provider>
```

```
  );
}

function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const value = useMemo(() => ({ theme, setTheme }), [theme]);
  return (
    <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>
  );
}
```

Now a notification change only re-renders notification consumers. Theme consumers are unaffected.

Advanced: Splitting State and Dispatch

For a single concern, you can split state (the current value) from dispatch (the updater function) into two contexts:

```jsx
const CountStateContext = React.createContext();
const CountDispatchContext = React.createContext();

function CountProvider({ children }) {
  const [count, setCount] = useState(0);

  return (
    <CountStateContext.Provider value={count}>
      <CountDispatchContext.Provider value={setCount}>
        {children}
      </CountDispatchContext.Provider>
    </CountStateContext.Provider>
  );
}

function useCountState() {
  const context = useContext(CountStateContext);
  if (context === undefined) {
    throw new Error('useCountState must be used within CountProvider');
  }
  return context;
}

function useCountDispatch() {
  const context = useContext(CountDispatchContext);
  if (context === undefined) {
    throw new Error('useCountDispatch must be used within CountProvider');
  }
  return context;
}
```

Why split? Because `setCount` is a stable reference (it never changes), the dispatch context never triggers re-renders. Components that only need to update the count (like a button) do not re-render when the count changes:

```jsx
// This component only dispatches -- it never re-renders when count changes
function IncrementButton() {
  const setCount = useCountDispatch();
  return <button onClick={() => setCount(c => c + 1)}>+1</button>;
}

// This component only reads -- it re-renders when count changes
function CountDisplay() {
  const count = useCountState();
  return <p>Count: {count}</p>;
}
```

Context + useReducer

For complex state with multiple actions, combine Context with `useReducer`:

```jsx
const TodoContext = React.createContext();

function todoReducer(state, action) {
  switch (action.type) {
    case 'ADD_TODO':
      return {
        ...state,
        todos: [...state.todos, {
          id: Date.now(),
          text: action.text,
          completed: false,
        }],
      };
    case 'TOGGLE_TODO':
      return {
        ...state,
        todos: state.todos.map(todo =>
          todo.id === action.id
            ? { ...todo, completed: !todo.completed }
            : todo
        ),
      };
    case 'DELETE_TODO':
      return {
        ...state,
```

```
        todos: state.todos.filter(todo => todo.id !== action.id),
      };
    case 'SET_FILTER':
      return { ...state, filter: action.filter };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function TodoProvider({ children }) {
  const [state, dispatch] = useReducer(todoReducer, {
    todos: [],
    filter: 'all',
  });

  const value = useMemo(() => ({ state, dispatch }), [state]);

  return (
    <TodoContext.Provider value={value}>
      {children}
    </TodoContext.Provider>
  );
}

function useTodos() {
  const context = useContext(TodoContext);
  if (!context) {
    throw new Error('useTodos must be used within a TodoProvider');
  }
  return context;
}

// Usage
function AddTodo() {
  const { dispatch } = useTodos();
  const [text, setText] = useState('');

  const handleAdd = () => {
    if (text.trim()) {
      dispatch({ type: 'ADD_TODO', text });
      setText('');
    }
  };

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={handleAdd}>Add</button>
    </div>
  );
}

function TodoList() {
  const { state, dispatch } = useTodos();
```

```
  const filteredTodos = state.todos.filter(todo => {
    if (state.filter === 'active') return !todo.completed;
    if (state.filter === 'completed') return todo.completed;
    return true;
  });

  return (
    <ul>
      {filteredTodos.map(todo => (
        <li key={todo.id}>
          <span
            style={{
              textDecoration: todo.completed ? 'line-through' : 'none'
            }}
            onClick={() => dispatch({ type: 'TOGGLE_TODO', id: todo.id })}
          >
            {todo.text}
          </span>
          <button onClick={() => dispatch({ type: 'DELETE_TODO', id: todo.id })}>
            Delete
          </button>
        </li>
      ))}
    </ul>
  );
}
```

When to Use / When NOT to Use

Use the Provider Pattern When

- Multiple components at different tree levels need the same data
- Prop drilling through 3+ levels of components that do not use the data
- The data is "global" to a subtree: theme, auth, locale, feature flags
- You are building a component library with compound components (Tabs, Accordions)

Do NOT Use the Provider Pattern When

- Only one or two components need the data -- just pass props
- The data changes very frequently (like mouse position at 60fps) -- context re-renders too many consumers
- You need to share data across different React roots or between React and non-React code -- use external state management

- You reach for it just to avoid passing props through one level -- that is not prop drilling, that is normal React

---

Common Mistakes

Mistake 1: Using Context for Frequently Changing Values

```jsx
// BAD -- mouse position changes 60 times per second
// Every consumer re-renders on every mouse move
function MouseProvider({ children }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });

  useEffect(() => {
    const handle = (e) => setPos({ x: e.clientX, y: e.clientY });
    window.addEventListener('mousemove', handle);
    return () => window.removeEventListener('mousemove', handle);
  }, []);

  return (
    <MouseContext.Provider value={pos}>
      {children}
    </MouseContext.Provider>
  );
}

// BETTER -- use a ref + subscription pattern, or a state management library
```

Mistake 2: Not Providing a Custom Hook

```jsx
// BAD -- consumers directly use useContext
function UserMenu() {
  const value = useContext(AuthContext);
  // No error if used outside provider -- just gets undefined
}

// GOOD -- custom hook with error boundary
function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within an AuthProvider');
  }
  return context;
}
```

Mistake 3: Putting Everything in One Context

As covered in the performance section: split unrelated state into separate contexts. Auth changes should not cause theme consumers to re-render.

Mistake 4: Forgetting the Default Value Semantics

```jsx
// The default value is used ONLY when there is no Provider above
const ThemeContext = React.createContext('light');

// This component uses 'light' because there is no provider
function OrphanedComponent() {
  const theme = useContext(ThemeContext); // 'light' (default)
}

// This component uses 'dark' because the provider overrides the default
function ProvidedComponent() {
  return (
    <ThemeContext.Provider value="dark">
      <Child />  {/* useContext returns 'dark' */}
    </ThemeContext.Provider>
  );
}
```

Best Practices

1. Always create a custom hook for each context (`useAuth`, `useTheme`). Never expose the raw context object.
2. Always validate context usage -- throw an error in the custom hook if the context is undefined (meaning no provider was found).
3. Memoize the context value with `useMemo` to prevent unnecessary consumer re-renders.
4. Split unrelated concerns into separate contexts. One context per domain: auth, theme, locale.
5. Consider splitting state from dispatch for contexts where some consumers only dispatch actions and never read state.
6. Co-locate the provider, context, and hook in one file. Export only the provider and hook, not the raw context.
7. Use Context for low-frequency state (theme, auth, locale). For high-frequency updates (animations, cursor position, real-time data), use external state management or refs.

8. Place providers as low as possible in the tree. If only a subtree needs the context, do not put the provider at the root.

Quick Summary

The Provider pattern uses React Context to make state available to any component in the tree without prop drilling. A provider component wraps a subtree and broadcasts state; any descendant can consume it via `useContext`. Multiple providers compose by nesting. The main performance concern is that all consumers re-render when the context value changes, which you mitigate by memoizing values, splitting contexts, and separating state from dispatch.

Key Points

- Context eliminates prop drilling: data skips intermediate components
- The pattern has three parts: `createContext`, a Provider component, and a `useContext` consumer hook
- Always create a custom hook that throws if used outside the provider
- Always memoize the context value with `useMemo`
- Split unrelated state into separate contexts to avoid unnecessary re-renders
- Splitting state from dispatch contexts prevents render-only consumers from causing update-only consumers to re-render (and vice versa)
- Context is best for low-frequency, "global" state: theme, auth, locale
- Do not use context for high-frequency data -- it re-renders all consumers on every change
- Compose multiple providers by nesting or with a `ComposeProviders` utility

---

Practice Questions

1. What is prop drilling, and at what point does it become a problem worth solving with Context?

2. Why is it important to memoize the context value with `useMemo`? What happens if you pass a new object literal directly to `Provider value`?
3. You have a single context providing `{ user, theme, notifications }`. Notifications update every 5 seconds. A component that only reads `theme` re-renders every 5 seconds. Why, and how do you fix it?
4. Explain the difference between the default value passed to `createContext` and the value passed to `<Provider value={...}>`. When does each one get used?
5. Why would you split a context into a "state context" and a "dispatch context"? Give a concrete example of two components that benefit from this split.

Exercises

Exercise 1: Build an Auth Provider

Build a complete `AuthProvider` with:

- `login(email, password)` function
- `logout()` function
- `user` state (null when logged out)
- `loading` state (true while checking session)
- A custom `useAuth` hook that throws if used outside the provider
- Memoized context value

Build two consumer components: a login form and a user profile dropdown.

Exercise 2: Build a Notification System

Build a `NotificationProvider` that manages a list of toast notifications:

- `addNotification(message, type)` -- adds a notification (types: success, error, warning)
- Notifications auto-dismiss after 5 seconds
- A `NotificationList` component that renders active notifications
- A custom `useNotification` hook

Split the context so that components that only add notifications do not re-render when the notification list changes.

Exercise 3: Performance Optimization

Start with a single "AppContext" that provides `{ user, theme, locale, sidebarOpen }`. Build four consumer components, one for each piece of state. Then:

1. Add `console.log` to each consumer to track re-renders
2. Change only the theme and observe that ALL consumers re-render
3. Refactor into separate contexts and verify that only the theme consumer re-renders

---

What Is Next?

The Provider pattern gives components access to shared state, but who decides how that state changes? In most cases, the provider dictates the state transitions -- the consumer calls `dispatch` or a setter, and the provider's reducer decides what happens. But what if you want the consumer to customize those transitions? In Chapter 22, you will learn the State Reducer pattern, which lets consumers override state transitions while the provider maintains sensible defaults.

Chapter 22: The State Reducer Pattern

What You Will Learn

- What the state reducer pattern is and why it exists
- How to implement customizable state transitions by accepting a user-supplied reducer
- The principle of inversion of control and how it applies to component state
- How to build a `useToggle` hook with a state reducer
- How to provide sensible defaults while letting consumers override any transition
- Real-world examples where this pattern shines

Why This Chapter Matters

Imagine you build a light switch component for a smart home app. It works perfectly: click to toggle on and off. Then a customer says, "I want the light to turn off automatically after 5 seconds." Another says, "I want to prevent turning it off while a security alarm is active." A third says, "I want clicking to cycle through brightness levels instead of just on/off."

You could add props for each request: `autoOff`, `preventOff`, `cycleBrightness`. But the requests keep coming, and each new prop adds complexity. Eventually, your component has 20 boolean props controlling every possible behavior, and the code is a tangled mess of conditional logic.

The state reducer pattern takes a different approach. Instead of anticipating every possible customization, you let the consumer replace your state transition logic with their own. You provide sensible defaults, and the consumer can override any transition for any action. It is like giving someone the light switch but saying, "Here is how the switch normally works. If you want it to work differently, just tell me what to do instead."

This is a powerful application of inversion of control: the component defines what actions exist, and the consumer defines how those actions affect state.

The Core Concept

A state reducer is a function that determines how state changes in response to actions. The pattern works like this:

1. Your component/hook has an internal reducer with default behavior
2. You accept an optional `reducer` (or `stateReducer`) from the consumer
3. When an action happens, the consumer's reducer runs instead of (or alongside) the default one
4. The consumer can modify, override, or extend any state transition

```
+-------------------------------------------------------------+
|  Without State Reducer:                                     |
|                                                             |
|  Action --> Internal Reducer --> New State                   |
|             (fixed behavior)                                |
|                                                             |
+-------------------------------------------------------------+
|  With State Reducer:                                        |
|                                                             |
|  Action --> Consumer's Reducer --> New State                 |
|             (customizable)                                  |
|             |                                               |
|             v                                               |
|             Can call the internal                           |
|             reducer for defaults                            |
+-------------------------------------------------------------+
```

Building useToggle with a State Reducer

Let us start with a simple `useToggle` hook and evolve it step by step.

Step 1: Basic useToggle (No Customization)

```jsx
function useToggle({ initialOn = false } = {}) {
  const [on, setOn] = useState(initialOn);

  const toggle = () => setOn(prev => !prev);
  const setOff = () => setOn(false);
  const setOnTrue = () => setOn(true);

  return { on, toggle, setOff, setOn: setOnTrue };
}

// Usage
function LightSwitch() {
  const { on, toggle } = useToggle();
  return (
    <button onClick={toggle}>
      Light is {on ? 'ON' : 'OFF'}
    </button>
  );
}
```

This works fine until someone needs custom behavior. What if they want to prevent toggling off? They cannot -- the logic is locked inside the hook.

Step 2: Convert to useReducer

First, refactor to use `useReducer` so we have a reducer function to customize:

```jsx
// Action types
const TOGGLE = 'TOGGLE';
const SET_ON = 'SET_ON';
const SET_OFF = 'SET_OFF';

// Default reducer -- the "normal" behavior
function toggleReducer(state, action) {
  switch (action.type) {
    case TOGGLE:
      return { on: !state.on };
    case SET_ON:
      return { on: true };
    case SET_OFF:
      return { on: false };
    default:
      throw new Error(`Unknown action type: ${action.type}`);
  }
}

function useToggle({ initialOn = false } = {}) {
  const [state, dispatch] = useReducer(toggleReducer, { on: initialOn });

  const toggle = () => dispatch({ type: TOGGLE });
```

```
  const setOff = () => dispatch({ type: SET_OFF });
  const setOn = () => dispatch({ type: SET_ON });

  return { on: state.on, toggle, setOff, setOn };
}
```

Same behavior, but now the state transitions are handled by a reducer function that we can make replaceable.

Step 3: Accept a Custom Reducer

Now the key step: let the consumer provide their own reducer:

```jsx
function useToggle({ initialOn = false, reducer = toggleReducer } = {}) {
  const [state, dispatch] = useReducer(reducer, { on: initialOn });

  const toggle = () => dispatch({ type: TOGGLE });
  const setOff = () => dispatch({ type: SET_OFF });
  const setOn = () => dispatch({ type: SET_ON });

  return { on: state.on, toggle, setOff, setOn };
}

// Export action types so consumers can reference them
useToggle.types = { TOGGLE, SET_ON, SET_OFF };

// Export the default reducer so consumers can call it for defaults
useToggle.reducer = toggleReducer;
```

Now a consumer can override behavior:

```jsx
// Custom reducer: prevent turning off
function preventOffReducer(state, action) {
  if (action.type === useToggle.types.SET_OFF) {
    return state; // Ignore the SET_OFF action
  }
  if (action.type === useToggle.types.TOGGLE && state.on) {
    return state; // Ignore toggle when already on
  }
  // For everything else, use default behavior
  return useToggle.reducer(state, action);
}

function AlarmLight() {
  const { on, toggle, setOn, setOff } = useToggle({
    reducer: preventOffReducer,
  });
```

```
  return (
    <div>
      <button onClick={toggle}>Toggle</button>
      <button onClick={setOn}>Force On</button>
      <button onClick={setOff}>Force Off (blocked!)</button>
      <p>Alarm light is {on ? 'ON' : 'OFF'}</p>
    </div>
  );
}
```

```
Default reducer:                      Custom reducer:

TOGGLE: on=true  -> on=false          TOGGLE: on=true  -> on=true (blocked!)
TOGGLE: on=false -> on=true           TOGGLE: on=false -> on=true
SET_ON:          -> on=true           SET_ON:          -> on=true
SET_OFF:         -> on=false          SET_OFF:         -> on=false (blocked!)
```

Real-World Example: useToggle with Maximum Click Count

A more practical example: a toggle that can only be toggled a certain number of times.

```jsx
function clickLimitReducer(state, action) {
  // Allow all default behavior, but track clicks
  if (action.type === useToggle.types.TOGGLE) {
    if (state.clickCount >= 4) {
      return state; // No more toggles allowed
    }
    return {
      ...useToggle.reducer(state, action),
      clickCount: (state.clickCount || 0) + 1,
    };
  }
  return useToggle.reducer(state, action);
}

function LimitedToggle() {
  const { on, toggle } = useToggle({
    reducer: clickLimitReducer,
  });

  return (
    <button onClick={toggle}>
```

```
      {on ? 'ON' : 'OFF'} (limited clicks)
    </button>
  );
}
```

Notice how the custom reducer extends the default behavior by adding `clickCount` to the state. It does not have to replace everything -- it can build on top of the defaults.

Pattern: Providing Both Changes and Action to the Reducer

A more powerful approach gives the consumer's reducer the proposed changes from the default reducer, so they can see what would happen and decide whether to allow it:

```jsx
function useToggle({ initialOn = false, reducer: userReducer } = {}) {
  function internalReducer(state, action) {
    // Calculate what the default reducer would produce
    const changes = toggleReducer(state, action);

    // If no custom reducer, use defaults
    if (!userReducer) {
      return changes;
    }

    // Let the consumer decide what to keep
    return userReducer(state, { ...action, changes });
  }

  const [state, dispatch] = useReducer(internalReducer, {
    on: initialOn,
  });

  const toggle = () => dispatch({ type: TOGGLE });
  const setOff = () => dispatch({ type: SET_OFF });
  const setOn = () => dispatch({ type: SET_ON });

  return { on: state.on, toggle, setOff, setOn };
}
```

Now the consumer's reducer receives the proposed changes along with the action:

```jsx
function myReducer(state, action) {
  // action.changes contains what the default reducer WOULD return

  if (action.type === TOGGLE && state.on) {
    // When toggling off, ask for confirmation instead
    if (!window.confirm('Are you sure you want to turn off?')) {
      return state; // Cancel the change
    }
  }

  // Accept the default changes
  return action.changes;
}

function ConfirmableSwitch() {
  const { on, toggle } = useToggle({ reducer: myReducer });

  return (
    <button onClick={toggle}>
      {on ? 'ON' : 'OFF'}
    </button>
  );
}
```

```
Flow with proposed changes:

1. User clicks toggle (on=true currently)
2. Internal reducer calculates: changes = { on: false }
3. Consumer's reducer receives:
   state = { on: true }
   action = { type: 'TOGGLE', changes: { on: false } }
4. Consumer decides:
   - Return action.changes to accept ({ on: false })
   - Return state to reject (stay { on: true })
   - Return something else to modify ({ on: false, reason: 'user' })
```

Inversion of Control

The state reducer pattern is a concrete example of inversion of control (IoC). In normal component design, the component controls everything:

```
Normal control flow:
Component decides what happens --> Consumer uses it as-is

Inverted control flow:
```

```
Component defines what actions exist
Consumer decides what happens for each action
```

Compare this to other IoC patterns you have seen:

Pattern	What Is Inverted
Render Props	Control of what is rendered
Compound Components	Control of structure and layout
State Reducer	Control of state transitions

The state reducer is the most powerful form of IoC because state drives everything in a UI. If you control how state changes, you control the entire behavior of the component.

A More Complete Example: useSelect

Here is a more complex hook that manages a select/dropdown component:

```jsx
const SELECT_ACTIONS = {
  OPEN: 'OPEN',
  CLOSE: 'CLOSE',
  SELECT_ITEM: 'SELECT_ITEM',
  HIGHLIGHT_ITEM: 'HIGHLIGHT_ITEM',
  CLEAR: 'CLEAR',
};

function selectReducer(state, action) {
  switch (action.type) {
    case SELECT_ACTIONS.OPEN:
      return { ...state, isOpen: true };
    case SELECT_ACTIONS.CLOSE:
      return { ...state, isOpen: false, highlightedIndex: -1 };
    case SELECT_ACTIONS.SELECT_ITEM:
      return {
        ...state,
        selectedItem: action.item,
        isOpen: false,
        highlightedIndex: -1,
      };
    case SELECT_ACTIONS.HIGHLIGHT_ITEM:
      return { ...state, highlightedIndex: action.index };
```

```
    case SELECT_ACTIONS.CLEAR:
      return { ...state, selectedItem: null };
    default:
      throw new Error(`Unknown action: ${action.type}`);
  }
}

function useSelect({ items, reducer: userReducer } = {}) {
  function internalReducer(state, action) {
    const changes = selectReducer(state, action);
    if (!userReducer) return changes;
    return userReducer(state, { ...action, changes });
  }

  const [state, dispatch] = useReducer(internalReducer, {
    isOpen: false,
    selectedItem: null,
    highlightedIndex: -1,
  });

  return {
    ...state,
    open: () => dispatch({ type: SELECT_ACTIONS.OPEN }),
    close: () => dispatch({ type: SELECT_ACTIONS.CLOSE }),
    selectItem: (item) =>
      dispatch({ type: SELECT_ACTIONS.SELECT_ITEM, item }),
    highlightItem: (index) =>
      dispatch({ type: SELECT_ACTIONS.HIGHLIGHT_ITEM, index }),
    clear: () => dispatch({ type: SELECT_ACTIONS.CLEAR }),
  };
}

useSelect.types = SELECT_ACTIONS;
useSelect.reducer = selectReducer;
```

Consumer customization: keep the dropdown open after selection (for multi-select behavior):

```jsx
function keepOpenReducer(state, action) {
  if (action.type === useSelect.types.SELECT_ITEM) {
    // Accept the selection but override isOpen to stay true
    return {
      ...action.changes,
      isOpen: true,  // Override: keep it open
    };
  }
  return action.changes;
}

function MultiSelect({ items }) {
  const { isOpen, selectedItem, open, selectItem } = useSelect({
    items,
    reducer: keepOpenReducer,
```

```
  });

  return (
    <div>
      <button onClick={open}>Select...</button>
      {isOpen && (
        <ul>
          {items.map(item => (
            <li key={item} onClick={() => selectItem(item)}>
              {item}
            </li>
          ))}
        </ul>
      )}
      <p>Selected: {selectedItem}</p>
    </div>
  );
}
```

When to Use / When NOT to Use

Use the State Reducer Pattern When

- You are building a reusable component or hook that different consumers need to customize
- You cannot predict all the ways consumers will want to modify behavior
- The component has well-defined actions but consumers need different responses to those actions
- You want to give consumers full control without sacrificing sensible defaults

Do NOT Use the State Reducer Pattern When

- The component is application-specific, not reusable -- just write the reducer you need
- The customization needs are simple enough for a prop (like `disabled` or `maxLength`)
- There are only one or two possible customizations -- props or callbacks are simpler
- Your team is not comfortable with reducers -- the pattern adds conceptual overhead

---

Common Mistakes

Mistake 1: Not Exposing the Default Reducer

```jsx
// BAD -- consumer has to reimplement all default behavior
function useToggle({ reducer }) {
  const [state, dispatch] = useReducer(reducer, { on: false });
  // ...
}

// Consumer must handle EVERY action type, even for default behavior
function myReducer(state, action) {
  switch (action.type) {
    case 'TOGGLE': return { on: !state.on };
    case 'SET_ON': return { on: true };
    case 'SET_OFF': return { on: false };
    // Must implement ALL defaults manually
  }
}

// GOOD -- export the default reducer so consumers can delegate
useToggle.reducer = toggleReducer;

function myReducer(state, action) {
  if (action.type === 'SET_OFF') return state; // Custom: block SET_OFF
  return useToggle.reducer(state, action);      // Default for everything else
}
```

Mistake 2: Not Exposing Action Types

```jsx
// BAD -- consumer uses magic strings
function myReducer(state, action) {
  if (action.type === 'TOGGLE') { ... }  // What if the type name changes?
}

// GOOD -- export action types as constants
useToggle.types = { TOGGLE, SET_ON, SET_OFF };

function myReducer(state, action) {
  if (action.type === useToggle.types.TOGGLE) { ... }
}
```

Mistake 3: Mutating State in the Reducer

```jsx
// WRONG -- mutates state directly
function myReducer(state, action) {
  state.on = !state.on;  // Mutation!
```

```
  return state;           // Same reference -- React does not re-render
}

// CORRECT -- return a new object
function myReducer(state, action) {
  return { ...state, on: !state.on };
}
```

Mistake 4: Overcomplicating Simple Cases

```jsx
// OVER-ENGINEERED -- a state reducer for a simple disabled toggle
function useToggle({ disabled, ...rest }) {
  // Just use a prop!
  return useToggle({
    ...rest,
    reducer: (state, action) => {
      if (disabled) return state;
      return toggleReducer(state, action);
    },
  });
}

// SIMPLER -- just check the prop
function useToggle({ initialOn = false, disabled = false } = {}) {
  const [on, setOn] = useState(initialOn);
  const toggle = () => { if (!disabled) setOn(prev => !prev); };
  return { on, toggle };
}
```

Use the state reducer pattern for open-ended customization, not for specific, predictable features.

Best Practices

1. Export the default reducer so consumers can delegate to it for actions they do not want to customize.
2. Export action type constants so consumers reference stable identifiers rather than magic strings.
3. Pass proposed changes to the consumer's reducer so they can see what would happen and selectively override.
4. Keep the action types stable -- they are your public API. Changing them is a breaking change.

5. Document each action type with what it does by default, so consumers know what they are overriding.
6. Make the reducer optional -- the hook should work with sensible defaults when no custom reducer is provided.
7. Do not mix state reducer with too many config props -- if you have a state reducer, let it handle behavioral customization rather than adding more boolean props.

Quick Summary

The state reducer pattern lets consumers customize how a component's state transitions work by providing their own reducer function. The component defines the actions; the consumer controls the responses. This gives maximum flexibility without the component needing to anticipate every possible use case. The consumer's reducer can accept defaults, reject changes, or modify transitions -- achieving true inversion of control over component behavior.

Key Points

- The state reducer pattern accepts a custom reducer from the consumer
- The component provides default behavior; the consumer overrides what they need
- Always export the default reducer and action types so consumers can build on top of defaults
- Passing proposed `changes` to the consumer's reducer lets them selectively override
- This is inversion of control applied to state transitions
- Use this for reusable hooks/components with unpredictable customization needs
- Do not use it for simple, predictable customization -- props are simpler
- Libraries like Downshift pioneered this pattern in the React ecosystem

---

Practice Questions

1. Explain the difference between adding a `disabled` prop to `useToggle` versus using the state reducer pattern to prevent toggling. When would you choose each approach?

2. Why is it important to export both the default reducer and the action type constants? What happens if a consumer does not have access to them?
3. What does "inversion of control" mean in the context of the state reducer pattern? How does it differ from inversion of control in render props?
4. A consumer's reducer always returns `action.changes` without modification. Is this any different from not providing a custom reducer at all? Explain.
5. You are building a `useForm` hook. What action types would you define, and how would the state reducer pattern let consumers customize form behavior?

Exercises

Exercise 1: useToggle with Confirmation

Build a `useToggle` hook using the state reducer pattern. Then create a consumer that:

- Allows toggling ON freely
- Requires `window.confirm()` before toggling OFF
- Tracks the total number of toggles in the state

Exercise 2: useCounter with Boundaries

Build a `useCounter` hook with `INCREMENT`, `DECREMENT`, and `RESET` actions. Export the default reducer and action types. Then:

1. Create a consumer that prevents the counter from going below 0
2. Create a consumer that limits the counter to a maximum of 10
3. Create a consumer that increments by 5 instead of 1

Exercise 3: useAccordion with Custom Rules

Build a `useAccordion` hook that manages which panels are open/closed, with actions like `TOGGLE_PANEL`, `OPEN_PANEL`, `CLOSE_PANEL`, `OPEN_ALL`, `CLOSE_ALL`. Then use the state reducer pattern to create:

1. An accordion where at most one panel can be open at a time
2. An accordion where the first panel can never be closed
3. An accordion with a "lock" feature that prevents any changes when locked

---

What Is Next?

The state reducer pattern lets consumers customize how state changes, but it still allows any state value. What if certain states should be impossible? What if going from "loading" to "success" requires passing through a specific transition, and jumping directly from "idle" to "error" is never valid? In Chapter 23, you will learn the State Machine pattern -- a way to define exactly which states exist and which transitions are allowed, eliminating entire categories of impossible bugs.

Chapter 23: The State Machine Pattern

What You Will Learn

- What finite state machines are and why they prevent impossible states
- How "boolean soup" creates bugs and how state machines eliminate them
- How to implement state machines with `useReducer`
- How to model multi-step flows (forms, wizards, checkout processes)
- An introduction to XState for complex state machines
- How to visualize and reason about your application's states

Why This Chapter Matters

Consider a data-fetching component. You might model its state with three booleans:

```jsx
const [isLoading, setIsLoading] = useState(false);
const [isError, setIsError] = useState(false);
const [isSuccess, setIsSuccess] = useState(false);
```

Three booleans means eight possible combinations. But how many are valid? Only four:

- Idle: `loading=false, error=false, success=false`
- Loading: `loading=true, error=false, success=false`
- Error: `loading=false, error=true, success=false`
- Success: `loading=false, error=false, success=true`

The other four combinations (like `loading=true, error=true, success=true`) should be impossible, but your code allows them. If you forget to set `isLoading` to `false` before setting `isError` to `true`, you end up in an invalid state: loading and errored at the same time. The UI

shows a spinner and an error message simultaneously.

State machines solve this by making impossible states unrepresentable. Instead of multiple booleans, you have one state variable that can only hold valid values, and explicit transitions that define which state can follow which.

This is not just theory. Every multi-step form, every authentication flow, every media player, and every checkout process is a state machine -- whether you model it as one or not. When you do not model it explicitly, you get bugs. When you do, you get clarity.

The Problem: Boolean Soup

"Boolean soup" is what happens when you model complex state with multiple independent boolean flags:

```jsx
function DataFetcher({ url }) {
  const [isIdle, setIsIdle] = useState(true);
  const [isLoading, setIsLoading] = useState(false);
  const [isError, setIsError] = useState(false);
  const [isSuccess, setIsSuccess] = useState(false);
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  const fetchData = async () => {
    setIsIdle(false);
    setIsLoading(true);
    setIsError(false);   // Did I forget any?
    setIsSuccess(false); // What if I miss one?

    try {
      const response = await fetch(url);
      const result = await response.json();
      setData(result);
      setIsLoading(false);  // Must remember to set this
      setIsSuccess(true);
    } catch (err) {
      setError(err);
      setIsLoading(false);  // Must remember here too
      setIsError(true);
    }
  };

  // Which combination of booleans is active RIGHT NOW?
  // Can you be sure isLoading and isError are never both true?
}
```

Each `setState` call is a separate update. If you forget one, you have an invalid state. If React batches updates differently than you expect, you might see a flash of an invalid combination.

```
Boolean soup: 2^4 = 16 possible combinations
Valid states: only 4

+----------+----------+----------+----------+--------+
| isIdle   | isLoading| isError  | isSuccess| Valid? |
+----------+----------+----------+----------+--------+
| true     | false    | false    | false    | YES    |
| false    | true     | false    | false    | YES    |
| false    | false    | true     | false    | YES    |
| false    | false    | false    | true     | YES    |
| true     | true     | false    | false    | NO     |
| false    | true     | true     | false    | NO     |
| false    | true     | false    | true     | NO     |
| ... 9 more invalid combinations ...        | NO     |
+----------+----------+----------+----------+--------+
```

The Solution: Finite State Machines

A finite state machine has:

1. A finite set of states -- the valid states your system can be in
2. An initial state -- where the system starts
3. Events (actions) -- things that happen that might cause a transition
4. Transitions -- rules that say "when in state X and event Y happens, move to state Z"

```jsx
// State machine approach: ONE variable, only valid values
function DataFetcher({ url }) {
  const [state, setState] = useState('idle');
  const [data, setData] = useState(null);
  const [error, setError] = useState(null);

  const fetchData = async () => {
    setState('loading');  // ONE update, always valid

    try {
      const result = await fetch(url).then(r => r.json());
      setData(result);
      setState('success');  // ONE update, always valid
    } catch (err) {
      setError(err);
      setState('error');  // ONE update, always valid
```

```
    }
  };

  // The state is ALWAYS one of: 'idle', 'loading', 'success', 'error'
  // No invalid combinations possible
}
```

```
State machine: only 4 valid states, explicit transitions

  +--------+     FETCH      +-----------+
  |  idle  | -------------> |  loading  |
  +--------+                +-----------+
                              /         \
                   SUCCESS  /           \  FAILURE
                           v             v
                   +-----------+   +-----------+
                   |  success  |   |   error   |
                   +-----------+   +-----------+
                           \             /
                    FETCH   \           /  FETCH
                             v         v
                           +-----------+
                           |  loading  |
                           +-----------+
```

Implementing State Machines with useReducer

`useReducer` is a natural fit for state machines because the reducer function maps directly to the transition table.

Basic Fetch State Machine

```jsx
function fetchReducer(state, event) {
  switch (state.status) {
    case 'idle':
      switch (event.type) {
        case 'FETCH':
          return { status: 'loading', data: null, error: null };
        default:
          return state; // Ignore events that do not apply in this state
      }

    case 'loading':
```

```
      switch (event.type) {
        case 'SUCCESS':
          return { status: 'success', data: event.data, error: null };
        case 'FAILURE':
          return { status: 'error', data: null, error: event.error };
        default:
          return state;
      }

    case 'success':
      switch (event.type) {
        case 'FETCH':
          return { status: 'loading', data: null, error: null };
        default:
          return state;
      }

    case 'error':
      switch (event.type) {
        case 'FETCH':
          return { status: 'loading', data: null, error: null };
        default:
          return state;
      }

    default:
      throw new Error(`Unknown state: ${state.status}`);
  }
}

function useFetch(url) {
  const [state, send] = useReducer(fetchReducer, {
    status: 'idle',
    data: null,
    error: null,
  });

  const fetchData = async () => {
    send({ type: 'FETCH' });
    try {
      const response = await fetch(url);
      const data = await response.json();
      send({ type: 'SUCCESS', data });
    } catch (error) {
      send({ type: 'FAILURE', error });
    }
  };

  return { ...state, fetchData };
}

// Usage
function UserProfile({ userId }) {
  const { status, data, error, fetchData } = useFetch(`/api/users/${userId}`);
```

```
  useEffect(() => {
    fetchData();
  }, [userId]);

  switch (status) {
    case 'idle':
      return <p>Ready to fetch</p>;
    case 'loading':
      return <Spinner />;
    case 'success':
      return <h1>{data.name}</h1>;
    case 'error':
      return (
        <div>
          <p>Error: {error.message}</p>
          <button onClick={fetchData}>Retry</button>
        </div>
      );
  }
}
```

Notice how the switch on `status` in the render covers exactly four cases. There are no impossible states to worry about.

Key Insight: Events Are Ignored in Wrong States

This is one of the most powerful features. If you dispatch `SUCCESS` while in the `idle` state, nothing happens -- the reducer returns the current state unchanged. No crash, no invalid state, just a graceful no-op.

```jsx
// In boolean soup, this causes a bug:
setIsSuccess(true);  // Oops, forgot to check if we are loading

// In a state machine, this is harmless:
send({ type: 'SUCCESS', data });
// If we are in 'idle', the reducer ignores this event
```

Modeling State Machines as Data

Instead of a large switch statement, you can represent the machine as a configuration object:

```jsx
const fetchMachine = {
  initial: 'idle',
  states: {
    idle: {
      on: {
        FETCH: 'loading',
      },
    },
    loading: {
      on: {
        SUCCESS: 'success',
        FAILURE: 'error',
      },
    },
    success: {
      on: {
        FETCH: 'loading',
      },
    },
    error: {
      on: {
        FETCH: 'loading',
      },
    },
  },
};

// Generic reducer that works with any machine config
function createMachineReducer(machine) {
  return function reducer(state, event) {
    const currentState = machine.states[state.status];
    const nextStatus = currentState?.on?.[event.type];

    if (!nextStatus) {
      return state; // Event not handled in this state
    }

    return {
      ...state,
      status: nextStatus,
      ...(event.data !== undefined && { data: event.data }),
      ...(event.error !== undefined && { error: event.error }),
    };
  };
}

// Usage
function useFetch(url) {
  const [state, send] = useReducer(
    createMachineReducer(fetchMachine),
    { status: fetchMachine.initial, data: null, error: null }
  );

  // ...same as before
}
```

This data-driven approach is easy to visualize, test, and modify. Adding a new state or transition is a one-line change in the config.

Real-World Example: Multi-Step Form

Multi-step forms are classic state machines. Each step is a state, and navigation between steps is a transition.

```
+-------+   NEXT   +----------+   NEXT   +---------+   SUBMIT   +---------+
| step1 | -------> |  step2   | -------> |  step3  | ---------> | submitting|
+-------+          +----------+          +---------+            +---------+
                     |                       |                     |      |
                     | BACK                  | BACK         SUCCESS|      |FAILURE
                     v                       v                     v      v
                   +-------+             +----------+     +--------+ +-------+
                   | step1 |             |  step2   |     |complete| | error |
                   +-------+             +----------+     +--------+ +-------+
                                                                     |
                                                                     | RETRY
                                                                     v
                                                              +---------+
                                                              |submitting|
                                                              +---------+
```

```jsx
const formMachine = {
  initial: 'step1',
  states: {
    step1: {
      on: { NEXT: 'step2' },
    },
    step2: {
      on: { NEXT: 'step3', BACK: 'step1' },
    },
    step3: {
      on: { SUBMIT: 'submitting', BACK: 'step2' },
    },
    submitting: {
      on: { SUCCESS: 'complete', FAILURE: 'error' },
    },
    complete: {
      on: {}, // Terminal state -- no transitions out
    },
    error: {
      on: { RETRY: 'submitting', BACK: 'step3' },
```

```
    },
  },
};

function formReducer(state, event) {
  const currentStateConfig = formMachine.states[state.step];
  const nextStep = currentStateConfig?.on?.[event.type];

  if (!nextStep) return state;

  // Merge form data on NEXT transitions
  const formData =
    event.type === 'NEXT' || event.type === 'SUBMIT'
      ? { ...state.formData, ...event.data }
      : state.formData;

  return {
    ...state,
    step: nextStep,
    formData,
    error: event.error || null,
  };
}

function useMultiStepForm() {
  const [state, send] = useReducer(formReducer, {
    step: formMachine.initial,
    formData: {},
    error: null,
  });

  return {
    step: state.step,
    formData: state.formData,
    error: state.error,
    next: (data) => send({ type: 'NEXT', data }),
    back: () => send({ type: 'BACK' }),
    submit: (data) => send({ type: 'SUBMIT', data }),
    success: () => send({ type: 'SUCCESS' }),
    failure: (error) => send({ type: 'FAILURE', error }),
    retry: () => send({ type: 'RETRY' }),
  };
}
```

Usage:

```jsx
function CheckoutForm() {
  const { step, formData, error, next, back, submit, success, failure, retry } =
    useMultiStepForm();

  switch (step) {
    case 'step1':
```

```
      return (
        <ShippingForm
          defaultValues={formData}
          onNext={(data) => next(data)}
        />
      );

    case 'step2':
      return (
        <PaymentForm
          defaultValues={formData}
          onNext={(data) => next(data)}
          onBack={back}
        />
      );

    case 'step3':
      return (
        <ReviewForm
          formData={formData}
          onSubmit={() => {
            submit(formData);
            submitOrder(formData).then(success).catch(failure);
          }}
          onBack={back}
        />
      );

    case 'submitting':
      return <Spinner message="Placing your order..." />;

    case 'complete':
      return <SuccessMessage orderId={formData.orderId} />;

    case 'error':
      return (
        <div>
          <p>Something went wrong: {error?.message}</p>
          <button onClick={retry}>Try Again</button>
          <button onClick={back}>Go Back</button>
        </div>
      );
  }
}
```

Every step renders exactly one view. There is no ambiguity about what the user sees. Going back from step 3 always goes to step 2. Submitting is only possible from step 3. These rules are enforced by the machine, not by developer discipline.

Introduction to XState

For complex state machines with nested states, parallel states, guards, and side effects, the XState library provides a complete solution.

Why XState?

Our `useReducer` approach works well for simple machines, but it lacks:

- Guards -- conditions that must be true for a transition to happen
- Actions -- side effects that run on transitions
- Nested states -- sub-states within a state (e.g., "authenticated" has sub-states "idle" and "fetching")
- Parallel states -- multiple independent state machines running simultaneously
- Visualization -- XState has tools that generate diagrams from your machine definition

Basic XState Example

```jsx
import { createMachine, assign } from 'xstate';
import { useMachine } from '@xstate/react';

const fetchMachine = createMachine({
  id: 'fetch',
  initial: 'idle',
  context: {
    data: null,
    error: null,
    retries: 0,
  },
  states: {
    idle: {
      on: {
        FETCH: 'loading',
      },
    },
    loading: {
      on: {
        SUCCESS: {
          target: 'success',
          actions: assign({ data: (_, event) => event.data }),
        },
        FAILURE: [
          {
            target: 'loading',
            guard: (context) => context.retries < 3,
            actions: assign({
```

```
              retries: (context) => context.retries + 1,
            }),
          },
          {
            target: 'error',
            actions: assign({ error: (_, event) => event.error }),
          },
        ],
      },
    },
    success: {
      on: {
        FETCH: {
          target: 'loading',
          actions: assign({ data: null, error: null, retries: 0 }),
        },
      },
    },
    error: {
      on: {
        RETRY: {
          target: 'loading',
          actions: assign({ error: null }),
        },
      },
    },
  },
});

function DataFetcher({ url }) {
  const [state, send] = useMachine(fetchMachine);

  useEffect(() => {
    if (state.matches('loading')) {
      fetch(url)
        .then(r => r.json())
        .then(data => send({ type: 'SUCCESS', data }))
        .catch(error => send({ type: 'FAILURE', error }));
    }
  }, [state, url, send]);

  if (state.matches('idle')) {
    return <button onClick={() => send({ type: 'FETCH' })}>Load</button>;
  }
  if (state.matches('loading')) {
    return <Spinner />;
  }
  if (state.matches('success')) {
    return <pre>{JSON.stringify(state.context.data, null, 2)}</pre>;
  }
  if (state.matches('error')) {
    return (
      <div>
        <p>Error: {state.context.error?.message}</p>
        <button onClick={() => send({ type: 'RETRY' })}>Retry</button>
```

```
      </div>
    );
  }
}
```

Key XState Features

```
+-----------------------------------------------+
|  Feature          | useReducer | XState      |
+-----------------------------------------------+
|  Basic states     |    Yes     |   Yes       |
|  Transitions      |    Yes     |   Yes       |
|  Guards           |    Manual  |   Built-in  |
|  Side effects     |    Manual  |   Built-in  |
|  Nested states    |    Hard    |   Built-in  |
|  Parallel states  |    Hard    |   Built-in  |
|  Visualization    |    No      |   Yes       |
|  DevTools         |    No      |   Yes       |
|  Serializable     |    No      |   Yes       |
|  Bundle size      |    0 KB    |   ~15 KB    |
+-----------------------------------------------+
```

When to Use XState vs useReducer

- useReducer: simple machines with 3-6 states, no guards, no side effects. Good enough for most UI components.
- XState: complex workflows with 10+ states, conditional transitions, nested sub-states, or when you need visualization and DevTools. Good for checkout flows, authentication, multi-step wizards, and video players.

Avoiding Boolean Soup: Before vs After

Before (Boolean Soup)

```jsx
function MediaPlayer() {
  const [isPlaying, setIsPlaying] = useState(false);
  const [isPaused, setIsPaused] = useState(false);
  const [isBuffering, setIsBuffering] = useState(false);
```

```
  const [isSeeking, setIsSeeking] = useState(false);
  const [hasEnded, setHasEnded] = useState(false);
  const [hasError, setHasError] = useState(false);

  const play = () => {
    setIsPlaying(true);
    setIsPaused(false);
    setHasEnded(false);
    setHasError(false);
    // Did I forget anything?
  };

  // What does the UI look like when isPlaying=true AND isBuffering=true?
  // What about isPlaying=true AND hasEnded=true?
  // How many combinations do I need to handle?
  // 2^6 = 64 possible combinations!
}
```

After (State Machine)

```jsx
const playerMachine = {
  initial: 'idle',
  states: {
    idle:      { on: { PLAY: 'playing' } },
    playing:   { on: { PAUSE: 'paused', BUFFER: 'buffering', END: 'ended', ERROR: 'erro...
    paused:    { on: { PLAY: 'playing', SEEK: 'seeking' } },
    buffering: { on: { LOADED: 'playing', ERROR: 'error' } },
    seeking:   { on: { SEEKED: 'playing', ERROR: 'error' } },
    ended:     { on: { PLAY: 'playing' } },
    error:     { on: { RETRY: 'idle' } },
  },
};

function MediaPlayer() {
  const [state, send] = useReducer(
    createMachineReducer(playerMachine),
    { status: 'idle' }
  );

  // Exactly 7 possible states. Each one is clear.
  switch (state.status) {
    case 'idle':      return <PlayButton onClick={() => send({ type: 'PLAY' })} />;
    case 'playing':   return <PauseButton onClick={() => send({ type: 'PAUSE' })} />;
    case 'paused':    return <PlayButton onClick={() => send({ type: 'PLAY' })} />;
    case 'buffering': return <Spinner />;
    case 'seeking':   return <SeekIndicator />;
    case 'ended':     return <ReplayButton onClick={() => send({ type: 'PLAY' })} />;
    case 'error':     return <RetryButton onClick={() => send({ type: 'RETRY' })} />;
  }
}
```

When to Use / When NOT to Use

Use State Machines When

- Your component has 3 or more distinct modes or steps
- You catch yourself using multiple booleans to track related states
- You need to prevent invalid state combinations
- The flow has clear sequential steps (wizard, checkout, onboarding)
- You want to visualize the entire flow for documentation or debugging
- Edge cases keep causing bugs because state transitions are not well-defined

Do NOT Use State Machines When

- The state is truly independent booleans (isOpen and isFullscreen are genuinely independent)
- There are only two states (a simple boolean toggle is fine)
- The overhead of defining a machine exceeds the complexity of the problem
- The state changes do not have meaningful transitions -- it is just data that updates

---

Common Mistakes

Mistake 1: Using Booleans When You Have Modes

```jsx
// BAD -- these are mutually exclusive modes, not independent flags
const [isViewing, setIsViewing] = useState(true);
const [isEditing, setIsEditing] = useState(false);
const [isSaving, setIsSaving] = useState(false);

// GOOD -- one variable for the current mode
const [mode, setMode] = useState('viewing');
// mode is 'viewing' | 'editing' | 'saving'
```

Mistake 2: Not Handling All States in the UI

```jsx
// BAD -- what if status is 'error'? Nothing renders.
function Display({ status, data }) {
  if (status === 'loading') return <Spinner />;
  if (status === 'success') return <Data data={data} />;
  // 'idle' and 'error' silently render nothing!
}

// GOOD -- explicit handling of every state
function Display({ status, data, error }) {
  switch (status) {
    case 'idle':    return <p>Click to load</p>;
    case 'loading': return <Spinner />;
    case 'success': return <Data data={data} />;
    case 'error':   return <Error error={error} />;
    default:
      throw new Error(`Unhandled status: ${status}`);
  }
}
```

Mistake 3: Transitions That Skip States

```jsx
// BAD -- jumping directly to 'success' from 'idle' skips 'loading'
function reducer(state, event) {
  if (event.type === 'SUCCESS') {
    return { status: 'success', data: event.data };
    // This allows idle -> success, which should require loading first
  }
}

// GOOD -- check current state before transitioning
function reducer(state, event) {
  if (state.status === 'loading' && event.type === 'SUCCESS') {
    return { status: 'success', data: event.data };
  }
  // SUCCESS in 'idle' state does nothing
}
```

Mistake 4: Trying to Set State Directly

```jsx
// BAD -- bypasses the state machine
const [state, send] = useReducer(reducer, initialState);
// Later:
state.status = 'error';  // Direct mutation! Bypasses transitions!

// GOOD -- always go through events
send({ type: 'FAILURE', error: new Error('timeout') });
```

Best Practices

1. Start by drawing the state diagram on paper or a whiteboard before writing code. List all states, then draw arrows for transitions.
2. Name states as nouns or adjectives, not verbs: `'idle'`, `'loading'`, `'error'` -- not `'load'`, `'fail'`.
3. Name events as verbs or past-tense: `'FETCH'`, `'SUCCESS'`, `'FAILURE'` -- representing things that happen.
4. Handle the default case in your switch: throw an error for unknown states so bugs surface immediately.
5. Use TypeScript to enforce valid states and events at compile time.
6. Ignore invalid events rather than throwing -- return the current state unchanged. This makes the system robust.
7. Start with useReducer and only move to XState when you need guards, nested states, or visualization.
8. Test the state machine independently from the UI. Feed it events and assert the resulting states. This is much easier than testing with rendered components.

Quick Summary

State machines model your system as a finite set of valid states with explicit transitions between them. Instead of multiple booleans that can combine into invalid states, you have one status variable that is always valid. `useReducer` is the natural tool for implementing state machines in React. For complex machines with guards, nested states, and side effects, XState provides a full-featured solution. The pattern eliminates entire categories of bugs by making impossible states unrepresentable.

Key Points

- Boolean soup (multiple flags) creates exponentially more state combinations, most of which are invalid

- State machines restrict state to a finite set of valid values
- Transitions define which events are allowed in which states
- Invalid events in a state machine are silently ignored, preventing bugs
- `useReducer` is the simplest way to implement state machines in React
- Machine configurations as data objects are easier to understand and modify than large switch statements
- XState extends the concept with guards, actions, nested states, parallel states, and DevTools
- Use `useReducer` for simple machines (3-6 states), XState for complex ones (10+ states)
- Multi-step forms, authentication flows, media players, and checkout processes are all state machines
- Always draw the state diagram first -- it serves as both design and documentation

---

Practice Questions

1. You have a component with `isLoading`, `isRetrying`, and `hasData` booleans. How many possible combinations exist? How many are valid? Rewrite the state as a state machine.
2. Explain why ignoring invalid events (returning current state) is better than throwing an error in a state machine reducer.
3. A multi-step form allows the user to navigate from step 3 directly to step 1. In a boolean approach (`currentStep` variable with free assignment), what bugs could this cause? How does a state machine prevent them?
4. Compare the `useReducer` state machine approach with XState. At what level of complexity would you switch from one to the other?
5. Draw a state diagram for a traffic light with states: `green`, `yellow`, `red`. What events trigger each transition? Is there a state that can transition to any other state, or are all transitions sequential?

Exercises

Exercise 1: Authentication State Machine

Model a user authentication flow as a state machine with these states:

- `loggedOut` -- user is not authenticated
- `loggingIn` -- credentials submitted, waiting for server
- `loggedIn` -- user is authenticated
- `loggingOut` -- logout request submitted
- `error` -- authentication failed

Define the transitions, implement it with `useReducer`, and build a simple login/logout UI that renders the correct view for each state.

Exercise 2: Multi-Step Signup Wizard

Build a 4-step signup wizard as a state machine:

1. Account -- email and password
2. Profile -- name and avatar
3. Preferences -- notification settings
4. Confirmation -- review and submit

Requirements:

- Users can go forward and backward between steps 1-3
- Step 4 (confirmation) transitions to `submitting`, then `success` or `error`
- Collect and preserve form data across all steps
- Prevent skipping steps (e.g., cannot go from step 1 to step 3 directly)

Exercise 3: Convert Boolean Soup to a State Machine

Take this boolean-soup component and refactor it into a state machine:

```jsx
function Upload() {
  const [file, setFile] = useState(null);
  const [isUploading, setIsUploading] = useState(false);
  const [isComplete, setIsComplete] = useState(false);
  const [isFailed, setIsFailed] = useState(false);
  const [progress, setProgress] = useState(0);

  // Refactor this into states: idle, selected, uploading, complete, failed
  // Define valid transitions between each state
}
```

Draw the state diagram, then implement it with `useReducer`. Verify that impossible states (like `isUploading=true` and `isComplete=true`) can no longer occur.

What Is Next?

State machines give you control over your component's state transitions. But components do not exist in isolation -- in real applications, many components need to share and coordinate state. In Chapter 24, you will learn the Flux and Redux pattern, which applies the reducer concept at the application level, managing global state through a single store with predictable, traceable updates.

Chapter 24: Flux/Redux Pattern

What You Will Learn

- What unidirectional data flow means and why it matters
- The Flux architecture: Actions, Dispatcher, Store, and View
- How Redux simplifies Flux with a single store and pure reducers
- Writing Redux code with Redux Toolkit (createSlice, configureStore)
- When Redux is overkill and what lighter alternatives exist
- How Zustand and Jotai compare to Redux
- Building a real-world e-commerce cart and multi-step wizard

Why This Chapter Matters

Imagine a busy restaurant kitchen. Orders come in from multiple waiters, cooks work on different dishes, and food goes out to various tables. Without a clear system, chaos happens: wrong dishes, missed orders, cold food.

Now imagine the same kitchen with a single order ticket system. Every order goes through one window, gets processed in order, and comes out predictably. That is what Flux and Redux do for your application state.

As frontend applications grow, state management becomes the hardest problem. You have user data, UI state, server responses, form inputs, and navigation state all tangled together. When a bug appears, you cannot tell which piece of code changed which piece of state. Redux gives you a strict, predictable system where every state change is traceable.

The Problem

Consider a social media dashboard. You have:

- A notification counter in the header
- A message list in the sidebar
- A post feed in the main area
- A user profile panel

When a new message arrives, the counter updates, the sidebar refreshes, and maybe the feed changes too. With two-way data binding, any component can change any state, and state changes can trigger more state changes:

```
TWO-WAY DATA FLOW (CHAOS)

  ┌─────────┐     ┌─────────┐     ┌─────────┐
  │ Model A │◄───►│ Model B │◄───►│ Model C │
  └─────────┘     └─────────┘     └─────────┘
       │▲              │▲              │▲
       ▼│              ▼│              ▼│
  ┌─────────┐     ┌─────────┐     ┌─────────┐
  │ View A  │◄───►│ View B  │◄───►│ View C  │
  └─────────┘     └─────────┘     └─────────┘

  Any view can change any model.
  Any model can change any other model.
  Result: unpredictable cascading updates.
```

```javascript
// Before: scattered state management
// Header.js
function Header() {
  const [notifications, setNotifications] = useState(0);

  // How does this sync with Sidebar?
  // Who updates this? Multiple places!
  useEffect(() => {
    api.onNewMessage(() => {
      setNotifications(n => n + 1);
    });
  }, []);

  return <div>Notifications: {notifications}</div>;
}

// Sidebar.js
function Sidebar() {
  const [messages, setMessages] = useState([]);
  const [unreadCount, setUnreadCount] = useState(0);

  // This count might disagree with Header's count!
  useEffect(() => {
```

```
    api.onNewMessage((msg) => {
      setMessages(prev => [...prev, msg]);
      setUnreadCount(c => c + 1);
    });
  }, []);

  return <div>Unread: {unreadCount}</div>;
}

// Both components listen independently.
// Counts can get out of sync.
// No single source of truth.
```

Output (the bug):

```
Header shows: Notifications: 5
Sidebar shows: Unread: 4
// They disagree! Which is correct?
```

The Solution: Unidirectional Data Flow

Flux Architecture

Facebook created Flux to solve this exact problem. The rule is simple: data flows in one direction only.

```
FLUX: UNIDIRECTIONAL DATA FLOW

  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
  │  Action  │──►│ Dispatcher │──►│  Store   │──►│   View   │
  └──────────┘    └──────────┘    └──────────┘    └──────────┘
       ▲                                                  │
       │                                                  │
       └──────────────────────────────────────────────────┘
                   User interaction creates
                      a new Action
```

Think of it like an assembly line:

1. Action: A description of what happened. "User clicked Add to Cart" or "Server returned new messages." It is just a plain object with a `type` and some data.

2. Dispatcher: The traffic cop. It receives every action and sends it to every store. There is only one dispatcher per app.
3. Store: Holds the state and the logic to update it. When it gets an action from the dispatcher, it decides how to change its state.
4. View: React components. They read from stores and render UI. When a user clicks something, the view creates a new action, and the cycle repeats.

```javascript
// Flux Action
const addToCartAction = {
  type: 'ADD_TO_CART',
  payload: {
    productId: 42,
    name: 'Wireless Headphones',
    price: 79.99
  }
};

// Flux Dispatcher (simplified)
class Dispatcher {
  constructor() {
    this.callbacks = [];        // Line 1: Array of registered store callbacks
  }

  register(callback) {
    this.callbacks.push(callback); // Line 2: Stores register themselves
  }

  dispatch(action) {
    this.callbacks.forEach(cb => cb(action)); // Line 3: Send action to ALL stores
  }
}

const AppDispatcher = new Dispatcher();

// Flux Store (simplified)
class CartStore {
  constructor() {
    this.items = [];            // Line 4: Internal state
    this.listeners = [];        // Line 5: View callbacks

    AppDispatcher.register((action) => {  // Line 6: Register with dispatcher
      switch (action.type) {
        case 'ADD_TO_CART':
          this.items = [...this.items, action.payload]; // Line 7: Update state
          this.emitChange();    // Line 8: Notify views
          break;
        case 'REMOVE_FROM_CART':
          this.items = this.items.filter(
            item => item.productId !== action.payload.productId
          );
          this.emitChange();
```

```
          break;
      }
    });
  }

  getItems() {
    return this.items;          // Line 9: Views read state through methods
  }

  emitChange() {
    this.listeners.forEach(fn => fn()); // Line 10: Tell views to re-render
  }

  addChangeListener(fn) {
    this.listeners.push(fn);   // Line 11: Views subscribe to changes
  }
}
```

Line-by-line explanation:

- Line 1: The dispatcher keeps a list of all store callbacks
- Line 2: Each store registers itself with the dispatcher
- Line 3: When an action is dispatched, EVERY store gets notified
- Lines 4-5: Store holds both state and a list of subscribed views
- Line 6: Store registers its handler with the global dispatcher
- Line 7: State updates are immutable (new array, not push)
- Line 8: After updating, store tells all views to re-check
- Line 9: Views never access internal state directly; they call getter methods
- Lines 10-11: Simple pub/sub for view updates

Redux: Flux Simplified

Redux took the Flux ideas and made three important simplifications:

1. Single Store: Instead of many stores, one store holds ALL state
2. Pure Reducers: Instead of store methods, pure functions handle state changes
3. No Dispatcher: The store itself handles dispatching

```
REDUX ARCHITECTURE

  ┌──────────────────────────────────────────────────┐
  │                    STORE                         │
```

```
│                                                          │
│    ┌──────────┐        dispatch      ┌──────────────┐    │
│    │  State   │◄─────────────────────│   Reducer    │    │
│    │ (Tree)   │                      │  (Pure Fn)   │    │
│    └──────────┘                      └──────▲───────┘    │
│         │                                    │            │
└─────────┼────────────────────────────────────┼────────────┘
          │                                    │
      subscribe                            dispatch
          │                                    │
          ▼                                    │
┌─────────────────┐                  ┌──────────────────┐
│   View (React)  │──────────────►   │     Action       │
│   Components    │   user           │  { type, data }  │
└─────────────────┘   event          └──────────────────┘
```

Three Principles of Redux

```
┌──────────────────────────────────────────────────────────┐
│            THE THREE PRINCIPLES OF REDUX                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  1. SINGLE SOURCE OF TRUTH                               │
│     The entire app state lives in one object             │
│     in one store.                                        │
│                                                          │
│  2. STATE IS READ-ONLY                                   │
│     The only way to change state is to dispatch          │
│     an action (a plain object describing what            │
│     happened).                                           │
│                                                          │
│  3. CHANGES ARE MADE WITH PURE FUNCTIONS                 │
│     Reducers are pure functions that take                │
│     (previousState, action) and return newState.         │
│     No side effects. No mutations.                       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Redux from Scratch (Plain JavaScript)

Before using any library, let us build a tiny Redux to understand the concept:

```javascript
// A minimal Redux implementation (about 25 lines)

function createStore(reducer, initialState) {
  let state = initialState;           // Line 1: Current state
```

```
  let listeners = [];                 // Line 2: Subscriber functions

  function getState() {
    return state;                     // Line 3: Read current state
  }

  function dispatch(action) {
    state = reducer(state, action);   // Line 4: Run reducer to get new state
    listeners.forEach(fn => fn());    // Line 5: Notify all subscribers
  }

  function subscribe(listener) {
    listeners.push(listener);         // Line 6: Add subscriber
    return () => {                    // Line 7: Return unsubscribe function
      listeners = listeners.filter(l => l !== listener);
    };
  }

  dispatch({ type: '@@INIT' });      // Line 8: Initialize with default state

  return { getState, dispatch, subscribe };
}
```

Line-by-line explanation:

- Line 1: State is just a variable inside a closure. Nobody can access it directly.
- Line 2: Array of functions to call when state changes.
- Line 3: `getState` is the ONLY way to read state.
- Line 4: The reducer receives current state and action, returns NEW state. The old state is never modified.
- Line 5: After state changes, every subscriber gets notified.
- Line 6: Components subscribe to know when to re-render.
- Line 7: Returns a cleanup function (like React's useEffect cleanup).
- Line 8: Dispatching a dummy action makes the reducer return its default state.

Now let us use it:

```javascript
// Step 1: Define a reducer (pure function)
function cartReducer(state = { items: [], total: 0 }, action) {
  switch (action.type) {
    case 'ADD_ITEM':
      const newItems = [...state.items, action.payload];
      const newTotal = newItems.reduce((sum, item) => sum + item.price, 0);
      return { items: newItems, total: newTotal };

    case 'REMOVE_ITEM':
      const filtered = state.items.filter(
```

```
        item => item.id !== action.payload.id
      );
      const filteredTotal = filtered.reduce(
        (sum, item) => sum + item.price, 0
      );
      return { items: filtered, total: filteredTotal };

    case 'CLEAR_CART':
      return { items: [], total: 0 };

    default:
      return state;  // Always return current state for unknown actions
  }
}

// Step 2: Create the store
const store = createStore(cartReducer);

// Step 3: Subscribe to changes
store.subscribe(() => {
  console.log('State changed:', store.getState());
});

// Step 4: Dispatch actions
store.dispatch({
  type: 'ADD_ITEM',
  payload: { id: 1, name: 'Laptop', price: 999 }
});
// Output: State changed: { items: [{ id: 1, name: 'Laptop', price: 999 }], total: 999 }

store.dispatch({
  type: 'ADD_ITEM',
  payload: { id: 2, name: 'Mouse', price: 29 }
});
// Output: State changed: { items: [...], total: 1028 }

store.dispatch({ type: 'CLEAR_CART' });
// Output: State changed: { items: [], total: 0 }
```

Redux Toolkit: The Modern Way

Writing raw Redux involves a lot of boilerplate: action types as constants, action creator functions, switch statements in reducers, immutable update logic. Redux Toolkit (RTK) eliminates all of that.

createSlice: Actions + Reducer in One

```javascript
// Before Redux Toolkit: verbose and error-prone

// action-types.js
const ADD_TO_CART = 'cart/addToCart';
const REMOVE_FROM_CART = 'cart/removeFromCart';
const UPDATE_QUANTITY = 'cart/updateQuantity';
const CLEAR_CART = 'cart/clearCart';

// action-creators.js
function addToCart(product) {
  return { type: ADD_TO_CART, payload: product };
}
function removeFromCart(productId) {
  return { type: REMOVE_FROM_CART, payload: productId };
}
function updateQuantity(productId, quantity) {
  return { type: UPDATE_QUANTITY, payload: { productId, quantity } };
}
function clearCart() {
  return { type: CLEAR_CART };
}

// reducer.js
function cartReducer(state = initialState, action) {
  switch (action.type) {
    case ADD_TO_CART:
      const existing = state.items.find(
        item => item.id === action.payload.id
      );
      if (existing) {
        return {
          ...state,
          items: state.items.map(item =>
            item.id === action.payload.id
              ? { ...item, quantity: item.quantity + 1 }
              : item
          )
        };
      }
      return {
        ...state,
        items: [...state.items, { ...action.payload, quantity: 1 }]
      };
    // ... more cases, each with careful spread operators
    default:
      return state;
  }
}

// That is THREE files and about 60 lines for basic cart logic.
```

```javascript
// After Redux Toolkit: clean and concise

import { createSlice } from '@reduxjs/toolkit';

const cartSlice = createSlice({
  name: 'cart',                    // Line 1: Prefix for action types
  initialState: {                  // Line 2: Starting state
    items: [],
    total: 0,
    itemCount: 0,
  },
  reducers: {                      // Line 3: Define reducers (and auto-create actions)
    addToCart(state, action) {      // Line 4: Looks like mutation, but is safe!
      const product = action.payload;
      const existingItem = state.items.find(item => item.id === product.id);

      if (existingItem) {
        existingItem.quantity += 1; // Line 5: Direct "mutation" — Immer handles it
      } else {
        state.items.push({         // Line 6: push() is fine here — Immer!
          ...product,
          quantity: 1
        });
      }

      // Line 7: Recalculate derived values
      state.total = state.items.reduce(
        (sum, item) => sum + item.price * item.quantity, 0
      );
      state.itemCount = state.items.reduce(
        (sum, item) => sum + item.quantity, 0
      );
    },

    removeFromCart(state, action) {
      const productId = action.payload;
      state.items = state.items.filter(item => item.id !== productId);
      state.total = state.items.reduce(
        (sum, item) => sum + item.price * item.quantity, 0
      );
      state.itemCount = state.items.reduce(
        (sum, item) => sum + item.quantity, 0
      );
    },

    updateQuantity(state, action) {
      const { productId, quantity } = action.payload;
      const item = state.items.find(item => item.id === productId);
      if (item) {
        item.quantity = quantity;   // Line 8: Direct assignment, safe with Immer
      }
      state.total = state.items.reduce(
        (sum, item) => sum + item.price * item.quantity, 0
      );
```

```
      state.itemCount = state.items.reduce(
        (sum, item) => sum + item.quantity, 0
      );
    },

    clearCart(state) {
      state.items = [];
      state.total = 0;
      state.itemCount = 0;
    },
  },
});

// Line 9: Auto-generated action creators
export const { addToCart, removeFromCart, updateQuantity, clearCart } =
  cartSlice.actions;

// Line 10: The reducer to give to the store
export default cartSlice.reducer;
```

Line-by-line explanation:

- Line 1: `name: 'cart'` means action types will be `'cart/addToCart'`, `'cart/removeFromCart'`, etc. You never type these strings yourself.
- Line 2: Initial state is defined right where the reducer lives.
- Line 3: Each key in `reducers` becomes both a reducer case AND an action creator.
- Lines 4-6: This looks like you are mutating state (push, direct assignment). But Redux Toolkit uses a library called Immer under the hood. Immer tracks your "mutations" and produces a new immutable state behind the scenes.
- Line 7: You can compute derived values right in the reducer.
- Line 8: `item.quantity = quantity` looks like mutation but Immer makes it safe.
- Line 9: `createSlice` auto-generates action creator functions.
- Line 10: Export the reducer to combine it into the store.

configureStore: Setting Up the Store

```javascript
import { configureStore } from '@reduxjs/toolkit';
import cartReducer from './cartSlice';
import userReducer from './userSlice';
import productsReducer from './productsSlice';

const store = configureStore({
  reducer: {                        // Line 1: Combine multiple slices
    cart: cartReducer,              // state.cart
    user: userReducer,              // state.user
```

```
    products: productsReducer,     // state.products
  },
  // Line 2: Redux Toolkit automatically adds:
  //   - Redux DevTools integration
  //   - redux-thunk middleware (for async actions)
  //   - Serializable state check (catches common mistakes)
  //   - Immutability check (catches accidental mutations)
});

export default store;
```

What configureStore does automatically:

```
┌──────────────────────────────────────────────────────────┐
│           configureStore AUTO-SETUP                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Without RTK, you need:          RTK does it for         │
│                                  you:                    │
│  ├─ combineReducers()      ──►  ✓ Automatic              │
│  ├─ applyMiddleware()      ──►  ✓ Automatic              │
│  ├─ Redux DevTools setup   ──►  ✓ Automatic              │
│  ├─ redux-thunk            ──►  ✓ Included               │
│  ├─ Immutability checks    ──►  ✓ Dev mode               │
│  └─ Serializability checks ──►  ✓ Dev mode               │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Using Redux in React Components

```javascript
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { addToCart, removeFromCart } from './cartSlice';

// Product List Component
function ProductList() {
  const products = useSelector(state => state.products.items); // Line 1
  const dispatch = useDispatch();                              // Line 2

  const handleAddToCart = (product) => {
    dispatch(addToCart(product));                               // Line 3
  };

  return (
    <div>
      <h2>Products</h2>
      {products.map(product => (
        <div key={product.id}>
```

```
          <span>{product.name} - ${product.price}</span>
          <button onClick={() => handleAddToCart(product)}>
            Add to Cart
          </button>
        </div>
      ))}
    </div>
  );
}

// Cart Component (completely separate, shares same state)
function Cart() {
  const { items, total, itemCount } = useSelector(state => state.cart); // Line 4
  const dispatch = useDispatch();

  return (
    <div>
      <h2>Cart ({itemCount} items)</h2>
      {items.map(item => (
        <div key={item.id}>
          <span>{item.name} x{item.quantity} = ${item.price * item.quantity}</span>
          <button onClick={() => dispatch(removeFromCart(item.id))}>
            Remove
          </button>
        </div>
      ))}
      <p>Total: ${total.toFixed(2)}</p>
    </div>
  );
}

// App setup
import { Provider } from 'react-redux';
import store from './store';

function App() {
  return (
    <Provider store={store}>       {/* Line 5 */}
      <ProductList />
      <Cart />
    </Provider>
  );
}
```

Line-by-line explanation:

- Line 1: `useSelector` reads a piece of state from the store. The component re-renders ONLY when this specific piece changes.
- Line 2: `useDispatch` returns the store's dispatch function.
- Line 3: Dispatch an action. `addToCart(product)` creates `{ type: 'cart/addToCart', payload: product }`.
- Line 4: Destructure multiple values from the cart state.

- Line 5: `Provider` makes the store available to all child components.

Async Actions with createAsyncThunk

Real apps need to fetch data from servers. Redux Toolkit provides `createAsyncThunk` for this:

```javascript
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Line 1: Define an async action
export const fetchProducts = createAsyncThunk(
  'products/fetchAll',            // Line 2: Action type prefix
  async (category, { rejectWithValue }) => {
    try {
      const response = await fetch(`/api/products?category=${category}`);
      if (!response.ok) {
        throw new Error('Failed to fetch products');
      }
      return await response.json(); // Line 3: This becomes action.payload
    } catch (error) {
      return rejectWithValue(error.message); // Line 4: This becomes action.payload on ...
    }
  }
);

const productsSlice = createSlice({
  name: 'products',
  initialState: {
    items: [],
    status: 'idle',    // 'idle' | 'loading' | 'succeeded' | 'failed'
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {    // Line 5: Handle async action states
    builder
      .addCase(fetchProducts.pending, (state) => {
        state.status = 'loading';  // Line 6: Request started
        state.error = null;
      })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.status = 'succeeded'; // Line 7: Request succeeded
        state.items = action.payload;
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.status = 'failed';    // Line 8: Request failed
        state.error = action.payload;
      });
  },
});
```

Line-by-line explanation:

- Line 1: `createAsyncThunk` wraps an async function and auto-dispatches pending/fulfilled/rejected actions.
- Line 2: This prefix generates three action types: `products/fetchAll/pending`, `products/fetchAll/fulfilled`, `products/fetchAll/rejected`.
- Line 3: Whatever you return from the async function becomes `action.payload` in the fulfilled case.
- Line 4: `rejectWithValue` lets you pass custom error data to the rejected case.
- Line 5: `extraReducers` handles actions defined outside the slice (like async thunks).
- Lines 6-8: Each async state gets its own handler. This pattern (status + error) is very common.

```javascript
// Using async thunk in a component
function ProductCatalog() {
  const dispatch = useDispatch();
  const { items, status, error } = useSelector(state => state.products);

  useEffect(() => {
    if (status === 'idle') {
      dispatch(fetchProducts('electronics'));
    }
  }, [status, dispatch]);

  if (status === 'loading') return <div>Loading products...</div>;
  if (status === 'failed') return <div>Error: {error}</div>;

  return (
    <div>
      {items.map(product => (
        <ProductCard key={product.id} product={product} />
      ))}
    </div>
  );
}
```

Output flow:

```
1. Component mounts
   → status is 'idle'
   → dispatch(fetchProducts('electronics'))
   → status becomes 'loading'
   → UI shows "Loading products..."

2. API responds successfully
   → status becomes 'succeeded'
   → items populated with product data
   → UI shows product cards

3. If API fails
```

```
    → status becomes 'failed'
    → error contains error message
    → UI shows "Error: Failed to fetch products"
```

Real-World Example 1: E-Commerce Cart

Let us build a complete e-commerce cart with Redux Toolkit:

```javascript
// features/cart/cartSlice.js
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';

// Async thunk to apply a coupon code
export const applyCoupon = createAsyncThunk(
  'cart/applyCoupon',
  async (couponCode, { getState, rejectWithValue }) => {
    const { cart } = getState();
    const response = await fetch('/api/coupons/validate', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        code: couponCode,
        cartTotal: cart.subtotal,
      }),
    });

    if (!response.ok) {
      const error = await response.json();
      return rejectWithValue(error.message);
    }

    return await response.json(); // { discount: 15, type: 'percentage' }
  }
);

const cartSlice = createSlice({
  name: 'cart',
  initialState: {
    items: [],
    subtotal: 0,
    discount: 0,
    tax: 0,
    total: 0,
    couponCode: null,
    couponStatus: 'idle', // 'idle' | 'loading' | 'valid' | 'invalid'
    couponError: null,
  },
```

```
  reducers: {
    addItem(state, action) {
      const product = action.payload;
      const existing = state.items.find(item => item.id === product.id);

      if (existing) {
        existing.quantity += 1;
      } else {
        state.items.push({ ...product, quantity: 1 });
      }
      recalculateTotals(state);
    },

    removeItem(state, action) {
      state.items = state.items.filter(item => item.id !== action.payload);
      recalculateTotals(state);
    },

    updateQuantity(state, action) {
      const { id, quantity } = action.payload;
      const item = state.items.find(item => item.id === id);
      if (item && quantity > 0) {
        item.quantity = quantity;
      } else if (item && quantity === 0) {
        state.items = state.items.filter(i => i.id !== id);
      }
      recalculateTotals(state);
    },

    clearCart(state) {
      state.items = [];
      state.subtotal = 0;
      state.discount = 0;
      state.tax = 0;
      state.total = 0;
      state.couponCode = null;
      state.couponStatus = 'idle';
    },
  },

  extraReducers: (builder) => {
    builder
      .addCase(applyCoupon.pending, (state) => {
        state.couponStatus = 'loading';
        state.couponError = null;
      })
      .addCase(applyCoupon.fulfilled, (state, action) => {
        state.couponStatus = 'valid';
        state.couponCode = action.meta.arg; // The coupon code string
        const { discount, type } = action.payload;

        if (type === 'percentage') {
          state.discount = state.subtotal * (discount / 100);
        } else {
          state.discount = discount;
```

```
        }
        recalculateTotals(state);
      })
      .addCase(applyCoupon.rejected, (state, action) => {
        state.couponStatus = 'invalid';
        state.couponError = action.payload;
        state.discount = 0;
        recalculateTotals(state);
      });
  },
});

// Helper function (not a reducer, just a utility)
function recalculateTotals(state) {
  state.subtotal = state.items.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );
  const TAX_RATE = 0.08;
  const afterDiscount = state.subtotal - state.discount;
  state.tax = afterDiscount * TAX_RATE;
  state.total = afterDiscount + state.tax;
}

export const { addItem, removeItem, updateQuantity, clearCart } =
  cartSlice.actions;
export default cartSlice.reducer;
```

```javascript
// components/CartSummary.jsx
import { useSelector, useDispatch } from 'react-redux';
import { removeItem, updateQuantity, clearCart, applyCoupon } from './cartSlice';

function CartSummary() {
  const cart = useSelector(state => state.cart);
  const dispatch = useDispatch();
  const [couponInput, setCouponInput] = useState('');

  return (
    <div className="cart-summary">
      <h2>Your Cart</h2>

      {cart.items.length === 0 ? (
        <p>Your cart is empty</p>
      ) : (
        <>
          {cart.items.map(item => (
            <div key={item.id} className="cart-item">
              <span>{item.name}</span>
              <input
                type="number"
                value={item.quantity}
                min="0"
```

```
                onChange={(e) =>
                  dispatch(updateQuantity({
                    id: item.id,
                    quantity: parseInt(e.target.value) || 0,
                  }))
                }
              />
              <span>${(item.price * item.quantity).toFixed(2)}</span>
              <button onClick={() => dispatch(removeItem(item.id))}>
                Remove
              </button>
            </div>
          ))}

          <div className="coupon-section">
            <input
              value={couponInput}
              onChange={(e) => setCouponInput(e.target.value)}
              placeholder="Enter coupon code"
            />
            <button
              onClick={() => dispatch(applyCoupon(couponInput))}
              disabled={cart.couponStatus === 'loading'}
            >
              {cart.couponStatus === 'loading' ? 'Checking...' : 'Apply'}
            </button>
            {cart.couponStatus === 'valid' && (
              <span className="success">Coupon applied!</span>
            )}
            {cart.couponStatus === 'invalid' && (
              <span className="error">{cart.couponError}</span>
            )}
          </div>

          <div className="totals">
            <div>Subtotal: ${cart.subtotal.toFixed(2)}</div>
            {cart.discount > 0 && (
              <div>Discount: -${cart.discount.toFixed(2)}</div>
            )}
            <div>Tax: ${cart.tax.toFixed(2)}</div>
            <div className="total">Total: ${cart.total.toFixed(2)}</div>
          </div>

          <button onClick={() => dispatch(clearCart())}>Clear Cart</button>
        </>
      )}
    </div>
  );
}
```

Real-World Example 2: Multi-Step Wizard

A checkout wizard with multiple steps, where each step's data needs to persist:

```javascript
// features/checkout/checkoutSlice.js
import { createSlice } from '@reduxjs/toolkit';

const checkoutSlice = createSlice({
  name: 'checkout',
  initialState: {
    currentStep: 0,
    steps: ['shipping', 'payment', 'review', 'confirmation'],
    data: {
      shipping: {
        firstName: '',
        lastName: '',
        address: '',
        city: '',
        zipCode: '',
        country: '',
      },
      payment: {
        method: 'credit-card', // 'credit-card' | 'paypal' | 'bank-transfer'
        cardLastFour: '',
        isValid: false,
      },
    },
    isComplete: false,
    errors: {},
  },

  reducers: {
    nextStep(state) {
      if (state.currentStep < state.steps.length - 1) {
        state.currentStep += 1;
      }
    },

    prevStep(state) {
      if (state.currentStep > 0) {
        state.currentStep -= 1;
      }
    },

    goToStep(state, action) {
      const step = action.payload;
      if (step >= 0 && step < state.steps.length) {
        state.currentStep = step;
      }
    },

    updateShipping(state, action) {
```

```
      state.data.shipping = { ...state.data.shipping, ...action.payload };
      // Clear errors for updated fields
      Object.keys(action.payload).forEach(key => {
        delete state.errors[`shipping.${key}`];
      });
    },

    updatePayment(state, action) {
      state.data.payment = { ...state.data.payment, ...action.payload };
    },

    setErrors(state, action) {
      state.errors = action.payload;
    },

    completeCheckout(state) {
      state.isComplete = true;
      state.currentStep = state.steps.length - 1;
    },

    resetCheckout(state) {
      return checkoutSlice.getInitialState();
    },
  },
});

export const {
  nextStep,
  prevStep,
  goToStep,
  updateShipping,
  updatePayment,
  setErrors,
  completeCheckout,
  resetCheckout,
} = checkoutSlice.actions;

export default checkoutSlice.reducer;
```

```javascript
// components/CheckoutWizard.jsx
function CheckoutWizard() {
  const { currentStep, steps, data, errors, isComplete } = useSelector(
    state => state.checkout
  );
  const dispatch = useDispatch();

  // Step indicator
  const StepIndicator = () => (
    <div className="steps">
      {steps.map((step, index) => (
        <div
          key={step}
```

```
          className={`step ${index === currentStep ? 'active' : ''}
                      ${index < currentStep ? 'completed' : ''}`}
          onClick={() => index < currentStep && dispatch(goToStep(index))}
        >
          {index + 1}. {step}
        </div>
      ))}
    </div>
  );

  // Render current step
  const renderStep = () => {
    switch (steps[currentStep]) {
      case 'shipping':
        return <ShippingForm data={data.shipping} errors={errors} />;
      case 'payment':
        return <PaymentForm data={data.payment} />;
      case 'review':
        return <ReviewOrder data={data} />;
      case 'confirmation':
        return <Confirmation />;
      default:
        return null;
    }
  };

  return (
    <div className="checkout-wizard">
      <StepIndicator />
      {renderStep()}
      <div className="navigation">
        {currentStep > 0 && !isComplete && (
          <button onClick={() => dispatch(prevStep())}>Back</button>
        )}
        {currentStep < steps.length - 2 && (
          <button onClick={() => dispatch(nextStep())}>Next</button>
        )}
      </div>
    </div>
  );
}
```

```
WIZARD STATE FLOW

  Step 0: Shipping     Step 1: Payment     Step 2: Review     Step 3: Done
  ┌──────────────┐     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
  │ First Name   │     │ Card / PP    │     │ Show all     │     │  Order       │
  │ Last Name    │──►  │ Validation   │──►  │ data from    │──►  │  Confirmed   │
  │ Address      │     │ Selection    │     │ steps 0-1    │     │  Thank you   │
  │ City / Zip   │     │              │     │ Edit links   │     │              │
  └──────────────┘     └──────────────┘     └──────────────┘     └──────────────┘
        │                    │                    │
        ▼                    ▼                    ▼
```

```
    updateShipping()   updatePayment()   completeCheckout()
         │                  │                  │
         └──────────────────┼──────────────────┘
                            │
                     All stored in ONE
                     Redux store slice
```

When Redux Is Overkill

Redux adds structure, but structure has a cost. Not every app needs it.

```
SHOULD YOU USE REDUX? (Decision Guide)

  Is your state shared across many components
  that are NOT in a parent-child relationship?
          │
    YES   │  NO ──► Use useState/useReducer + Context
          │
          ▼
  Do you need to track EVERY state change
  for debugging or undo/redo?
          │
    YES   │  NO ──► Consider Zustand or Context
          │
          ▼
  Is your team large (5+ frontend devs)?
          │
    YES   │  NO ──► Zustand or Jotai may be simpler
          │
          ▼
  Does your app have complex data flows
  (e.g., real-time updates, offline sync)?
          │
    YES   │  NO ──► Reconsider if you really need Redux
          │
          ▼
  USE REDUX (with Redux Toolkit)
```

Signs Redux is Overkill

- Your app has 5-10 components total
- State is mostly local (form inputs, toggles, modals)
- Only 2-3 pieces of state need to be shared
- You are the only developer

- You are building a prototype or MVP

Signs You Need Redux (or similar)

- Multiple unrelated components need the same data
- State changes need to be logged and debugged
- Complex state transitions (dependent updates)
- Undo/redo functionality
- Offline support and state persistence
- Large team needing clear state management patterns

Lighter Alternatives

Zustand: Bear Minimum State Management

Zustand is much simpler than Redux while keeping a similar single-store approach:

```javascript
// Zustand: entire cart store in about 30 lines
import { create } from 'zustand';

const useCartStore = create((set, get) => ({
  // State
  items: [],
  total: 0,

  // Actions (just functions that call set)
  addItem: (product) => {
    set((state) => {
      const existing = state.items.find(item => item.id === product.id);
      let newItems;

      if (existing) {
        newItems = state.items.map(item =>
          item.id === product.id
            ? { ...item, quantity: item.quantity + 1 }
            : item
        );
      } else {
        newItems = [...state.items, { ...product, quantity: 1 }];
      }

      return {
        items: newItems,
```

```
        total: newItems.reduce(
          (sum, item) => sum + item.price * item.quantity, 0
        ),
      };
    });
  },

  removeItem: (productId) => {
    set((state) => {
      const newItems = state.items.filter(item => item.id !== productId);
      return {
        items: newItems,
        total: newItems.reduce(
          (sum, item) => sum + item.price * item.quantity, 0
        ),
      };
    });
  },

  clearCart: () => set({ items: [], total: 0 }),

  // Computed value using get()
  getItemCount: () => {
    return get().items.reduce((sum, item) => sum + item.quantity, 0);
  },
}));

// Using in a component - no Provider needed!
function Cart() {
  const items = useCartStore(state => state.items);
  const total = useCartStore(state => state.total);
  const removeItem = useCartStore(state => state.removeItem);

  return (
    <div>
      {items.map(item => (
        <div key={item.id}>
          {item.name} x{item.quantity}
          <button onClick={() => removeItem(item.id)}>Remove</button>
        </div>
      ))}
      <p>Total: ${total}</p>
    </div>
  );
}
```

Jotai: Atomic State Management

Jotai takes a completely different approach. Instead of one big store, you create small atoms of state:

```javascript
import { atom, useAtom } from 'jotai';

// Define atoms (individual pieces of state)
const cartItemsAtom = atom([]);

// Derived atom (computed from other atoms)
const cartTotalAtom = atom((get) => {
  const items = get(cartItemsAtom);
  return items.reduce(
    (sum, item) => sum + item.price * item.quantity, 0
  );
});

const cartCountAtom = atom((get) => {
  const items = get(cartItemsAtom);
  return items.reduce((sum, item) => sum + item.quantity, 0);
});

// Write atom (action)
const addToCartAtom = atom(
  null, // no read value
  (get, set, product) => {
    const items = get(cartItemsAtom);
    const existing = items.find(item => item.id === product.id);

    if (existing) {
      set(cartItemsAtom, items.map(item =>
        item.id === product.id
          ? { ...item, quantity: item.quantity + 1 }
          : item
      ));
    } else {
      set(cartItemsAtom, [...items, { ...product, quantity: 1 }]);
    }
  }
);

// Using in components
function CartCount() {
  const [count] = useAtom(cartCountAtom);
  return <span>Cart ({count})</span>;
}

function AddButton({ product }) {
  const [, addToCart] = useAtom(addToCartAtom);
  return <button onClick={() => addToCart(product)}>Add</button>;
}
```

Comparison Table

```
┌──────────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Feature          │ Redux TK     │ Zustand      │ Jotai        │ Context      │
├──────────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Bundle Size      │ ~11 kB       │ ~1.5 kB      │ ~3 kB        │ 0 kB         │
│ Boilerplate      │ Medium       │ Low          │ Low          │ Low          │
│ Learning Curve   │ High         │ Low          │ Medium       │ Low          │
│ DevTools         │ Excellent    │ Good         │ Good         │ Basic        │
│ Middleware       │ Built-in     │ Plugin       │ None         │ Manual       │
│ Async Support    │ Thunks       │ Built-in     │ Built-in     │ Manual       │
│ Provider Needed  │ Yes          │ No           │ Yes          │ Yes          │
│ Re-render Perf   │ Good         │ Great        │ Great        │ Poor*        │
│ Team Scale       │ Large        │ Any          │ Any          │ Small        │
│ Undo/Redo        │ Easy         │ Possible     │ Hard         │ Hard         │
│ Time Travel      │ Yes          │ No           │ No           │ No           │
│ Best For         │ Large        │ Medium       │ Small-       │ Simple       │
│                  │ apps         │ apps         │ Medium       │ sharing      │
└──────────────────┴──────────────┴──────────────┴──────────────┴──────────────┘

* Context causes all consumers to re-render on any change.
  The others can do selective subscriptions.
```

When to Use / When NOT to Use

Use Flux/Redux When

- Multiple unrelated components share state
- You need time-travel debugging
- State changes are complex and depend on each other
- Team is large and needs clear conventions
- You need middleware for logging, analytics, or error tracking
- Undo/redo is a requirement

Do NOT Use When

- State is mostly local to components
- Your app is small (less than 10 routes)
- You can solve the problem with useReducer + Context
- You are prototyping and need to move fast
- Server state is your main concern (use React Query instead)

Common Mistakes

Mistake 1: Putting Everything in Redux

```javascript
// BAD: UI state does not belong in Redux
const uiSlice = createSlice({
  name: 'ui',
  initialState: {
    isModalOpen: false,        // This is local state!
    tooltipText: '',           // This is local state!
    inputValue: '',            // This is definitely local state!
    isDropdownExpanded: false,  // Local state!
  },
  // ...
});

// GOOD: Only shared, app-wide state goes in Redux
// Use useState for UI state that belongs to one component
function Modal() {
  const [isOpen, setIsOpen] = useState(false); // Local is fine
  return (/* ... */);
}
```

Mistake 2: Mutating State Without Immer

```javascript
// BAD: Mutating state in a plain reducer (outside RTK)
function reducer(state, action) {
  state.items.push(action.payload); // MUTATION! Redux will not detect the change
  return state; // Same reference, no re-render
}

// GOOD: Return a new object
function reducer(state, action) {
  return {
    ...state,
    items: [...state.items, action.payload],
  };
}

// ALSO GOOD: Use RTK createSlice (Immer handles it)
// state.items.push(action.payload) IS safe inside createSlice
```

Mistake 3: Deriving State Incorrectly

```javascript
// BAD: Storing derived values that can get out of sync
const slice = createSlice({
  name: 'cart',
  initialState: {
    items: [],
    totalPrice: 0,    // Derived from items — risk of getting out of sync
    totalCount: 0,    // Also derived
  },
  reducers: {
    addItem(state, action) {
      state.items.push(action.payload);
      // Forgot to update totalPrice! Now it is wrong.
    },
  },
});

// GOOD: Use selectors to derive values
const selectCartTotal = (state) =>
  state.cart.items.reduce(
    (sum, item) => sum + item.price * item.quantity, 0
  );

// Or if the calculation is expensive, use createSelector (memoized)
import { createSelector } from '@reduxjs/toolkit';

const selectCartItems = (state) => state.cart.items;
const selectCartTotal = createSelector(
  [selectCartItems],
  (items) => items.reduce(
    (sum, item) => sum + item.price * item.quantity, 0
  )
);
```

Mistake 4: Not Using Selectors

```javascript
// BAD: Accessing deep state paths everywhere
function Component() {
  const name = useSelector(
    state => state.user.profile.settings.display.name
  );
  // If state shape changes, you must update EVERY component
}

// GOOD: Centralize selectors near the slice
// userSlice.js
export const selectDisplayName = (state) =>
  state.user.profile.settings.display.name;

// Component.js
function Component() {
```

```
  const name = useSelector(selectDisplayName);
  // If state shape changes, update ONE selector
}
```

Best Practices

1. Use Redux Toolkit: Never write raw Redux in new projects. RTK eliminates boilerplate and prevents common bugs.
2. Normalize State Shape: Store data like a database. Use IDs as keys, not nested arrays.

```javascript
// Instead of: { users: [{ id: 1, name: 'Alice' }, { id: 2, name: 'Bob' }] }
// Use: { users: { byId: { 1: { name: 'Alice' }, 2: { name: 'Bob' } }, allIds: [1, 2...
```

3. Keep Reducers Pure: No API calls, no random numbers, no Date.now() inside reducers. Use thunks for side effects.
4. Write Selectors: Put selectors next to the slice. Use `createSelector` for expensive computations.
5. One Slice Per Feature: Organize by feature (cart, user, products), not by type (actions, reducers, selectors).
6. Do Not Over-Subscribe: Select only the data you need. Selecting too much causes unnecessary re-renders.

```javascript
// BAD: re-renders on ANY cart change
const cart = useSelector(state => state.cart);

// GOOD: re-renders only when itemCount changes
const itemCount = useSelector(state => state.cart.itemCount);
```

7. Use TypeScript: Redux Toolkit has excellent TypeScript support. Type your state, actions, and selectors.

Quick Summary

Flux introduced unidirectional data flow: Actions describe events, the Dispatcher routes them, Stores update state, and Views render it. Redux simplified this to one store, pure reducer functions, and dispatched actions. Redux Toolkit (RTK) is the modern way to write Redux,

eliminating boilerplate with createSlice and configureStore while using Immer for safe "mutations." For smaller apps, Zustand offers a simpler API with no boilerplate, and Jotai provides atomic state management. Choose based on app size, team size, and debugging needs.

Key Points

- Unidirectional data flow means data moves in one direction: Action -> Reducer -> Store -> View -> Action. This makes state changes predictable and debuggable.
- Actions are plain objects that describe what happened (`{ type: 'cart/addItem', payload: product }`). They never contain logic.
- Reducers are pure functions: `(state, action) => newState`. Same input always produces same output, no side effects.
- Redux Toolkit is the standard way to write Redux today. It includes Immer (safe mutations), auto-generated action creators, and DevTools integration.
- createSlice combines action types, action creators, and reducer logic in one place.
- configureStore automatically sets up middleware, DevTools, and development checks.
- createAsyncThunk handles async operations with automatic pending/fulfilled/rejected states.
- Not every app needs Redux. Consider Zustand for medium apps or Jotai for atomic state. Use plain Context for simple state sharing.

Practice Questions

1. What are the three principles of Redux? Explain each one in your own words.
2. Why does Redux require reducers to be pure functions? What would happen if a reducer called an API or used `Math.random()`?
3. You have a component that only needs the cart item count, but it re-renders whenever any cart state changes. How would you fix this?
4. Compare Redux Toolkit's `createSlice` with writing raw Redux. What three things does `createSlice` generate automatically?
5. Your team is building a small marketing website with 5 pages, a contact form, and a newsletter signup. Someone suggests using Redux. What would you recommend instead, and why?

Exercises

Exercise 1: Todo App with Redux Toolkit

Build a todo application using Redux Toolkit. Requirements:

- Add, toggle complete, and delete todos
- Filter by: All, Active, Completed
- Show count of remaining todos
- Use createSlice for the todo logic
- Use selectors for filtered todos and counts

Hints: Create a `todosSlice` with an items array and a filter string. Use `createSelector` for the filtered list.

Exercise 2: Multi-Step Registration Wizard

Build a 3-step registration form (Personal Info, Preferences, Review) where:

- Each step's data persists in Redux when moving between steps
- Users can go back and edit previous steps
- The Review step shows all data from previous steps
- A "Submit" button on Review dispatches an async thunk
- Handle loading and error states

Hints: Create a `registrationSlice` similar to the checkout example. Use `extraReducers` for the async submission.

Exercise 3: Redux vs Zustand Comparison

Implement the same feature (a simple shopping cart with add/remove/clear) using:

1. Redux Toolkit
2. Zustand

Compare: lines of code, number of files, ease of setup, and developer experience. Write a short paragraph explaining which you preferred and why.

What Is Next?

Now that you understand how to manage application state predictably, the next chapter explores Rendering Patterns. You will learn how CSR, SSR, SSG, and ISR work, when to use each one, and how modern frameworks like Next.js decide how to render your pages. State management and rendering strategy together determine how your app feels to users.

Chapter 25: Rendering Patterns -- CSR, SSR, SSG, ISR, and Beyond

What You Will Learn

- The four core rendering strategies: CSR, SSR, SSG, and ISR
- How hydration works and why it matters
- What streaming SSR brings to the table
- How React Server Components change the game
- A decision flowchart for choosing the right rendering pattern
- Trade-offs of each approach with real performance data

Why This Chapter Matters

When you build a React application, one of the most impactful architectural decisions you make is how and where your HTML gets generated. This single choice affects:

- How fast users see content (First Contentful Paint)
- How fast they can interact with it (Time to Interactive)
- How well search engines find your pages (SEO)
- How much your servers cost to run
- How complex your deployment is

Most tutorials teach you Client-Side Rendering (CSR) first because it is the simplest. But in production, CSR is often the wrong default. Understanding all the rendering patterns lets you pick the right tool for each page in your application.

Think of it like cooking methods. You could microwave everything, but sometimes you need to grill, sometimes bake, and sometimes serve it raw. Each method exists because it solves a different problem best.

Pattern 1: Client-Side Rendering (CSR)

The Problem

You want to build a highly interactive application like a dashboard, a chat app, or an internal tool. The content is dynamic and personalized. SEO does not matter much.

The Solution

Ship an almost-empty HTML file to the browser. Let JavaScript download, execute, and build the entire UI on the client.

How CSR Works

```
+------------------+     +------------------+     +------------------+
|   Browser asks   |     |  Server sends    |     | Browser runs JS  |
|   for page       | --> |  empty HTML +    | --> | and builds the   |
|                  |     |  JS bundle       |     | entire DOM       |
+------------------+     +------------------+     +------------------+
                                                          |
                                                          v
                                                  +------------------+
                                                  | User finally     |
                                                  | sees content     |
                                                  +------------------+

Timeline:
|-- HTML downloaded --|-- JS downloaded --|-- JS executes --|-- Content visible --|
0s                   0.2s                1.5s               2.5s
                                                   (example times)
```

Code Example

A typical CSR setup with React and Vite:

```html
<!-- index.html - This is ALL the server sends -->
<!DOCTYPE html>
<html>
<head>
  <title>My App</title>
```

```
</head>
<body>
  <div id="root"></div>
  <!-- The entire app is in this JS bundle -->
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

```jsx
// src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<App />);
```

```jsx
// src/App.jsx
import { useState, useEffect } from 'react';

function App() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Data fetching happens AFTER JavaScript loads and runs
    fetch('/api/products')
      .then(res => res.json())
      .then(data => {
        setProducts(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Loading products...</div>;

  return (
    <div>
      <h1>Products</h1>
      {products.map(p => (
        <div key={p.id}>{p.name} - ${p.price}</div>
      ))}
    </div>
  );
}

// Output (what the user sees):
// First: "Loading products..." (after JS downloads and executes)
// Then:  "Products"
//        "Widget A - $29.99"
//        "Widget B - $49.99"
```

What Crawlers See

```
View Source (what Google's crawler initially sees):

<html>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>

// Almost nothing! No product names, no prices, no content.
// Google CAN execute JavaScript, but it is slower and less reliable.
```

Pattern 2: Server-Side Rendering (SSR)

The Problem

You need good SEO. You want users to see content immediately, not wait for JavaScript to download and execute. But the content is dynamic and changes per request.

The Solution

Run React on the server for every request. Generate the full HTML there, send it to the browser, then "hydrate" it with JavaScript to make it interactive.

How SSR Works

```
+------------------+     +------------------+     +------------------+
|   Browser asks   |     | Server runs React|     | Browser receives |
|   for page       | --> | and generates    | --> | FULL HTML with   |
|                  |     | complete HTML    |     | all content      |
+------------------+     +------------------+     +------------------+
                                                          |
                                                          v
                                                +------------------+
                                                | User sees content|
                                                | IMMEDIATELY      |
                                                +------------------+
                                                          |
                                                          v
                                                +------------------+
```

```
                                                  | JS downloads and |
                                                  | "hydrates" the   |
                                                  | page (adds       |
                                                  | interactivity)   |
                                                  +------------------+

Timeline:
|-- Server renders --|-- HTML arrives --|-- Content visible --|-- JS hydrates --|
0s                  0.3s               0.5s                  2.0s
                                   (user sees content!)
```

Code Example

Using Next.js (the most popular SSR framework for React):

```jsx
// pages/products.jsx (Next.js Pages Router)

// This function runs on the SERVER for every request
export async function getServerSideProps() {
  // Fetch data on the server - fast because server is close to database
  const res = await fetch('https://api.example.com/products');
  const products = await res.json();

  return {
    props: { products }, // Passed to the component as props
  };
}

function ProductsPage({ products }) {
  // This component renders on BOTH server and client
  return (
    <div>
      <h1>Products</h1>
      {products.map(p => (
        <div key={p.id}>
          <h2>{p.name}</h2>
          <p>${p.price}</p>
          <button onClick={() => addToCart(p.id)}>
            Add to Cart
          </button>
        </div>
      ))}
    </div>
  );
}

export default ProductsPage;

// Output (what the user sees):
// IMMEDIATELY (before JS loads):
```

```
//   "Products"
//   "Widget A"
//   "$29.99"
//   [Add to Cart] (button visible but NOT clickable yet)
//
// AFTER hydration (JS loads):
//   [Add to Cart] (button NOW clickable)
```

What Crawlers See

```
View Source (what Google's crawler sees):

<html>
<body>
  <div id="root">
    <div>
      <h1>Products</h1>
      <div>
        <h2>Widget A</h2>
        <p>$29.99</p>
        <button>Add to Cart</button>
      </div>
      <div>
        <h2>Widget B</h2>
        <p>$49.99</p>
        <button>Add to Cart</button>
      </div>
    </div>
  </div>
</body>
</html>

// Full content is right there in the HTML. Perfect for SEO.
```

Understanding Hydration

Hydration is one of the most misunderstood concepts in modern React. Let us break it down.

What Is Hydration?

When the server sends pre-rendered HTML, the browser displays it instantly. But it is just static HTML. Buttons do not work. Forms do not submit. Nothing is interactive.

Hydration is the process where React takes over that static HTML, attaches event listeners, and makes everything interactive.

```
Before Hydration:                    After Hydration:
+------------------------+          +------------------------+
|  <button>Add to Cart   |          |  <button>Add to Cart   |
|                        |          |                        |
|  Status: DEAD HTML     |          |  Status: ALIVE React   |
|  Click? Nothing happens|          |  Click? Adds to cart!  |
+------------------------+          +------------------------+

The HTML looks the same. But now React "owns" it.
```

How Hydration Works Under the Hood

```jsx
// Step 1: Server renders React to HTML string
import { renderToString } from 'react-dom/server';
const html = renderToString(<App />);
// Result: '<div><h1>Hello</h1><button>Click me</button></div>'

// Step 2: Server sends that HTML to the browser
// Browser displays it immediately (fast!)

// Step 3: Client-side React "hydrates" the existing HTML
import { hydrateRoot } from 'react-dom/client';
hydrateRoot(document.getElementById('root'), <App />);

// React walks through the existing DOM
// It attaches event listeners
// It sets up state and effects
// The HTML MUST match what React would render
```

The Hydration Mismatch Problem

```jsx
// BAD: This will cause a hydration mismatch
function Greeting() {
  // On server: renders "Hello" at 3:00 PM server time
  // On client: renders "Hello" at 3:01 PM user time
  return <p>Current time: {new Date().toLocaleTimeString()}</p>;
}
// React will warn: "Text content did not match"

// GOOD: Defer client-only values to useEffect
function Greeting() {
  const [time, setTime] = useState(null);
```

```
  useEffect(() => {
    // useEffect only runs on the client, after hydration
    setTime(new Date().toLocaleTimeString());
  }, []);

  return <p>Current time: {time ?? 'Loading...'}</p>;
}
```

Pattern 3: Static Site Generation (SSG)

The Problem

You have content that does not change often. Blog posts, documentation, marketing pages. You want maximum performance and minimal server costs.

The Solution

Generate all HTML at build time. Every page becomes a static file. Serve them from a CDN. No server needed at runtime.

How SSG Works

```
BUILD TIME (once):
+-------------------+     +-------------------+     +-------------------+
|  Developer runs   |     | Build process     |     | Static HTML       |
|  "npm run build"  | --> | renders EVERY     | --> | files ready to    |
|                   |     | page to HTML      |     | deploy            |
+-------------------+     +-------------------+     +-------------------+

RUNTIME (every request):
+-------------------+     +-------------------+     +-------------------+
|   Browser asks    |     | CDN serves        |     | User sees page    |
|   for page        | --> | pre-built HTML    | --> | INSTANTLY          |
|                   |     | (no computation)  |     | (fastest possible)|
+-------------------+     +-------------------+     +-------------------+

Timeline:
|-- CDN responds --|-- Content visible --|
0s                0.1s                  0.2s
                (blazing fast!)
```

Code Example

```jsx
// pages/blog/[slug].jsx (Next.js Pages Router)

// Runs at BUILD TIME - tells Next.js which pages to generate
export async function getStaticPaths() {
  const res = await fetch('https://api.example.com/posts');
  const posts = await res.json();

  // Generate a page for each blog post
  const paths = posts.map(post => ({
    params: { slug: post.slug },
  }));

  return { paths, fallback: false };
}

// Runs at BUILD TIME for each path
export async function getStaticProps({ params }) {
  const res = await fetch(
    `https://api.example.com/posts/${params.slug}`
  );
  const post = await res.json();

  return {
    props: { post },
  };
}

function BlogPost({ post }) {
  return (
    <article>
      <h1>{post.title}</h1>
      <p>Published: {post.date}</p>
      <div dangerouslySetInnerHTML={{ __html: post.content }} />
    </article>
  );
}

export default BlogPost;

// At build time, this generates files like:
//   /blog/my-first-post.html
//   /blog/react-patterns.html
//   /blog/javascript-tips.html
// Each is a complete HTML file served from CDN
```

The Stale Content Problem

```
SSG Problem:

Build at 9:00 AM --> HTML generated with current data
User visits at 3:00 PM --> sees 9:00 AM data
Author fixes typo at 3:30 PM --> still old data
Need to rebuild and redeploy --> 5 minute build process

For a blog with 10,000 posts, rebuilding everything
for a single typo fix is wasteful.
```

This leads us to the next pattern.

Pattern 4: Incremental Static Regeneration (ISR)

The Problem

SSG is fast but stale. SSR is fresh but slow. You want both: the speed of static files with the freshness of server rendering.

The Solution

Serve static pages, but regenerate them in the background after a specified time period. Users always get fast static responses. The content stays reasonably fresh.

How ISR Works

```
Initial Build:
  Same as SSG - generate all pages as static HTML

First Request (within revalidation window):
+------------------+     +------------------+
|   Browser asks   |     | CDN serves       |
|   for page       | --> | cached static    |  (fast, possibly stale)
|                  |     | HTML             |
+------------------+     +------------------+

First Request AFTER revalidation window:
+------------------+     +------------------+     +------------------+
|   Browser asks   |     | CDN serves       |     | Server rebuilds  |
|   for page       | --> | STALE cached     | --> | page in the      |
|                  |     | HTML (still fast) |     | BACKGROUND       |
+------------------+     +------------------+     +------------------+
                                                          |
```

```
                                                          v
                                                  +-----------------+
                                                  | Next request    |
                                                  | gets FRESH page |
                                                  +-----------------+

This is called "stale-while-revalidate" strategy.
```

Code Example

```jsx
// pages/products/[id].jsx

export async function getStaticPaths() {
  const res = await fetch('https://api.example.com/products');
  const products = await res.json();

  const paths = products.map(p => ({
    params: { id: p.id.toString() },
  }));

  return {
    paths,
    fallback: 'blocking', // Generate new pages on-demand
  };
}

export async function getStaticProps({ params }) {
  const res = await fetch(
    `https://api.example.com/products/${params.id}`
  );
  const product = await res.json();

  return {
    props: { product },
    revalidate: 60, // Regenerate this page every 60 seconds
  };
}

function ProductPage({ product }) {
  return (
    <div>
      <h1>{product.name}</h1>
      <p>${product.price}</p>
      <p>In Stock: {product.inStock ? 'Yes' : 'No'}</p>
    </div>
  );
}

export default ProductPage;
```

```
// How it plays out:
//
// 9:00:00 AM - Build generates page with price $29.99
// 9:00:30 AM - User visits, gets cached $29.99 (fast!)
// 9:00:45 AM - Admin changes price to $24.99
// 9:01:00 AM - User visits, gets cached $29.99 (stale but fast!)
//              Background: server regenerates page with $24.99
// 9:01:05 AM - User visits, gets fresh $24.99 page (fast!)
```

On-Demand Revalidation

Instead of time-based revalidation, you can trigger rebuilds manually:

```jsx
// pages/api/revalidate.js
export default async function handler(req, res) {
  // Verify the request is legitimate
  if (req.query.secret !== process.env.REVALIDATION_SECRET) {
    return res.status(401).json({ message: 'Invalid token' });
  }

  try {
    // Rebuild a specific page immediately
    await res.revalidate(`/products/${req.query.id}`);
    return res.json({ revalidated: true });
  } catch (err) {
    return res.status(500).send('Error revalidating');
  }
}

// Usage: When admin updates a product in the CMS,
// the CMS webhook calls:
// POST /api/revalidate?secret=MY_SECRET&id=42
// And only that product page gets regenerated.
```

Streaming SSR

The Problem with Traditional SSR

Traditional SSR has a waterfall problem:

```
Traditional SSR Waterfall:

Server:  [---Fetch ALL data---][---Render ALL HTML---]
```

```
Network:                                                [---Send ALL HTML---]
Browser:                                                                     [---Show ALL...

Nothing is shown until EVERYTHING is ready.
If one API call is slow, the entire page is delayed.

Example: Product page needs:
  1. Product details (50ms)
  2. Reviews (2000ms)     <-- Slow API!
  3. Recommendations (100ms)

Total wait: 2150ms before user sees ANYTHING.
```

The Solution: Streaming SSR

Send HTML in chunks as each part becomes ready.

```
Streaming SSR:

Server:  [Fetch product (50ms)]
         [----------Render product shell----------]
Network:                        [Send shell HTML]
Browser:                                          [Show product!]

Server:  [----------Fetch reviews (2000ms)---------]
         [-----Render reviews-----]
Network:                           [Stream reviews HTML]
Browser:                                                [Show reviews!]

User sees the product in 50ms instead of waiting 2150ms!
```

Code Example (React 18+ with Next.js App Router)

```jsx
// app/products/[id]/page.jsx
import { Suspense } from 'react';

// This component can be async - it is a Server Component
async function ProductDetails({ id }) {
  const product = await fetch(
    `https://api.example.com/products/${id}`
  ).then(r => r.json());

  return (
    <div>
      <h1>{product.name}</h1>
      <p>${product.price}</p>
```

```
    </div>
  );
}

// This slow component will stream in later
async function ProductReviews({ id }) {
  // This API is slow - takes 2 seconds
  const reviews = await fetch(
    `https://api.example.com/products/${id}/reviews`
  ).then(r => r.json());

  return (
    <div>
      <h2>Reviews ({reviews.length})</h2>
      {reviews.map(r => (
        <div key={r.id}>
          <strong>{r.author}</strong>: {r.text}
        </div>
      ))}
    </div>
  );
}

// The page component uses Suspense for streaming
export default function ProductPage({ params }) {
  return (
    <div>
      {/* This renders and sends immediately */}
      <Suspense fallback={<p>Loading product...</p>}>
        <ProductDetails id={params.id} />
      </Suspense>

      {/* This streams in when ready, showing fallback first */}
      <Suspense fallback={<p>Loading reviews...</p>}>
        <ProductReviews id={params.id} />
      </Suspense>
    </div>
  );
}

// Output sequence:
// 1. Browser receives: product details + "Loading reviews..."
// 2. Browser receives: actual reviews (streamed in later)
// User sees useful content almost immediately!
```

React Server Components (RSC)

The Problem

Even with SSR, we ship all the React component code to the client for hydration. A product page might import a Markdown renderer, a date formatting library, and a syntax highlighter. The user downloads all of that JavaScript even though it only runs once on the server.

The Solution

React Server Components run only on the server. They never ship JavaScript to the client. They can directly access databases, file systems, and APIs. Client Components handle interactivity.

```
Traditional SSR:
+---------------------------+
|  Server renders HTML      |
|  Client downloads ALL JS  |
|  Client hydrates ALL      |
+---------------------------+
JS Bundle: [ProductPage + ReviewList + MarkdownRenderer + DateLib]
           (everything, even non-interactive parts)

React Server Components:
+---------------------------+
|  Server Components stay   |
|  on server (zero JS sent) |
|  Client Components sent   |
|  only for interactive bits|
+---------------------------+
JS Bundle: [AddToCartButton + ReviewForm]
           (only interactive parts!)
```

Code Example

```jsx
// app/products/[id]/page.jsx
// This is a SERVER Component by default in Next.js App Router
// It NEVER runs in the browser. Zero JavaScript sent for this.

import { db } from '@/lib/database';      // Direct DB access!
import { formatDate } from '@/lib/utils';  // Not sent to client
import AddToCartButton from './AddToCartButton'; // Client Component

export default async function ProductPage({ params }) {
  // Query database directly - no API needed!
  const product = await db.products.findById(params.id);

  return (
    <div>
      <h1>{product.name}</h1>
```

```
      <p>${product.price}</p>
      <p>Added: {formatDate(product.createdAt)}</p>

      {/* Only THIS component's JS is sent to the browser */}
      <AddToCartButton productId={product.id} />
    </div>
  );
}
```

```jsx
// app/products/[id]/AddToCartButton.jsx
'use client'; // This directive marks it as a Client Component

import { useState } from 'react';

export default function AddToCartButton({ productId }) {
  const [added, setAdded] = useState(false);

  async function handleClick() {
    await fetch('/api/cart', {
      method: 'POST',
      body: JSON.stringify({ productId }),
    });
    setAdded(true);
  }

  return (
    <button onClick={handleClick}>
      {added ? 'Added!' : 'Add to Cart'}
    </button>
  );
}

// Only this small component ships JavaScript to the browser.
// The entire ProductPage, database library, date formatting
// library -- none of that goes to the client.
```

Server vs Client Component Rules

```
+-------------------------------+-----------------------------------+
|     Server Components         |      Client Components            |
+-------------------------------+-----------------------------------+
| Default in App Router         | Must add 'use client' directive   |
| Can access database/fs        | Cannot access server resources    |
| Can use async/await           | Cannot be async                   |
| Cannot use useState           | Can use useState                  |
| Cannot use useEffect          | Can use useEffect                 |
| Cannot use event handlers     | Can use onClick, onChange, etc.   |
| Cannot use browser APIs       | Can use window, document, etc.   |
| Zero JavaScript to client     | JavaScript sent to client         |
```

```
| Can import Client Components  | Cannot import Server Components   |
+-------------------------------+-----------------------------------+
```

Comparing All Rendering Patterns

```
+-------+------------------+-----------------+---------+----------+
|Pattern| When HTML is     | Data Freshness  | Speed   | SEO      |
|       | Generated        |                 |         |          |
+-------+------------------+-----------------+---------+----------+
| CSR   | In browser       | Always fresh    | Slowest | Poor     |
|       | (at runtime)     | (fetches live)  | initial |          |
+-------+------------------+-----------------+---------+----------+
| SSR   | On server        | Always fresh    | Fast    | Great    |
|       | (per request)    | (per request)   | initial |          |
+-------+------------------+-----------------+---------+----------+
| SSG   | At build time    | Stale until     | Fastest | Great    |
|       | (once)           | next build      |         |          |
+-------+------------------+-----------------+---------+----------+
| ISR   | At build time +  | Mostly fresh    | Fast    | Great    |
|       | background regen | (configurable)  |         |          |
+-------+------------------+-----------------+---------+----------+
| Stream| On server        | Always fresh    | Fast    | Great    |
| SSR   | (per request,    | (per request)   | initial |          |
|       |  in chunks)      |                 | (progr.)|          |
+-------+------------------+-----------------+---------+----------+
| RSC   | Server (server   | Fresh for       | Minimal | Great    |
|       | parts) + Client  | server parts    | JS sent |          |
|       | (interactive)    |                 |         |          |
+-------+------------------+-----------------+---------+----------+
```

Decision Flowchart

Use this flowchart to choose the right rendering pattern:

```
                    START
                      |
                      v
               Does SEO matter?
               /             \
           No                 Yes
```

```
            |                  |
            v                  v
      Is it highly        Does content change
      interactive?        per request?
      (dashboard,         /              \
       internal tool)   Yes               No
            |            |                |
            v            v                v
          CSR          SSR or        Does content change
                     Streaming SSR    frequently?
                                     /          \
                                   Yes           No
                                    |             |
                                    v             v
                                   ISR           SSG


      Additional considerations:

      Large JS bundle?  --> Add React Server Components
      Slow API calls?   --> Use Streaming SSR with Suspense
      Need offline?     --> CSR with Service Worker
      E-commerce?       --> ISR (products) + SSR (cart) + SSG (about)
```

Real-World Use Case: E-Commerce Site

A real e-commerce site uses multiple rendering patterns on different pages:

```jsx
// PATTERN MIX for an e-commerce site:

// Landing page --> SSG (rarely changes, must be fast)
// app/page.jsx (Server Component, built at deploy time)

// Product listing --> ISR (products change, but not every second)
// app/products/page.jsx with revalidate: 300

// Product detail --> ISR + Streaming SSR
// app/products/[id]/page.jsx
//   - Product info: ISR (revalidate: 60)
//   - Reviews: Streamed in with Suspense
//   - "Add to Cart": Client Component

// Shopping cart --> CSR (user-specific, no SEO needed)
// app/cart/page.jsx with 'use client'

// Checkout --> SSR (must be fresh, needs auth)
```

```
// app/checkout/page.jsx (Server Component with auth check)

// Blog posts --> SSG (content does not change)
// app/blog/[slug]/page.jsx (built at deploy time)
```

```jsx
// Example: Product page mixing patterns
// app/products/[id]/page.jsx

import { Suspense } from 'react';
import { db } from '@/lib/database';
import AddToCart from './AddToCart';         // Client Component
import Reviews from './Reviews';             // Server Component (async)
import Recommendations from './Recommendations';

// ISR: Regenerate every 60 seconds
export const revalidate = 60;

export default async function ProductPage({ params }) {
  // Server Component: direct DB access, zero client JS
  const product = await db.products.findById(params.id);

  return (
    <main>
      {/* Rendered immediately */}
      <section>
        <h1>{product.name}</h1>
        <p className="price">${product.price}</p>
        <p>{product.description}</p>

        {/* Only interactive part ships JS */}
        <AddToCart product={product} />
      </section>

      {/* Streams in when ready */}
      <Suspense fallback={<ReviewsSkeleton />}>
        <Reviews productId={product.id} />
      </Suspense>

      {/* Also streams independently */}
      <Suspense fallback={<RecommendationsSkeleton />}>
        <Recommendations category={product.category} />
      </Suspense>
    </main>
  );
}
```

When to Use / When NOT to Use

Use CSR When

- Building dashboards and admin panels
- Internal tools behind authentication
- Highly interactive apps (chat, collaborative editors)
- SEO does not matter

Do NOT Use CSR When

- SEO matters (blog, e-commerce, marketing pages)
- First load performance is critical
- Users are on slow connections or devices

Use SSR When

- Content is personalized per user (logged-in pages)
- Data must be fresh on every request
- SEO matters and content changes frequently

Do NOT Use SSR When

- Content rarely changes (use SSG or ISR instead)
- You cannot afford server costs for every request
- Pages are simple and static

Use SSG When

- Content rarely changes (docs, blog, marketing)
- Maximum performance is needed
- You want minimal hosting costs (static hosting)

Do NOT Use SSG When

- Content changes frequently
- You have thousands of pages that rebuild slowly
- Content is personalized per user

Use ISR When

- Content changes but not on every request

- You have many pages (e-commerce catalogs)
- You want static performance with reasonable freshness

Common Mistakes

Mistake 1: Using CSR for Everything

```
WRONG thinking:
"Create React App uses CSR and it works fine for development,
 so I will use it for production too."

PROBLEM:
- Users stare at a blank screen for 2-3 seconds
- Search engines may not index your content
- Performance scores tank on Lighthouse

FIX:
Evaluate each page. Most public-facing pages benefit from
SSR, SSG, or ISR. Reserve CSR for interactive authenticated pages.
```

Mistake 2: Using SSR When SSG Would Work

```
WRONG:
export async function getServerSideProps() {
  const aboutContent = await fetchCMSContent('about-page');
  return { props: { aboutContent } };
}
// The about page renders on every request, even though
// it changes once a month.

RIGHT:
export async function getStaticProps() {
  const aboutContent = await fetchCMSContent('about-page');
  return {
    props: { aboutContent },
    revalidate: 3600, // Check for updates hourly
  };
}
// Static page, regenerated hourly. Much faster.
```

Mistake 3: Ignoring Hydration Mismatches

```jsx
// WRONG: Different output on server vs client
function Component() {
  return <p>Window width: {window.innerWidth}</p>;
  // Crashes on server! window does not exist there.
}

// ALSO WRONG: Conditional rendering based on typeof window
function Component() {
  if (typeof window !== 'undefined') {
    return <p>Client: {window.innerWidth}</p>;
  }
  return <p>Server</p>;
  // Different HTML on server vs client = hydration mismatch
}

// RIGHT: Use useEffect for client-only values
function Component() {
  const [width, setWidth] = useState(0);

  useEffect(() => {
    setWidth(window.innerWidth);
    const handleResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', handleResize);
    return () => window.removeEventListener('resize', handleResize);
  }, []);

  return <p>Window width: {width || 'Loading...'}</p>;
}
```

Best Practices

1. Mix rendering patterns per page -- Use the right strategy for each page, not one strategy for the entire app.
2. Default to Server Components -- In the App Router, keep components as Server Components unless they need interactivity. This minimizes client JavaScript.
3. Push 'use client' down the tree -- Mark the smallest possible component as a Client Component, not entire pages.
4. Use Suspense boundaries strategically -- Wrap slow data fetches in Suspense to stream content progressively.
5. Set appropriate revalidation intervals -- For ISR, match the revalidation time to how often your data actually changes.
6. Monitor hydration errors in development -- React shows hydration mismatch warnings in the console. Fix them immediately.

7. Use `loading.jsx` in App Router -- Next.js App Router automatically wraps pages in Suspense when you add a `loading.jsx` file.
8. Profile before optimizing -- Use Lighthouse, Web Vitals, and your framework's built-in analytics to measure actual performance.

Quick Summary

Rendering patterns determine where and when your HTML is generated. CSR builds everything in the browser (simple but slow). SSR generates HTML on the server per request (fresh but server-intensive). SSG pre-builds pages at deploy time (fastest but stale). ISR combines static speed with background regeneration. Streaming SSR sends HTML in chunks as data becomes available. React Server Components keep non-interactive code on the server, dramatically reducing client JavaScript. Most production apps mix multiple patterns based on each page's needs.

Key Points

- CSR: Browser builds HTML. Simple setup. Poor initial load and SEO.
- SSR: Server builds HTML per request. Great SEO. Server cost per request.
- SSG: Build step generates HTML. Fastest delivery. Content can go stale.
- ISR: Static + background regeneration. Fast with configurable freshness.
- Hydration: Process of making server-rendered HTML interactive on the client.
- Streaming SSR: Send HTML in chunks. Users see content progressively.
- React Server Components: Server-only components that ship zero JavaScript.
- Mix patterns: Use the right rendering strategy for each page in your app.

Practice Questions

1. You are building a news website. The homepage shows trending articles that update every few minutes. Individual articles rarely change after publication. Which rendering patterns would you use for the homepage and for article pages? Why?

2. A developer uses `getServerSideProps` for their company's "About Us" page that changes once a year. What is wrong with this approach, and what would you recommend instead?
3. Explain the concept of hydration to a non-technical person. Why does a page sometimes look loaded but not respond to clicks?
4. A React component renders `Math.random()` in its JSX. What problem will this cause with SSR, and how would you fix it?
5. You have a product page where the product details load in 50ms but customer reviews take 3 seconds. How would streaming SSR improve the user experience compared to traditional SSR?

Exercises

Exercise 1: Rendering Pattern Audit

Take a website you use regularly (Amazon, Wikipedia, your company's site). For each major page type (homepage, listing, detail, cart, account), identify which rendering pattern would be ideal. Create a table showing:

- Page type
- Recommended rendering pattern
- Reasoning (SEO needs, data freshness, interactivity)

Exercise 2: Convert CSR to SSR

Take a simple CSR React component that fetches data in `useEffect` and convert it to use `getServerSideProps` in Next.js. Then convert it again to use `getStaticProps` with ISR. Compare the three approaches by measuring First Contentful Paint.

Exercise 3: Server and Client Component Split

You have a blog post page with: a header (static), article content (from CMS), a "like" button (interactive), and a comment section (interactive with form). Draw a component tree and mark each component as Server or Client. Explain why you made each choice.

What Is Next?

Now that you understand how to render your pages efficiently, the next challenge is making sure you do not send too much JavaScript to the browser in the first place. In Chapter 26, we will explore Code Splitting and Lazy Loading -- techniques that break your JavaScript bundle into smaller pieces and load them only when needed.

Chapter 26: Code Splitting and Lazy Loading

What You Will Learn

- Why large JavaScript bundles hurt performance
- How `React.lazy` and `Suspense` work together for component-level splitting
- Route-based code splitting strategies
- How `dynamic import()` works under the hood
- Prefetching techniques to make lazy loading invisible to users
- How to analyze your bundle and find what to split
- Practical patterns for real applications

Why This Chapter Matters

Imagine you walk into a library to read one specific book. But instead of handing you that book, the librarian says, "Hold on, let me load every single book in the building onto this cart first. Then you can pick yours." That is what happens when you ship a single massive JavaScript bundle to the browser.

Your users came for one page. But you are forcing them to download JavaScript for every page, every feature, every component in your entire application before they can interact with anything.

Code splitting fixes this. It breaks your application into smaller chunks that load on demand. Users download only the code they need right now. The rest loads later, in the background, when they actually need it.

The difference is dramatic. A well-split application can reduce initial load time by 50-70%. For users on slow connections or low-end devices, it can mean the difference between your app being usable or abandoned.

The Bundle Size Problem

Before Code Splitting

```
Your Application:
+-----------------------------------------------------------------+
|                    main.bundle.js (2.4 MB)                      |
|                                                                 |
|  HomePage     ProductPage    AdminDashboard    UserSettings    |
|  ChartLib     MarkdownLib    DatePickerLib     PDFGenerator    |
|  LoginForm    SignupFlow     PaymentForm       ReportViewer    |
+-----------------------------------------------------------------+

User visits homepage:
  Downloads: 2.4 MB of JavaScript
  Uses: ~200 KB of it
  Wasted: ~2.2 MB downloaded for nothing

Timeline:
|-------- Download 2.4 MB --------|--- Parse ---|-- Execute --|-- Interactive --|
0s                                2s            3s            4s               4.5s
```

After Code Splitting

```
Your Application:
+-------------------+  +-------------------+  +-------------------+
| main.chunk.js     |  | products.chunk.js |  | admin.chunk.js    |
| (150 KB)          |  | (300 KB)          |  | (800 KB)          |
| Core + HomePage   |  | ProductPage       |  | AdminDashboard    |
|                   |  | ChartLib          |  | ReportViewer      |
+-------------------+  +-------------------+  +-------------------+

User visits homepage:
  Downloads: 150 KB of JavaScript
  Uses: 150 KB of it
  Wasted: 0 KB

Timeline:
|-- Download 150KB --|-- Parse --|-- Interactive --|
0s                  0.3s        0.5s              0.8s
```

React.lazy and Suspense

The Problem

You have a component that is heavy (it imports a large library) or rarely used (like an admin panel). You do not want it in your main bundle.

The Solution

Use `React.lazy` to defer loading a component until it is actually rendered. Use `Suspense` to show a fallback while it loads.

Before (Eager Loading)

```jsx
// All imports are bundled together, loaded upfront
import HomePage from './pages/HomePage';
import ProductPage from './pages/ProductPage';
import AdminDashboard from './pages/AdminDashboard'; // Heavy! 800KB
import UserSettings from './pages/UserSettings';

function App() {
  return (
    <Routes>
      <Route path="/" element={<HomePage />} />
      <Route path="/products" element={<ProductPage />} />
      <Route path="/admin" element={<AdminDashboard />} />
      <Route path="/settings" element={<UserSettings />} />
    </Routes>
  );
}

// Bundle: ONE file containing ALL pages
// main.bundle.js: 2.4 MB
// User downloading admin code even if they never visit /admin
```

After (Lazy Loading)

```jsx
import { lazy, Suspense } from 'react';

// These imports happen ONLY when the component is first rendered
const HomePage = lazy(() => import('./pages/HomePage'));
const ProductPage = lazy(() => import('./pages/ProductPage'));
const AdminDashboard = lazy(() => import('./pages/AdminDashboard'));
const UserSettings = lazy(() => import('./pages/UserSettings'));
```

```
function App() {
  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/products" element={<ProductPage />} />
        <Route path="/admin" element={<AdminDashboard />} />
        <Route path="/settings" element={<UserSettings />} />
      </Routes>
    </Suspense>
  );
}

function LoadingSpinner() {
  return (
    <div className="loading">
      <div className="spinner" />
      <p>Loading...</p>
    </div>
  );
}

// Output when user navigates to /admin:
// 1. Shows <LoadingSpinner /> immediately
// 2. Downloads admin.chunk.js in the background
// 3. Replaces spinner with <AdminDashboard /> when ready
```

How It Works Under the Hood

```
React.lazy(() => import('./AdminDashboard'))

Step 1: React encounters <AdminDashboard /> in the tree
Step 2: Calls the factory function: () => import('./AdminDashboard')
Step 3: import() returns a Promise
Step 4: React "suspends" - throws a Promise up to nearest Suspense
Step 5: Suspense catches it, shows fallback
Step 6: When Promise resolves, React re-renders with the real component

+------------------+        +------------------+
|   Suspense       |        |   Suspense       |
|   +-----------+  |  -->   |   +-----------+  |
|   | Loading...|  |        |   | Dashboard |  |
|   +-----------+  |        |   +-----------+  |
|   (fallback)     |        |   (resolved)     |
+------------------+        +------------------+
```

Route-Based Code Splitting

The Pattern

The most natural place to split code is at route boundaries. Each page becomes its own chunk.

```jsx
// routes.jsx
import { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Layout from './components/Layout'; // Keep in main bundle

// Lazy load all page components
const Home = lazy(() => import('./pages/Home'));
const Products = lazy(() => import('./pages/Products'));
const ProductDetail = lazy(() => import('./pages/ProductDetail'));
const Cart = lazy(() => import('./pages/Cart'));
const Checkout = lazy(() => import('./pages/Checkout'));
const Account = lazy(() => import('./pages/Account'));

function PageLoader() {
  return (
    <div className="page-loader">
      <div className="skeleton-header" />
      <div className="skeleton-body" />
    </div>
  );
}

export default function AppRoutes() {
  return (
    <BrowserRouter>
      <Layout>
        <Suspense fallback={<PageLoader />}>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/products" element={<Products />} />
            <Route path="/products/:id" element={<ProductDetail />} />
            <Route path="/cart" element={<Cart />} />
            <Route path="/checkout" element={<Checkout />} />
            <Route path="/account" element={<Account />} />
          </Routes>
        </Suspense>
      </Layout>
    </BrowserRouter>
  );
}

// Generated chunks:
//   main.js          - Layout, Router, core utils (always loaded)
```

```
//   home.chunk.js    - Home page (loaded on /)
//   products.chunk.js - Products page (loaded on /products)
//   detail.chunk.js  - Product detail (loaded on /products/:id)
//   cart.chunk.js    - Cart page (loaded on /cart)
//   checkout.chunk.js - Checkout (loaded on /checkout)
//   account.chunk.js - Account (loaded on /account)
```

Per-Route Suspense Boundaries

```jsx
// More granular: different loading states per route
export default function AppRoutes() {
  return (
    <BrowserRouter>
      <Layout>
        <Routes>
          <Route
            path="/"
            element={
              <Suspense fallback={<HeroSkeleton />}>
                <Home />
              </Suspense>
            }
          />
          <Route
            path="/products"
            element={
              <Suspense fallback={<ProductGridSkeleton />}>
                <Products />
              </Suspense>
            }
          />
          <Route
            path="/checkout"
            element={
              <Suspense fallback={<CheckoutSkeleton />}>
                <Checkout />
              </Suspense>
            }
          />
        </Routes>
      </Layout>
    </BrowserRouter>
  );
}

// Each route gets a contextual loading skeleton instead
// of a generic spinner. Better user experience!
```

Dynamic import() Deep Dive

`import()` is the JavaScript feature that makes code splitting possible. It is not React-specific.

Basic Dynamic Import

```jsx
// Static import (bundled together, loaded immediately)
import { calculateTax } from './tax-calculator';

// Dynamic import (separate chunk, loaded on demand)
async function handleCheckout(cart) {
  // tax-calculator.js is only downloaded when checkout happens
  const { calculateTax } = await import('./tax-calculator');
  const tax = calculateTax(cart.subtotal, cart.state);
  return cart.subtotal + tax;
}

// Output:
// 1. User browses products - tax-calculator.js NOT downloaded
// 2. User clicks "Checkout"
// 3. tax-calculator.chunk.js downloads (e.g., 45 KB)
// 4. Tax is calculated
// 5. Checkout continues
```

Conditional Dynamic Imports

```jsx
// Load different modules based on conditions
async function loadEditor(format) {
  let editor;

  if (format === 'markdown') {
    // Only loads markdown editor when needed
    editor = await import('./editors/MarkdownEditor');
  } else if (format === 'richtext') {
    // Only loads rich text editor when needed
    editor = await import('./editors/RichTextEditor');
  } else {
    editor = await import('./editors/PlainTextEditor');
  }

  return editor.default;
}

// Usage in a component
function DocumentEditor({ format }) {
  const [Editor, setEditor] = useState(null);
```

```
  useEffect(() => {
    loadEditor(format).then(EditorComponent => {
      setEditor(() => EditorComponent);
    });
  }, [format]);

  if (!Editor) return <p>Loading editor...</p>;
  return <Editor />;
}
```

Named Exports with Dynamic Import

```jsx
// The module exports multiple things
// utils/charts.js
export function BarChart() { /* ... */ }
export function LineChart() { /* ... */ }
export function PieChart() { /* ... */ }

// Dynamic import with named exports
const ChartComponent = lazy(() =>
  import('./utils/charts').then(module => ({
    // React.lazy expects a default export
    // This trick converts a named export to default
    default: module.BarChart,
  }))
);

// Or create a wrapper for cleaner syntax
function lazyNamed(importFn, name) {
  return lazy(() =>
    importFn().then(module => ({ default: module[name] }))
  );
}

const BarChart = lazyNamed(
  () => import('./utils/charts'),
  'BarChart'
);
const PieChart = lazyNamed(
  () => import('./utils/charts'),
  'PieChart'
);
```

Prefetching: Making Lazy Loading Invisible

The Problem with Naive Lazy Loading

```
Without prefetching:

User hovers over "Products" link
User clicks "Products" link
  |--- Download products.chunk.js (300ms) ---|
  |--- Parse and execute (100ms) ---|
  |--- Render (50ms) ---|
                                              User sees page (450ms later)

The user sees a loading spinner for almost half a second.
On slow connections, it could be 2-3 seconds.
```

Solution: Prefetch on Hover

```jsx
import { lazy, Suspense, useCallback } from 'react';

// Store the import function so we can call it early
const importProducts = () => import('./pages/Products');

// Create the lazy component
const Products = lazy(importProducts);

function Navigation() {
  const prefetchProducts = useCallback(() => {
    // Start loading the chunk when user hovers over the link
    importProducts();
  }, []);

  return (
    <nav>
      <Link to="/">Home</Link>
      <Link
        to="/products"
        onMouseEnter={prefetchProducts}  // Prefetch on hover!
        onFocus={prefetchProducts}        // Also on keyboard focus
      >
        Products
      </Link>
    </nav>
  );
}

// Timeline WITH prefetching:
//
// User hovers over "Products" (starts download)
//   |--- Download products.chunk.js (300ms) ---|
// User clicks 200ms later (chunk already downloading!)
```

```
//   |--- Remaining download (100ms) ---|
//   |--- Parse + Render (150ms) ---|
//                                        User sees page (250ms)
//
// vs WITHOUT prefetching: 450ms
// Savings: ~200ms (feels instant)
```

Prefetch with Intersection Observer

```jsx
// Prefetch when a component scrolls into view
import { useEffect, useRef, lazy, Suspense } from 'react';

const HeavyChart = lazy(() => import('./components/HeavyChart'));

function Dashboard() {
  const chartRef = useRef(null);
  const prefetched = useRef(false);

  useEffect(() => {
    const observer = new IntersectionObserver(
      ([entry]) => {
        if (entry.isIntersecting && !prefetched.current) {
          // User is scrolling toward the chart area
          import('./components/HeavyChart');
          prefetched.current = true;
          observer.disconnect();
        }
      },
      { rootMargin: '200px' } // Start loading 200px before visible
    );

    if (chartRef.current) {
      observer.observe(chartRef.current);
    }

    return () => observer.disconnect();
  }, []);

  return (
    <div>
      <h1>Dashboard</h1>
      <div>...other content...</div>

      {/* Sentinel element triggers prefetch */}
      <div ref={chartRef}>
        <Suspense fallback={<ChartSkeleton />}>
          <HeavyChart />
        </Suspense>
      </div>
    </div>
```

```
  );
}
```

Prefetch with Webpack Magic Comments

```jsx
// Webpack-specific prefetch hints
const AdminPanel = lazy(() =>
  import(
    /* webpackPrefetch: true */
    './pages/AdminPanel'
  )
);
// Webpack adds <link rel="prefetch"> to the HTML
// Browser downloads in idle time, lowest priority

const Checkout = lazy(() =>
  import(
    /* webpackPreload: true */
    './pages/Checkout'
  )
);
// Webpack adds <link rel="preload"> to the HTML
// Browser downloads immediately, high priority

// You can also name your chunks for easier debugging
const Products = lazy(() =>
  import(
    /* webpackChunkName: "products-page" */
    './pages/Products'
  )
);
// Creates: products-page.chunk.js (instead of random hash)
```

Component-Level Code Splitting

Not all splitting needs to happen at the route level. Heavy components within a page are great candidates too.

```jsx
import { lazy, Suspense, useState } from 'react';

// Heavy components loaded on demand
const MarkdownPreview = lazy(() => import('./MarkdownPreview'));
```

```
const CodeEditor = lazy(() => import('./CodeEditor'));
const ImageCropper = lazy(() => import('./ImageCropper'));

function PostEditor() {
  const [showPreview, setShowPreview] = useState(false);
  const [showImageCrop, setShowImageCrop] = useState(false);

  return (
    <div>
      <h2>Write a Post</h2>

      {/* Code editor loads when PostEditor mounts */}
      <Suspense fallback={<div>Loading editor...</div>}>
        <CodeEditor language="markdown" />
      </Suspense>

      {/* Preview only loads when user clicks the button */}
      <button onClick={() => setShowPreview(!showPreview)}>
        {showPreview ? 'Hide Preview' : 'Show Preview'}
      </button>

      {showPreview && (
        <Suspense fallback={<div>Loading preview...</div>}>
          <MarkdownPreview />
        </Suspense>
      )}

      {/* Image cropper loads only when user adds an image */}
      <button onClick={() => setShowImageCrop(true)}>
        Add Image
      </button>

      {showImageCrop && (
        <Suspense fallback={<div>Loading image tools...</div>}>
          <ImageCropper onCrop={handleCrop} />
        </Suspense>
      )}
    </div>
  );
}

// Bundle result:
// main.js            - PostEditor shell (~5 KB)
// code-editor.chunk  - CodeEditor + dependencies (~200 KB)
// md-preview.chunk   - MarkdownPreview + parser (~150 KB)
// image-crop.chunk   - ImageCropper + canvas lib (~100 KB)
//
// Users who never preview or crop images save 250 KB!
```

Bundle Analysis

Before you split code, you need to know what is in your bundle.

Using webpack-bundle-analyzer

```bash
# Install
npm install --save-dev webpack-bundle-analyzer

# For Create React App (using source-map-explorer)
npm install --save-dev source-map-explorer

# Add to package.json scripts
# "analyze": "source-map-explorer 'build/static/js/*.js'"

# Run
npm run build
npm run analyze
```

Reading the Analysis

```
Bundle Analysis Output (simplified):

+-----------------------------------------------------------------+
|                     main.bundle.js (2.4 MB)                     |
+-----------------------------------------------------------------+
|                                                                 |
|  +------------------+  +------------------+  +--------------+ |
|  | node_modules     |  | src/pages        |  | src/utils    | |
|  | (1.8 MB - 75%)   |  | (400 KB - 17%)   |  | (200 KB)    | |
|  |                  |  |                  |  |              | |
|  | moment.js  500KB |  | Admin    300KB   |  |              | |
|  | lodash     200KB |  | Products 50KB    |  |              | |
|  | chart.js   400KB |  | Home     30KB    |  |              | |
|  | react-pdf  300KB |  | Settings 20KB    |  |              | |
|  | others     400KB |  |                  |  |              | |
|  +------------------+  +------------------+  +--------------+ |
+-----------------------------------------------------------------+

Findings:
1. moment.js (500KB) - Replace with date-fns (30KB) or dayjs (7KB)
2. lodash (200KB) - Import individual functions: import get from 'lodash/get'
3. chart.js (400KB) - Lazy load, only used on Admin page
4. react-pdf (300KB) - Lazy load, only used in Reports
5. Admin page (300KB) - Lazy load, only 5% of users are admins
```

Actionable Steps After Analysis

```jsx
// BEFORE: Everything in main bundle
import moment from 'moment';           // 500 KB!
import _ from 'lodash';                 // 200 KB!
import { Chart } from 'chart.js';       // 400 KB!

// AFTER: Optimized imports + lazy loading

// Step 1: Replace heavy libraries with lighter alternatives
import { format, parseISO } from 'date-fns'; // 30 KB (tree-shakable)

// Step 2: Import only what you need from lodash
import get from 'lodash/get';           // 2 KB instead of 200 KB
import debounce from 'lodash/debounce'; // 1 KB

// Step 3: Lazy load heavy components
const ChartDashboard = lazy(() => import('./ChartDashboard'));
// chart.js only downloads when ChartDashboard renders

// Result: Main bundle went from 2.4 MB to 350 KB
```

Vite Bundle Analysis

```bash
# For Vite projects
npm install --save-dev rollup-plugin-visualizer

# vite.config.js
import { visualizer } from 'rollup-plugin-visualizer';

export default {
  plugins: [
    visualizer({
      open: true,           // Auto-open in browser
      gzipSize: true,       // Show gzipped sizes
      brotliSize: true,     // Show brotli sizes
    }),
  ],
};

# Run build to generate report
npm run build
# Opens an interactive treemap in your browser
```

Real-World Use Case: SaaS Dashboard

```jsx
// A SaaS app with many features, but users only use a few at a time

import { lazy, Suspense } from 'react';
import { Routes, Route, NavLink } from 'react-router-dom';

// Core: always loaded (small)
import AppShell from './components/AppShell';
import Sidebar from './components/Sidebar';

// Features: lazy loaded
const Dashboard = lazy(() => import('./features/Dashboard'));
const Analytics = lazy(() =>
  import(/* webpackChunkName: "analytics" */ './features/Analytics')
);
const Reports = lazy(() =>
  import(/* webpackChunkName: "reports" */ './features/Reports')
);
const TeamManagement = lazy(() =>
  import(/* webpackChunkName: "team" */ './features/TeamManagement')
);
const BillingSettings = lazy(() =>
  import(/* webpackChunkName: "billing" */ './features/BillingSettings')
);
const Integrations = lazy(() =>
  import(/* webpackChunkName: "integrations" */ './features/Integrations')
);

// Loading component with skeleton matching each feature
function FeatureLoader({ feature }) {
  return (
    <div className="feature-loader">
      <div className="skeleton-nav" />
      <div className="skeleton-content" />
      <p>Loading {feature}...</p>
    </div>
  );
}

// Wrapper that adds Suspense to lazy routes
function LazyRoute({ component: Component, name }) {
  return (
    <Suspense fallback={<FeatureLoader feature={name} />}>
      <Component />
    </Suspense>
  );
}

export default function App() {
  return (
    <AppShell>
      <Sidebar
        links={[
          { to: '/', label: 'Dashboard', prefetch: Dashboard },
          { to: '/analytics', label: 'Analytics', prefetch: Analytics },
```

```
          { to: '/reports', label: 'Reports', prefetch: Reports },
          { to: '/team', label: 'Team', prefetch: TeamManagement },
          { to: '/billing', label: 'Billing', prefetch: BillingSettings },
        ]}
      />
      <main>
        <Routes>
          <Route
            path="/"
            element={<LazyRoute component={Dashboard} name="Dashboard" />}
          />
          <Route
            path="/analytics"
            element={<LazyRoute component={Analytics} name="Analytics" />}
          />
          <Route
            path="/reports"
            element={<LazyRoute component={Reports} name="Reports" />}
          />
          <Route
            path="/team"
            element={<LazyRoute component={TeamManagement} name="Team" />}
          />
          <Route
            path="/billing"
            element={<LazyRoute component={BillingSettings} name="Billing" />}
          />
        </Routes>
      </main>
    </AppShell>
  );
}

// Bundle output:
//   main.js         - 120 KB (AppShell + Sidebar + Router)
//   analytics.js    - 280 KB (Chart.js + analytics components)
//   reports.js      - 350 KB (PDF generation + report templates)
//   team.js         - 80 KB  (team management UI)
//   billing.js      - 95 KB  (Stripe integration + forms)
//   integrations.js - 60 KB  (webhook config UI)
//
// Total: 985 KB, but user downloads only ~200 KB initially
```

When to Use / When NOT to Use

Use Code Splitting When

- Your bundle exceeds 200 KB (gzipped)

- You have features only some users access (admin panels)
- You have heavy dependencies used on specific pages (chart libraries, PDF generators)
- Your app has many routes
- Initial load time is a priority

Do NOT Use Code Splitting When

- Your entire app is small (under 50 KB gzipped)
- The overhead of loading spinners would hurt UX more than the bundle size
- You are building a single-page tool with no navigation
- Every feature is used on every page load

Common Mistakes

Mistake 1: Splitting Too Aggressively

```jsx
// WRONG: Splitting a tiny 2KB component
const Button = lazy(() => import('./Button'));
// The chunk loading overhead (HTTP request, parsing)
// is MORE expensive than just including 2KB in the main bundle.

// RIGHT: Only split substantial components (50KB+)
const ChartDashboard = lazy(() => import('./ChartDashboard'));
// This component pulls in chart.js (400KB). Worth splitting!
```

Mistake 2: No Fallback or Poor Fallback

```jsx
// WRONG: No Suspense wrapper
const Page = lazy(() => import('./Page'));
function App() {
  return <Page />; // CRASHES! "A component suspended while rendering,
                   //  but no Suspense boundary was found"
}

// WRONG: Generic, unhelpful fallback
<Suspense fallback={<div>...</div>}>
  <Page />
</Suspense>
```

```
// RIGHT: Contextual skeleton fallback
<Suspense fallback={<PageSkeleton />}>
  <Page />
</Suspense>
```

Mistake 3: No Error Handling for Failed Chunks

```jsx
// Network can fail! Chunks might not load.
// WRONG: No error handling
const Page = lazy(() => import('./Page'));
// If network fails, user sees blank screen or crash

// RIGHT: Combine with Error Boundary (Chapter 29)
import { ErrorBoundary } from 'react-error-boundary';

function ChunkErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div>
      <p>Failed to load this section.</p>
      <button onClick={resetErrorBoundary}>Try Again</button>
    </div>
  );
}

function App() {
  return (
    <ErrorBoundary FallbackComponent={ChunkErrorFallback}>
      <Suspense fallback={<PageSkeleton />}>
        <Page />
      </Suspense>
    </ErrorBoundary>
  );
}
```

Mistake 4: Not Handling Stale Chunks After Deployment

```
Problem: User has tab open for hours. You deploy new code.
Old chunk URLs no longer exist on the server.
User navigates to a lazy-loaded route --> 404 on chunk file!

Solution: Catch chunk load errors and prompt refresh.
```

```jsx
// Retry logic for chunk loading failures
function lazyWithRetry(importFn) {
```

```
  return lazy(() =>
    importFn().catch(error => {
      // If chunk fails to load (likely stale deployment)
      console.error('Chunk load failed:', error);

      // Option 1: Retry once
      return importFn().catch(() => {
        // Option 2: Force reload on second failure
        window.location.reload();
        // Return a dummy component to prevent crash
        return { default: () => null };
      });
    })
  );
}

const Products = lazyWithRetry(() => import('./pages/Products'));
```

Best Practices

1. Start with route-based splitting -- It gives the biggest wins with the least effort.
2. Analyze before splitting -- Use bundle analysis tools to find the biggest chunks first.
3. Set a bundle budget -- Aim for under 150 KB gzipped for the initial bundle.
4. Prefetch likely next pages -- Use hover/focus prefetching for navigation links.
5. Use contextual loading states -- Skeleton screens that match the incoming content are better than generic spinners.
6. Handle errors gracefully -- Wrap lazy components in Error Boundaries for network failures.
7. Name your chunks -- Use `webpackChunkName` or Vite's `manualChunks` for readable chunk names in DevTools.
8. Test on slow connections -- Throttle your network in DevTools to see what lazy loading actually feels like.
9. Measure the impact -- Compare Lighthouse scores before and after code splitting.

Quick Summary

Code splitting breaks your JavaScript bundle into smaller chunks that load on demand. `React.lazy` and `Suspense` make this easy at the component level. Route-based splitting is the

most impactful starting point. Dynamic `import()` lets you load any module on demand, not just React components. Prefetching on hover or with Intersection Observer makes lazy loading feel instant. Bundle analysis tools help you find the biggest optimization opportunities. Always handle chunk load errors gracefully and provide meaningful loading states.

Key Points

- Bundle size matters -- Large bundles directly increase time-to-interactive.
- React.lazy + Suspense -- The standard way to lazy load React components.
- Route-based splitting -- Split at page boundaries for maximum impact with minimal effort.
- dynamic import() -- JavaScript feature that loads modules on demand (not React-specific).
- Prefetching -- Load chunks before the user needs them (on hover, on visible, idle time).
- Bundle analysis -- Identify what is large, what is unused, what should be split.
- Error handling -- Chunks can fail to load. Always have a recovery strategy.
- Do not over-split -- Splitting tiny components adds overhead without benefit.

Practice Questions

1. A developer uses `React.lazy` for every single component in their app, including small 1 KB utility components. What problems will this cause, and what is a better approach?
2. Explain the difference between `webpackPrefetch` and `webpackPreload`. When would you use each one?
3. Your app's main bundle is 1.2 MB. After bundle analysis, you find that 400 KB comes from `moment.js` (used everywhere) and 300 KB comes from `chart.js` (used only on the analytics page). What is your optimization strategy for each?
4. A user reports that navigating between pages in your app shows a loading spinner for 2-3 seconds. The lazy-loaded chunks are only 50 KB each. What could be causing this, and how would you fix it?
5. After deploying a new version of your app, some users report blank screens when navigating. What is happening, and how do you prevent it?

Exercises

Exercise 1: Split an Existing App

Take a React app (or create a simple one with 4-5 routes). Measure the initial bundle size. Then apply route-based code splitting with `React.lazy` and `Suspense`. Measure again. Document the before and after bundle sizes and loading times.

Exercise 2: Build a Prefetching System

Create a `PrefetchLink` component that wraps React Router's `Link`. It should prefetch the target page's chunk on hover (with a 100ms debounce to avoid prefetching on quick mouse movements). Test it by adding artificial delay to your chunks and comparing the perceived load time.

Exercise 3: Bundle Analysis Report

Run a bundle analysis on a project (yours or an open-source one). Create a report listing:

- The 5 largest dependencies and their sizes
- Which of those could be replaced with smaller alternatives
- Which components should be lazy loaded
- Estimated savings from each optimization

What Is Next?

Code splitting helps you send less JavaScript to the browser. But even with a smaller bundle, rendering thousands of items (like a long product list or data table) can bring the browser to its knees. In Chapter 27, we will explore the Virtualization Pattern -- a technique that renders only the items currently visible on screen, making lists of any size feel fast.

Chapter 27: The Virtualization Pattern

What You Will Learn

- Why rendering thousands of DOM elements kills performance
- How virtualization works (the "window" concept)
- The math behind window calculation
- How to use `react-window` for list and grid virtualization
- Combining infinite scroll with virtualization
- Variable-height item handling
- When virtualization is overkill

Why This Chapter Matters

Open your email inbox. You might have 10,000 emails, but Gmail does not create 10,000 DOM elements. Open Twitter. You might scroll through hundreds of tweets, but your browser is not rendering them all at once. Open a spreadsheet with 100,000 rows. Your browser would crash if it tried to render every cell.

These applications all use the same trick: virtualization. They only render the items you can actually see on screen, plus a small buffer above and below. As you scroll, items that leave the viewport are destroyed, and new items entering the viewport are created.

Without virtualization, a list of 10,000 items creates 10,000 DOM nodes. Each one consumes memory, triggers layout calculations, and slows down every interaction. With virtualization, that same list might only have 20 DOM nodes at any time, recycled as you scroll.

The difference is not subtle. It is the difference between an app that freezes and an app that feels butter-smooth.

The Problem: Too Many DOM Nodes

What Happens Without Virtualization

```jsx
// Rendering 10,000 items naively
function ProductList({ products }) {
  return (
    <div className="product-list">
      {products.map(product => (
        <div key={product.id} className="product-card">
          <img src={product.image} alt={product.name} />
          <h3>{product.name}</h3>
          <p>${product.price}</p>
          <button>Add to Cart</button>
        </div>
      ))}
    </div>
  );
}

// With 10,000 products:
// - Creates 10,000 <div> containers
// - Creates 10,000 <img> elements (all start loading!)
// - Creates 10,000 <h3> elements
// - Creates 10,000 <p> elements
// - Creates 10,000 <button> elements
// = 50,000+ DOM nodes
//
// Result:
// - Initial render: 3-5 seconds
// - Memory usage: 500+ MB
// - Scrolling: janky, laggy
// - Page becomes unresponsive
```

```
Without Virtualization:

Viewport (what user sees):
+------------------------+
|  Item 45               |  <-- Visible
|  Item 46               |  <-- Visible
|  Item 47               |  <-- Visible
|  Item 48               |  <-- Visible
|  Item 49               |  <-- Visible
+------------------------+

But the DOM contains ALL items:
  Item 1     <-- Rendered but invisible (above viewport)
```

```
  Item 2     <-- Rendered but invisible
  ...
  Item 44    <-- Rendered but invisible
  Item 45    <-- VISIBLE
  Item 46    <-- VISIBLE
  Item 47    <-- VISIBLE
  Item 48    <-- VISIBLE
  Item 49    <-- VISIBLE
  Item 50    <-- Rendered but invisible (below viewport)
  ...
  Item 9,999 <-- Rendered but invisible
  Item 10,000<-- Rendered but invisible

DOM nodes: 10,000 (but only 5 are visible!)
```

The Solution: Only Render What Is Visible

How Virtualization Works

```
With Virtualization:

                     Overscan (buffer above)
                     +-------------------------+
                     |  Item 43                |  <-- In DOM but not visible
                     |  Item 44                |  <-- In DOM but not visible
                     +-------------------------+
Viewport:            +-------------------------+
                     |  Item 45                |  <-- Visible
                     |  Item 46                |  <-- Visible
                     |  Item 47                |  <-- Visible
                     |  Item 48                |  <-- Visible
                     |  Item 49                |  <-- Visible
                     +-------------------------+
                     +-------------------------+
                     |  Item 50                |  <-- In DOM but not visible
                     |  Item 51                |  <-- In DOM but not visible
                     +-------------------------+
                     Overscan (buffer below)

DOM nodes: 9 (instead of 10,000!)
Items 1-42 and 52-10,000 do NOT exist in the DOM.

The container div has a calculated total height to
maintain correct scrollbar behavior.
```

The Window Calculation

```
Key Variables:
  containerHeight = 500px    (visible area height)
  itemHeight      = 50px     (each item's height)
  scrollTop       = 2200px   (how far user has scrolled)
  totalItems      = 10,000
  overscan        = 2        (extra items above and below)

Calculations:
  startIndex = floor(scrollTop / itemHeight)
             = floor(2200 / 50)
             = 44

  visibleCount = ceil(containerHeight / itemHeight)
               = ceil(500 / 50)
               = 10

  endIndex = startIndex + visibleCount
           = 44 + 10
           = 54

  With overscan:
    renderStart = max(0, startIndex - overscan)
                = max(0, 44 - 2)
                = 42

    renderEnd = min(totalItems - 1, endIndex + overscan)
              = min(9999, 54 + 2)
              = 56

  Items rendered: 42 through 56 = 15 items
  Total height of container: 10,000 * 50 = 500,000px
  (This makes the scrollbar look correct)

  Each rendered item is positioned absolutely:
    Item 42: top = 42 * 50 = 2100px
    Item 43: top = 43 * 50 = 2150px
    ...and so on
```

Building a Simple Virtual List from Scratch

Understanding the concept by building it ourselves:

```jsx
import { useState, useRef, useCallback } from 'react';

function SimpleVirtualList({ items, itemHeight, containerHeight }) {
  const [scrollTop, setScrollTop] = useState(0);
  const containerRef = useRef(null);

  // Calculate which items are visible
  const startIndex = Math.max(
    0,
    Math.floor(scrollTop / itemHeight) - 2 // 2 items overscan
  );
  const endIndex = Math.min(
    items.length,
    Math.ceil((scrollTop + containerHeight) / itemHeight) + 2
  );

  // Total height to make scrollbar work correctly
  const totalHeight = items.length * itemHeight;

  // Only the visible items (plus overscan)
  const visibleItems = items.slice(startIndex, endIndex);

  const handleScroll = useCallback((e) => {
    setScrollTop(e.target.scrollTop);
  }, []);

  return (
    <div
      ref={containerRef}
      onScroll={handleScroll}
      style={{
        height: containerHeight,
        overflow: 'auto',
        position: 'relative',
      }}
    >
      {/* Spacer div creates correct total scroll height */}
      <div style={{ height: totalHeight }}>
        {visibleItems.map((item, index) => (
          <div
            key={startIndex + index}
            style={{
              position: 'absolute',
              top: (startIndex + index) * itemHeight,
              height: itemHeight,
              width: '100%',
            }}
          >
            {item.name} - ${item.price}
          </div>
        ))}
      </div>
    </div>
  );
```

```
}

// Usage:
function App() {
  // Generate 10,000 items
  const items = Array.from({ length: 10000 }, (_, i) => ({
    id: i,
    name: `Product ${i + 1}`,
    price: (Math.random() * 100).toFixed(2),
  }));

  return (
    <SimpleVirtualList
      items={items}
      itemHeight={50}
      containerHeight={500}
    />
  );
}

// Output:
// A scrollable list showing ~14 items at a time
// Scrollbar indicates 10,000 items
// Only ~14 DOM nodes exist at any moment
// Scrolling is smooth because React only updates ~14 elements
```

Using react-window

Building virtualization from scratch is educational, but for production use a battle-tested library. `react-window` is the standard choice.

Fixed-Size List

```jsx
import { FixedSizeList } from 'react-window';

// Row component receives index and style
function ProductRow({ index, style, data }) {
  const product = data[index];

  return (
    <div style={style} className="product-row">
      <span className="product-name">{product.name}</span>
      <span className="product-price">${product.price}</span>
      <button className="add-btn">Add to Cart</button>
    </div>
```

```
  );
}

function ProductList({ products }) {
  return (
    <FixedSizeList
      height={600}          // Viewport height
      width="100%"           // Full width
      itemCount={products.length}  // Total items
      itemSize={60}          // Each row height in pixels
      itemData={products}    // Passed to each Row as 'data' prop
      overscanCount={5}      // Render 5 extra items above/below
    >
      {ProductRow}
    </FixedSizeList>
  );
}

// Output:
// A 600px tall scrollable area
// Shows ~10 product rows at a time
// Smooth scrolling through all 10,000 products
// Only ~20 DOM nodes (10 visible + 5 overscan top + 5 bottom)
```

Fixed-Size Grid

```jsx
import { FixedSizeGrid } from 'react-window';

function ProductCard({ columnIndex, rowIndex, style, data }) {
  const index = rowIndex * data.columnCount + columnIndex;

  if (index >= data.items.length) {
    return <div style={style} />; // Empty cell
  }

  const product = data.items[index];

  return (
    <div style={style} className="product-card">
      <div className="card-inner">
        <img
          src={product.thumbnail}
          alt={product.name}
          loading="lazy"
        />
        <h3>{product.name}</h3>
        <p>${product.price}</p>
      </div>
    </div>
  );
```

```
}

function ProductGrid({ products }) {
  const columnCount = 4;
  const rowCount = Math.ceil(products.length / columnCount);

  return (
    <FixedSizeGrid
      height={800}
      width={1200}
      columnCount={columnCount}
      columnWidth={300}
      rowCount={rowCount}
      rowHeight={350}
      itemData={{ items: products, columnCount }}
      overscanRowCount={2}
      overscanColumnCount={1}
    >
      {ProductCard}
    </FixedSizeGrid>
  );
}

// Output:
// A 4-column grid of product cards
// Each card is 300x350 pixels
// Only visible cards (plus overscan) exist in the DOM
// Scrolling through 2,500 rows (10,000 products / 4 columns)
```

Variable-Size List

When items have different heights:

```jsx
import { VariableSizeList } from 'react-window';
import { useRef, useCallback } from 'react';

function MessageList({ messages }) {
  const listRef = useRef(null);

  // Calculate height for each message based on content
  const getItemSize = useCallback(
    (index) => {
      const message = messages[index];
      const baseHeight = 60; // padding + metadata
      const textLines = Math.ceil(message.text.length / 50);
      const textHeight = textLines * 20;
      const imageHeight = message.hasImage ? 200 : 0;
      return baseHeight + textHeight + imageHeight;
    },
    [messages]
```

```
  );

  return (
    <VariableSizeList
      ref={listRef}
      height={600}
      width="100%"
      itemCount={messages.length}
      itemSize={getItemSize}    // Function instead of fixed number
      estimatedItemSize={100}   // Helps with scrollbar accuracy
      overscanCount={3}
    >
      {({ index, style }) => {
        const message = messages[index];
        return (
          <div style={style} className="message">
            <div className="message-header">
              <strong>{message.author}</strong>
              <span>{message.time}</span>
            </div>
            <p>{message.text}</p>
            {message.hasImage && (
              <img src={message.imageUrl} alt="attachment" />
            )}
          </div>
        );
      }}
    </VariableSizeList>
  );
}

// Output:
// Chat-like message list
// Each message has different height based on text length
// Long messages take more space, short ones take less
// Still only renders visible messages
```

Infinite Scroll + Virtualization

The real power comes from combining infinite scroll (loading more data as you scroll) with virtualization (only rendering visible items).

```jsx
import { FixedSizeList } from 'react-window';
import InfiniteLoader from 'react-window-infinite-loader';
import { useState, useCallback, useRef } from 'react';
```

```
function InfiniteProductList() {
  const [products, setProducts] = useState([]);
  const [hasMore, setHasMore] = useState(true);
  const [isLoading, setIsLoading] = useState(false);
  const pageRef = useRef(1);

  // Total count: loaded items + 1 placeholder for "loading" row
  const itemCount = hasMore ? products.length + 1 : products.length;

  // Check if a specific item has been loaded
  const isItemLoaded = useCallback(
    (index) => !hasMore || index < products.length,
    [hasMore, products.length]
  );

  // Load more items when user scrolls near the end
  const loadMoreItems = useCallback(async () => {
    if (isLoading) return;
    setIsLoading(true);

    const response = await fetch(
      `/api/products?page=${pageRef.current}&limit=50`
    );
    const newProducts = await response.json();

    if (newProducts.length < 50) {
      setHasMore(false);
    }

    setProducts(prev => [...prev, ...newProducts]);
    pageRef.current += 1;
    setIsLoading(false);
  }, [isLoading]);

  // Row renderer
  const Row = useCallback(
    ({ index, style }) => {
      if (!isItemLoaded(index)) {
        return (
          <div style={style} className="loading-row">
            Loading more products...
          </div>
        );
      }

      const product = products[index];
      return (
        <div style={style} className="product-row">
          <span>{product.name}</span>
          <span>${product.price}</span>
        </div>
      );
    },
    [products, isItemLoaded]
  );
```

```
  return (
    <InfiniteLoader
      isItemLoaded={isItemLoaded}
      itemCount={itemCount}
      loadMoreItems={loadMoreItems}
      threshold={10}  // Start loading when 10 items from the end
    >
      {({ onItemsRendered, ref }) => (
        <FixedSizeList
          ref={ref}
          height={600}
          width="100%"
          itemCount={itemCount}
          itemSize={60}
          onItemsRendered={onItemsRendered}
          overscanCount={5}
        >
          {Row}
        </FixedSizeList>
      )}
    </InfiniteLoader>
  );
}

// Output behavior:
// 1. Initial load: fetches first 50 products, renders ~15 visible
// 2. User scrolls down to item 40
// 3. InfiniteLoader triggers loadMoreItems (threshold=10)
// 4. Next 50 products load while user keeps scrolling
// 5. Only ~25 DOM nodes exist at any time
// 6. User can scroll through 100,000+ products smoothly
```

```
Infinite Scroll + Virtualization Flow:

Products loaded:    [1-50]
DOM nodes:          [Items 1-15] (visible + overscan)
Scrollbar:          |||                                  (short)

User scrolls to item 40:
Products loaded:    [1-50]  --> triggers load --> [1-100]
DOM nodes:          [Items 35-55] (recycled)
Scrollbar:          ||||||||                             (growing)

User scrolls to item 90:
Products loaded:    [1-100] --> triggers load --> [1-150]
DOM nodes:          [Items 85-105] (recycled)
Scrollbar:          |||||||||||||                        (growing)

Key insight: DOM always has ~20 nodes regardless of total items!
```

Real-World Use Case: Data Table with 100K Rows

```jsx
import { FixedSizeList } from 'react-window';
import { useState, useMemo, useCallback } from 'react';

function VirtualDataTable({ data, columns }) {
  const [sortColumn, setSortColumn] = useState(null);
  const [sortDirection, setSortDirection] = useState('asc');
  const [filter, setFilter] = useState('');

  // Filter and sort data (memoized for performance)
  const processedData = useMemo(() => {
    let result = data;

    // Apply filter
    if (filter) {
      const lowerFilter = filter.toLowerCase();
      result = result.filter(row =>
        columns.some(col =>
          String(row[col.key]).toLowerCase().includes(lowerFilter)
        )
      );
    }

    // Apply sort
    if (sortColumn) {
      result = [...result].sort((a, b) => {
        const aVal = a[sortColumn];
        const bVal = b[sortColumn];
        const modifier = sortDirection === 'asc' ? 1 : -1;
        if (aVal < bVal) return -1 * modifier;
        if (aVal > bVal) return 1 * modifier;
        return 0;
      });
    }

    return result;
  }, [data, filter, sortColumn, sortDirection, columns]);

  const handleSort = useCallback((column) => {
    setSortColumn(prev => {
      if (prev === column) {
        setSortDirection(d => (d === 'asc' ? 'desc' : 'asc'));
        return column;
      }
      setSortDirection('asc');
      return column;
    });
  }, []);

  // Table header (not virtualized - always visible)
  const Header = () => (
```

```
    <div className="table-header">
      {columns.map(col => (
        <div
          key={col.key}
          className="header-cell"
          onClick={() => handleSort(col.key)}
          style={{ width: col.width }}
        >
          {col.label}
          {sortColumn === col.key && (
            <span>{sortDirection === 'asc' ? ' ^' : ' v'}</span>
          )}
        </div>
      ))}
    </div>
  );

  // Virtualized row
  const Row = useCallback(
    ({ index, style }) => {
      const row = processedData[index];
      return (
        <div style={style} className="table-row">
          {columns.map(col => (
            <div
              key={col.key}
              className="table-cell"
              style={{ width: col.width }}
            >
              {row[col.key]}
            </div>
          ))}
        </div>
      );
    },
    [processedData, columns]
  );

  return (
    <div className="virtual-table">
      <div className="table-toolbar">
        <input
          type="text"
          placeholder="Filter..."
          value={filter}
          onChange={e => setFilter(e.target.value)}
        />
        <span>{processedData.length.toLocaleString()} rows</span>
      </div>

      <Header />

      <FixedSizeList
        height={600}
        width="100%"
```

```
        itemCount={processedData.length}
        itemSize={40}
        overscanCount={10}
      >
        {Row}
      </FixedSizeList>
    </div>
  );
}

// Usage:
const columns = [
  { key: 'id', label: 'ID', width: 80 },
  { key: 'name', label: 'Name', width: 200 },
  { key: 'email', label: 'Email', width: 250 },
  { key: 'department', label: 'Department', width: 150 },
  { key: 'salary', label: 'Salary', width: 120 },
];

// 100,000 rows, but DOM only has ~25 at any time
<VirtualDataTable data={hundredThousandRows} columns={columns} />

// Output:
// +------+----------------+--------------------+-----------+--------+
// | ID   | Name           | Email              | Dept      | Salary |
// +------+----------------+--------------------+-----------+--------+
// | 1    | Alice Johnson  | alice@example.com  | Eng       | 95000  |
// | 2    | Bob Smith      | bob@example.com    | Sales     | 72000  |
// | ...  | ...            | ...                | ...       | ...    |
// +------+----------------+--------------------+-----------+--------+
// Filter, sort, and scroll through 100K rows instantly
```

When to Use / When NOT to Use

Use Virtualization When

- Rendering lists with 100+ items that cause visible lag
- Building data tables with thousands of rows
- Creating infinite-scroll feeds (social media, news)
- Displaying large datasets (logs, analytics, search results)
- Mobile apps where memory is constrained

Do NOT Use Virtualization When

- Lists have fewer than 50 items (the overhead is not worth it)

- All items must be in the DOM for accessibility or SEO (search engines cannot scroll)
- Items need to be measured by the browser for complex layouts (CSS Grid)
- You need `Ctrl+F` browser search to find text in the list
- Printing the page requires all items visible

Common Mistakes

Mistake 1: Creating Components Inside the Render Function

```jsx
// WRONG: Row component created on every render
function ProductList({ products }) {
  return (
    <FixedSizeList
      height={600}
      itemCount={products.length}
      itemSize={50}
    >
      {/* This creates a NEW function on every render */}
      {({ index, style }) => (
        <div style={style}>{products[index].name}</div>
      )}
    </FixedSizeList>
  );
  // Every scroll event re-creates ALL visible row components!
}

// RIGHT: Stable Row component defined outside or memoized
const Row = React.memo(({ index, style, data }) => (
  <div style={style}>{data[index].name}</div>
));

function ProductList({ products }) {
  return (
    <FixedSizeList
      height={600}
      itemCount={products.length}
      itemSize={50}
      itemData={products}
    >
      {Row}
    </FixedSizeList>
  );
}
```

Mistake 2: Forgetting the `style` Prop

```jsx
// WRONG: Not applying the style prop
function Row({ index, style, data }) {
  return (
    <div className="row">
      {/* Missing style prop! Items stack on top of each other */}
      {data[index].name}
    </div>
  );
}

// RIGHT: Always spread the style prop
function Row({ index, style, data }) {
  return (
    <div style={style} className="row">
      {data[index].name}
    </div>
  );
}
// The style prop contains position: absolute, top, left, width, height
// It is how react-window positions each item correctly
```

Mistake 3: Not Using `itemData` for External Data

```jsx
// WRONG: Closing over external data in the Row component
function ProductList({ products, onSelect }) {
  return (
    <FixedSizeList
      height={600}
      itemCount={products.length}
      itemSize={50}
    >
      {({ index, style }) => (
        <div style={style} onClick={() => onSelect(products[index])}>
          {products[index].name}
        </div>
      )}
    </FixedSizeList>
  );
  // products and onSelect changes cause ALL rows to re-render
}

// RIGHT: Pass through itemData
const Row = React.memo(({ index, style, data }) => (
  <div style={style} onClick={() => data.onSelect(data.products[index])}>
    {data.products[index].name}
  </div>
));

function ProductList({ products, onSelect }) {
```

```
  const itemData = useMemo(
    () => ({ products, onSelect }),
    [products, onSelect]
  );

  return (
    <FixedSizeList
      height={600}
      itemCount={products.length}
      itemSize={50}
      itemData={itemData}
    >
      {Row}
    </FixedSizeList>
  );
}
```

Best Practices

1. Measure first -- Profile your list with React DevTools or Chrome DevTools before adding virtualization. If 200 items render fine without it, do not add complexity.
2. Use `overscanCount` -- A value of 3-5 prevents blank flashes during fast scrolling. Too high defeats the purpose of virtualization.
3. Memoize row components -- Wrap row components in `React.memo` to prevent unnecessary re-renders.
4. Pass data through `itemData` -- Instead of closing over variables, use the `itemData` prop. It integrates with `react-window`'s optimization.
5. Use `estimatedItemSize` -- For `VariableSizeList`, provide a good estimate for accurate scrollbar sizing.
6. Lazy load images -- Add `loading="lazy"` to images inside virtualized rows. Even though the row is not visible initially, the image should still load on demand.
7. Handle window resize -- If your container is responsive, reset the list cache with `list.resetAfterIndex(0)` when dimensions change.
8. Test on low-end devices -- Virtualization benefits are most visible on slower devices. Test there.

Quick Summary

The Virtualization Pattern renders only the items currently visible in the viewport, plus a small buffer. This reduces the DOM from potentially tens of thousands of nodes to just a handful, dramatically improving memory usage and rendering performance. The core math involves calculating which items fall within the scroll window and positioning them absolutely within a container whose total height represents all items. Libraries like `react-window` handle the complexity for you. Combined with infinite scroll, virtualization makes it possible to display datasets of any size without performance degradation.

Key Points

- The problem: Rendering thousands of DOM nodes causes lag, high memory use, and jank.
- The solution: Only create DOM nodes for visible items plus a small overscan buffer.
- Window calculation: `startIndex = floor(scrollTop / itemHeight)`, render from there.
- react-window: Production library for list and grid virtualization in React.
- FixedSizeList: All items have the same height (simplest, most performant).
- VariableSizeList: Items have different heights (needs a size function).
- Infinite scroll: Combine with `react-window-infinite-loader` for paginated data.
- Overscan: Extra items above/below viewport prevent blank flashes during fast scroll.

Practice Questions

1. You have a list of 500 items. Each item is a simple text row. Should you use virtualization? What factors would influence your decision?
2. Explain why the total container height must equal `itemCount * itemHeight` even though most items are not in the DOM. What would happen if you did not set this height?
3. A developer reports that their virtualized list shows blank rows when scrolling quickly. What is likely causing this, and how would you fix it?
4. Why is it important to define the Row component outside the parent component (or memoize it with `useCallback`/`React.memo`) when using `react-window`?
5. Your virtualized list needs to support keyboard navigation where the user can press arrow keys to move selection up and down. What additional logic would you need to implement?

Exercises

Exercise 1: Build a Virtual List from Scratch

Without using any library, build a virtualized list component that:

- Accepts an array of items and an item height
- Calculates which items to render based on scroll position
- Positions items with absolute positioning
- Includes an overscan of 3 items above and below
- Handles the scroll event efficiently

Exercise 2: Product Catalog with react-window

Create a product catalog that displays 5,000 products in a virtualized grid (4 columns). Each product card should show an image, name, price, and an "Add to Cart" button. Implement filtering by name and sorting by price, ensuring the virtualized list updates correctly.

Exercise 3: Infinite Chat History

Build a chat application interface that loads message history as the user scrolls up (reverse infinite scroll). Use `react-window` with `VariableSizeList` since messages have different lengths. New messages should appear at the bottom and the scroll position should stay stable when older messages load above.

What Is Next?

Virtualization handles the problem of too many items. But even with few items, your components might re-render too often, wasting CPU cycles recalculating values that have not changed. In Chapter 28, we will explore the Memoization Pattern -- using `React.memo`, `useMemo`, and `useCallback` to skip unnecessary work and keep your app responsive.

Chapter 28: The Memoization Pattern

What You Will Learn

- What memoization is and how it works conceptually
- How `React.memo` prevents unnecessary component re-renders
- When and how to use `useMemo` for expensive calculations
- When and how to use `useCallback` for stable function references
- Why premature memoization can make things worse
- How to profile first and memoize second
- The real cost of memoization

Why This Chapter Matters

Memoization is one of the most misunderstood concepts in React. Some developers sprinkle `React.memo`, `useMemo`, and `useCallback` everywhere, thinking they are "optimizing" their code. Others avoid them entirely because they heard they are unnecessary.

Both extremes are wrong.

Think of memoization like insulation in a house. In a cold climate, insulation saves you money on heating. In a tropical climate, it is a waste. And even in a cold climate, you insulate the walls, not the mailbox.

The key insight is: memoization is medicine, not vitamins. You should not take it preventatively. You should diagnose a performance problem first, then apply memoization where it actually helps.

This chapter teaches you how to diagnose, when to apply, and where each memoization tool fits.

What Is Memoization?

Memoization is caching the result of a function based on its inputs. If the inputs have not changed, return the cached result instead of recalculating.

```
Without Memoization:

  calculate(5, 10) --> runs computation --> returns 50    (100ms)
  calculate(5, 10) --> runs computation --> returns 50    (100ms)
  calculate(5, 10) --> runs computation --> returns 50    (100ms)
  Total: 300ms for the same result three times

With Memoization:

  calculate(5, 10) --> runs computation --> caches (5,10)=50 --> returns 50 (100ms)
  calculate(5, 10) --> cache hit! --> returns 50                            (0.01ms)
  calculate(5, 10) --> cache hit! --> returns 50                            (0.01ms)
  Total: ~100ms. Saved ~200ms.

  calculate(5, 20) --> new inputs --> runs computation --> caches --> returns 100
```

Pure JavaScript Memoization

```jsx
// A simple memoization function
function memoize(fn) {
  const cache = new Map();

  return function (...args) {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      console.log('Cache hit!');
      return cache.get(key);
    }

    console.log('Computing...');
    const result = fn(...args);
    cache.set(key, result);
    return result;
  };
}

// Example: Expensive Fibonacci calculation
```

```
function fibonacci(n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}

const memoizedFib = memoize(fibonacci);

memoizedFib(40); // Computing... (takes ~1 second)
                 // Output: 102334155

memoizedFib(40); // Cache hit! (instant)
                 // Output: 102334155
```

React.memo: Preventing Component Re-Renders

The Problem

In React, when a parent component re-renders, all its children re-render too, even if their props have not changed.

```jsx
function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>
        Count: {count}
      </button>

      {/* This re-renders EVERY TIME count changes */}
      {/* Even though it receives NO props related to count */}
      <ExpensiveChart data={staticData} />
    </div>
  );
}

function ExpensiveChart({ data }) {
  console.log('ExpensiveChart rendered'); // Logs on EVERY click!
  // Imagine this does complex SVG calculations
  return <svg>{/* complex chart */}</svg>;
}

// Output when clicking the button 3 times:
// "ExpensiveChart rendered"  (initial)
// "ExpensiveChart rendered"  (click 1 - unnecessary!)
```

```
// "ExpensiveChart rendered"  (click 2 - unnecessary!)
// "ExpensiveChart rendered"  (click 3 - unnecessary!)
```

The Solution: React.memo

```jsx
// Wrap the component with React.memo
const ExpensiveChart = React.memo(function ExpensiveChart({ data }) {
  console.log('ExpensiveChart rendered');
  return <svg>{/* complex chart */}</svg>;
});

function App() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>
        Count: {count}
      </button>

      {/* Now this only re-renders when `data` actually changes */}
      <ExpensiveChart data={staticData} />
    </div>
  );
}

// Output when clicking the button 3 times:
// "ExpensiveChart rendered"  (initial render only!)
// (no more logs - React.memo prevents re-renders)
```

How React.memo Works

```
Without React.memo:
  Parent renders --> React renders ALL children
  Parent renders --> React renders ALL children (again)

With React.memo:
  Parent renders --> React checks: did props change?
                     YES --> render child
                     NO  --> skip child, reuse last result

  React.memo does a SHALLOW comparison of props:
    { data: [1,2,3] } === { data: [1,2,3] }  --> FALSE (different reference)
    { data: sameArray } === { data: sameArray } --> TRUE (same reference)
    { name: "Alice" } === { name: "Alice" } --> TRUE (same primitive value)
```

Custom Comparison Function

```jsx
// Default: React.memo uses shallow equality (Object.is for each prop)
const MemoizedComponent = React.memo(MyComponent);

// Custom: You can provide your own comparison function
const MemoizedComponent = React.memo(MyComponent, (prevProps, nextProps) => {
  // Return TRUE if props are EQUAL (skip re-render)
  // Return FALSE if props are DIFFERENT (re-render)

  // Only re-render if the `id` or `name` changed
  // Ignore changes to `lastUpdated` timestamp
  return (
    prevProps.id === nextProps.id &&
    prevProps.name === nextProps.name
  );
});

// Usage example:
function UserCard({ id, name, lastUpdated, onlineStatus }) {
  console.log(`UserCard ${name} rendered`);
  return (
    <div>
      <h3>{name}</h3>
      <span>{onlineStatus}</span>
    </div>
  );
}

const MemoizedUserCard = React.memo(UserCard, (prev, next) => {
  // Only re-render when name or onlineStatus changes
  // Skip re-render for lastUpdated changes (cosmetic, not visual)
  return (
    prev.name === next.name &&
    prev.onlineStatus === next.onlineStatus
  );
});
```

---

useMemo: Caching Expensive Calculations

The Problem

Some calculations are expensive. If their inputs have not changed, running them again is wasted work.

```jsx
function ProductList({ products, filterText }) {
  // This runs on EVERY render, even if products and filterText
  // have not changed (e.g., parent re-rendered for unrelated reason)
  const filteredProducts = products
    .filter(p => p.name.toLowerCase().includes(filterText.toLowerCase()))
    .sort((a, b) => a.price - b.price)
    .map(p => ({
      ...p,
      discountedPrice: calculateComplexDiscount(p),
    }));

  return (
    <ul>
      {filteredProducts.map(p => (
        <li key={p.id}>{p.name} - ${p.discountedPrice}</li>
      ))}
    </ul>
  );
}
// With 10,000 products, this filtering + sorting + discount calculation
// might take 50-100ms. Running it 60 times per second is a problem.
```

The Solution: useMemo

```jsx
function ProductList({ products, filterText }) {
  // Only recalculates when products OR filterText changes
  const filteredProducts = useMemo(() => {
    console.log('Recalculating filtered products...');
    return products
      .filter(p =>
        p.name.toLowerCase().includes(filterText.toLowerCase())
      )
      .sort((a, b) => a.price - b.price)
      .map(p => ({
        ...p,
        discountedPrice: calculateComplexDiscount(p),
      }));
  }, [products, filterText]); // Dependency array

  return (
    <ul>
      {filteredProducts.map(p => (
        <li key={p.id}>{p.name} - ${p.discountedPrice}</li>
      ))}
    </ul>
  );
}

// Output:
// First render: "Recalculating filtered products..."
```

```
// Parent re-renders (unrelated state change): (no log - cached!)
// filterText changes: "Recalculating filtered products..."
// products array changes: "Recalculating filtered products..."
```

useMemo for Stable Object References

```jsx
// PROBLEM: New object created every render, breaks React.memo on child
function Parent() {
  const [count, setCount] = useState(0);

  // New object on EVERY render, even though values are the same
  const style = { color: 'blue', fontSize: 16 };

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>
        Count: {count}
      </button>
      {/* MemoizedChild re-renders because style is new object */}
      <MemoizedChild style={style} />
    </div>
  );
}

// FIX: Memoize the object
function Parent() {
  const [count, setCount] = useState(0);

  // Same object reference unless values change
  const style = useMemo(
    () => ({ color: 'blue', fontSize: 16 }),
    [] // No dependencies = never recalculated
  );

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>
        Count: {count}
      </button>
      {/* MemoizedChild skips re-render because style reference is stable */}
      <MemoizedChild style={style} />
    </div>
  );
}
```

useCallback: Stable Function References

The Problem

Functions defined inside a component are recreated on every render. This breaks `React.memo` on children that receive those functions as props.

```jsx
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

  // NEW function created on every render
  const handleDelete = (id) => {
    setTodos(prev => prev.filter(t => t.id !== id));
  };

  // NEW function created on every render
  const handleToggle = (id) => {
    setTodos(prev =>
      prev.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  };

  return (
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} />
      {todos.map(todo => (
        // Even with React.memo on TodoItem, it re-renders
        // because handleDelete and handleToggle are new functions
        <TodoItem
          key={todo.id}
          todo={todo}
          onDelete={handleDelete}
          onToggle={handleToggle}
        />
      ))}
    </div>
  );
}

const TodoItem = React.memo(function TodoItem({ todo, onDelete, onToggle }) {
  console.log(`Rendering todo: ${todo.text}`);
  return (
    <div>
      <input
        type="checkbox"
        checked={todo.done}
        onChange={() => onToggle(todo.id)}
      />
      <span>{todo.text}</span>
      <button onClick={() => onDelete(todo.id)}>Delete</button>
    </div>
  );
});
```

```
// Output when typing in input (which changes `input` state):
// "Rendering todo: Buy groceries"    <-- unnecessary!
// "Rendering todo: Walk the dog"     <-- unnecessary!
// "Rendering todo: Read book"        <-- unnecessary!
// All todos re-render because function props are new references
```

The Solution: useCallback

```jsx
function TodoList() {
  const [todos, setTodos] = useState([]);
  const [input, setInput] = useState('');

  // Stable function reference - only changes when dependencies change
  const handleDelete = useCallback((id) => {
    setTodos(prev => prev.filter(t => t.id !== id));
  }, []); // No dependencies needed - uses functional updater

  const handleToggle = useCallback((id) => {
    setTodos(prev =>
      prev.map(t => (t.id === id ? { ...t, done: !t.done } : t))
    );
  }, []); // No dependencies needed - uses functional updater

  return (
    <div>
      <input value={input} onChange={e => setInput(e.target.value)} />
      {todos.map(todo => (
        <TodoItem
          key={todo.id}
          todo={todo}
          onDelete={handleDelete}   // Same reference every render!
          onToggle={handleToggle}   // Same reference every render!
        />
      ))}
    </div>
  );
}

// Output when typing in input:
// (nothing logged - React.memo works because function refs are stable!)
```

When Memoization Helps vs. When It Hurts

Profile First, Memoize Second

```
Step 1: Notice a performance issue (laggy typing, janky scrolling)
Step 2: Open React DevTools Profiler
Step 3: Record the interaction
Step 4: Find which components re-render unnecessarily
Step 5: Apply memoization to THOSE specific components
Step 6: Profile again to verify improvement

Do NOT do this:
Step 1: "I heard memoization makes things faster"
Step 2: Add React.memo to every component
Step 3: Add useMemo and useCallback to everything
Step 4: Wonder why the code is harder to read and not faster
```

The Cost of Memoization

```
React.memo cost:
  + Shallow comparison of all props on every render
  + Memory to store the previous render result
  + Added code complexity

  Worth it when:
    Component is expensive to render (complex DOM, calculations)
    Component renders often with the same props
    Component is deep in the tree with many expensive children

  NOT worth it when:
    Component is simple (a few divs and text)
    Props almost always change (comparison cost wasted)
    Component is at the top of the tree (re-renders are rare)

useMemo cost:
  + Dependency array comparison on every render
  + Memory to store the cached value
  + Added code complexity

  Worth it when:
    Calculation takes measurable time (>1ms)
    Result is passed to React.memo'd children as a prop
    Calculation runs frequently with same inputs

  NOT worth it when:
    Calculation is trivial (simple math, string concat)
    Dependencies change on every render (never hits cache)
    Value is only used in this component (no child optimization)

useCallback cost:
  + Dependency array comparison on every render
  + Memory for the closure
  + Added code complexity

  Worth it when:
```

```
    Function is passed to React.memo'd children
    Function is a dependency of useEffect or other hooks
    Function is used in a virtualized list with many items

  NOT worth it when:
    Function is passed to native HTML elements (<button onClick>)
    No child uses React.memo
    Dependencies change every render
```

Examples of Premature Optimization

```jsx
// BAD: Memoizing trivial values
function UserProfile({ user }) {
  // This addition takes nanoseconds. useMemo overhead > savings.
  const age = useMemo(
    () => new Date().getFullYear() - user.birthYear,
    [user.birthYear]
  );

  // This string concat is instant. useMemo adds overhead.
  const fullName = useMemo(
    () => `${user.firstName} ${user.lastName}`,
    [user.firstName, user.lastName]
  );

  return (
    <div>
      <h1>{fullName}</h1>
      <p>Age: {age}</p>
    </div>
  );
}

// GOOD: Just compute it directly
function UserProfile({ user }) {
  const age = new Date().getFullYear() - user.birthYear;
  const fullName = `${user.firstName} ${user.lastName}`;

  return (
    <div>
      <h1>{fullName}</h1>
      <p>Age: {age}</p>
    </div>
  );
}
```

```jsx
// BAD: useCallback for native element handlers
function SearchBox() {
```

```
  const [query, setQuery] = useState('');

  // Wrapping in useCallback is pointless here
  // <input> is a native element, not a React.memo'd component
  const handleChange = useCallback((e) => {
    setQuery(e.target.value);
  }, []);

  return <input value={query} onChange={handleChange} />;
}

// GOOD: Just use a plain function
function SearchBox() {
  const [query, setQuery] = useState('');
  return (
    <input
      value={query}
      onChange={(e) => setQuery(e.target.value)}
    />
  );
}
```

Profiling with React DevTools

Step-by-Step Profiling

```
1. Install React DevTools browser extension
2. Open your app and go to the "Profiler" tab
3. Click "Start profiling" (record button)
4. Perform the slow interaction (type, scroll, click)
5. Click "Stop profiling"
6. Examine the results

What to look for:
+-------------------------------------------------------------------+
| Profiler Results                                                   |
+-------------------------------------------------------------------+
| Component          | Renders | Time    | Why                       |
+-------------------------------------------------------------------+
| App                | 5       | 2ms     | State changed             |
| Header             | 5       | 0.5ms   | Parent rendered           | <-- candidate
| ProductList        | 5       | 45ms    | Parent rendered           | <-- problem!
|   ProductCard (x50)| 250     | 40ms    | Parent rendered           | <-- problem!
| Footer             | 5       | 0.3ms   | Parent rendered           | <-- candidate
+-------------------------------------------------------------------+

Analysis:
```

```
- ProductList renders 5 times even though products did not change
- Each render causes 50 ProductCards to re-render
- Total wasted: ~200ms of rendering for nothing

Solution:
1. Wrap ProductList with React.memo
2. Wrap ProductCard with React.memo
3. Use useCallback for onAddToCart passed from App to ProductList
```

Highlight Updates in React DevTools

```
In React DevTools Settings:
  Check "Highlight updates when components render"

Now every re-render flashes a colored border:
  Green  = fast render (< 16ms)
  Yellow = moderate render (16-100ms)
  Red    = slow render (> 100ms)

When typing in a search box, you should see:
  - Search input: green flash (expected)
  - Product cards: NO flash (they should not re-render!)

If product cards flash, they are re-rendering unnecessarily.
```

Real-World Use Case: Dashboard with Multiple Widgets

```jsx
import React, { useState, useMemo, useCallback, memo } from 'react';

// Expensive chart component
const SalesChart = memo(function SalesChart({ data, timeRange }) {
  console.log('SalesChart rendered');

  // Expensive: processes thousands of data points
  const chartData = useMemo(() => {
    return data
      .filter(d => d.date >= timeRange.start && d.date <= timeRange.end)
      .reduce((acc, d) => {
        const month = d.date.substring(0, 7);
        acc[month] = (acc[month] || 0) + d.amount;
```

```
        return acc;
      }, {});
  }, [data, timeRange]);

  return (
    <div className="chart">
      {Object.entries(chartData).map(([month, total]) => (
        <div key={month} className="bar" style={{ height: total / 100 }}>
          {month}: ${total}
        </div>
      ))}
    </div>
  );
});

// Expensive table component
const TopProducts = memo(function TopProducts({ products, onSelect }) {
  console.log('TopProducts rendered');

  const sortedProducts = useMemo(
    () => [...products].sort((a, b) => b.revenue - a.revenue).slice(0, 10),
    [products]
  );

  return (
    <table>
      <thead>
        <tr><th>Product</th><th>Revenue</th></tr>
      </thead>
      <tbody>
        {sortedProducts.map(p => (
          <tr key={p.id} onClick={() => onSelect(p.id)}>
            <td>{p.name}</td>
            <td>${p.revenue.toLocaleString()}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
});

// Simple stat card - NOT memoized (too cheap to bother)
function StatCard({ label, value }) {
  return (
    <div className="stat-card">
      <h4>{label}</h4>
      <p className="stat-value">{value}</p>
    </div>
  );
}

// Dashboard orchestrator
function Dashboard() {
  const [salesData] = useState(generateSalesData);     // 50K records
  const [products] = useState(generateProductData);     // 1000 products
```

```
  const [timeRange, setTimeRange] = useState(defaultRange);
  const [selectedProduct, setSelectedProduct] = useState(null);
  const [sidebarOpen, setSidebarOpen] = useState(true);

  // Stable callback for child components
  const handleProductSelect = useCallback((id) => {
    setSelectedProduct(id);
  }, []);

  // Memoize time range to avoid breaking SalesChart memo
  const stableTimeRange = useMemo(
    () => timeRange,
    [timeRange.start, timeRange.end]
  );

  // Cheap calculations - no need for useMemo
  const totalRevenue = salesData.reduce((sum, d) => sum + d.amount, 0);
  const avgOrder = totalRevenue / salesData.length;

  return (
    <div className="dashboard">
      <button onClick={() => setSidebarOpen(s => !s)}>
        Toggle Sidebar
      </button>

      {/* These are cheap - re-render is fine */}
      <StatCard label="Total Revenue" value={`$${totalRevenue}`} />
      <StatCard label="Average Order" value={`$${avgOrder.toFixed(2)}`} />

      {/* These are expensive - memo prevents re-render when sidebar toggles */}
      <SalesChart data={salesData} timeRange={stableTimeRange} />
      <TopProducts products={products} onSelect={handleProductSelect} />
    </div>
  );
}

// Output when toggling sidebar:
// (No "SalesChart rendered" or "TopProducts rendered" logs)
// Only the sidebar and stat cards re-render. Dashboard stays fast.
//
// Output when changing time range:
// "SalesChart rendered" (expected - timeRange changed)
// (No "TopProducts rendered" - products did not change)
```

When to Use / When NOT to Use

Use React.memo When

- Component re-renders frequently with the same props

- Component is expensive to render (complex DOM, calculations)
- Component has many children that also re-render
- Component is in a list rendered by a virtualized container

Do NOT Use React.memo When

- Component is simple and renders fast anyway
- Props change on almost every render (comparison cost is wasted)
- Component is the root or near-root (re-renders are infrequent)

Use useMemo When

- Calculation takes measurable time (filter/sort/transform large arrays)
- Result is used as a prop for a memoized child component
- Result is used as a dependency in another hook
- Creating a new object/array that breaks child memoization

Do NOT Use useMemo When

- Calculation is trivial (basic math, simple string operations)
- Dependencies change on every render (cache never hits)
- Value is only used locally in simple rendering

Use useCallback When

- Function is passed to a React.memo'd child component
- Function is in a dependency array of useEffect/useMemo
- Function is passed to many items in a virtualized list

Do NOT Use useCallback When

- Function is passed only to native HTML elements
- No child component uses React.memo
- Function dependencies change every render

Common Mistakes

Mistake 1: Memoizing Everything

```jsx
// WRONG: Memoizing trivial values and simple components
const Title = React.memo(({ text }) => <h1>{text}</h1>);

function App() {
  const title = useMemo(() => 'Hello World', []);
  const handleClick = useCallback(() => console.log('click'), []);

  return (
    <div>
      <Title text={title} />
      <button onClick={handleClick}>Click</button>
    </div>
  );
}
// All this memoization adds complexity for ZERO performance gain.
// <h1> rendering is essentially free.
```

Mistake 2: Missing Dependencies

```jsx
// WRONG: Stale closure - count will always be 0
const handleClick = useCallback(() => {
  console.log(count); // Always logs 0!
  setCount(count + 1); // Always sets to 1!
}, []); // Missing 'count' in dependencies

// RIGHT: Use functional updater to avoid dependency
const handleClick = useCallback(() => {
  setCount(prev => prev + 1); // Always correct!
}, []); // No dependencies needed with functional updater

// OR include the dependency
const handleClick = useCallback(() => {
  console.log(count);
  setCount(count + 1);
}, [count]); // But this changes on every count update
```

Mistake 3: React.memo with Inline Objects/Functions

```jsx
// WRONG: React.memo is useless here because props are new every render
const MemoizedChild = React.memo(ChildComponent);

function Parent() {
  return (
```

```
    <MemoizedChild
      style={{ color: 'red' }}          // New object every render!
      onClick={() => console.log('hi')} // New function every render!
      config={{ theme: 'dark' }}        // New object every render!
    />
  );
  // React.memo compares props, but EVERY prop is a new reference.
  // The memo wrapper does comparison work + renders anyway. Worse than no memo!
}

// RIGHT: Stabilize the props
function Parent() {
  const style = useMemo(() => ({ color: 'red' }), []);
  const handleClick = useCallback(() => console.log('hi'), []);
  const config = useMemo(() => ({ theme: 'dark' }), []);

  return (
    <MemoizedChild
      style={style}
      onClick={handleClick}
      config={config}
    />
  );
}
```

Best Practices

1. Profile before memoizing -- Use React DevTools Profiler to find actual bottlenecks. Do not guess.
2. Start without memoization -- Write your components normally first. Add memoization only when profiling shows a problem.
3. Memoize expensive components, not cheap ones -- A component that renders a complex chart is worth memoizing. A component that renders `<h1>` is not.
4. Use functional state updaters with useCallback -- `setCount(prev => prev + 1)` avoids the need to include `count` in dependencies.
5. Keep dependency arrays accurate -- Never lie about dependencies. Use the ESLint `exhaustive-deps` rule.
6. Consider component structure -- Sometimes restructuring components (lifting state up or pushing it down) eliminates the need for memoization entirely.
7. Measure the impact -- After adding memoization, profile again. If there is no measurable improvement, remove it. Simpler code is better code.

8. Memoize at the right level -- If a parent passes props to 50 children, memoize the children (not each individual prop calculation).

Quick Summary

Memoization in React comes in three forms: `React.memo` (skip re-rendering a component if props have not changed), `useMemo` (cache an expensive calculation between renders), and `useCallback` (cache a function reference between renders). All three exist to avoid unnecessary work. But they are not free -- they add memory overhead, comparison costs, and code complexity. The correct approach is to profile first, identify actual performance bottlenecks, then apply memoization surgically to the specific components and calculations that benefit from it. Premature memoization makes code harder to read without improving performance.

Key Points

- React.memo: Wraps a component to skip re-render when props are unchanged (shallow comparison).
- useMemo: Caches a computed value; recalculates only when dependencies change.
- useCallback: Caches a function reference; creates a new function only when dependencies change.
- Profile first: Use React DevTools Profiler to find real performance problems before optimizing.
- Premature optimization hurts: Memoization adds overhead. Only apply it where measured performance gains exist.
- Shallow comparison: React.memo and useMemo compare by reference, not deep equality.
- Stable references matter: Inline objects and functions break React.memo on children.
- Functional updaters: `setState(prev => ...)` avoids stale closures and extra dependencies.

Practice Questions

1. A developer wraps every component in their app with `React.memo`. What are the potential downsides of this approach?
2. You have a component that receives a `user` object as a prop. The parent recreates this object on every render (`{name: 'Alice', age: 30}`), even though the values do not change. How would you fix this to work with `React.memo`?
3. Explain the difference between `useMemo` and `useCallback`. Could you implement `useCallback` using `useMemo`? Show how.
4. A search component filters a list of 100 items on every keystroke. A developer wraps the filter logic in `useMemo`. Is this optimization worth it? Why or why not?
5. Your React DevTools Profiler shows that a `DataTable` component re-renders 20 times in 3 seconds, each taking 50ms. The props do not change between renders. What steps would you take to fix this?

Exercises

Exercise 1: Find the Bottleneck

Create a dashboard with 4 widgets: a counter button, a list of 1000 items, a chart (simulate with a loop), and a text input. Without any memoization, type in the input and observe the lag. Use React DevTools Profiler to identify which widgets re-render unnecessarily. Then apply targeted memoization and measure the improvement.

Exercise 2: Memoization Audit

Take an existing React project (yours or open-source). Search for all uses of `React.memo`, `useMemo`, and `useCallback`. For each one, determine whether it is:

- Necessary and effective
- Unnecessary (premature optimization)
- Broken (dependencies are wrong or inline objects break the memo)

Exercise 3: Refactor vs. Memoize

Build a parent component with a text input and a child that displays an expensive list. First, fix the unnecessary re-renders using memoization. Then, refactor the component structure to achieve the same result WITHOUT any memoization (hint: push the state down to a sibling

component). Compare the two approaches.

What Is Next?

Memoization prevents wasted rendering work. But what happens when something goes truly wrong -- a JavaScript error that crashes your component? Without proper handling, one error can take down your entire application. In Chapter 29, we will learn the Error Boundary Pattern -- how to catch errors at component boundaries, display graceful fallback UIs, and recover without losing the entire page.

Chapter 29: The Error Boundary Pattern

What You Will Learn

- Why JavaScript errors in one component can crash your entire React app
- How Error Boundaries catch errors at component boundaries
- How to build fallback UIs that keep the app usable
- Strategic placement of Error Boundaries in your component tree
- Recovery and retry patterns
- Using the `react-error-boundary` library for production apps
- Error logging and monitoring integration

Why This Chapter Matters

Imagine you are browsing an e-commerce site. You click on a product. The product's review section has a bug -- maybe a reviewer left a malformed emoji that breaks the text parser. Without error handling, the entire page crashes. The product image, the price, the "Add to Cart" button -- everything gone. You see a blank white screen.

Now imagine the same scenario with Error Boundaries. The review section shows a friendly message: "Could not load reviews. Click to retry." But the product image, price, and "Add to Cart" button all work perfectly. You can still buy the product.

Error Boundaries are the difference between "our site crashed" and "one small section had an issue." In the real world, errors happen. APIs return unexpected data. Third-party scripts fail. Edge cases slip through testing. The question is not whether errors will happen, but how gracefully your application handles them.

The Problem: One Error Crashes Everything

Without Error Boundaries

```jsx
function App() {
  return (
    <div>
      <Header />
      <ProductDetails />
      <ReviewSection />    {/* BUG: crashes here */}
      <Recommendations />
      <Footer />
    </div>
  );
}

function ReviewSection() {
  const reviews = getReviews();
  // Oops! reviews is null because API failed
  return (
    <div>
      <h2>Reviews ({reviews.length})</h2>  {/* TypeError: null.length */}
      {reviews.map(r => <Review key={r.id} {...r} />)}
    </div>
  );
}
```

```
What the user sees WITHOUT Error Boundaries:

+----------------------------------------------------+
|                                                    |
|              BLANK WHITE SCREEN                    |
|                                                    |
|     (or React's error overlay in development)      |
|                                                    |
|     The ENTIRE app is gone.                        |
|     Header? Gone. Product? Gone. Footer? Gone.     |
|     All because of one null reference in reviews.|
|                                                    |
+----------------------------------------------------+
```

```
What the user sees WITH Error Boundaries:

+----------------------------------------------------+
|  [Logo]  Home  Products  Cart                      |  <-- Header works!
+----------------------------------------------------+
|  Product Name                     $49.99           |  <-- Product works!
|  [Product Image]                  [Add to Cart]    |
|                                                    |
```

```
+-----------------------------------------------------+
|  +-----------------------------------------------+|
|  |  Could not load reviews.                      ||  <-- Graceful fallback
|  |  [Try Again]                                  ||
|  +-----------------------------------------------+|
+-----------------------------------------------------+
|  You might also like:                             |  <-- Recommendations work!
|  [Item 1] [Item 2] [Item 3]                      |
+-----------------------------------------------------+
|  Footer  |  Contact  |  Terms                     |  <-- Footer works!
+-----------------------------------------------------+
```

How Error Boundaries Work

Error Boundaries are React components that catch JavaScript errors in their child component tree, log those errors, and display a fallback UI instead of crashing the entire app.

The Class Component Approach

Error Boundaries MUST be class components. This is one of the few remaining cases where React requires a class component (as of React 18, there is no hook equivalent).

```jsx
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  // Called when a child throws an error during rendering
  static getDerivedStateFromError(error) {
    // Update state so next render shows fallback UI
    return { hasError: true, error };
  }

  // Called after an error is caught - for logging
  componentDidCatch(error, errorInfo) {
    // Log to error reporting service
    console.error('Error caught by boundary:', error);
    console.error('Component stack:', errorInfo.componentStack);

    // In production, send to monitoring service:
    // logErrorToService(error, errorInfo);
  }
```

```
  render() {
    if (this.state.hasError) {
      // Render fallback UI
      return (
        <div className="error-fallback">
          <h2>Something went wrong</h2>
          <p>{this.state.error?.message}</p>
          <button
            onClick={() => this.setState({ hasError: false, error: null })}
          >
            Try Again
          </button>
        </div>
      );
    }

    // No error: render children normally
    return this.props.children;
  }
}

// Usage:
function App() {
  return (
    <div>
      <Header />
      <ErrorBoundary>
        <ProductDetails />
      </ErrorBoundary>
      <ErrorBoundary>
        <ReviewSection />
      </ErrorBoundary>
      <Footer />
    </div>
  );
}

// If ReviewSection crashes:
// - ErrorBoundary catches the error
// - Shows "Something went wrong" ONLY in the review area
// - Header, ProductDetails, and Footer keep working
```

What Errors Are Caught (and What Are NOT)

```
Error Boundaries CATCH:
  + Errors during rendering (return statement)
  + Errors in lifecycle methods
  + Errors in constructors of child components

Error Boundaries DO NOT CATCH:
  - Event handler errors (use try/catch instead)
```

```
  - Async errors (promises, setTimeout)
  - Server-side rendering errors
  - Errors in the Error Boundary component itself

+--------------------------------+--------------------------------+
|        CAUGHT                  |        NOT CAUGHT              |
+--------------------------------+--------------------------------+
| function Child() {             | function Child() {             |
|   // Error during render       |   function handleClick() {     |
|   const x = null;              |     // Error in event handler  |
|   return <p>{x.name}</p>;      |     throw new Error('oops');   |
| }                              |   }                            |
|                                |   return (                     |
|                                |     <button onClick={          |
|                                |       handleClick}>Click       |
|                                |     </button>                  |
|                                |   );                           |
|                                | }                              |
+--------------------------------+--------------------------------+
```

Handling Event Handler Errors

```jsx
// Error Boundaries do NOT catch event handler errors.
// Use try/catch for those.

function DeleteButton({ itemId, onDelete }) {
  const [error, setError] = useState(null);

  async function handleClick() {
    try {
      await deleteItem(itemId);
      onDelete(itemId);
    } catch (err) {
      // Handle the error in component state
      setError(err.message);
      // Or log it
      logErrorToService(err);
    }
  }

  if (error) {
    return (
      <div className="error-inline">
        <p>Failed to delete: {error}</p>
        <button onClick={() => setError(null)}>Dismiss</button>
      </div>
    );
  }

  return <button onClick={handleClick}>Delete</button>;
}
```

Fallback UI Strategies

Strategy 1: Simple Error Message

```jsx
function SimpleErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert" className="error-simple">
      <p>Something went wrong.</p>
      <button onClick={resetErrorBoundary}>Try Again</button>
    </div>
  );
}
```

Strategy 2: Contextual Fallback

```jsx
// Different fallbacks for different parts of the app
function ReviewErrorFallback() {
  return (
    <div className="review-error">
      <p>Reviews could not be loaded right now.</p>
      <p>You can still purchase this product.</p>
    </div>
  );
}

function ChartErrorFallback() {
  return (
    <div className="chart-error">
      <div className="chart-placeholder">
        <span>Chart unavailable</span>
      </div>
      <p>Data visualization is temporarily unavailable.</p>
    </div>
  );
}

function SidebarErrorFallback() {
  return (
    <div className="sidebar-error">
      <p>Navigation error. <a href="/">Go to homepage</a></p>
    </div>
  );
}
```

Strategy 3: Full Error Details (Development)

```jsx
function DevelopmentErrorFallback({ error, errorInfo }) {
  if (process.env.NODE_ENV === 'production') {
    return <SimpleErrorFallback />;
  }

  return (
    <div className="error-dev" style={{
      padding: '20px',
      background: '#fff0f0',
      border: '2px solid red',
      borderRadius: '8px',
      margin: '10px',
    }}>
      <h3 style={{ color: 'red' }}>Development Error</h3>
      <pre style={{ fontSize: '14px', overflow: 'auto' }}>
        {error.message}
      </pre>
      <details>
        <summary>Component Stack</summary>
        <pre style={{ fontSize: '12px' }}>
          {errorInfo?.componentStack}
        </pre>
      </details>
    </div>
  );
}
```

Placement Strategies

Where you place Error Boundaries dramatically affects the user experience.

```
Strategy 1: Top-Level Only (minimal protection)

<App>
  <ErrorBoundary>        <-- Catches everything, but replaces EVERYTHING
    <Header />
    <Sidebar />
    <MainContent />
    <Footer />
  </ErrorBoundary>
</App>

Problem: Any error replaces the entire app with a fallback.
Use case: Last resort. Better than a white screen, but not great.
```

```
Strategy 2: Section-Level (recommended starting point)

<App>
  <ErrorBoundary>
    <Header />
  </ErrorBoundary>
  <div className="layout">
    <ErrorBoundary>
      <Sidebar />
    </ErrorBoundary>
    <ErrorBoundary>
      <MainContent />            <-- Error here only affects main content
    </ErrorBoundary>
  </div>
  <ErrorBoundary>
    <Footer />
  </ErrorBoundary>
</App>

Better: Error in sidebar does not break main content.


Strategy 3: Feature-Level (best user experience)

<App>
  <Header />
  <ProductPage>
    <ErrorBoundary fallback={<ImagePlaceholder />}>
      <ProductImages />
    </ErrorBoundary>
    <ProductInfo />             <-- No boundary needed (simple, unlikely to error)
    <ErrorBoundary fallback={<ReviewsUnavailable />}>
      <ReviewSection />
    </ErrorBoundary>
    <ErrorBoundary fallback={<RecsUnavailable />}>
      <Recommendations />
    </ErrorBoundary>
  </ProductPage>
  <Footer />
</App>

Best: Each feature fails independently. Users can still buy the product
even if reviews or recommendations break.
```

Recommended Approach: Layered Boundaries

```jsx
function App() {
  return (
```

```
    // Layer 1: App-level boundary (last resort)
    <ErrorBoundary fallback={<FullPageError />}>
      <Layout>
        {/* Layer 2: Page-level boundary */}
        <ErrorBoundary fallback={<PageError />}>
          <Routes>
            <Route path="/" element={<HomePage />} />
            <Route path="/products/:id" element={
              // Layer 3: Feature-level boundaries
              <ProductPage />
            } />
          </Routes>
        </ErrorBoundary>
      </Layout>
    </ErrorBoundary>
  );
}

function ProductPage({ productId }) {
  return (
    <div>
      <ProductInfo productId={productId} />

      <ErrorBoundary fallback={<p>Reviews unavailable</p>}>
        <ReviewSection productId={productId} />
      </ErrorBoundary>

      <ErrorBoundary fallback={<p>Recommendations unavailable</p>}>
        <Recommendations productId={productId} />
      </ErrorBoundary>
    </div>
  );
}
```

Recovery and Retry Patterns

Basic Retry with Key Reset

```jsx
class ErrorBoundaryWithRetry extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }

  static getDerivedStateFromError(error) {
```

```
    return { hasError: true, error };
  }

  componentDidCatch(error, errorInfo) {
    console.error('Boundary caught:', error, errorInfo);
  }

  handleRetry = () => {
    this.setState({ hasError: false, error: null });
  };

  render() {
    if (this.state.hasError) {
      return this.props.fallback({
        error: this.state.error,
        retry: this.handleRetry,
      });
    }
    return this.props.children;
  }
}

// Usage:
<ErrorBoundaryWithRetry
  fallback={({ error, retry }) => (
    <div className="error-panel">
      <h3>Something went wrong</h3>
      <p>{error.message}</p>
      <button onClick={retry}>Retry</button>
    </div>
  )}
>
  <ReviewSection />
</ErrorBoundaryWithRetry>

// When user clicks "Retry":
// 1. State resets to hasError: false
// 2. React attempts to render ReviewSection again
// 3. If the error was transient (network hiccup), it works
// 4. If the error persists, boundary catches it again
```

Retry with Automatic Fallback Escalation

```jsx
function RetryableSection({ children, maxRetries = 3 }) {
  const [retryCount, setRetryCount] = useState(0);

  // Changing the key forces React to remount the ErrorBoundary
  // and its children, giving them a fresh start
  const handleRetry = () => {
    setRetryCount(prev => prev + 1);
```

```
  };

  if (retryCount >= maxRetries) {
    return (
      <div className="max-retries">
        <p>This section is currently unavailable.</p>
        <p>Please try refreshing the page.</p>
        <button onClick={() => window.location.reload()}>
          Refresh Page
        </button>
      </div>
    );
  }

  return (
    <ErrorBoundaryWithRetry
      key={retryCount}  // Fresh boundary on each retry
      fallback={({ error, retry }) => (
        <div>
          <p>Error: {error.message}</p>
          <p>Attempt {retryCount + 1} of {maxRetries}</p>
          <button onClick={() => { handleRetry(); retry(); }}>
            Retry ({maxRetries - retryCount - 1} attempts left)
          </button>
        </div>
      )}
    >
      {children}
    </ErrorBoundaryWithRetry>
  );
}

// Usage:
<RetryableSection maxRetries={3}>
  <ReviewSection />
</RetryableSection>

// Behavior:
// Error occurs --> "Retry (2 attempts left)"
// User clicks --> Retries. If error again:
// "Retry (1 attempt left)"
// User clicks --> Retries. If error again:
// "Retry (0 attempts left)"
// User clicks --> "This section is currently unavailable. Refresh Page"
```

Using react-error-boundary Library

The `react-error-boundary` library provides a production-ready Error Boundary with built-in retry, reset, and logging features.

Installation and Basic Usage

```bash
npm install react-error-boundary
```

```jsx
import { ErrorBoundary } from 'react-error-boundary';

// Fallback component receives error and reset function
function ErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div role="alert" className="error-fallback">
      <h3>Something went wrong</h3>
      <pre>{error.message}</pre>
      <button onClick={resetErrorBoundary}>Try Again</button>
    </div>
  );
}

function App() {
  return (
    <ErrorBoundary
      FallbackComponent={ErrorFallback}
      onError={(error, errorInfo) => {
        // Log to monitoring service
        console.error('Logged error:', error, errorInfo);
      }}
      onReset={() => {
        // Reset any state that might have caused the error
        // e.g., clear a cache, reset a query
        console.log('Error boundary reset');
      }}
    >
      <Dashboard />
    </ErrorBoundary>
  );
}
```

Reset on Prop Change

```jsx
import { ErrorBoundary } from 'react-error-boundary';

function ProductPage() {
  const { productId } = useParams();

  return (
    <ErrorBoundary
      FallbackComponent={ErrorFallback}
      resetKeys={[productId]}
```

```
      // When productId changes, automatically reset the boundary
      // This is useful when navigating between products
    >
      <ProductDetails id={productId} />
      <Reviews productId={productId} />
    </ErrorBoundary>
  );
}

// Behavior:
// User visits /products/1 --> Reviews crash
// User sees error fallback
// User navigates to /products/2 --> productId changes
// ErrorBoundary auto-resets --> tries rendering Reviews for product 2
// If product 2's reviews work fine, user never sees the error again
```

useErrorBoundary Hook

```jsx
import { useErrorBoundary } from 'react-error-boundary';

function ReviewSection({ productId }) {
  const { showBoundary } = useErrorBoundary();
  const [reviews, setReviews] = useState([]);

  useEffect(() => {
    fetchReviews(productId)
      .then(setReviews)
      .catch(error => {
        // This catches ASYNC errors and sends them
        // to the nearest Error Boundary!
        showBoundary(error);
      });
  }, [productId, showBoundary]);

  return (
    <div>
      {reviews.map(r => (
        <div key={r.id}>
          <strong>{r.author}</strong>: {r.text}
        </div>
      ))}
    </div>
  );
}

// Now async errors (which Error Boundaries normally miss)
// are properly caught and display the fallback UI.

// Usage:
<ErrorBoundary FallbackComponent={ErrorFallback}>
```

```
  <ReviewSection productId={42} />
</ErrorBoundary>
```

Inline Fallback

```jsx
import { ErrorBoundary } from 'react-error-boundary';

function Dashboard() {
  return (
    <div className="dashboard-grid">
      <ErrorBoundary fallback={<div>Revenue chart unavailable</div>}>
        <RevenueChart />
      </ErrorBoundary>

      <ErrorBoundary fallback={<div>User stats unavailable</div>}>
        <UserStats />
      </ErrorBoundary>

      <ErrorBoundary
        fallback={<div>Activity feed unavailable</div>}
      >
        <ActivityFeed />
      </ErrorBoundary>
    </div>
  );
}

// Each widget fails independently.
// If RevenueChart crashes, UserStats and ActivityFeed keep working.
```

Error Boundary + Code Splitting

Error Boundaries pair perfectly with lazy-loaded components (Chapter 26):

```jsx
import { lazy, Suspense } from 'react';
import { ErrorBoundary } from 'react-error-boundary';

const AdminPanel = lazy(() => import('./AdminPanel'));

function AdminRoute() {
  return (
    <ErrorBoundary
      FallbackComponent={({ error, resetErrorBoundary }) => (
```

```
        <div>
          <h2>Failed to load Admin Panel</h2>
          <p>
            {error.message.includes('Loading chunk')
              ? 'Network error. Check your connection.'
              : error.message}
          </p>
          <button onClick={resetErrorBoundary}>Retry</button>
        </div>
      )}
    >
      <Suspense fallback={<div>Loading admin panel...</div>}>
        <AdminPanel />
      </Suspense>
    </ErrorBoundary>
  );
}

// Flow:
// 1. User navigates to /admin
// 2. Suspense shows "Loading admin panel..."
// 3a. Chunk loads successfully --> AdminPanel renders
// 3b. Network fails --> ErrorBoundary catches --> shows retry button
// 4. User clicks retry --> ErrorBoundary resets --> Suspense tries again
```

Real-World Use Case: Error Monitoring Integration

```jsx
import { ErrorBoundary } from 'react-error-boundary';

// Error logging service (e.g., Sentry, LogRocket, Datadog)
function logError(error, errorInfo) {
  // In production, send to your error monitoring service
  if (process.env.NODE_ENV === 'production') {
    // Example: Sentry
    // Sentry.captureException(error, {
    //   extra: { componentStack: errorInfo.componentStack }
    // });

    // Example: Custom API
    fetch('/api/errors', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        message: error.message,
        stack: error.stack,
        componentStack: errorInfo.componentStack,
```

```
        url: window.location.href,
        timestamp: new Date().toISOString(),
        userAgent: navigator.userAgent,
      }),
    }).catch(() => {
      // Silently fail - do not crash the error handler!
    });
  } else {
    console.error('Error Boundary caught:', error);
    console.error('Component stack:', errorInfo.componentStack);
  }
}

function AppErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="app-error" role="alert">
      <h1>Something went wrong</h1>
      <p>
        We have been notified and are looking into it.
        Please try again or contact support if the problem persists.
      </p>
      <div className="error-actions">
        <button onClick={resetErrorBoundary}>
          Try Again
        </button>
        <button onClick={() => window.location.href = '/'}>
          Go to Homepage
        </button>
      </div>
      {process.env.NODE_ENV !== 'production' && (
        <details>
          <summary>Error Details</summary>
          <pre>{error.message}</pre>
          <pre>{error.stack}</pre>
        </details>
      )}
    </div>
  );
}

function App() {
  return (
    <ErrorBoundary
      FallbackComponent={AppErrorFallback}
      onError={logError}
      onReset={() => {
        // Clear any cached state that might cause the error again
        // queryClient.clear(); // React Query
        // store.dispatch(resetState()); // Redux
      }}
    >
      <Router>
        <Layout>
          <ErrorBoundary
            FallbackComponent={PageErrorFallback}
```

```
            onError={logError}
          >
            <AppRoutes />
          </ErrorBoundary>
        </Layout>
      </Router>
    </ErrorBoundary>
  );
}
```

When to Use / When NOT to Use

Use Error Boundaries When

- Wrapping route-level page components
- Around third-party components you do not control
- Around data-dependent sections (API responses, user-generated content)
- Around complex components with many edge cases
- With lazy-loaded components (Suspense + ErrorBoundary)
- At the app root as a last-resort catch-all

Do NOT Use Error Boundaries When

- For event handler errors (use try/catch)
- For async errors in useEffect (use try/catch + state, or `useErrorBoundary`)
- Around simple, static components that are unlikely to error
- As a replacement for proper input validation
- As a substitute for fixing the actual bug

Common Mistakes

Mistake 1: Only Using One Top-Level Boundary

```jsx
// WRONG: One boundary replaces the ENTIRE app on any error
function App() {
  return (
    <ErrorBoundary fallback={<h1>App crashed</h1>}>
      <Header />
      <Sidebar />
      <MainContent />
      <Footer />
    </ErrorBoundary>
  );
}
// A bug in Sidebar kills Header, MainContent, and Footer too!

// RIGHT: Granular boundaries
function App() {
  return (
    <ErrorBoundary fallback={<h1>App crashed</h1>}>
      <Header />
      <div className="layout">
        <ErrorBoundary fallback={<p>Sidebar error</p>}>
          <Sidebar />
        </ErrorBoundary>
        <ErrorBoundary fallback={<p>Content error</p>}>
          <MainContent />
        </ErrorBoundary>
      </div>
      <Footer />
    </ErrorBoundary>
  );
}
```

Mistake 2: Assuming Async Errors Are Caught

```jsx
// WRONG: Error Boundary will NOT catch this
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(setUser);
    // If fetch fails, error is swallowed! No boundary catches it.
  }, [userId]);

  return <div>{user?.name}</div>;
}

// RIGHT: Use useErrorBoundary hook or state-based error handling
import { useErrorBoundary } from 'react-error-boundary';
```

```
function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const { showBoundary } = useErrorBoundary();

  useEffect(() => {
    fetch(`/api/users/${userId}`)
      .then(res => {
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        return res.json();
      })
      .then(setUser)
      .catch(showBoundary); // Sends error to nearest ErrorBoundary
  }, [userId, showBoundary]);

  return <div>{user?.name}</div>;
}
```

Mistake 3: Not Providing a Reset Mechanism

```jsx
// WRONG: User is stuck on the error screen forever
<ErrorBoundary fallback={<p>Error occurred</p>}>
  <Widget />
</ErrorBoundary>
// The user has no way to recover!

// RIGHT: Always provide a way to retry or navigate away
<ErrorBoundary
  FallbackComponent={({ resetErrorBoundary }) => (
    <div>
      <p>Something went wrong</p>
      <button onClick={resetErrorBoundary}>Retry</button>
      <a href="/">Go Home</a>
    </div>
  )}
>
  <Widget />
</ErrorBoundary>
```

Best Practices

1. Layer your boundaries -- Use a top-level boundary as a last resort, section-level boundaries for major areas, and feature-level boundaries for independent widgets.
2. Match fallback UI to context -- A review section error should show "Reviews unavailable," not a generic "Something went wrong."

3. Always provide recovery -- Give users a "Retry" button, a link to the homepage, or a way to refresh.
4. Log errors to a monitoring service -- Use the `onError` callback to send errors to Sentry, Datadog, LogRocket, or a custom endpoint.
5. Use `resetKeys` for navigation -- When route params change, auto-reset the boundary so stale errors do not persist.
6. Pair with Suspense -- Error Boundaries and Suspense work together for lazy-loaded components: Suspense handles the loading state, Error Boundary handles failures.
7. Handle async errors explicitly -- Use `useErrorBoundary` from `react-error-boundary` to send async errors to the nearest boundary.
8. Test your error boundaries -- Intentionally throw errors in development to verify fallback UIs look correct and recovery works.

Quick Summary

Error Boundaries are React components that catch JavaScript errors anywhere in their child component tree. They prevent one broken component from crashing the entire application. Built with `getDerivedStateFromError` and `componentDidCatch` lifecycle methods (class components only), they display a fallback UI while the rest of the app continues working. The `react-error-boundary` library provides a production-ready implementation with retry, reset, and async error support. Strategic placement at app, page, and feature levels provides layered protection. Always pair Error Boundaries with error logging, meaningful fallback UIs, and recovery mechanisms.

Key Points

- Error Boundaries catch render errors in child components, not event handlers or async code.
- Class components only -- React has no hook equivalent for Error Boundaries (use `react-error-boundary` library).
- Granular placement -- More boundaries = more resilient UI. Each section fails independently.
- Fallback UI should be contextual -- Tell users what failed and what they can still do.
- Recovery matters -- Always provide retry, navigation, or refresh options.

- react-error-boundary -- Production library with `resetKeys`, `useErrorBoundary`, and `onError` logging.
- Pair with Suspense -- Handle both loading and error states for lazy-loaded components.
- Async errors need special handling -- Use `useErrorBoundary` hook or try/catch with state.

Practice Questions

1. Why must Error Boundaries be class components? What would a hypothetical `useErrorBoundary` hook need to do to work?
2. A developer puts one Error Boundary at the root of their app and considers error handling "done." What are the problems with this approach?
3. You have a dashboard with 6 independent widgets. One widget crashes. With no Error Boundaries, what happens? With one boundary around all widgets? With one boundary per widget?
4. An Error Boundary does not catch an error thrown inside a `useEffect` callback. Why? How would you handle this using the `react-error-boundary` library?
5. Your app uses code splitting with `React.lazy`. A user on a flaky network connection cannot download a chunk. How would you combine Error Boundaries and Suspense to handle this gracefully?

Exercises

Exercise 1: Build an Error Boundary from Scratch

Create a reusable Error Boundary class component that:

- Catches errors and shows a customizable fallback
- Provides a retry button that resets the error state
- Logs errors to the console with the component stack trace
- Accepts a `fallback` prop (either a React element or a render function)

Test it by creating a component that intentionally throws an error.

Exercise 2: Granular Error Boundaries

Build a dashboard page with 4 widgets (Stats, Chart, Activity Feed, Notifications). Each widget should have its own Error Boundary with a contextual fallback. Add a "Chaos Mode" button that randomly breaks one widget. Verify that the other 3 keep working when one fails.

Exercise 3: Async Error Recovery

Using `react-error-boundary`, build a data-fetching component that:

- Fetches data from a simulated API (use `setTimeout` + `Math.random()` for failures)
- Sends fetch errors to the nearest Error Boundary using `useErrorBoundary`
- Shows a fallback with "Retry" button on failure
- Automatically resets when the fetch URL changes (use `resetKeys`)
- Limits retries to 3 attempts, then shows a "Contact Support" message

---

What Is Next?

Error Boundaries protect individual applications from crashing. But what happens when your application grows so large that one team cannot manage it all? In Chapter 30, we will explore Micro-Frontends -- an architectural pattern where independent teams build, deploy, and maintain separate pieces of a frontend application, composing them into a seamless whole.

Chapter 30: Micro-Frontends

What You Will Learn

- What micro-frontends are and why large organizations adopt them
- How Module Federation enables sharing code between independently deployed apps
- Using Web Components for framework-agnostic composition
- Routing-based composition strategies
- Communication patterns between micro-frontends
- The real trade-offs: what you gain and what you pay
- When micro-frontends are the right (and wrong) choice

Why This Chapter Matters

You work at a growing company. Two years ago, your React app was built and maintained by a single team of five developers. Now the company has fifty frontend developers organized into eight teams: checkout, product catalog, search, user accounts, analytics dashboard, marketing pages, admin tools, and design system.

All fifty developers push code to the same repository. A bug in the checkout flow blocks a deploy of new search features. The analytics team wants to use a charting library that conflicts with one the product team uses. Every pull request touches shared code, causing merge conflicts. Deploys take 45 minutes because the entire monolith must be built and tested.

Micro-frontends solve this problem by splitting the monolithic frontend into independent applications, each owned and deployed by a separate team. The checkout team ships checkout. The search team ships search. They compose into one seamless experience for the user.

Think of it like a shopping mall. Each store (micro-frontend) is independently owned, decorated, and staffed. But to the shopper (user), it feels like one cohesive experience. The

mall (shell application) provides the shared infrastructure: hallways, parking, security.

What Are Micro-Frontends?

The Core Idea

```
Monolithic Frontend:
+-------------------------------------------------------------------+
|                    One Application                                |
|                    One Repository                                 |
|                    One Deploy Pipeline                            |
|                    One Team (or many teams stepping on each other)|
|                                                                   |
|  [Header] [Search] [Products] [Cart] [Checkout] [Account]       |
|  [Admin]  [Analytics] [Marketing]                                 |
+-------------------------------------------------------------------+

Micro-Frontend Architecture:
+-------------------------------------------------------------------+
|  Shell Application (routing, shared layout, authentication)      |
+-------------------------------------------------------------------+
|                    |                    |                        |
|  +--------------+  |  +--------------+ |  +--------------+       |
|  | Search MFE   |  |  | Product MFE  | |  | Cart MFE     |       |
|  | Team: Search |  |  | Team: Catalog| |  | Team: Checkout|      |
|  | Deploy: Own  |  |  | Deploy: Own  | |  | Deploy: Own  |       |
|  | React 18     |  |  | React 18     | |  | React 18     |       |
|  +--------------+  |  +--------------+ |  +--------------+       |
|                    |                    |                        |
|  +--------------+  |  +--------------+ |  +--------------+       |
|  | Account MFE  |  |  | Admin MFE    | |  | Analytics MFE|       |
|  | Team: Users  |  |  | Team: Admin  | |  | Team: Data   |       |
|  | Deploy: Own  |  |  | Deploy: Own  | |  | Deploy: Own  |       |
|  | React 18     |  |  | Vue 3        | |  | React 18     |       |
|  +--------------+  |  +--------------+ |  +--------------+       |
+-------------------------------------------------------------------+

Each micro-frontend:
  - Has its own repository
  - Has its own CI/CD pipeline
  - Can be deployed independently
  - Is owned by one team
  - Can even use a different framework (but usually does not)
```

Composition Approach 1: Module Federation (Webpack 5)

Module Federation lets independently built and deployed applications share code at runtime. One application can dynamically load components from another application that is running on a different server.

How Module Federation Works

```
Build Time:
  App A builds and deploys to https://products.example.com
  App B builds and deploys to https://cart.example.com
  Shell builds and deploys to https://www.example.com

Runtime:
  User visits https://www.example.com
  Shell loads
  Shell needs ProductList --> fetches module from products.example.com
  Shell needs CartWidget  --> fetches module from cart.example.com

+-------------------+         +-------------------+
|  Shell App        |         | Products App      |
|  (Host)           |  <---   | (Remote)          |
|                   |  loads  |                   |
|  Uses:            |         |  Exposes:         |
|  - ProductList    |         |  - ProductList    |
|  - CartWidget     |         |  - ProductDetail  |
+-------------------+         +-------------------+
         |
         |  loads
         v
+-------------------+
| Cart App          |
| (Remote)          |
|                   |
|  Exposes:         |
|  - CartWidget     |
|  - CartPage       |
+-------------------+
```

Setting Up Module Federation

Remote App (Products Team):

```js
// products-app/webpack.config.js
const { ModuleFederationPlugin } = require('webpack').container;

module.exports = {
  // ...other webpack config
  plugins: [
    new ModuleFederationPlugin({
      name: 'productsApp',          // Unique name for this app
      filename: 'remoteEntry.js',    // Entry point file for consumers

      // Components this app shares with others
      exposes: {
        './ProductList': './src/components/ProductList',
        './ProductDetail': './src/components/ProductDetail',
      },

      // Shared dependencies (avoid loading React twice)
      shared: {
        react: { singleton: true, requiredVersion: '^18.0.0' },
        'react-dom': { singleton: true, requiredVersion: '^18.0.0' },
      },
    }),
  ],
};
```

```jsx
// products-app/src/components/ProductList.jsx
// This component is developed and tested independently
import { useState, useEffect } from 'react';

export default function ProductList({ onProductClick }) {
  const [products, setProducts] = useState([]);

  useEffect(() => {
    fetch('/api/products')
      .then(res => res.json())
      .then(setProducts);
  }, []);

  return (
    <div className="product-grid">
      {products.map(product => (
        <div
          key={product.id}
          className="product-card"
          onClick={() => onProductClick(product.id)}
        >
          <img src={product.image} alt={product.name} />
          <h3>{product.name}</h3>
          <p>${product.price}</p>
        </div>
      ))}
    </div>
```

```
  );
}
```

Shell App (Host):

```js
// shell-app/webpack.config.js
const { ModuleFederationPlugin } = require('webpack').container;

module.exports = {
  plugins: [
    new ModuleFederationPlugin({
      name: 'shell',

      // Remote apps to load from
      remotes: {
        productsApp: 'productsApp@https://products.example.com/remoteEntry.js',
        cartApp: 'cartApp@https://cart.example.com/remoteEntry.js',
      },

      shared: {
        react: { singleton: true, requiredVersion: '^18.0.0' },
        'react-dom': { singleton: true, requiredVersion: '^18.0.0' },
      },
    }),
  ],
};
```

```jsx
// shell-app/src/App.jsx
import React, { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { ErrorBoundary } from 'react-error-boundary';
import Header from './components/Header';

// Load components from remote apps at RUNTIME
const ProductList = lazy(() => import('productsApp/ProductList'));
const ProductDetail = lazy(() => import('productsApp/ProductDetail'));
const CartPage = lazy(() => import('cartApp/CartPage'));

function MicroFrontendError({ error, resetErrorBoundary }) {
  return (
    <div className="mfe-error">
      <p>This section is temporarily unavailable.</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

function App() {
  return (
    <BrowserRouter>
```

```
      <Header />
      <main>
        <ErrorBoundary FallbackComponent={MicroFrontendError}>
          <Suspense fallback={<div>Loading...</div>}>
            <Routes>
              <Route path="/" element={<ProductList />} />
              <Route path="/products/:id" element={<ProductDetail />} />
              <Route path="/cart" element={<CartPage />} />
            </Routes>
          </Suspense>
        </ErrorBoundary>
      </main>
    </BrowserRouter>
  );
}

export default App;

// How it works at runtime:
// 1. User visits https://www.example.com/
// 2. Shell app loads (Header + Router)
// 3. Route "/" matches --> React.lazy triggers
// 4. Browser fetches https://products.example.com/remoteEntry.js
// 5. ProductList component loads and renders
// 6. User navigates to /cart
// 7. Browser fetches https://cart.example.com/remoteEntry.js
// 8. CartPage component loads and renders
//
// Products team can deploy new ProductList without touching the shell!
```

Composition Approach 2: Web Components

Web Components are a browser-native way to create custom HTML elements. They work with any framework (or no framework), making them a natural boundary for micro-frontends.

Creating a Micro-Frontend as a Web Component

```jsx
// search-mfe/src/SearchWidget.jsx
import React, { useState } from 'react';

function SearchWidget() {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState([]);

  async function handleSearch(e) {
```

```
    e.preventDefault();
    const res = await fetch(`/api/search?q=${query}`);
    const data = await res.json();
    setResults(data);
  }

  return (
    <div className="search-widget">
      <form onSubmit={handleSearch}>
        <input
          type="text"
          value={query}
          onChange={e => setQuery(e.target.value)}
          placeholder="Search products..."
        />
        <button type="submit">Search</button>
      </form>
      <ul>
        {results.map(r => (
          <li key={r.id}>{r.name} - ${r.price}</li>
        ))}
      </ul>
    </div>
  );
}
```

```jsx
// search-mfe/src/web-component.jsx
// Wrap the React component as a Web Component
import React from 'react';
import ReactDOM from 'react-dom/client';
import SearchWidget from './SearchWidget';

class SearchWidgetElement extends HTMLElement {
  connectedCallback() {
    // Called when element is added to the DOM
    const mountPoint = document.createElement('div');
    this.attachShadow({ mode: 'open' }).appendChild(mountPoint);

    // Mount React inside the Shadow DOM
    this.root = ReactDOM.createRoot(mountPoint);
    this.root.render(<SearchWidget />);
  }

  disconnectedCallback() {
    // Called when element is removed from the DOM
    this.root.unmount();
  }
}

// Register the custom element
customElements.define('search-widget', SearchWidgetElement);
```

```
// Now ANY page can use it:
// <search-widget></search-widget>
// Works in React, Vue, Angular, or plain HTML!
```

Using Web Component Micro-Frontends

```html
<!-- Shell application (could be plain HTML, React, Vue, anything) -->
<!DOCTYPE html>
<html>
<head>
  <title>Our Store</title>
  <!-- Load micro-frontend scripts -->
  <script src="https://search.example.com/search-widget.js" async></script>
  <script src="https://cart.example.com/cart-icon.js" async></script>
</head>
<body>
  <header>
    <h1>Our Store</h1>
    <!-- Use micro-frontends as custom HTML elements -->
    <search-widget></search-widget>
    <cart-icon user-id="123"></cart-icon>
  </header>
  <main id="content">
    <!-- Main content loaded based on route -->
  </main>
</body>
</html>
```

```
Web Component Isolation:

+-------------------------------------------------------------+
|  Host Page DOM                                              |
|  +-----------------------------------------------------+  |
|  |  <search-widget>                                    |  |
|  |    #shadow-root (isolates CSS and DOM)               |  |
|  |    +-----------------------------------------------+ |  |
|  |    |  React app running inside                    | |  |
|  |    |  Own styles, own state, own lifecycle        | |  |
|  |    |  Cannot be affected by host page CSS         | |  |
|  |    +-----------------------------------------------+ |  |
|  +-----------------------------------------------------+  |
|                                                             |
|  Host page styles do NOT leak into shadow DOM               |
|  Shadow DOM styles do NOT leak into host page               |
+-------------------------------------------------------------+
```

Composition Approach 3: Routing-Based Composition

The simplest approach: different URLs load entirely different applications. A reverse proxy or CDN routes requests to the correct app.

```
Routing-Based Composition:

User visits:                     Served by:
www.example.com/               --> Marketing App (Next.js)
www.example.com/products/*     --> Products App (React)
www.example.com/cart           --> Cart App (React)
www.example.com/account/*      --> Account App (React)
www.example.com/admin/*        --> Admin App (Vue)

+------------------+
|  Reverse Proxy   |     (Nginx, Cloudflare, or CDN)
|  / Load Balancer |
+--------+---------+
         |
    +----+----+----+----+
    |    |    |    |    |
    v    v    v    v    v
  Mktg Prod  Cart Acct Admin
  App  App   App  App  App
```

Nginx Configuration Example

```nginx
# nginx.conf
server {
    listen 80;
    server_name www.example.com;

    # Marketing pages (Next.js on port 3000)
    location / {
        proxy_pass http://marketing-app:3000;
    }

    # Product catalog (React on port 3001)
    location /products {
        proxy_pass http://products-app:3001;
    }

    # Shopping cart (React on port 3002)
    location /cart {
        proxy_pass http://cart-app:3002;
```

```
    }

    # Account management (React on port 3003)
    location /account {
        proxy_pass http://account-app:3003;
    }

    # Admin panel (Vue on port 3004)
    location /admin {
        proxy_pass http://admin-app:3004;
    }
}
```

Shared Header via Server-Side Includes

```html
<!-- Each micro-frontend includes the shared header -->
<!-- products-app/index.html -->
<!DOCTYPE html>
<html>
<head>
  <title>Products</title>
  <link rel="stylesheet" href="/shared/header.css" />
</head>
<body>
  <!-- Shared header loaded from CDN -->
  <div id="shared-header"></div>
  <script src="https://cdn.example.com/shared-header.js"></script>

  <!-- This app's content -->
  <div id="root"></div>
  <script src="/products-bundle.js"></script>
</body>
</html>
```

Communication Between Micro-Frontends

Micro-frontends need to communicate sometimes. A product being added to the cart needs to update the cart icon. Here are the main patterns:

Pattern 1: Custom Events

```jsx
// Product MFE: dispatches event when product is added to cart
function ProductCard({ product }) {
  function handleAddToCart() {
    // Add to cart via API
    fetch('/api/cart', {
      method: 'POST',
      body: JSON.stringify({ productId: product.id }),
    });

    // Notify other micro-frontends via custom event
    window.dispatchEvent(
      new CustomEvent('cart:item-added', {
        detail: {
          productId: product.id,
          name: product.name,
          price: product.price,
        },
      })
    );
  }

  return (
    <div className="product-card">
      <h3>{product.name}</h3>
      <button onClick={handleAddToCart}>Add to Cart</button>
    </div>
  );
}

// Cart MFE: listens for cart events
function CartIcon() {
  const [itemCount, setItemCount] = useState(0);

  useEffect(() => {
    function handleItemAdded(event) {
      console.log('Item added:', event.detail.name);
      setItemCount(prev => prev + 1);
    }

    window.addEventListener('cart:item-added', handleItemAdded);
    return () => {
      window.removeEventListener('cart:item-added', handleItemAdded);
    };
  }, []);

  return (
    <div className="cart-icon">
      Cart ({itemCount})
    </div>
  );
}
```

Pattern 2: Shared State via URL

```jsx
// Search MFE: puts search query in URL
function SearchBar() {
  const [searchParams, setSearchParams] = useSearchParams();
  const query = searchParams.get('q') || '';

  function handleSearch(newQuery) {
    setSearchParams({ q: newQuery });
    // URL changes to /products?q=shoes
    // Product MFE reads the query from URL
  }

  return (
    <input
      value={query}
      onChange={e => handleSearch(e.target.value)}
      placeholder="Search..."
    />
  );
}

// Product MFE: reads search query from URL
function ProductList() {
  const [searchParams] = useSearchParams();
  const query = searchParams.get('q') || '';

  const products = useProducts(query);

  return (
    <div>
      {query && <p>Results for: {query}</p>}
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}
```

Pattern 3: Shared API/Service Layer

```jsx
// shared-services/cart-service.js
// Deployed as a shared npm package or loaded from CDN

class CartService {
  constructor() {
    this.listeners = new Set();
  }

  async addItem(productId) {
    const response = await fetch('/api/cart', {
      method: 'POST',
      body: JSON.stringify({ productId }),
```

```
    });
    const cart = await response.json();
    this.notify(cart);
    return cart;
  }

  subscribe(callback) {
    this.listeners.add(callback);
    return () => this.listeners.delete(callback);
  }

  notify(cart) {
    this.listeners.forEach(cb => cb(cart));
  }
}

// Singleton instance shared across micro-frontends
export const cartService = new CartService();

// Product MFE uses it:
import { cartService } from '@shared/cart-service';
function AddToCartButton({ productId }) {
  return (
    <button onClick={() => cartService.addItem(productId)}>
      Add to Cart
    </button>
  );
}

// Cart MFE uses it:
import { cartService } from '@shared/cart-service';
function CartBadge() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    return cartService.subscribe(cart => setCount(cart.items.length));
  }, []);
  return <span className="badge">{count}</span>;
}
```

Trade-Offs: What You Gain and What You Pay

```
+-------------------------------+-------------------------------+
|         GAINS                 |         COSTS                 |
+-------------------------------+-------------------------------+
| Independent deployments       | More infrastructure to manage |
| Team autonomy                 | Cross-team coordination       |
| Smaller, focused codebases    | Duplicated dependencies       |
| Isolated failures             | Harder debugging across MFEs  |
```

```
| Tech stack flexibility        | Inconsistent UX if not careful|
| Faster CI/CD per team         | Complex routing and state     |
| Parallel development          | Performance overhead (multiple|
|                               |  frameworks, extra JS loaded) |
| Scale teams independently     | Shared design system needed   |
+-------------------------------+-------------------------------+
```

The Dependency Duplication Problem

```
Monolith:
  react: loaded ONCE (42 KB)
  react-dom: loaded ONCE (130 KB)
  Total React cost: 172 KB

Naive Micro-Frontends (3 MFEs, each bundles React):
  MFE 1: react + react-dom (172 KB)
  MFE 2: react + react-dom (172 KB)
  MFE 3: react + react-dom (172 KB)
  Total React cost: 516 KB  (3x the monolith!)

Module Federation with shared dependencies:
  Shared: react + react-dom (172 KB, loaded ONCE)
  MFE 1: own code only (50 KB)
  MFE 2: own code only (80 KB)
  MFE 3: own code only (60 KB)
  Total: 362 KB (much better, some overhead remains)
```

The Consistency Challenge

```
Without a shared design system:

  Search MFE:    [  Search  ]   (blue button, 14px font)
  Product MFE:   [Add to Cart]  (green button, 16px font)
  Cart MFE:      [ Checkout ]   (blue button, 15px font)

  Users notice the inconsistency. It feels broken.

With a shared design system:

  All MFEs import from @company/design-system
  Search MFE:    [  Search  ]   (same button style)
  Product MFE:   [Add to Cart]  (same button style)
  Cart MFE:      [ Checkout ]   (same button style)

  The design system is versioned. Teams can upgrade
  at their own pace, but all buttons look the same.
```

Real-World Use Case: E-Commerce Platform

```
Architecture for a large e-commerce company:

+-------------------------------------------------------------------+
|  Shell Application                                                |
|  - Authentication                                                 |
|  - Routing                                                        |
|  - Shared Header/Footer                                           |
|  - Design System CSS                                              |
|  - Analytics SDK                                                  |
+-------------------------------------------------------------------+
         |              |              |              |
         v              v              v              v
+------------+  +------------+  +------------+  +------------+
| Search MFE |  | Catalog MFE|  | Cart MFE   |  | Account MFE|
|            |  |            |  |            |  |            |
| Features:  |  | Features:  |  | Features:  |  | Features:  |
| - Search   |  | - Browse   |  | - Cart     |  | - Profile  |
| - Autocmpl |  | - Filters  |  | - Checkout |  | - Orders   |
| - Filters  |  | - Detail   |  | - Payment  |  | - Wishlist |
|            |  | - Compare  |  |            |  |            |
| Team: 4 dev|  | Team: 6 dev|  | Team: 5 dev|  | Team: 3 dev|
| Deploy: 5x |  | Deploy: 3x |  | Deploy: 2x |  | Deploy: 1x |
| /day       |  | /day       |  | /day       |  | /week      |
+------------+  +------------+  +------------+  +------------+

Communication:
  Search --> Catalog: URL params (?q=shoes&category=running)
  Catalog --> Cart: Custom Event (cart:item-added)
  Cart --> Account: Shared Auth Token
  All --> Shell: Custom Events for header badge updates
```

```jsx
// shell-app/src/App.jsx
import { lazy, Suspense } from 'react';
import { ErrorBoundary } from 'react-error-boundary';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import SharedHeader from './components/SharedHeader';
import SharedFooter from './components/SharedFooter';
import { AuthProvider } from './auth/AuthProvider';

// Remote micro-frontends via Module Federation
const SearchPage = lazy(() => import('searchApp/SearchPage'));
const CatalogPage = lazy(() => import('catalogApp/CatalogPage'));
const ProductPage = lazy(() => import('catalogApp/ProductPage'));
const CartPage = lazy(() => import('cartApp/CartPage'));
const CheckoutPage = lazy(() => import('cartApp/CheckoutPage'));
```

```
const AccountPage = lazy(() => import('accountApp/AccountPage'));

function MFEFallback({ name }) {
  return (
    <div className="mfe-loading">
      <div className="skeleton" />
      <p>Loading {name}...</p>
    </div>
  );
}

function MFEError({ error, resetErrorBoundary }) {
  return (
    <div className="mfe-error">
      <p>This section could not be loaded.</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

function MFEWrapper({ children, name }) {
  return (
    <ErrorBoundary FallbackComponent={MFEError}>
      <Suspense fallback={<MFEFallback name={name} />}>
        {children}
      </Suspense>
    </ErrorBoundary>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <SharedHeader />
        <main>
          <Routes>
            <Route path="/search" element={
              <MFEWrapper name="Search">
                <SearchPage />
              </MFEWrapper>
            } />
            <Route path="/products" element={
              <MFEWrapper name="Catalog">
                <CatalogPage />
              </MFEWrapper>
            } />
            <Route path="/products/:id" element={
              <MFEWrapper name="Product">
                <ProductPage />
              </MFEWrapper>
            } />
            <Route path="/cart" element={
              <MFEWrapper name="Cart">
                <CartPage />
```

```
              </MFEWrapper>
            } />
            <Route path="/checkout" element={
              <MFEWrapper name="Checkout">
                <CheckoutPage />
              </MFEWrapper>
            } />
            <Route path="/account/*" element={
              <MFEWrapper name="Account">
                <AccountPage />
              </MFEWrapper>
            } />
          </Routes>
        </main>
        <SharedFooter />
      </BrowserRouter>
    </AuthProvider>
  );
}
```

When to Use / When NOT to Use

Use Micro-Frontends When

- You have multiple teams (3+) working on the same frontend
- Teams are blocked by each other's deploy schedules
- The monolith codebase has become unmanageable (build times, merge conflicts)
- Different parts of the app have very different release cadences
- Your organization is structured around independent product teams

Do NOT Use Micro-Frontends When

- You have a small team (under 10 developers)
- A single team maintains the entire frontend
- The application is small or medium-sized
- You do not have the infrastructure to support multiple deploy pipelines
- You are using it because it sounds cool (the most common wrong reason)
- Your team does not have experience managing distributed systems

Common Mistakes

Mistake 1: Adopting Micro-Frontends Too Early

```
WRONG thinking:
"We have 5 developers and one React app. Let us split it into
 micro-frontends so it scales better."

REALITY:
5 developers do not need micro-frontends. The overhead of managing
multiple repositories, deploy pipelines, shared dependencies, and
cross-app communication far outweighs any benefit.

RULE OF THUMB:
If you have fewer than 3 independent teams with different
release schedules, a well-organized monolith is better.
```

Mistake 2: No Shared Design System

```
WRONG: Each team builds their own buttons, forms, and layouts.
RESULT: The app looks like a Frankenstein monster of different
        UIs stitched together. Users notice immediately.

RIGHT: Invest in a shared design system (component library)
       that all teams consume via npm.
       @company/design-system provides Button, Input, Modal, etc.
       All micro-frontends use the same visual components.
```

Mistake 3: Tight Coupling Between Micro-Frontends

```
WRONG: Cart MFE imports a function directly from Product MFE.
       Now they cannot deploy independently.

RIGHT: Micro-frontends communicate through:
       - Custom Events (loose coupling)
       - URL parameters (stateless)
       - Shared services with stable APIs

If you cannot deploy one MFE without deploying another,
you do not have micro-frontends. You have a distributed monolith.
That is the worst of both worlds.
```

Best Practices

1. Start with a monolith -- Build your app as a monolith first. Split into micro-frontends only when organizational pain demands it.
2. Split by business domain -- Each micro-frontend should own a complete business capability (search, checkout, account), not a technical layer (header, sidebar).
3. Share a design system -- Invest in a versioned component library that all teams consume. Visual consistency is non-negotiable.
4. Use Module Federation for React-to-React -- When all micro-frontends use React, Module Federation provides the smoothest experience with shared dependencies.
5. Communicate loosely -- Use Custom Events, URL params, or a shared event bus. Never import code directly between micro-frontends.
6. Each MFE must work in isolation -- Every micro-frontend should be runnable as a standalone app for development and testing.
7. Monitor bundle sizes -- Track the total JavaScript loaded. Micro-frontends can accidentally increase the total bundle if shared dependencies are duplicated.
8. Establish clear contracts -- Define APIs between micro-frontends (event names, URL param formats) and version them.

Quick Summary

Micro-frontends split a monolithic frontend into independent applications, each owned and deployed by a separate team. The three main composition approaches are Module Federation (Webpack 5 runtime module sharing), Web Components (framework-agnostic custom elements), and routing-based composition (reverse proxy routing to separate apps). Communication between micro-frontends uses Custom Events, URL parameters, or shared services. The pattern provides independent deployments and team autonomy but adds infrastructure complexity, potential dependency duplication, and coordination overhead. Micro-frontends are the right choice for large organizations with multiple independent teams; they are overkill for small teams.

Key Points

- Micro-frontends split one big frontend into independently deployable applications.

- Module Federation (Webpack 5) shares code between apps at runtime with dependency deduplication.
- Web Components provide framework-agnostic composition with Shadow DOM isolation.
- Routing-based composition uses a reverse proxy to route URLs to different apps (simplest approach).
- Communication between micro-frontends should be loose: Custom Events, URL params, or shared services.
- Shared design system is mandatory for visual consistency across teams.
- Do not adopt too early -- The overhead is only justified for large organizations with multiple independent teams.
- Distributed monolith is worse than a real monolith. If MFEs cannot deploy independently, you have the worst of both worlds.

Practice Questions

1. Your company has 8 frontend developers on one team. The CTO wants to adopt micro-frontends because a competitor uses them. What would you advise, and why?
2. Explain the difference between Module Federation and routing-based micro-frontends. When would you choose one over the other?
3. Two micro-frontends need to share the user's authentication status. Describe three different ways they could communicate this, and which you would recommend.
4. A micro-frontend architecture is loading React three times (once per MFE). How does Module Federation's `shared` configuration solve this, and what is the `singleton` option?
5. Your micro-frontend architecture works well, but users complain that the search page looks different from the product page (different button styles, font sizes). What architectural solution would you implement?

Exercises

Exercise 1: Module Federation Setup

Create two separate React applications using Webpack 5:

- App A (Host): A shell with navigation

- App B (Remote): A simple widget (counter or todo list)

Configure Module Federation so App A loads the widget from App B at runtime. Verify that you can update App B's widget and see the changes in App A without rebuilding App A.

Exercise 2: Web Component Wrapper

Take an existing React component and wrap it as a Web Component using `customElements.define`. Verify that it works when inserted into a plain HTML page with no React. Pass attributes to the Web Component and map them to React props.

Exercise 3: Communication Protocol

Build two micro-frontends on one page (use iframes or Module Federation):

- A "Product Picker" that lets users select products
- A "Cart Summary" that shows selected products

Implement communication between them using Custom Events. Define a clear event contract (event names, payload shapes) and handle cases where one MFE loads before the other.

What Is Next?

You now understand patterns from individual components to entire application architectures. In Chapter 31, we bring everything together in a Project Pattern Library -- a complete React application that applies 10+ design patterns from this book, showing how they work in concert in a real codebase.

Chapter 31: Project Pattern Library -- A Complete React Application

What You Will Learn

- How to combine 10+ design patterns in a single, cohesive React application
- How each pattern solves a specific problem in a real project
- The architecture of a production-quality React application
- How patterns interact and complement each other
- A full codebase walkthrough with explanations

Why This Chapter Matters

Throughout this book, you have learned patterns one at a time. Each chapter focused on a single pattern in isolation. But real applications do not use just one pattern. They use many patterns working together, each solving a different problem.

This chapter is the capstone. We build a complete Task Management Application (like a simplified Jira or Trello) that uses patterns from across the book. You will see how patterns compose, where they fit in the architecture, and how they solve real problems you will face in production.

Think of the previous chapters as learning individual musical instruments. This chapter is the orchestra concert where all the instruments play together.

Application Overview

We are building TaskFlow, a task management application with these features:

- Create, edit, and delete tasks
- Organize tasks into boards (To Do, In Progress, Done)
- Filter and search tasks
- User notifications
- Theme switching (light/dark)
- Error recovery
- Performance optimization for large task lists

Architecture Diagram

```
+-----------------------------------------------------------------+
|                     TaskFlow Application                        |
+-----------------------------------------------------------------+
|                                                                 |
|  Patterns Used:                                                 |
|  1.  Module Pattern          - API service organization         |
|  2.  Factory Pattern         - Task and notification creation   |
|  3.  Observer Pattern        - Event system for notifications   |
|  4.  Compound Components     - Board and Column components      |
|  5.  Provider Pattern        - Theme and Auth context           |
|  6.  Custom Hooks            - Reusable stateful logic          |
|  7.  State Machine           - Task status transitions          |
|  8.  Memoization             - Optimized list rendering         |
|  9.  Error Boundary          - Graceful error handling          |
|  10. Code Splitting          - Lazy-loaded routes               |
|  11. Controlled Components   - Form handling                    |
|  12. Container/Presentational- Data/UI separation               |
|                                                                 |
+-----------------------------------------------------------------+

Component Tree:
+-----------------------------------------------------------------+
|  <ErrorBoundary>                          [Error Boundary Pattern]|
|    <ThemeProvider>                         [Provider Pattern]     |
|      <AuthProvider>                        [Provider Pattern]     |
|        <NotificationProvider>              [Observer Pattern]     |
|          <AppShell>                                               |
|            <Suspense>                      [Code Splitting]      |
|              <Routes>                                             |
|                <BoardPage>                                        |
|                  <Board>                   [Compound Components]  |
|                    <Board.Column>                                 |
|                      <TaskList>            [Memoization]          |
|                        <TaskCard>          [State Machine]        |
|                <TaskFormPage>              [Controlled Components]|
|                <SettingsPage>              [Code Splitting]       |
|              </Routes>                                            |
|            </Suspense>                                            |
|          </AppShell>                                              |
|        </NotificationProvider>                                    |
```

```
|      </AuthProvider>                                            |
|    </ThemeProvider>                                              |
|  </ErrorBoundary>                                               |
+-----------------------------------------------------------------+
```

Pattern 1: Module Pattern -- API Service

The Module Pattern organizes our API layer into clean, self-contained modules with clear public interfaces.

```jsx
// src/services/taskService.js
// MODULE PATTERN: Encapsulates API logic behind a clean interface

const API_BASE = '/api';

// Private helper (not exported)
async function request(endpoint, options = {}) {
  const response = await fetch(`${API_BASE}${endpoint}`, {
    headers: {
      'Content-Type': 'application/json',
      ...options.headers,
    },
    ...options,
  });

  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || `HTTP ${response.status}`);
  }

  return response.json();
}

// Public interface
export const taskService = {
  getAll() {
    return request('/tasks');
  },

  getById(id) {
    return request(`/tasks/${id}`);
  },

  create(taskData) {
    return request('/tasks', {
      method: 'POST',
```

```
      body: JSON.stringify(taskData),
    });
  },

  update(id, updates) {
    return request(`/tasks/${id}`, {
      method: 'PATCH',
      body: JSON.stringify(updates),
    });
  },

  delete(id) {
    return request(`/tasks/${id}`, {
      method: 'DELETE',
    });
  },

  search(query) {
    return request(`/tasks/search?q=${encodeURIComponent(query)}`);
  },
};

// Usage elsewhere:
// import { taskService } from './services/taskService';
// const tasks = await taskService.getAll();
// const newTask = await taskService.create({ title: 'Fix bug' });
```

Pattern 2: Factory Pattern -- Creating Tasks and Notifications

The Factory Pattern creates objects with consistent structure without exposing the creation logic.

```jsx
// src/factories/taskFactory.js
// FACTORY PATTERN: Creates task objects with proper defaults and validation

let nextId = 1;

export function createTask({
  title,
  description = '',
  priority = 'medium',
  assignee = null,
  status = 'todo',
}) {
```

```
  // Validation
  if (!title || title.trim().length === 0) {
    throw new Error('Task title is required');
  }

  const validPriorities = ['low', 'medium', 'high', 'urgent'];
  if (!validPriorities.includes(priority)) {
    throw new Error(`Invalid priority: ${priority}`);
  }

  const validStatuses = ['todo', 'in_progress', 'in_review', 'done'];
  if (!validStatuses.includes(status)) {
    throw new Error(`Invalid status: ${status}`);
  }

  return {
    id: `task-${nextId++}`,
    title: title.trim(),
    description: description.trim(),
    priority,
    assignee,
    status,
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
  };
}

// src/factories/notificationFactory.js
// FACTORY PATTERN: Creates notification objects

let notifId = 1;

export function createNotification(type, message, options = {}) {
  const defaults = {
    info: { icon: 'info', duration: 3000, color: 'blue' },
    success: { icon: 'check', duration: 2000, color: 'green' },
    warning: { icon: 'alert', duration: 5000, color: 'yellow' },
    error: { icon: 'error', duration: 0, color: 'red' }, // 0 = no auto-dismiss
  };

  const config = defaults[type] || defaults.info;

  return {
    id: `notif-${notifId++}`,
    type,
    message,
    icon: options.icon || config.icon,
    duration: options.duration ?? config.duration,
    color: config.color,
    createdAt: Date.now(),
  };
}

// Usage:
// const task = createTask({ title: 'Implement login', priority: 'high' });
```

```
// const notif = createNotification('success', 'Task created!');
```

Pattern 3: Observer Pattern -- Event System

The Observer Pattern provides a publish/subscribe system for cross-component communication.

```jsx
// src/events/eventBus.js
// OBSERVER PATTERN: Decoupled communication between components

class EventBus {
  constructor() {
    this.listeners = new Map();
  }

  on(event, callback) {
    if (!this.listeners.has(event)) {
      this.listeners.set(event, new Set());
    }
    this.listeners.get(event).add(callback);

    // Return unsubscribe function
    return () => {
      this.listeners.get(event)?.delete(callback);
    };
  }

  emit(event, data) {
    const callbacks = this.listeners.get(event);
    if (callbacks) {
      callbacks.forEach(cb => cb(data));
    }
  }

  off(event, callback) {
    this.listeners.get(event)?.delete(callback);
  }
}

export const eventBus = new EventBus();

// Event constants for type safety
export const EVENTS = {
  TASK_CREATED: 'task:created',
  TASK_UPDATED: 'task:updated',
  TASK_DELETED: 'task:deleted',
```

```
  TASK_MOVED: 'task:moved',
  NOTIFICATION: 'notification',
};

// Usage in components:
// eventBus.emit(EVENTS.TASK_CREATED, { task });
// const unsub = eventBus.on(EVENTS.TASK_CREATED, (data) => { ... });
// unsub(); // cleanup
```

Pattern 4: Compound Components -- Board Layout

The Compound Components Pattern creates a flexible board with columns that share implicit state.

```jsx
// src/components/Board/Board.jsx
// COMPOUND COMPONENTS PATTERN: Board and Column share context

import { createContext, useContext, useMemo } from 'react';

const BoardContext = createContext(null);

function Board({ children, onTaskMove }) {
  const contextValue = useMemo(
    () => ({ onTaskMove }),
    [onTaskMove]
  );

  return (
    <BoardContext.Provider value={contextValue}>
      <div className="board">
        {children}
      </div>
    </BoardContext.Provider>
  );
}

function Column({ status, title, children }) {
  const { onTaskMove } = useContext(BoardContext);

  function handleDrop(e) {
    e.preventDefault();
    const taskId = e.dataTransfer.getData('taskId');
    onTaskMove(taskId, status);
  }

  function handleDragOver(e) {
```

```
    e.preventDefault();
  }

  return (
    <div
      className="board-column"
      onDrop={handleDrop}
      onDragOver={handleDragOver}
    >
      <h2 className="column-header">{title}</h2>
      <div className="column-content">
        {children}
      </div>
    </div>
  );
}

// Attach sub-components
Board.Column = Column;

export default Board;

// Usage:
// <Board onTaskMove={handleTaskMove}>
//   <Board.Column status="todo" title="To Do">
//     {todoTasks.map(t => <TaskCard key={t.id} task={t} />)}
//   </Board.Column>
//   <Board.Column status="in_progress" title="In Progress">
//     {inProgressTasks.map(t => <TaskCard key={t.id} task={t} />)}
//   </Board.Column>
//   <Board.Column status="done" title="Done">
//     {doneTasks.map(t => <TaskCard key={t.id} task={t} />)}
//   </Board.Column>
// </Board>
```

Pattern 5: Provider Pattern -- Theme and Auth

The Provider Pattern makes values available to any component in the tree without prop drilling.

```jsx
// src/providers/ThemeProvider.jsx
// PROVIDER PATTERN: Theme context available app-wide

import { createContext, useContext, useState, useMemo } from 'react';

const ThemeContext = createContext(null);
```

```
const themes = {
  light: {
    background: '#ffffff',
    surface: '#f5f5f5',
    text: '#1a1a1a',
    primary: '#3b82f6',
    border: '#e5e7eb',
  },
  dark: {
    background: '#1a1a2e',
    surface: '#16213e',
    text: '#e0e0e0',
    primary: '#60a5fa',
    border: '#374151',
  },
};

export function ThemeProvider({ children }) {
  const [themeName, setThemeName] = useState(() => {
    return localStorage.getItem('theme') || 'light';
  });

  const value = useMemo(() => ({
    theme: themes[themeName],
    themeName,
    toggleTheme: () => {
      setThemeName(prev => {
        const next = prev === 'light' ? 'dark' : 'light';
        localStorage.setItem('theme', next);
        return next;
      });
    },
  }), [themeName]);

  return (
    <ThemeContext.Provider value={value}>
      <div style={{
        backgroundColor: value.theme.background,
        color: value.theme.text,
        minHeight: '100vh',
      }}>
        {children}
      </div>
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme must be used within ThemeProvider');
  }
  return context;
}
```

```
// src/providers/AuthProvider.jsx
// PROVIDER PATTERN: Auth state available app-wide

import { createContext, useContext, useState, useMemo, useCallback } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(false);

  const login = useCallback(async (email, password) => {
    setLoading(true);
    try {
      const response = await fetch('/api/auth/login', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
      });
      const userData = await response.json();
      setUser(userData);
      return userData;
    } finally {
      setLoading(false);
    }
  }, []);

  const logout = useCallback(() => {
    setUser(null);
    localStorage.removeItem('token');
  }, []);

  const value = useMemo(
    () => ({ user, loading, login, logout, isAuthenticated: !!user }),
    [user, loading, login, logout]
  );

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
}
```

Pattern 6: Custom Hooks -- Reusable Logic

Custom Hooks extract and share stateful logic across components.

```jsx
// src/hooks/useTasks.js
// CUSTOM HOOKS PATTERN: Reusable data fetching and task management logic

import { useState, useEffect, useCallback } from 'react';
import { taskService } from '../services/taskService';
import { eventBus, EVENTS } from '../events/eventBus';
import { createNotification } from '../factories/notificationFactory';

export function useTasks() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  // Fetch all tasks
  useEffect(() => {
    let cancelled = false;

    async function fetchTasks() {
      try {
        setLoading(true);
        const data = await taskService.getAll();
        if (!cancelled) {
          setTasks(data);
          setError(null);
        }
      } catch (err) {
        if (!cancelled) {
          setError(err.message);
        }
      } finally {
        if (!cancelled) {
          setLoading(false);
        }
      }
    }

    fetchTasks();
    return () => { cancelled = true; };
  }, []);

  const addTask = useCallback(async (taskData) => {
    const newTask = await taskService.create(taskData);
    setTasks(prev => [...prev, newTask]);
    eventBus.emit(EVENTS.TASK_CREATED, { task: newTask });
    eventBus.emit(EVENTS.NOTIFICATION,
      createNotification('success', `Task "${newTask.title}" created`)
    );
    return newTask;
```

```
  }, []);

  const updateTask = useCallback(async (id, updates) => {
    const updated = await taskService.update(id, updates);
    setTasks(prev => prev.map(t => (t.id === id ? updated : t)));
    eventBus.emit(EVENTS.TASK_UPDATED, { task: updated });
    return updated;
  }, []);

  const deleteTask = useCallback(async (id) => {
    await taskService.delete(id);
    setTasks(prev => prev.filter(t => t.id !== id));
    eventBus.emit(EVENTS.TASK_DELETED, { taskId: id });
    eventBus.emit(EVENTS.NOTIFICATION,
      createNotification('info', 'Task deleted')
    );
  }, []);

  const moveTask = useCallback(async (taskId, newStatus) => {
    const updated = await taskService.update(taskId, { status: newStatus });
    setTasks(prev => prev.map(t => (t.id === taskId ? updated : t)));
    eventBus.emit(EVENTS.TASK_MOVED, { task: updated, newStatus });
  }, []);

  return { tasks, loading, error, addTask, updateTask, deleteTask, moveTask };
}

// src/hooks/useSearch.js
// CUSTOM HOOKS PATTERN: Debounced search logic

import { useState, useEffect, useRef } from 'react';

export function useSearch(items, searchFields, delay = 300) {
  const [query, setQuery] = useState('');
  const [results, setResults] = useState(items);
  const timeoutRef = useRef(null);

  useEffect(() => {
    if (!query.trim()) {
      setResults(items);
      return;
    }

    clearTimeout(timeoutRef.current);
    timeoutRef.current = setTimeout(() => {
      const lowerQuery = query.toLowerCase();
      const filtered = items.filter(item =>
        searchFields.some(field =>
          String(item[field]).toLowerCase().includes(lowerQuery)
        )
      );
      setResults(filtered);
    }, delay);

    return () => clearTimeout(timeoutRef.current);
```

```
  }, [query, items, searchFields, delay]);

  return { query, setQuery, results };
}

// src/hooks/useNotifications.js
// CUSTOM HOOKS PATTERN: Notification management with Observer

import { useState, useEffect, useCallback } from 'react';
import { eventBus, EVENTS } from '../events/eventBus';

export function useNotifications() {
  const [notifications, setNotifications] = useState([]);

  useEffect(() => {
    const unsub = eventBus.on(EVENTS.NOTIFICATION, (notification) => {
      setNotifications(prev => [...prev, notification]);

      // Auto-dismiss if duration > 0
      if (notification.duration > 0) {
        setTimeout(() => {
          setNotifications(prev =>
            prev.filter(n => n.id !== notification.id)
          );
        }, notification.duration);
      }
    });

    return unsub;
  }, []);

  const dismiss = useCallback((id) => {
    setNotifications(prev => prev.filter(n => n.id !== id));
  }, []);

  return { notifications, dismiss };
}
```

Pattern 7: State Machine -- Task Status Transitions

The State Machine Pattern ensures tasks can only transition through valid statuses.

```jsx
// src/stateMachines/taskStatusMachine.js
// STATE MACHINE PATTERN: Valid task status transitions

const taskStatusMachine = {
```

```
  todo: {
    START_WORK: 'in_progress',
  },
  in_progress: {
    SUBMIT_REVIEW: 'in_review',
    MOVE_BACK: 'todo',
  },
  in_review: {
    APPROVE: 'done',
    REQUEST_CHANGES: 'in_progress',
  },
  done: {
    REOPEN: 'todo',
  },
};

export function getNextStatus(currentStatus, action) {
  const transitions = taskStatusMachine[currentStatus];
  if (!transitions) {
    throw new Error(`Unknown status: ${currentStatus}`);
  }

  const nextStatus = transitions[action];
  if (!nextStatus) {
    throw new Error(
      `Invalid action "${action}" for status "${currentStatus}". ` +
      `Valid actions: ${Object.keys(transitions).join(', ')}`
    );
  }

  return nextStatus;
}

export function getAvailableActions(currentStatus) {
  const transitions = taskStatusMachine[currentStatus];
  if (!transitions) return [];

  return Object.entries(transitions).map(([action, targetStatus]) => ({
    action,
    targetStatus,
    label: formatActionLabel(action),
  }));
}

function formatActionLabel(action) {
  const labels = {
    START_WORK: 'Start Working',
    SUBMIT_REVIEW: 'Submit for Review',
    MOVE_BACK: 'Move Back to To Do',
    APPROVE: 'Approve',
    REQUEST_CHANGES: 'Request Changes',
    REOPEN: 'Reopen',
  };
  return labels[action] || action;
}
```

```
// Visualization of the state machine:
//
//   +--------+    START_WORK    +--------------+
//   |  todo  | --------------> | in_progress  |
//   +--------+                 +--------------+
//       ^                        |          ^
//       |  REOPEN      SUBMIT_   |          | REQUEST_
//       |              REVIEW    v          | CHANGES
//   +--------+               +-----------+
//   |  done  | <------------ | in_review |
//   +--------+    APPROVE    +-----------+
```

jsx

```jsx
// src/components/TaskCard/TaskActions.jsx
// Using the state machine in a component

import { getAvailableActions, getNextStatus } from '../../stateMachines/taskStatusMachi...

function TaskActions({ task, onStatusChange }) {
  const availableActions = getAvailableActions(task.status);

  function handleAction(action) {
    const newStatus = getNextStatus(task.status, action);
    onStatusChange(task.id, newStatus);
  }

  return (
    <div className="task-actions">
      {availableActions.map(({ action, label }) => (
        <button
          key={action}
          onClick={() => handleAction(action)}
          className="action-button"
        >
          {label}
        </button>
      ))}
    </div>
  );
}

export default TaskActions;

// Output for a task with status "in_progress":
// [Submit for Review]  [Move Back to To Do]
//
// Output for a task with status "in_review":
// [Approve]  [Request Changes]
//
// There is NO way to go directly from "todo" to "done"!
// The state machine enforces the workflow.
```

Pattern 8: Memoization -- Optimized Rendering

The Memoization Pattern prevents unnecessary re-renders in the task list.

```jsx
// src/components/TaskList/TaskList.jsx
// MEMOIZATION PATTERN: Optimized list rendering

import React, { useMemo, useCallback, memo } from 'react';
import TaskCard from '../TaskCard/TaskCard';

const MemoizedTaskCard = memo(TaskCard);

function TaskList({ tasks, status, onStatusChange, onDelete }) {
  // Memoize filtered tasks - recalculates only when tasks or status change
  const filteredTasks = useMemo(
    () => tasks.filter(t => t.status === status),
    [tasks, status]
  );

  // Stable callback references for memoized children
  const handleStatusChange = useCallback(
    (taskId, newStatus) => onStatusChange(taskId, newStatus),
    [onStatusChange]
  );

  const handleDelete = useCallback(
    (taskId) => onDelete(taskId),
    [onDelete]
  );

  if (filteredTasks.length === 0) {
    return (
      <div className="empty-list">
        <p>No tasks here yet.</p>
      </div>
    );
  }

  return (
    <div className="task-list">
      {filteredTasks.map(task => (
        <MemoizedTaskCard
          key={task.id}
          task={task}
          onStatusChange={handleStatusChange}
          onDelete={handleDelete}
        />
      ))}
    </div>
```

```
  );
}

export default memo(TaskList);

// Why memoization matters here:
// - When user types in the search box, App re-renders
// - Without memo, ALL TaskCards re-render (even if tasks have not changed)
// - With memo, only the TaskList that actually changed re-renders
// - With 100+ tasks, this prevents 100+ unnecessary re-renders per keystroke
```

Pattern 9: Error Boundary -- Graceful Failures

The Error Boundary Pattern prevents one broken component from crashing the entire app.

```jsx
// src/components/ErrorBoundary/AppErrorBoundary.jsx
// ERROR BOUNDARY PATTERN: Layered error handling

import { ErrorBoundary } from 'react-error-boundary';

function AppErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="app-error" role="alert">
      <h1>Application Error</h1>
      <p>Something unexpected happened. We are working on fixing it.</p>
      <div className="error-actions">
        <button onClick={resetErrorBoundary}>Try Again</button>
        <button onClick={() => window.location.reload()}>
          Reload Page
        </button>
      </div>
    </div>
  );
}

function BoardErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="board-error" role="alert">
      <p>Could not load this board section.</p>
      <button onClick={resetErrorBoundary}>Retry</button>
    </div>
  );
}

// Usage in the app:
function App() {
```

```
  return (
    // App-level boundary (last resort)
    <ErrorBoundary FallbackComponent={AppErrorFallback}>
      <ThemeProvider>
        <AuthProvider>
          <AppShell>
            {/* Board-level boundaries (granular) */}
            <ErrorBoundary FallbackComponent={BoardErrorFallback}>
              <BoardPage />
            </ErrorBoundary>
          </AppShell>
        </AuthProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}
```

Pattern 10: Code Splitting -- Lazy Routes

The Code Splitting Pattern loads page-level components only when navigated to.

```jsx
// src/App.jsx
// CODE SPLITTING PATTERN: Lazy-loaded routes

import { lazy, Suspense } from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { ErrorBoundary } from 'react-error-boundary';
import { ThemeProvider } from './providers/ThemeProvider';
import { AuthProvider } from './providers/AuthProvider';
import AppShell from './components/AppShell/AppShell';
import LoadingPage from './components/Loading/LoadingPage';

// Lazy load page components
const BoardPage = lazy(() => import('./pages/BoardPage'));
const TaskFormPage = lazy(() => import('./pages/TaskFormPage'));
const SettingsPage = lazy(() => import('./pages/SettingsPage'));
const AnalyticsPage = lazy(() => import('./pages/AnalyticsPage'));

function AppErrorFallback({ error, resetErrorBoundary }) {
  return (
    <div className="app-error" role="alert">
      <h1>Something went wrong</h1>
      <button onClick={resetErrorBoundary}>Try Again</button>
    </div>
  );
}
```

```
export default function App() {
  return (
    <ErrorBoundary FallbackComponent={AppErrorFallback}>
      <ThemeProvider>
        <AuthProvider>
          <BrowserRouter>
            <AppShell>
              <Suspense fallback={<LoadingPage />}>
                <Routes>
                  <Route path="/" element={<BoardPage />} />
                  <Route path="/tasks/new" element={<TaskFormPage />} />
                  <Route path="/tasks/:id/edit" element={<TaskFormPage />} />
                  <Route path="/settings" element={<SettingsPage />} />
                  <Route path="/analytics" element={<AnalyticsPage />} />
                </Routes>
              </Suspense>
            </AppShell>
          </BrowserRouter>
        </AuthProvider>
      </ThemeProvider>
    </ErrorBoundary>
  );
}

// Bundle output:
// main.js         - App shell, providers, router (~80 KB)
// board.chunk.js  - Board page + task components (~120 KB)
// form.chunk.js   - Task form + validation (~60 KB)
// settings.chunk  - Settings page (~40 KB)
// analytics.chunk - Analytics + charts (~200 KB)
//
// Initial load: only 80 KB instead of 500 KB
```

Patterns 11 & 12: Controlled Components and Container/Presentational

```jsx
// src/pages/TaskFormPage.jsx
// CONTAINER/PRESENTATIONAL PATTERN: Separates data logic from UI
// CONTROLLED COMPONENTS PATTERN: Form state managed by React

import { useState, useEffect } from 'react';
import { useParams, useNavigate } from 'react-router-dom';
import { useTasks } from '../hooks/useTasks';
import { createTask } from '../factories/taskFactory';
```

```
import TaskForm from '../components/TaskForm/TaskForm';

// CONTAINER: handles data and logic
export default function TaskFormPage() {
  const { id } = useParams();
  const navigate = useNavigate();
  const { tasks, addTask, updateTask } = useTasks();
  const [submitting, setSubmitting] = useState(false);

  const existingTask = id ? tasks.find(t => t.id === id) : null;
  const isEditing = !!existingTask;

  async function handleSubmit(formData) {
    setSubmitting(true);
    try {
      if (isEditing) {
        await updateTask(id, formData);
      } else {
        const task = createTask(formData); // Factory pattern
        await addTask(task);
      }
      navigate('/');
    } catch (error) {
      console.error('Failed to save task:', error);
    } finally {
      setSubmitting(false);
    }
  }

  return (
    <div className="task-form-page">
      <h1>{isEditing ? 'Edit Task' : 'New Task'}</h1>
      <TaskForm
        initialValues={existingTask}
        onSubmit={handleSubmit}
        submitting={submitting}
      />
    </div>
  );
}

// src/components/TaskForm/TaskForm.jsx
// PRESENTATIONAL + CONTROLLED COMPONENT: Pure UI with controlled inputs

import { useState } from 'react';

export default function TaskForm({ initialValues, onSubmit, submitting }) {
  const [title, setTitle] = useState(initialValues?.title || '');
  const [description, setDescription] = useState(
    initialValues?.description || ''
  );
  const [priority, setPriority] = useState(
    initialValues?.priority || 'medium'
  );
  const [errors, setErrors] = useState({});
```

```
function validate() {
  const newErrors = {};
  if (!title.trim()) newErrors.title = 'Title is required';
  if (title.length > 100) newErrors.title = 'Title must be under 100 characters';
  setErrors(newErrors);
  return Object.keys(newErrors).length === 0;
}

function handleSubmit(e) {
  e.preventDefault();
  if (!validate()) return;
  onSubmit({ title, description, priority });
}

return (
  <form onSubmit={handleSubmit} className="task-form">
    <div className="form-field">
      <label htmlFor="title">Title *</label>
      <input
        id="title"
        type="text"
        value={title}
        onChange={e => setTitle(e.target.value)}
        className={errors.title ? 'input-error' : ''}
        disabled={submitting}
      />
      {errors.title && (
        <span className="error-message">{errors.title}</span>
      )}
    </div>

    <div className="form-field">
      <label htmlFor="description">Description</label>
      <textarea
        id="description"
        value={description}
        onChange={e => setDescription(e.target.value)}
        rows={4}
        disabled={submitting}
      />
    </div>

    <div className="form-field">
      <label htmlFor="priority">Priority</label>
      <select
        id="priority"
        value={priority}
        onChange={e => setPriority(e.target.value)}
        disabled={submitting}
      >
        <option value="low">Low</option>
        <option value="medium">Medium</option>
        <option value="high">High</option>
        <option value="urgent">Urgent</option>
```

```
        </select>
      </div>

      <button type="submit" disabled={submitting}>
        {submitting ? 'Saving...' : 'Save Task'}
      </button>
    </form>
  );
}
```

Putting It All Together: The Board Page

```jsx
// src/pages/BoardPage.jsx
// This page uses nearly ALL the patterns together

import { useMemo, useCallback } from 'react';
import { ErrorBoundary } from 'react-error-boundary';
import Board from '../components/Board/Board';
import TaskList from '../components/TaskList/TaskList';
import { useTasks } from '../hooks/useTasks';
import { useSearch } from '../hooks/useSearch';

export default function BoardPage() {
  const { tasks, loading, error, moveTask, updateTask, deleteTask } = useTasks();
  const { query, setQuery, results } = useSearch(tasks, ['title', 'description']);

  // Memoize grouped tasks
  const columns = useMemo(() => ({
    todo: results.filter(t => t.status === 'todo'),
    in_progress: results.filter(t => t.status === 'in_progress'),
    in_review: results.filter(t => t.status === 'in_review'),
    done: results.filter(t => t.status === 'done'),
  }), [results]);

  // Stable callbacks for memoized children
  const handleMove = useCallback(
    (taskId, newStatus) => moveTask(taskId, newStatus),
    [moveTask]
  );

  const handleStatusChange = useCallback(
    (taskId, newStatus) => updateTask(taskId, { status: newStatus }),
    [updateTask]
  );

  const handleDelete = useCallback(
```

```
    (taskId) => deleteTask(taskId),
    [deleteTask]
  );

  if (loading) return <BoardSkeleton />;
  if (error) return <BoardError message={error} />;

  return (
    <div className="board-page">
      {/* Search bar */}
      <div className="board-toolbar">
        <input
          type="text"
          placeholder="Search tasks..."
          value={query}
          onChange={e => setQuery(e.target.value)}
          className="search-input"
        />
        <span className="task-count">
          {results.length} task{results.length !== 1 ? 's' : ''}
        </span>
      </div>

      {/* Board with compound components */}
      <Board onTaskMove={handleMove}>
        <Board.Column status="todo" title="To Do">
          <ErrorBoundary fallback={<p>Error loading tasks</p>}>
            <TaskList
              tasks={columns.todo}
              status="todo"
              onStatusChange={handleStatusChange}
              onDelete={handleDelete}
            />
          </ErrorBoundary>
        </Board.Column>

        <Board.Column status="in_progress" title="In Progress">
          <ErrorBoundary fallback={<p>Error loading tasks</p>}>
            <TaskList
              tasks={columns.in_progress}
              status="in_progress"
              onStatusChange={handleStatusChange}
              onDelete={handleDelete}
            />
          </ErrorBoundary>
        </Board.Column>

        <Board.Column status="in_review" title="In Review">
          <ErrorBoundary fallback={<p>Error loading tasks</p>}>
            <TaskList
              tasks={columns.in_review}
              status="in_review"
              onStatusChange={handleStatusChange}
              onDelete={handleDelete}
            />
```

```
          </ErrorBoundary>
        </Board.Column>

        <Board.Column status="done" title="Done">
          <ErrorBoundary fallback={<p>Error loading tasks</p>}>
            <TaskList
              tasks={columns.done}
              status="done"
              onStatusChange={handleStatusChange}
              onDelete={handleDelete}
            />
          </ErrorBoundary>
        </Board.Column>
      </Board>
    </div>
  );
}

function BoardSkeleton() {
  return (
    <div className="board-skeleton">
      {[1, 2, 3, 4].map(i => (
        <div key={i} className="column-skeleton">
          <div className="skeleton-header" />
          <div className="skeleton-card" />
          <div className="skeleton-card" />
        </div>
      ))}
    </div>
  );
}

function BoardError({ message }) {
  return (
    <div className="board-error">
      <h2>Could not load board</h2>
      <p>{message}</p>
      <button onClick={() => window.location.reload()}>Reload</button>
    </div>
  );
}
```

File Structure

```
src/
+-- App.jsx                          # Root: ErrorBoundary + Providers + Router
+-- index.jsx                        # Entry point
|
```

```
+-- pages/                          # Route-level components (lazy loaded)
|   +-- BoardPage.jsx               # Main board view
|   +-- TaskFormPage.jsx            # Create/edit task form
|   +-- SettingsPage.jsx            # User settings
|   +-- AnalyticsPage.jsx           # Task analytics
|
+-- components/                     # Reusable UI components
|   +-- AppShell/
|   |   +-- AppShell.jsx            # Layout shell with nav
|   +-- Board/
|   |   +-- Board.jsx               # Compound component (Board + Column)
|   +-- TaskCard/
|   |   +-- TaskCard.jsx            # Individual task card
|   |   +-- TaskActions.jsx         # Status transition buttons (State Machine)
|   +-- TaskList/
|   |   +-- TaskList.jsx            # Memoized task list
|   +-- TaskForm/
|   |   +-- TaskForm.jsx            # Controlled form component
|   +-- Notifications/
|   |   +-- NotificationToast.jsx   # Toast notification UI
|   +-- Loading/
|   |   +-- LoadingPage.jsx         # Full-page loading state
|   +-- ErrorBoundary/
|       +-- AppErrorBoundary.jsx    # Error fallback components
|
+-- providers/                      # Context providers
|   +-- ThemeProvider.jsx           # Theme context
|   +-- AuthProvider.jsx            # Auth context
|
+-- hooks/                          # Custom hooks
|   +-- useTasks.js                 # Task CRUD operations
|   +-- useSearch.js                # Debounced search
|   +-- useNotifications.js         # Notification management
|
+-- services/                       # API services
|   +-- taskService.js              # Module pattern for API calls
|
+-- factories/                      # Object factories
|   +-- taskFactory.js              # Create task objects
|   +-- notificationFactory.js      # Create notification objects
|
+-- stateMachines/                  # State machines
|   +-- taskStatusMachine.js        # Task status transitions
|
+-- events/                         # Event system
    +-- eventBus.js                 # Observer pattern implementation
```

Pattern Interaction Map

```
User creates a task:

1. TaskFormPage (Container)
   --> calls createTask() from taskFactory (FACTORY)
   --> calls addTask() from useTasks hook (CUSTOM HOOK)

2. useTasks hook
   --> calls taskService.create() (MODULE)
   --> emits TASK_CREATED event via eventBus (OBSERVER)
   --> emits NOTIFICATION event via eventBus (OBSERVER)

3. NotificationToast component
   --> receives notification via useNotifications hook (CUSTOM HOOK)
   --> shows "Task created!" toast

4. BoardPage re-renders
   --> useMemo recalculates column grouping (MEMOIZATION)
   --> Board.Column receives new tasks (COMPOUND COMPONENT)
   --> MemoizedTaskCard renders only new task (MEMOIZATION)
   --> TaskActions shows valid transitions (STATE MACHINE)

5. If any component crashes
   --> ErrorBoundary catches it (ERROR BOUNDARY)
   --> Only that column shows error, rest works fine

All of this is within:
   ThemeProvider (PROVIDER) for consistent styling
   AuthProvider (PROVIDER) for user context
   Lazy-loaded routes (CODE SPLITTING) for fast initial load
```

Quick Summary

This chapter demonstrated how 12 design patterns work together in a single React application. The Module Pattern organizes API services. The Factory Pattern creates consistent objects. The Observer Pattern enables decoupled communication. Compound Components build flexible layouts. The Provider Pattern shares global state. Custom Hooks extract reusable logic. The State Machine Pattern enforces valid transitions. Memoization optimizes rendering performance. Error Boundaries provide graceful failure handling. Code Splitting reduces initial bundle size. Controlled Components manage form state. The Container/Presentational split separates concerns. Each pattern solves a specific problem, and together they create a maintainable, performant, production-quality application.

Key Points

- Patterns complement each other -- No single pattern solves everything. Combine them.
- Module Pattern for API organization keeps service logic contained and testable.
- Factory Pattern ensures objects are created consistently with proper defaults.
- Observer Pattern (EventBus) enables communication without tight coupling.
- Compound Components create flexible, declarative APIs for complex layouts.
- Provider Pattern eliminates prop drilling for cross-cutting concerns.
- Custom Hooks are the primary way to share stateful logic in modern React.
- State Machine enforces business rules at the code level, preventing invalid states.
- Memoization should target expensive components identified through profiling.
- Error Boundaries should be layered: app-level, page-level, and feature-level.
- Code Splitting at route boundaries gives the best effort-to-impact ratio.

Practice Questions

1. In the TaskFlow application, trace the flow from when a user clicks "Start Working" on a task to when the board UI updates. Which patterns are involved at each step?
2. Why does the BoardPage use `useMemo` to group tasks by column instead of filtering in the render? What would happen without memoization when the user types in the search box?
3. The EventBus (Observer Pattern) and the Provider Pattern both share data across components. When would you choose one over the other?
4. If the task status machine allowed a direct transition from "todo" to "done", what code change would you make? How does the State Machine Pattern make this change safe?
5. Explain how Error Boundaries and Code Splitting work together in the App component. What happens if the AnalyticsPage chunk fails to load?

Exercises

Exercise 1: Add a New Pattern

Extend the TaskFlow application with the Decorator Pattern (Chapter 9). Create a `withPriorityHighlight` higher-order component that wraps `TaskCard` and adds a colored border based on the task's priority (red for urgent, orange for high, default for others).

Exercise 2: Build Your Own Project

Choose a different domain (recipe manager, workout tracker, or expense tracker) and build a small application using at least 8 of the patterns covered in this book. Create the file structure, identify which pattern solves each problem, and draw the pattern interaction map.

Exercise 3: Pattern Audit

Take an existing React project (yours or open-source) and audit it for patterns. For each feature, identify:

- Which pattern is currently being used (if any)
- Whether a different pattern would be more appropriate
- Where a missing pattern would add value

Create a table mapping features to recommended patterns with justifications.

What Is Next?

You have now seen every pattern in this book in action, both individually and working together. Chapter 32 provides a Glossary of all key terms, giving you a quick reference to look up any concept from the book.

Chapter 32: Glossary

What You Will Learn

- Quick definitions for 60+ key terms used throughout this book
- Which chapters cover each concept in depth
- A reference you can return to whenever you encounter an unfamiliar term

Why This Chapter Matters

Design patterns come with a lot of vocabulary. When you are reading code, reviewing pull requests, or discussing architecture with your team, having precise definitions at your fingertips saves time and prevents miscommunication. This glossary is your quick-reference companion to the rest of the book.

Terms are organized alphabetically. Each entry includes a concise definition and a reference to the chapter where the concept is covered in detail.

A

Abstract Factory -- A pattern that provides an interface for creating families of related objects without specifying their concrete classes. Related to the Factory Pattern. See Chapter 11.

Anti-pattern -- A common approach that appears to solve a problem but actually creates more problems than it solves. Examples include prop drilling, god components, and premature optimization. See Chapter 1.

App Shell -- The minimal HTML, CSS, and JavaScript needed to render the layout of an application (header, sidebar, navigation) before content loads. Used in combination with code splitting and lazy loading. See Chapters 25, 26.

Async Component -- A React Server Component that uses `async/await` directly in the component body. Only available in Server Components (Next.js App Router). See Chapter 25.

B

Boundary (Error Boundary) -- A React component that catches JavaScript errors in its child component tree, logs the error, and displays a fallback UI instead of crashing the entire application. See Chapter 29.

Bundle -- The combined JavaScript file(s) produced by a build tool (Webpack, Vite, Rollup) from your source code and dependencies. Smaller bundles lead to faster page loads. See Chapter 26.

Bundle Analysis -- The process of inspecting what is inside your JavaScript bundle to identify large dependencies, unused code, and code splitting opportunities. Tools include `webpack-bundle-analyzer` and `rollup-plugin-visualizer`. See Chapter 26.

C

Client Component -- In the React Server Components model, a component marked with `'use client'` that runs in the browser. Can use hooks (`useState`, `useEffect`), event handlers, and browser APIs. Ships JavaScript to the client. See Chapter 25.

Client-Side Rendering (CSR) -- A rendering strategy where the browser downloads an empty HTML page and JavaScript builds the entire UI on the client. Simple to set up but has slower initial load and poor SEO. See Chapter 25.

Code Splitting -- The technique of breaking a JavaScript bundle into smaller chunks that load on demand, reducing initial page load time. Implemented with `React.lazy`, `Suspense`, and dynamic `import()`. See Chapter 26.

Compound Components -- A pattern where a parent component and its children share implicit state through React Context, providing a flexible and declarative API. Example: `<Select>`, `<Select.Option>`. See Chapter 19.

Composition -- Building complex components by combining simpler ones, as opposed to inheritance. The primary method of code reuse in React. See Chapter 14.

Container Component -- A component that handles data fetching, state management, and business logic, passing data to presentational components for rendering. See Chapter 15.

Controlled Component -- A form element whose value is controlled by React state. The component's value is set by a state variable and updated via an `onChange` handler. See Chapter 20.

Context (React Context) -- A React API that allows passing data through the component tree without manually threading props at every level. Used by the Provider Pattern. See Chapters 14, 21.

Custom Hook -- A JavaScript function whose name starts with `use` that encapsulates reusable stateful logic by composing built-in React hooks. See Chapter 18.

D

Decorator Pattern -- A pattern that wraps a component to add behavior or modify its output without changing the original component's code. Higher-Order Components are a React implementation. See Chapter 9.

Dependency Array -- The second argument to hooks like `useEffect`, `useMemo`, and `useCallback`. React re-runs the hook only when values in this array change. See Chapters 18, 28.

Dependency Injection -- A technique where a component receives its dependencies (services, utilities) from the outside rather than creating them internally. The Provider Pattern is a form of dependency injection. See Chapter 21.

Dynamic Import -- The `import()` expression that loads a JavaScript module at runtime rather than at build time. Returns a Promise. The foundation of code splitting. See Chapter 26.

E

Eager Loading -- Loading all code upfront when the page loads, before it is needed. The opposite of lazy loading. See Chapter 26.

Error Boundary -- A React class component that implements `getDerivedStateFromError` and/or `componentDidCatch` to catch rendering errors in child components and display a fallback UI. See Chapter 29.

Event Bus -- An implementation of the Observer Pattern that provides a central hub for publishing and subscribing to events, enabling decoupled communication between components. See Chapters 4, 5.

Event Delegation -- A technique where a single event listener on a parent element handles events from multiple child elements, reducing the number of event listeners in the DOM.

F

Facade Pattern -- A pattern that provides a simplified interface to a complex subsystem. In frontend development, an API service module is a common facade. See Chapter 10.

Factory Pattern -- A pattern that creates objects without specifying their exact class or constructor. Useful for creating consistent data structures with defaults and validation. See Chapter 11.

Fallback UI -- The user interface displayed when a component fails (Error Boundary fallback) or while a component is loading (Suspense fallback). See Chapters 26, 29.

First Contentful Paint (FCP) -- A performance metric measuring the time from page load to when the first piece of content (text, image) appears on screen. Affected by rendering pattern choice. See Chapter 25.

Flux -- An application architecture for managing state with unidirectional data flow: Action, Dispatcher, Store, View. The precursor to Redux. See Chapter 24.

G

Gang of Four (GoF) -- The four authors (Gamma, Helm, Johnson, Vlissides) of the seminal 1994 book on design patterns. They documented 23 object-oriented patterns. See Chapter 1.

getDerivedStateFromError -- A static lifecycle method in React class components that returns updated state when a child component throws an error. Used by Error Boundaries. See Chapter 29.

H

Higher-Order Component (HOC) -- A function that takes a component and returns a new component with additional behavior or props. Example: `withAuth(Component)`. Largely replaced by Custom Hooks. See Chapter 16.

Hydration -- The process where client-side React attaches event listeners and state to server-rendered HTML, making it interactive. Occurs after SSR delivers static HTML to the browser. See Chapter 25.

Hydration Mismatch -- A warning (or error) that occurs when the HTML rendered on the server does not match what React would render on the client. Common causes include using `Date`, `Math.random()`, or `window` in render. See Chapter 25.

I

Incremental Static Regeneration (ISR) -- A rendering strategy that serves statically generated pages but regenerates them in the background after a configurable time interval. Combines SSG speed with SSR freshness. See Chapter 25.

Infinite Scroll -- A UI pattern where more content loads automatically as the user scrolls toward the bottom of the page. Often combined with virtualization for performance. See Chapter 27.

Intersection Observer -- A browser API that detects when an element enters or exits the viewport. Used for lazy loading images, infinite scroll triggers, and prefetching. See Chapters 26, 27.

Iterator Pattern -- A pattern that provides a way to access elements of a collection sequentially without exposing the underlying data structure. In JavaScript, implemented via the iterable protocol (`Symbol.iterator`). See Chapter 7.

K

Key (React Key) -- A special string attribute used when rendering lists of elements. Helps React identify which items changed, were added, or were removed. Must be stable and unique among siblings.

L

Lazy Loading -- Deferring the loading of resources (code, images, data) until they are actually needed. Reduces initial page load time. See Chapter 26.

M

Mediator Pattern -- A pattern where a central object coordinates communication between multiple components, preventing direct references between them. Reduces coupling in complex UIs. See Chapter 12.

Memoization -- Caching the result of a function call based on its arguments. In React, implemented via `React.memo` (components), `useMemo` (values), and `useCallback` (functions). See Chapter 28.

Micro-Frontend -- An architectural pattern where a frontend application is composed of independently developed, tested, and deployed smaller applications, each owned by a separate team. See Chapter 30.

Mixin Pattern -- A pattern that adds functionality to a class or object by copying properties from another object. In modern React, replaced by Custom Hooks and composition. See Chapter 13.

Module Federation -- A Webpack 5 feature that allows independently built applications to share code at runtime. Used in micro-frontend architectures to load remote components dynamically. See Chapter 30.

Module Pattern -- A pattern that encapsulates related code (variables, functions) into a single unit with a clear public API, hiding internal implementation details. In JavaScript, implemented via ES Modules. See Chapter 2.

O

Observer Pattern -- A pattern where an object (subject) maintains a list of dependents (observers) and notifies them of state changes. Foundation for event systems and reactive programming. See Chapter 4.

Overscan -- In virtualization, the extra items rendered above and below the visible viewport to prevent blank flashes during fast scrolling. See Chapter 27.

P

Prefetching -- Loading resources before they are needed, typically during idle time or on user hover, so they are available instantly when requested. See Chapter 26.

Presentational Component -- A component that is purely concerned with how things look. Receives data via props and has no direct data fetching or complex state logic. See Chapter 15.

Prop Drilling -- Passing props through multiple levels of components that do not use them, just to reach a deeply nested child. Solved by the Provider Pattern (Context). See Chapter 21.

Provider Pattern -- A pattern that uses React Context to make values available to any component in the tree without prop drilling. Common for themes, authentication, and localization. See Chapter 21.

Proxy Pattern -- A pattern that provides a surrogate or placeholder for another object to control access to it. In JavaScript, implemented with the `Proxy` built-in object. See Chapter 8.

Publish/Subscribe (Pub/Sub) -- A messaging pattern where publishers emit events without knowing who will receive them, and subscribers listen for events without knowing who sent them. A decoupled variant of the Observer Pattern. See Chapter 5.

R

React.lazy -- A React function that lets you define a component loaded via dynamic import. Used with `Suspense` for code splitting. See Chapter 26.

React.memo -- A higher-order component that memoizes a component, preventing re-renders when props have not changed (shallow comparison). See Chapter 28.

React Server Components (RSC) -- A React feature where components run exclusively on the server, sending zero JavaScript to the client. Can directly access databases and file systems. See Chapter 25.

Reducer -- A pure function that takes the current state and an action, and returns the new state. Used with `useReducer` and in Redux/Flux architectures. See Chapters 22, 24.

Redux -- A state management library implementing the Flux architecture with a single store, pure reducer functions, and middleware. See Chapter 24.

Render Props -- A pattern where a component receives a function as a prop (or as `children`) that returns React elements, giving the parent control over what is rendered. See Chapter 17.

Rendering Pattern -- A strategy for generating HTML: Client-Side Rendering (CSR), Server-Side Rendering (SSR), Static Site Generation (SSG), or Incremental Static Regeneration (ISR). See Chapter 25.

Revalidation -- In ISR, the process of regenerating a static page in the background after a specified time period or on-demand trigger. See Chapter 25.

S

Server Component -- A React component that runs only on the server by default (in the App Router). Cannot use hooks or event handlers. Ships zero JavaScript to the client. See Chapter 25.

Server-Side Rendering (SSR) -- A rendering strategy where the server generates complete HTML for each request and sends it to the browser. Provides fast initial content and good SEO. See Chapter 25.

Shadow DOM -- A browser API that encapsulates a component's DOM and styles, preventing them from leaking into or being affected by the rest of the page. Used with Web Components. See Chapter 30.

Shallow Comparison -- Comparing object properties only one level deep (by reference for objects, by value for primitives). Used by `React.memo`, `useMemo`, and `useCallback`. See Chapter 28.

Singleton Pattern -- A pattern that restricts a class or module to a single instance. In JavaScript modules, every `import` returns the same reference, making modules natural singletons. See Chapter 3.

Skeleton Screen -- A placeholder UI that mimics the layout of the actual content while it loads, providing a better perceived performance than a spinner. Used as Suspense fallbacks. See Chapters 25, 26.

Stale-While-Revalidate -- A caching strategy that returns cached (potentially stale) data immediately while fetching fresh data in the background. The basis for ISR. See Chapter 25.

State Machine -- A model of computation where a system can be in exactly one of a finite number of states at any time, with defined transitions between states triggered by events. See Chapter 23.

State Reducer Pattern -- A pattern where a component accepts a custom reducer function from the parent, allowing the parent to control or modify state transitions. See Chapter 22.

Static Site Generation (SSG) -- A rendering strategy where HTML pages are generated at build time and served as static files from a CDN. The fastest delivery method for content that does not change often. See Chapter 25.

Strategy Pattern -- A pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can choose which algorithm to use at runtime. See Chapter 6.

Streaming SSR -- A Server-Side Rendering technique where HTML is sent to the browser in chunks as each section becomes ready, rather than waiting for the entire page. Uses React's `Suspense` for boundaries. See Chapter 25.

Suspense -- A React component that lets you specify a loading fallback while child components are not yet ready to render (loading lazy components or fetching data in Server Components). See Chapters 25, 26.

T

Time to Interactive (TTI) -- A performance metric measuring how long it takes from page load until the page is fully interactive (event handlers are attached, UI responds to input). See Chapter 25.

Tree Shaking -- A build optimization that removes unused code (dead code) from the final bundle. Relies on ES Module static import/export syntax. See Chapter 26.

U

Uncontrolled Component -- A form element that manages its own internal state via the DOM. Values are read using `ref` rather than React state. Simpler but less flexible than controlled components. See Chapter 20.

useCallback -- A React hook that returns a memoized version of a callback function. The function only changes when its dependencies change. See Chapter 28.

useEffect -- A React hook that runs side effects (data fetching, subscriptions, DOM manipulation) after render. Replaces lifecycle methods like `componentDidMount` and `componentDidUpdate`.

useMemo -- A React hook that memoizes a computed value. The value is only recalculated when its dependencies change. See Chapter 28.

useReducer -- A React hook for managing complex state logic using a reducer function. An alternative to `useState` when state updates depend on previous state or involve multiple sub-values. See Chapters 22, 24.

useState -- A React hook that adds state to functional components. Returns a state value and a setter function.

V

Virtual DOM -- React's in-memory representation of the real DOM. React compares the virtual DOM before and after state changes (reconciliation) and updates only the parts of the real DOM that changed.

Virtual List -- A list component that only renders the items currently visible in the viewport (plus a small buffer), dramatically reducing DOM nodes for large datasets. See Chapter 27.

Virtualization -- The technique of rendering only the visible portion of a large dataset, creating and destroying DOM elements as the user scrolls. Implemented by libraries like `react-window`. See Chapter 27.

W

Web Component -- A set of browser-native APIs (Custom Elements, Shadow DOM, HTML Templates) for creating reusable, encapsulated custom HTML elements. Framework-agnostic. See Chapter 30.

Webpack -- A module bundler that compiles JavaScript modules and their dependencies into static assets (bundles). Supports code splitting, Module Federation, and various optimizations. See Chapters 26, 30.

Window (Virtualization) -- The visible portion of a virtualized list. The "window" determines which items to render based on the scroll position, container height, and item size. See Chapter 27.

Quick Reference by Chapter

Chapter	Key Terms
1	Design Pattern, Anti-pattern, Gang of Four
2	Module Pattern, Encapsulation, ES Modules
3	Singleton Pattern
4	Observer Pattern
5	Publish/Subscribe, Event Bus
6	Strategy Pattern
7	Iterator Pattern
8	Proxy Pattern
9	Decorator Pattern
10	Facade Pattern
11	Factory Pattern
12	Mediator Pattern
13	Mixin Pattern
14	Composition, Component Composition
15	Container Component, Presentational Component
16	Higher-Order Component (HOC)
17	Render Props
18	Custom Hook
19	Compound Components
20	Controlled Component, Uncontrolled Component
21	Provider Pattern, Context, Prop Drilling
22	State Reducer Pattern
23	State Machine
24	Flux, Redux, Reducer
25	CSR, SSR, SSG, ISR, Hydration, Streaming SSR, RSC, Server Component, Client Component

26	Code Splitting, Lazy Loading, React.lazy, Suspense, Dynamic Import, Prefetching, Bundle Analysis
27	Virtualization, Virtual List, Overscan, Window, Infinite Scroll
28	Memoization, React.memo, useMemo, useCallback, Shallow Comparison
29	Error Boundary, Fallback UI, getDerivedStateFromError, componentDidCatch
30	Micro-Frontend, Module Federation, Web Component, Shadow DOM
31	Project Pattern Library (all patterns combined)